OpenArmX 双臂机器人系统

使用说明书

User Manual

产品型号: OpenArmX Bimanual Robot System
文档版本: Version 1.0

成都长数机器人有限公司
Chengdu Changshu Robotics Co., Ltd.

openarmrobot@gmail.com
+86-17746530375
https://openarmx.com/
天津经济技术开发区西区新业八街 11 号华诚机械厂

发布日期: 2025 年 10 月

适用系统: ROS2 Humble / Ubuntu 22.04 LTS

版权声明

本使用说明书的版权归**成都长数机器人有限公司**所有。

使用许可

√ **允许** 用户自由阅读、学习和内部培训使用

√ **允许** 在合法购买产品后复制备份本文档

× **禁止** 未经授权的商业性分发或转售

× **禁止** 删除或修改文档中的版权信息

× **禁止** 将本文档用于任何非法目的

免责声明

1. 本文档内容如有更新, 恕不另行通知, 请访问官网获取最新版本
2. 成都长数机器人有限公司对因使用本文档而产生的任何直接或间接损失不承担责任
3. 使用本产品前, 请务必完整阅读安全声明章节
4. 如因操作不当造成的人身伤害或财产损失, 责任由使用者自行承担

技术支持

如需技术支持或发现文档错误, 请通过以下方式联系我们:

- 技术支持邮箱: openarmrobot@gmail.com
- 技术支持电话/微信: +86-17746530375
- 在线文档: https://openarmx.com/

Copyright © 2025 Chengdu Changshu Robotics Co., Ltd. All Rights Reserved.

目录

1 安全警告 40
1.1 [警告] 重要安全警告 . . . 40
1.2 总体安全事项 . . . 40
1.2.1 适用对象 . . . 40
1.2.2 使用环境要求 . . . 40
1.2.3 电源安全 . . . 40
1.3 机械危险 . . . 40
1.3.1 运动危险 . . . 40
1.3.2 夹持危险 . . . 41
1.3.3 碰撞危险 . . . 41
1.4 电气危险 . . . 41
1.4.1 触电危险 . . . 41
1.4.2 短路危险 . . . 41
1.4.3 CAN 总线安全 . . . 41
1.5 软件与控制危险 . . . 41
1.5.1 误操作风险 . . . 41
1.5.2 控制软件故障 . . . 42
1.5.3 网络安全 . . . 42
1.6 紧急停止 . . . 42
1.6.1 急停按钮 . . . 42
1.6.2 紧急停止操作 . . . 42
1.6.3 急停后处理 . . . 42
1.7 特定操作警告 . . . 43
1.7.1 首次通电 . . . 43
1.7.2 电机零位设置 . . . 43
1.7.3 轨迹执行 . . . 43
1.7.4 示教模式 . . . 43
1.8 维护与检修安全 . . . 43
1.8.1 日常维护 . . . 43
1.8.2 部件更换 . . . 44
1.8.3 软件更新 . . . 44
1.9 禁止事项 . . . 44
1.9.1 [×] 严格禁止以下行为: . . . 44
1.10 风险等级说明 . . . 44
1.11 用户责任 . . . 45
1.12 紧急联系方式 . . . 45

2 操作规范 45
2.1 正确操作流程与规范 . . . 45
2.2 操作人员要求 . . . 45
2.2.1 资质要求 . . . 45
2.2.2 健康要求 . . . 45
2.2.3 着装要求 . . . 46
2.3 开机前检查 . . . 46
2.3.1 环境检查清单 . . . 46
2.3.2 设备检查清单 . . . 46
2.3.3 软件检查清单 . . . 46
2.4 标准开机流程 . . . 47
2.4.1 步骤 1：系统准备 . . . 47
2.4.2 步骤 2：启动 CAN 接口 . . . 47

2.4.3 步骤 3：上电 47
2.4.4 步骤 4：使能电机 48
2.4.5 步骤 5：检查电机状态 48
2.4.6 步骤 6：回零（首次启动或零位丢失时） 48
2.5 日常操作规范 48
2.5.1 单电机测试 48
2.5.2 MoveIt 运动规划 49
2.5.3 轨迹录制 49
2.5.4 示教模式 50
2.6 参数设置规范 50
2.6.1 KP/KD 参数设置 50
2.6.2 速度和加速度限制 51
2.6.3 参数调整原则 51
2.7 异常情况处理规范 51
2.7.1 立即急停的情况 51
2.7.2 急停后检查流程 52
2.7.3 常见异常处理 52
2.8 关机规范 53
2.8.1 标准关机流程 53
2.8.2 紧急关机 54
2.9 维护规范 54
2.9.1 日常维护（每次使用后） 54
2.9.2 定期维护（每月） 54
2.9.3 专业维护（每季度） 54
2.10 记录与文档 54
2.10.1 操作日志 54
2.10.2 参数变更记录 54
2.11 禁止操作清单 55
2.11.1 [×] 以下操作严格禁止： 55
2.12 培训要求 55
2.12.1 理论培训 55
2.12.2 实操培训 55
2.12.3 考核要求 55

3 紧急处理 56
3.1 紧急情况应急处理指南 56
3.2 紧急停止 56
3.2.1 急停按钮位置与使用 56
3.2.2 需要立即急停的情况 56
3.2.3 急停后立即行动 57
3.3 火灾应急 57
3.3.1 预防措施 57
3.3.2 火灾处理流程 57
3.4 触电应急 58
3.4.1 触电预防 58
3.4.2 触电救援 58
3.5 机械伤害应急 58
3.5.1 夹伤/挤压伤 58
3.5.2 碰撞伤 59
3.6 设备故障应急 59
3.6.1 电机失控 59
3.6.2 CAN 通信故障 60

3.6.3 电机过热 60
3.6.4 零位丢失 61
3.7 软件故障应急 61
3.7.1 控制软件崩溃 61
3.7.2 参数配置错误 62
3.8 应急联系方式 62
3.8.1 紧急救援 62
3.8.2 技术支持 62
3.8.3 报告内容模板 63
3.9 应急物资清单 63
3.9.1 必备物资 63
3.9.2 推荐物资 63
3.10 应急演练 63
3.10.1 演练频率 63
3.10.2 演练项目 64
3.10.3 演练评估 64
3.11 事故后处理 64
3.11.1 事故调查 64
3.11.2 设备恢复 64
3.11.3 预防措施 65
3.12 应急响应流程图 65

4 责任声明 66
4.1 使用责任与免责条款 66
4.2 总则 66
4.3 产品定义 66
4.3.1 产品范围 66
4.3.2 产品性质 66
4.4 用户责任 67
4.4.1 使用前责任 67
4.4.2 使用中责任 67
4.4.3 安全责任 67
4.5 制造商责任 68
4.5.1 责任限制 68
4.6 质量保证 68
4.6.1 质保范围 68
4.6.2 质保排除 68
4.6.3 质保流程 68
4.7 免责条款 69
4.7.1 人身伤害免责 69
4.7.2 财产损失免责 69
4.7.3 软件免责 70
4.7.4 第三方组件 70
4.8 使用限制 70
4.8.1 禁止用途 70
4.8.2 出口限制 70
4.9 知识产权 70
4.9.1 所有权 70
4.9.2 开源许可 71
4.10 责任限额 71
4.10.1 赔偿上限 71
4.10.2 诉讼时效 71

4.11 法律适用 71
4.11.1 适用法律 71
4.11.2 争议解决 71
4.12 声明变更 72
4.12.1 修订权利 72
4.12.2 通知方式 72
4.12.3 生效方式 72
4.13 用户确认 72
4.14 联系方式 72

5 产品介绍 73
5.1 OpenArmX 双臂机器人系统产品介绍 73
5.2 产品概述 73
5.2.1 什么是 OpenArmX 73
5.2.2 产品定位 73
5.3 技术规格 73
5.3.1 机械参数 73
5.3.2 电机配置 73
5.3.3 控制系统 74
5.4 功能特性 74
5.4.1 核心功能 74
5.4.2 开发友好性 75
5.5 应用场景 75
5.5.1 研究与教学 75
5.5.2 工业应用原型 76
5.5.3 创新应用 76
5.6 系统优势 76
5.6.1 技术优势 76
5.6.2 开源优势 76
5.7 系统局限性 77
5.7.1 技术限制 77
5.7.2 使用限制 77
5.7.3 应用限制 77
5.8 产品组成 77
5.8.1 硬件清单 77
5.8.2 软件清单 78
5.9 与同类产品对比 78
5.9.1 对比商业机器人 78
5.9.2 对比开源机器人 79
5.10 发展路线 79
5.10.1 当前版本 (V1.0) 79
5.10.2 计划中的功能 79
5.11 获取与支持 80
5.11.1 获取产品 80
5.11.2 技术支持 80
5.11.3 培训服务 80
5.12 安全与责任 80
5.12.1 安全声明 80
5.12.2 免责条款 80
5.13 总结 81
5.13.1 适合您吗? 81
5.13.2 核心价值 81

6 系统架构 81
6.1 OpenArmX 系统架构详解 . . . 81
6.2 总体架构 . . . 82
6.2.1 系统分层架构 . . . 82
6.3 硬件架构 . . . 83
6.3.1 机械结构 . . . 83
6.3.2 电机拓扑 . . . 84
6.3.3 电气系统 . . . 85
6.4 软件架构 . . . 86
6.4.1 ROS2 节点架构 . . . 86
6.4.2 话题 (Topic) 架构 . . . 87
6.4.3 控制器架构 . . . 88
6.5 数据流 . . . 89
6.5.1 读取流程 (从硬件到软件) . . . 89
6.5.2 写入流程 (从软件到硬件) . . . 91
6.5.3 完整控制循环 . . . 92
6.6 技术栈 . . . 93
6.6.1 软件技术栈 . . . 93
6.6.2 开发语言 . . . 94
6.6.3 关键库依赖 . . . 94
6.7 文件系统结构 . . . 94
6.7.1 工作空间组织 . . . 94
6.7.2 配置文件位置 . . . 95
6.8 通信协议 . . . 96
6.8.1 CAN 协议 . . . 96
6.8.2 ROS2 消息类型 . . . 97
6.9 性能参数 . . . 98
6.9.1 控制性能 . . . 98
6.9.2 通信性能 . . . 98
6.9.3 计算资源 . . . 98
6.10 扩展性设计 . . . 98
6.10.1 硬件扩展 . . . 98
6.10.2 软件扩展 . . . 99
6.10.3 多机器人协作 . . . 100
6.11 安全架构 . . . 100
6.11.1 多层安全机制 . . . 100
6.11.2 故障处理流程 . . . 101
6.12 总结 . . . 102

7 电机配置说明 102
7.1 Robstride 电机配置详解 . . . 102
7.2 电机型号概述 . . . 102
7.2.1 Robstride 电机系列 . . . 102
7.2.2 电机技术参数 . . . 103
7.3 CAN 总线配置 . . . 105
7.3.1 CAN 通道分配 . . . 105
7.3.2 电机 ID 映射表 . . . 105
7.3.3 CAN 总线参数 . . . 106
7.4 电机控制参数 . . . 106
7.4.1 MIT 模式参数 . . . 106
7.4.2 推荐 KP/KD 参数 . . . 107
7.5 关节限位 . . . 108

7.5.1 软件限位配置 . . . 108
7.5.2 机械限位 . . . 110
7.6 零位定义 . . . 110
7.6.1 零位姿态 . . . 110
7.6.2 设置零位 . . . 111
7.6.3 验证零位 . . . 112
7.7 电机状态监控 . . . 112
7.7.1 状态检查工具 . . . 112
7.7.2 关键状态参数 . . . 113
7.7.3 错误码说明 . . . 113
7.8 电机参数配置文件 . . . 113
7.8.1 配置文件位置 . . . 113
7.8.2 motor_types.yaml . . . 114
7.8.3 motor_mapping.yaml . . . 115
7.9 常见问题 . . . 116
7.9.1 电机 ID 冲突 . . . 116
7.9.2 电机无响应 . . . 116
7.9.3 电机抖动 . . . 116
7.9.4 电机过热 . . . 116
7.10 维护与保养 . . . 117
7.10.1 日常检查 (每次使用前) . . . 117
7.10.2 定期维护 (每月) . . . 117
7.10.3 专业维护 (每季度) . . . 117
7.11 电机更换 . . . 117
7.11.1 更换流程 . . . 117
7.12 总结 . . . 118
7.12.1 快速参考 . . . 118
7.12.2 最佳实践 . . . 118

8 硬件要求 119
8.1 OpenArmX 系统硬件要求 . . . 119
8.2 控制计算机要求 . . . 119
8.2.1 最低配置 . . . 119
8.2.2 推荐配置 . . . 119
8.2.3 专业配置 . . . 119
8.3 CAN 通信设备 . . . 120
8.3.1 CAN-USB 转换器 . . . 120
8.3.2 CAN 线缆 . . . 120
8.3.3 CAN 接口接线 . . . 121
8.4 电源系统 . . . 122
8.4.1 主电源要求 . . . 122
8.4.2 电源容量计算 . . . 122
8.4.3 配电建议 . . . 123
8.5 安全设备 . . . 123
8.5.1 急停系统 . . . 123
8.5.2 防护设施 . . . 124
8.5.3 个人防护装备 (PPE) . . . 124
8.6 工作空间要求 . . . 125
8.6.1 物理空间 . . . 125
8.6.2 环境要求 . . . 126
8.6.3 工作台面 . . . 126
8.7 可选设备 . . . 127

8.7.1 显示设备 . . . 127
8.7.2 输入设备 . . . 127
8.7.3 网络设备 . . . 127
8.7.4 扩展传感器 . . . 127
8.8 硬件检查清单 . . . 127
8.8.1 购买清单 . . . 127
8.8.2 收货检查 . . . 128
8.9 硬件兼容性 . . . 129
8.9.1 已测试配置 . . . 129
8.9.2 不推荐设备 . . . 129
8.10 硬件连接示意图 . . . 129
8.10.1 完整系统连接 . . . 129
8.10.2 急停电路连接 . . . 130
8.11 总结 . . . 131
8.11.1 核心硬件要求 . . . 131
8.11.2 采购建议 . . . 131

9 系统依赖安装 131
9.1 系统环境和依赖库安装指南 . . . 131
9.2 操作系统安装 . . . 131
9.2.1 Ubuntu 22.04 LTS 安装 . . . 131
9.2.2 系统初始化 . . . 132
9.3 ROS2 Humble 安装 . . . 133
9.3.1 设置 ROS2 软件源 . . . 133
9.3.2 安装 ROS2 Humble . . . 134
9.3.3 配置 ROS2 环境 . . . 135
9.4 MoveIt 2 安装 . . . 136
9.4.1 安装 MoveIt 2 . . . 136
9.5 ros2_control 安装 . . . 137
9.5.1 安装 ros2_control 框架 . . . 137
9.6 CAN 总线工具安装 . . . 137
9.6.1 安装 SocketCAN 工具 . . . 137
9.6.2 加载 CAN 内核模块 . . . 138
9.6.3 安装 Python CAN 库 . . . 138
9.7 Python 依赖安装 . . . 138
9.7.1 基础 Python 包 . . . 138
9.7.2 OpenArmX 所需 Python 包 . . . 139
9.8 编译工具链 . . . 140
9.8.1 C++ 编译环境 . . . 140
9.8.2 CMake . . . 140
9.8.3 colcon 构建工具 . . . 141
9.9 其他依赖库 . . . 141
9.9.1 Eigen (线性代数库) . . . 141
9.9.2 Boost . . . 141
9.9.3 URDF/Xacro 相关 . . . 141
9.10 可选软件 . . . 142
9.10.1 开发工具 . . . 142
9.10.2 可视化工具 . . . 142
9.11 Git 配置 . . . 143
9.11.1 配置 Git . . . 143
9.11.2 配置 SSH 密钥 (可选) . . . 143
9.12 系统优化 (可选) . . . 143

9.12.1 实时性能优化 . . . 143
9.12.2 网络延迟优化 . . . 144
9.13 安装验证 . . . 144
9.13.1 验证清单 . . . 144
9.13.2 预期输出 . . . 146
9.14 常见问题 . . . 147
9.14.1 rosdep init 失败 . . . 147
9.14.2 pip 安装速度慢 . . . 148
9.14.3 CAN 模块加载失败 . . . 148
9.15 卸载与清理 . . . 148
9.15.1 卸载 ROS2 . . . 148
9.15.2 清理系统 . . . 149
9.16 总结 . . . 149
9.16.1 安装顺序总结 . . . 149
9.16.2 安装时间估计 . . . 149
9.16.3 下一步 . . . 150

10 工作空间构建 150
10.1 OpenArmX ROS2 工作空间构建指南 . . . 150
10.2 工作空间概念 . . . 150
10.2.1 ROS2 工作空间结构 . . . 150
10.2.2 工作空间流程 . . . 151
10.3 创建工作空间 . . . 151
10.3.1 创建工作空间目录 . . . 151
10.4 克隆源代码 . . . 151
10.4.1 获取 OpenArmX 代码 . . . 151
10.4.2 验证源代码结构 . . . 152
10.4.3 检查 package.xml 文件 . . . 152
10.5 安装依赖 . . . 153
10.5.1 使用 rosdep 安装系统依赖 . . . 153
10.5.2 安装 Python 依赖 . . . 153
10.6 编译工作空间 . . . 154
10.6.1 首次编译 . . . 154
10.6.2 预期编译输出 . . . 154
10.6.3 编译错误处理 . . . 155
10.7 配置环境变量 . . . 156
10.7.1 加载工作空间环境 . . . 156
10.7.2 环境变量说明 . . . 156
10.7.3 验证环境配置 . . . 157
10.8 测试编译结果 . . . 157
10.8.1 测试 URDF 模型 . . . 157
10.8.2 测试节点启动 . . . 157
10.9 工作空间管理 . . . 158
10.9.1 清理编译文件 . . . 158
10.9.2 增量编译 . . . 158
10.9.3 并行编译优化 . . . 159
10.10 更新代码 . . . 159
10.10.1 拉取最新代码 . . . 159
10.10.2 更新后重新编译 . . . 159
10.11 多工作空间管理 . . . 160
10.11.1 Overlay 概念 . . . 160
10.11.2 创建 Overlay 工作空间 . . . 160

10.12 常用 colcon 命令 . . . 161
10.12.1 编译命令 . . . 161
10.12.2 测试命令 . . . 161
10.12.3 信息查询 . . . 162
10.13 工作空间备份与迁移 . . . 162
10.13.1 备份工作空间 . . . 162
10.13.2 迁移到其他计算机 . . . 162
10.14 故障排查 . . . 163
10.14.1 colcon build 失败 . . . 163
10.14.2 找不到包 . . . 164
10.14.3 环境变量冲突 . . . 164
10.15 开发工作流 . . . 165
10.15.1 典型开发流程 . . . 165
10.15.2 快速测试脚本 . . . 165
10.16 性能优化 . . . 166
10.16.1 编译加速 . . . 166
10.16.2 symlink-install . . . 167
10.17 总结 . . . 167
10.17.1 完整工作空间构建流程 . . . 167
10.17.2 关键检查点 . . . 168
10.17.3 下一步 . . . 168

11 CAN 总线连接 169
11.1 CAN 总线物理连接指南 . . . 169
11.2 CAN 总线概述 . . . 169
11.2.1 为什么使用双 CAN 总线 . . . 169
11.2.2 CAN 总线基本原理 . . . 169
11.3 硬件准备 . . . 169
11.3.1 所需设备清单 . . . 169
11.3.2 CAN 转换器选择 . . . 170
11.4 CAN 线缆连接 . . . 171
11.4.1 DB9 接口针脚定义 . . . 171
11.4.2 电机端接线 . . . 171
11.4.3 完整连接拓扑 . . . 172
11.4.4 菊花链 vs 星型拓扑 . . . 172
11.5 终端电阻配置 . . . 173
11.5.1 为什么需要终端电阻 . . . 173
11.5.2 终端电阻位置 . . . 173
11.5.3 检查终端电阻 . . . 173
11.5.4 外接终端电阻 . . . 174
11.6 实际接线步骤 . . . 174
11.6.1 准备工作 . . . 174
11.6.2 连接 CAN 转换器到电脑 . . . 175
11.6.3 连接 CAN 线缆到电机 . . . 175
11.6.4 电源连接 . . . 176
11.7 接线验证 . . . 176
11.7.1 视觉检查 . . . 176
11.7.2 电阻测量 . . . 176
11.7.3 上电测试 . . . 177
11.8 CAN 接口配置 . . . 177
11.8.1 加载 CAN 驱动 . . . 177
11.8.2 配置 can0 和 can1 . . . 178

11.8.3 验证 CAN 接口 . . . 179
11.9 通信测试 . . . 179
11.9.1 监听 CAN 总线 . . . 179
11.9.2 发送测试帧 . . . 179
11.9.3 检查电机响应 . . . 180
11.10 常见问题排查 . . . 180
11.10.1 CAN 接口无法启动 . . . 180
11.10.2 CAN 总线错误率高 . . . 181
11.10.3 部分电机无响应 . . . 182
11.10.4 CAN 总线风暴 (Bus-Off) . . . 182
11.11 安全注意事项 . . . 183
11.11.1 接线安全 . . . 183
11.11.2 调试安全 . . . 183
11.12 最佳实践 . . . 184
11.12.1 线缆管理 . . . 184
11.12.2 维护建议 . . . 184
11.12.3 故障预防 . . . 184
11.13 总结 . . . 184
11.13.1 连接检查清单 . . . 184
11.13.2 下一步 . . . 185

12 CAN 接口配置 185
12.1 SocketCAN 接口配置详解 . . . 185
12.2 SocketCAN 简介 . . . 185
12.2.1 什么是 SocketCAN . . . 185
12.2.2 工作原理 . . . 186
12.3 CAN 模块加载 . . . 186
12.3.1 检查内核支持 . . . 186
12.3.2 加载 CAN 模块 . . . 186
12.3.3 开机自动加载 . . . 187
12.4 CAN 接口配置命令 . . . 187
12.4.1 基本命令格式 . . . 187
12.4.2 常用参数 . . . 188
12.5 配置 can0 和 can1 . . . 188
12.5.1 基本配置 . . . 188
12.5.2 使用 Python 脚本配置 (推荐) . . . 188
12.5.3 高级配置选项 . . . 189
12.6 查看 CAN 接口状态 . . . 190
12.6.1 基本状态查看 . . . 190
12.6.2 统计信息查看 . . . 190
12.7 CAN 接口管理 . . . 191
12.7.1 重启 CAN 接口 . . . 191
12.7.2 手动重启 (Bus-Off 恢复) . . . 191
12.7.3 关闭 CAN 接口 . . . 191
12.8 波特率配置 . . . 192
12.8.1 标准波特率 . . . 192
12.8.2 降低波特率 (故障排查) . . . 192
12.8.3 自定义位时序 . . . 192
12.9 错误处理 . . . 193
12.9.1 CAN 状态机 . . . 193
12.9.2 错误计数器 . . . 193
12.9.3 Bus-Off 恢复 . . . 193

12.10 虚拟 CAN(vcan) . . . 194
12.10.1 创建虚拟 CAN 接口 . . . 194
12.10.2 使用 vcan 测试 . . . 194
12.10.3 删除 vcan . . . 195
12.11 网络命名空间 (高级) . . . 195
12.11.1 隔离 CAN 接口 . . . 195
12.12 性能优化 . . . 195
12.12.1 接收缓冲区大小 . . . 195
12.12.2 CPU 亲和性 . . . 195
12.12.3 实时优先级 . . . 196
12.13 故障排查 . . . 196
12.13.1 设备不存在 . . . 196
12.13.2 权限不足 . . . 196
12.13.3 波特率设置失败 . . . 197
12.14 监控和调试 . . . 197
12.14.1 实时监控脚本 . . . 197
12.14.2 错误日志记录 . . . 198
12.15 自动化配置 . . . 198
12.15.1 创建 systemd 服务 . . . 198
12.15.2 udev 规则 (自动配置) . . . 199
12.16 总结 . . . 200
12.16.1 常用命令速查 . . . 200
12.16.2 推荐配置 . . . 200
12.16.3 下一步 . . . 201

13 电机 ID 配置 201
13.1 电机 ID 设置和零位校准指南 . . . 201
13.2 电机 ID 概述 . . . 201
13.2.1 ID 分配方案 . . . 201
13.2.2 ID 的重要性 . . . 202
13.3 查看电机 ID . . . 202
13.3.1 使用电机状态检查脚本 . . . 202
13.3.2 使用 candump 监听 . . . 203
13.4 零位概念 . . . 203
13.4.1 什么是零位 . . . 203
13.4.2 为什么要设置零位 . . . 203
13.4.3 零位丢失的影响 . . . 203
13.5 零位设置步骤 . . . 203
13.5.1 准备工作 . . . 203
13.5.2 调整到零位姿态 . . . 204
13.5.3 使用 GUI 设置零位 . . . 204
13.5.4 使用命令行脚本设置 . . . 204
13.5.5 零位设置原理 . . . 205
13.6 验证零位 . . . 206
13.6.1 回零测试 . . . 206
13.6.2 检查关节角度 . . . 206
13.6.3 使用 RViz 可视化验证 . . . 206
13.7 常见问题 . . . 207
13.7.1 设置零位失败 . . . 207
13.7.2 零位设置后仍然偏差大 . . . 207
13.7.3 部分电机零位正确, 部分错误 . . . 208
13.8 定期维护 . . . 208

13.8.1 零位检查频率 208
13.8.2 零位漂移监控 208
13.8.3 记录零位历史 209
13.9 高级操作 209
13.9.1 单独设置某个电机零位 209
13.9.2 批量设置零位 209
13.9.3 零位偏移补偿 210
13.10 安全注意事项 210
13.10.1 设置零位前 210
13.10.2 设置零位中 210
13.10.3 设置零位后 211
13.11 故障排查 211
13.11.1 电机 ID 冲突 211
13.11.2 零位无法保存 211
13.12 总结 212
13.12.1 零位设置流程总结 212
13.12.2 关键点 212
13.12.3 下一步 212

14 硬件故障排查 212
14.1 硬件问题诊断与解决指南 212
14.2 故障排查流程 213
14.2.1 系统化排查方法 213
14.2.2 安全第一原则 213
14.3 电源问题 213
14.3.1 完全无电 213
14.3.2 电压不稳定 214
14.3.3 急停无法解除 214
14.4 CAN 通信问题 215
14.4.1 CAN 接口无法启动 215
14.4.2 CAN 通信间歇性中断 216
14.4.3 Bus-Off 状态 217
14.4.4 左右臂反向 (PCAN 特有) 218
14.5 电机问题 218
14.5.1 单个电机无响应 218
14.5.2 电机抖动 219
14.5.3 电机过热 220
14.5.4 电机失控 221
14.5.5 电机无力 221
14.6 机械问题 222
14.6.1 关节卡顿 222
14.6.2 机械松动 223
14.6.3 机械碰撞痕迹 223
14.7 传感器问题 224
14.7.1 编码器故障 224
14.7.2 温度传感器异常 224
14.8 电气干扰 225
14.8.1 现象 225
14.8.2 减少干扰措施 225
14.9 诊断工具 225
14.9.1 必备工具 225
14.9.2 软件工具 226

14.10 故障记录 . . . 226
14.10.1 故障日志模板 . . . 226
14.11 常见故障速查表 . . . 227
14.12 何时联系技术支持 . . . 227
14.12.1 建议联系的情况 . . . 227
14.12.2 联系方式 . . . 228
14.12.3 准备信息 . . . 228
14.13 总结 . . . 228
14.13.1 故障排查原则 . . . 228
14.13.2 预防性维护 . . . 228

15 CAN 总线管理 229
15.1 CAN 总线启动和管理 . . . 229
15.2 CAN 管理脚本概述 . . . 229
15.2.1 脚本位置 . . . 229
15.2.2 脚本功能 . . . 229
15.3 启动 CAN 接口 . . . 229
15.3.1 使用 en_all_can.py . . . 229
15.3.2 脚本内部实现 . . . 230
15.3.3 验证启动成功 . . . 230
15.4 关闭 CAN 接口 . . . 231
15.4.1 使用 dis_all_can.py . . . 231
15.4.2 何时关闭 CAN . . . 231
15.5 自动化管理 . . . 231
15.5.1 开机自动启动 . . . 231
15.5.2 快捷命令设置 . . . 232
15.6 故障排查 . . . 233
15.6.1 启动失败 . . . 233
15.6.2 权限问题 . . . 233
15.6.3 PCAN 设备反向 . . . 233
15.7 监控 CAN 状态 . . . 234
15.7.1 实时状态监控 . . . 234
15.7.2 检查通信 . . . 234
15.8 高级配置 . . . 235
15.8.1 修改波特率 . . . 235
15.8.2 启用自动重启 . . . 235
15.9 总结 . . . 235
15.9.1 常用命令 . . . 235
15.9.2 最佳实践 . . . 236

16 电机使能与初始化 236
16.1 电机使能、禁用和初始化操作 . . . 236
16.2 电机使能概述 . . . 236
16.2.1 什么是使能 . . . 236
16.2.2 使能管理脚本 . . . 236
16.3 使能电机 . . . 237
16.3.1 使用 en_all_motors.py . . . 237
16.3.2 使能时的现象 . . . 237
16.3.3 安全注意事项 . . . 237
16.4 禁用电机 . . . 238
16.4.1 使用 dis_all_motors.py . . . 238
16.4.2 禁用后的现象 . . . 238

16.4.3 何时禁用电机 . . . 238
16.5 零位设置 . . . 239
16.5.1 使用 set_motor_zeros_setMIT.py . . . 239
16.5.2 零位设置失败 . . . 240
16.5.3 验证零位 . . . 240
16.6 回零操作 . . . 241
16.6.1 使用 control_motor_gohome.py . . . 241
16.6.2 回零安全检查 . . . 241
16.7 启动和关闭流程 . . . 242
16.7.1 标准启动流程 . . . 242
16.7.2 标准关闭流程 . . . 242
16.7.3 紧急关闭流程 . . . 243
16.8 批量脚本 . . . 243
16.8.1 创建一键启动脚本 . . . 243
16.8.2 创建一键关闭脚本 . . . 244
16.9 常见问题 . . . 245
16.9.1 使能失败 . . . 245
16.9.2 使能后立即掉线 . . . 245
16.9.3 零位设置后仍然偏差 . . . 245
16.10 图形化管理界面 . . . 246
16.10.1 使用启动 _ 关闭 _ 电机.py . . . 246
16.11 总结 . . . 246
16.11.1 关键命令 . . . 246
16.11.2 最佳实践 . . . 247

17 电机状态监控 247
17.1 电机状态检查和参数读取 . . . 247
17.2 电机状态检查 . . . 247
17.2.1 使用 check_motor_status.py . . . 247
17.3 状态参数解读 . . . 248
17.3.1 位置 (Position) . . . 248
17.3.2 速度 (Velocity) . . . 248
17.3.3 力矩 (Torque) . . . 248
17.3.4 温度 (Temperature) . . . 248
17.3.5 错误码 (Error Code) . . . 249
17.4 持续监控 . . . 249
17.4.1 实时监控脚本 . . . 249
17.4.2 温度监控 . . . 249
17.4.3 记录到日志 . . . 250
17.5 异常诊断 . . . 250
17.5.1 温度异常 . . . 250
17.5.2 位置异常 . . . 250
17.5.3 通信异常 . . . 251
17.6 总结 . . . 251

18 单电机测试 252
18.1 单个电机运动测试 . . . 252
18.2 test_motor_one.py 使用 . . . 252
18.3 参数调整 . . . 252
18.3.1 KP 参数影响 . . . 252
18.3.2 KD 参数影响 . . . 252
18.4 常见测试场景 . . . 253

18.4.1 位置精度测试 253
18.4.2 重复定位测试 253

19 多电机测试 253
19.1 多电机协调运动测试 253
19.2 多电机测试概述 253
19.2.1 测试目的 253
19.2.2 测试范围 254
19.3 单臂多关节测试 254
19.3.1 肩部三关节测试 254
19.3.2 整臂 7 关节测试 256
19.4 双臂同步测试 258
19.4.1 双臂镜像运动 258
19.4.2 双臂协同抓取 260
19.5 性能测试 263
19.5.1 同步性测试 263
19.5.2 负载测试 265
19.6 测试最佳实践 267
19.6.1 测试前准备 267
19.6.2 测试流程 267
19.6.3 数据记录 267
19.7 常见问题 268
19.7.1 电机不同步 268
19.7.2 CAN 总线拥塞 268
19.7.3 某个电机异常 269
19.8 进阶测试 269
19.8.1 插补轨迹测试 269
19.8.2 力控协同测试 270
19.9 总结 271
19.9.1 多电机测试要点 271
19.9.2 性能指标 271
19.9.3 下一步 272

20 回零操作 272
20.1 机器人回零操作详解 272
20.2 回零概述 272
20.2.1 什么是回零 272
20.2.2 回零的作用 272
20.2.3 回零前提条件 273
20.3 使用 control_motor_gohome.py 273
20.3.1 基本用法 273
20.3.2 脚本工作流程 274
20.3.3 参数说明 275
20.4 分步回零逻辑 275
20.4.1 为什么分阶段 275
20.4.2 回零顺序 276
20.4.3 平滑运动 276
20.5 手动实现回零 277
20.5.1 单臂手动回零 277
20.5.2 双臂同步回零 279
20.6 回零中的安全考虑 281
20.6.1 避免碰撞 281

20.6.2 超时保护 . . . 281
20.6.3 紧急停止 . . . 282
20.7 验证回零结果 . . . 283
20.7.1 位置精度检查 . . . 283
20.7.2 视觉检查 . . . 284
20.7.3 重复性测试 . . . 284
20.8 常见问题 . . . 286
20.8.1 回零后位置偏差大 . . . 286
20.8.2 回零过程中卡住 . . . 286
20.8.3 回零速度过快/过慢 . . . 287
20.9 回零的最佳实践 . . . 287
20.9.1 日常使用 . . . 287
20.9.2 定期验证 . . . 288
20.9.3 记录与维护 . . . 288
20.10 总结 . . . 289
20.10.1 回零要点 . . . 289
20.10.2 安全检查清单 . . . 289
20.10.3 下一步 . . . 289

21 启动 MoveIt 291
21.1 MoveIt 运动规划系统启动 . . . 291
21.2 MoveIt 简介 . . . 291
21.2.1 什么是 MoveIt . . . 291
21.2.2 OpenArmX 的 MoveIt 配置 . . . 291
21.3 启动 MoveIt Demo . . . 291
21.3.1 基本启动 . . . 291
21.3.2 启动参数 . . . 292
21.3.3 启动检查 . . . 292
21.4 RViz 界面布局 . . . 292
21.4.1 主要组件 . . . 292
21.4.2 MoveIt 控制面板 . . . 293
21.4.3 3D 显示设置 . . . 293
21.5 Planning Group 选择 . . . 293
21.5.1 可用的 Planning Group . . . 293
21.5.2 选择 Planning Group . . . 293
21.6 基本运动规划 . . . 294
21.6.1 使用交互标记 . . . 294
21.6.2 使用随机目标 . . . 294
21.6.3 使用预定义姿态 . . . 294
21.7 参数调整 . . . 295
21.7.1 规划器选择 . . . 295
21.7.2 速度缩放 . . . 295
21.7.3 加速度缩放 . . . 295
21.8 安全注意事项 . . . 296
21.8.1 规划前检查 . . . 296
21.8.2 执行前检查 . . . 296
21.8.3 执行中监控 . . . 296
21.9 常见问题 . . . 296
21.9.1 MoveIt 无法启动 . . . 296
21.9.2 规划失败 . . . 297
21.9.3 执行失败 . . . 297
21.10 Demo 模式 vs 实际硬件 . . . 298

21.10.1 Demo 模式 (仿真) . . . 298
21.10.2 实际硬件模式 . . . 298
21.11 高级选项 . . . 298
21.11.1 添加场景对象 . . . 298
21.11.2 保存规划轨迹 . . . 299
21.11.3 自定义配置 . . . 299
21.12 总结 . . . 299
21.12.1 快速启动流程 . . . 299
21.12.2 最佳实践 . . . 300
21.12.3 下一步 . . . 300

22 RViz 界面操作 300
22.1 RViz 可视化界面详细操作指南 . . . 300
22.2 RViz 界面概览 . . . 300
22.2.1 主界面布局 . . . 300
22.2.2 主要区域功能 . . . 301
22.3 Displays 面板详解 . . . 301
22.3.1 显示项结构 . . . 301
22.3.2 常用显示项配置 . . . 302
22.3.3 显示项开关 . . . 303
22.4 MotionPlanning 面板详解 . . . 304
22.4.1 Planning 标签 . . . 304
22.4.2 Scene 标签 . . . 307
22.4.3 Joints 标签 . . . 308
22.5 3D 视图操作 . . . 309
22.5.1 视角控制 . . . 309
22.5.2 交互标记使用 . . . 310
22.5.3 轨迹可视化 . . . 311
22.6 实用操作技巧 . . . 311
22.6.1 快速规划流程 . . . 311
22.6.2 避免规划失败 . . . 312
22.6.3 保存和恢复配置 . . . 312
22.7 高级功能 . . . 313
22.7.1 多规划组协同 . . . 313
22.7.2 路径优化 . . . 313
22.7.3 碰撞检查配置 . . . 314
22.8 常见问题 . . . 314
22.8.1 交互标记不出现 . . . 314
22.8.2 规划的轨迹不显示 . . . 315
22.8.3 Execute 无反应 . . . 315
22.9 总结 . . . 315
22.9.1 RViz 关键操作 . . . 315
22.9.2 界面导航速查 . . . 316
22.9.3 下一步 . . . 316

23 单臂运动规划 316
23.1 单臂轨迹规划技巧与应用 . . . 316
23.2 单臂规划概述 . . . 317
23.2.1 规划空间 . . . 317
23.2.2 单臂运动特点 . . . 317
23.3 使用 RViz 进行单臂规划 . . . 318
23.3.1 基本规划流程 . . . 318

23.3.2 方式 1: 交互标记规划 . . . 319
23.3.3 方式 2: 随机目标规划 . . . 320
23.3.4 方式 3: 关节滑块规划 . . . 321
23.4 使用 Python API 规划 . . . 322
23.4.1 基本脚本结构 . . . 322
23.4.2 笛卡尔路径规划 . . . 325
23.4.3 完整抓取示例 . . . 327
23.5 规划技巧 . . . 332
23.5.1 提高规划成功率 . . . 332
23.5.2 避免碰撞 . . . 333
23.5.3 速度和加速度控制 . . . 333
23.6 常见任务示例 . . . 334
23.6.1 扫描轨迹 . . . 334
23.6.2 圆形轨迹 . . . 335
23.7 总结 . . . 335
23.7.1 单臂规划要点 . . . 335
23.7.2 最佳实践 . . . 336
23.7.3 下一步 . . . 336

24 双臂独立控制 336
24.1 双臂协调运动与独立控制 . . . 336
24.2 双臂控制概述 . . . 336
24.2.1 控制模式 . . . 336
24.2.2 Planning Groups . . . 337
24.3 RViz 双臂独立控制 . . . 337
24.3.1 分别规划左右臂 . . . 337
24.3.2 示例: 对称摆放 . . . 338
24.3.3 夹爪控制 . . . 339
24.4 Python 双臂协调编程 . . . 339
24.4.1 基本双臂控制类 . . . 339
24.4.2 双臂协同抓取 . . . 344
24.4.3 镜像运动 . . . 350
24.5 碰撞避免 . . . 352
24.5.1 自碰撞检测 . . . 352
24.5.2 运动时避碰 . . . 352
24.6 同步控制技巧 . . . 353
24.6.1 时间同步 . . . 353
24.6.2 位置同步 . . . 354
24.7 实用应用场景 . . . 354
24.7.1 双臂装配 . . . 354
24.7.2 双臂传递 . . . 355
24.8 常见问题 . . . 356
24.8.1 双臂运动不同步 . . . 356
24.8.2 碰撞检测过于敏感 . . . 356
24.8.3 规划失败率高 . . . 357
24.9 总结 . . . 357
24.9.1 双臂控制要点 . . . 357
24.9.2 最佳实践 . . . 357
24.9.3 下一步 . . . 357

25 连续录制 359
25.1 连续录制机器人运动轨迹 . . . 359

25.2 轨迹录制概述 . . . 359
25.2.1 录制功能 . . . 359
25.2.2 录制脚本 . . . 359
25.3 连续录制使用 . . . 359
25.3.1 启动连续录制 . . . 359
25.3.2 停止录制 . . . 360
25.3.3 录制参数 . . . 360
25.4 录制工作流 . . . 360
25.4.1 准备工作 . . . 360
25.4.2 执行录制 . . . 361
25.4.3 验证录制 . . . 362
25.5 轨迹文件格式 . . . 362
25.5.1 YAML 结构 . . . 362
25.5.2 字段说明 . . . 363
25.6 录制技巧 . . . 363
25.6.1 平滑轨迹录制 . . . 363
25.6.2 录制前准备 . . . 364
25.6.3 录制质量检查 . . . 364
25.7 常见问题 . . . 364
25.7.1 录制文件过大 . . . 364
25.7.2 轨迹不平滑 . . . 364
25.7.3 录制中断 . . . 365
25.8 高级功能 . . . 365
25.8.1 指定保存路径 . . . 365
25.8.2 添加标注信息 . . . 365
25.8.3 合并多个轨迹 . . . 366
25.9 总结 . . . 366
25.9.1 录制流程 . . . 366
25.9.2 最佳实践 . . . 367
25.9.3 下一步 . . . 367

26 轨迹回放 367
26.1 录制轨迹的回放与执行 . . . 367
26.2 轨迹回放概述 . . . 367
26.2.1 回放原理 . . . 367
26.2.2 回放方式 . . . 368
26.3 使用 playback 节点回放 . . . 368
26.3.1 基本用法 . . . 368
26.3.2 参数说明 . . . 369
26.4 Python 脚本回放 . . . 370
26.4.1 基本回放脚本 . . . 370
26.4.2 循环回放 . . . 375
26.4.3 带反馈的回放 . . . 376
26.5 回放前准备 . . . 377
26.5.1 系统检查 . . . 377
26.5.2 轨迹文件验证 . . . 378
26.5.3 安全准备 . . . 378
26.6 回放技巧 . . . 378
26.6.1 速度调整 . . . 378
26.6.2 部分回放 . . . 379
26.6.3 镜像回放 . . . 380
26.7 回放监控 . . . 381

26.7.1 实时监控 . . . 381
26.7.2 误差监控 . . . 382
26.8 常见问题 . . . 383
26.8.1 回放失败 . . . 383
26.8.2 回放不平滑 . . . 384
26.8.3 轨迹与录制不符 . . . 385
26.9 高级功能 . . . 385
26.9.1 轨迹编辑后回放 . . . 385
26.9.2 轨迹拼接 . . . 386
26.10 总结 . . . 387
26.10.1 回放要点 . . . 387
26.10.2 最佳实践 . . . 387
26.10.3 下一步 . . . 387

27 轨迹文件格式 388
27.1 轨迹 YAML 文件结构详解 . . . 388
27.2 文件格式概述 . . . 388
27.2.1 YAML 简介 . . . 388
27.2.2 轨迹文件总体结构 . . . 389
27.3 Header 字段 . . . 389
27.3.1 header 结构 . . . 389
27.4 joint_names 字段 . . . 390
27.4.1 关节名称列表 . . . 390
27.5 points 字段 (核心) . . . 392
27.5.1 轨迹点结构 . . . 392
27.5.2 字段详解 . . . 393
27.6 完整示例文件 . . . 394
27.6.1 最小示例 (左臂 3 个点) . . . 394
27.6.2 带速度和力矩的示例 . . . 396
27.7 文件编辑 . . . 397
27.7.1 手动编辑 . . . 397
27.7.2 Python 编辑 . . . 397
27.8 轨迹分析 . . . 401
27.8.1 基本统计 . . . 401
27.8.2 可视化 . . . 403
27.9 格式验证 . . . 404
27.9.1 验证脚本 . . . 404
27.10 常见问题 . . . 405
27.10.1 YAML 解析错误 . . . 405
27.10.2 文件过大 . . . 406
27.10.3 关节限位超出 . . . 406
27.11 总结 . . . 407
27.11.1 文件格式要点 . . . 407
27.11.2 编辑最佳实践 . . . 407
27.11.3 下一步 . . . 408

28 控制器类型 409
28.1 ros2_control 控制器类型介绍 . . . 409
28.2 控制器概述 . . . 409
28.2.1 什么是 ros2_control . . . 409
28.2.2 OpenArmX 控制器配置 . . . 409
28.3 控制器类型 . . . 410

28.3.1 JointStateBroadcaster . . . 410
28.3.2 JointTrajectoryController . . . 410
28.3.3 PositionController . . . 412
28.3.4 VelocityController . . . 413
28.3.5 EffortController (力矩控制) . . . 414
28.4 控制器对比 . . . 415
28.4.1 功能对比表 . . . 415
28.4.2 控制接口对比 . . . 415
28.5 控制器选择指南 . . . 416
28.5.1 应用场景推荐 . . . 416
28.5.2 控制器组合 . . . 417
28.6 控制器配置文件 . . . 418
28.6.1 配置文件结构 . . . 418
28.6.2 关键参数说明 . . . 419
28.7 控制器管理 . . . 421
28.7.1 查看控制器状态 . . . 421
28.7.2 控制器生命周期 . . . 421
28.7.3 加载/卸载控制器 . . . 422
28.8 控制器切换 . . . 422
28.8.1 切换控制器 . . . 422
28.8.2 Python 切换示例 . . . 423
28.9 常见问题 . . . 425
28.9.1 控制器启动失败 . . . 425
28.9.2 控制器冲突 . . . 425
28.9.3 控制频率过低 . . . 425
28.10 总结 . . . 425
28.10.1 控制器选择要点 . . . 425
28.10.2 配置最佳实践 . . . 426
28.10.3 下一步 . . . 426

29 参数调优 426
29.1 控制器 KP/KD 参数调优指南 . . . 426
29.2 参数调优概述 . . . 426
29.2.1 什么是 KP/KD . . . 426
29.2.2 OpenArmX 控制架构 . . . 427
29.3 参数范围与推荐值 . . . 428
29.3.1 电机类型对应参数 . . . 428
29.3.2 应用场景参数表 . . . 429
29.4 参数调优方法 . . . 429
29.4.1 经验调参法 . . . 429
29.4.2 Ziegler-Nichols 法 . . . 430
29.4.3 实验调参步骤 . . . 430
29.4.4 可视化调参 . . . 432
29.5 参数配置方式 . . . 433
29.5.1 硬件接口层配置 . . . 433
29.5.2 运行时动态调整 . . . 435
29.6 调优案例 . . . 437
29.6.1 案例 1: 示教模式参数 . . . 437
29.6.2 案例 2: 高速运动震荡 . . . 437
29.6.3 案例 3: 负载运动下垂 . . . 438
29.6.4 案例 4: 精确定位 . . . 439
29.7 不同控制器的参数调优 . . . 439

29.7.1 JointTrajectoryController . . . 439
29.7.2 PositionController . . . 440
29.7.3 EffortController (力控) . . . 440
29.8 调参工具 . . . 441
29.8.1 自动调参脚本 . . . 441
29.8.2 实时调参界面 . . . 444
29.9 常见问题 . . . 446
29.9.1 震荡问题 . . . 446
29.9.2 响应慢 . . . 446
29.9.3 稳态误差 . . . 447
29.9.4 噪声/抖动 . . . 447
29.10 总结 . . . 447
29.10.1 调优要点 . . . 447
29.10.2 推荐流程 . . . 447
29.10.3 下一步 . . . 448

30 控制模式说明 448
30.1 MIT 模式与 CSP 模式对比 . . . 448
30.2 控制模式概述 . . . 448
30.2.1 Robstride 支持的模式 . . . 448
30.3 MIT 模式详解 . . . 449
30.3.1 MIT 模式原理 . . . 449
30.3.2 MIT 模式特点 . . . 450
30.3.3 MIT 模式应用 . . . 451
30.4 CSP 模式详解 . . . 452
30.4.1 CSP 模式原理 . . . 452
30.4.2 CSP 模式特点 . . . 453
30.4.3 CSP 模式应用 . . . 454
30.5 MIT vs CSP 对比 . . . 454
30.5.1 功能对比表 . . . 454
30.5.2 性能对比 . . . 454
30.5.3 应用场景对比 . . . 455
30.6 其他控制模式 . . . 456
30.6.1 CSV 模式 (速度控制) . . . 456
30.6.2 CST 模式 (力矩控制) . . . 456
30.6.3 混合模式 . . . 457
30.7 OpenArmX 控制模式配置 . . . 457
30.7.1 当前配置 . . . 457
30.7.2 切换到 CSP 模式 . . . 457
30.8 模式选择指南 . . . 458
30.8.1 决策流程图 . . . 458
30.8.2 推荐配置 . . . 459
30.9 实际应用案例 . . . 459
30.9.1 案例 1: 柔顺装配 (MIT) . . . 459
30.9.2 案例 2: 示教录制 (MIT) . . . 461
30.9.3 案例 3: 高精度定位 (CSP 或高增益 MIT) . . . 463
30.10 常见问题 . . . 464
30.10.1 MIT 模式调参困难 . . . 464
30.10.2 CSP 模式缺少灵活性 . . . 464
30.10.3 力矩控制不稳定 . . . 464
30.11 总结 . . . 465
30.11.1 控制模式选择 . . . 465

30.11.2MIT 模式要点 465
30.11.3下一步 465

31 控制器管理 466
31.1 ros2_control 控制器高级管理 466
31.2 控制器管理概述 466
31.2.1 Controller Manager 466
31.2.2 控制器生命周期 466
31.3 控制器命令行管理 467
31.3.1 列出控制器 467
31.3.2 查看硬件接口 468
31.3.3 加载控制器 468
31.3.4 卸载控制器 469
31.3.5 切换控制器状态 469
31.3.6 切换控制器 469
31.4 Python 管理接口 470
31.4.1 基本管理类 470
31.4.2 控制器状态监控 475
31.5 控制器切换策略 477
31.5.1 零宕机切换 477
31.5.2 停止-启动切换 478
31.5.3 备用控制器 479
31.6 批量控制器管理 480
31.6.1 批量启动 480
31.6.2 批量停止 480
31.6.3 Python 批量管理 481
31.7 控制器诊断 483
31.7.1 诊断检查 483
31.7.2 日志监控 485
31.8 高级管理技巧 485
31.8.1 控制器参数热更新 485
31.8.2 控制器性能监控 486
31.9 常见问题 488
31.9.1 控制器加载失败 488
31.9.2 控制器无法启动 488
31.9.3 切换控制器失败 489
31.10总结 489
31.10.1管理要点 489
31.10.2最佳实践 490
31.10.3下一步 490

32 力控与柔顺控制 491
32.1 力控制与柔顺控制应用 491
32.2 力控概述 491
32.2.1 什么是力控 491
32.2.2 OpenArmX 力控能力 491
32.3 阻抗控制原理 492
32.3.1 阻抗方程 492
32.3.2 刚度调节 492
32.4 力矩控制 493
32.4.1 纯力矩控制 493
32.4.2 力矩前馈 494

32.4.3 恒力控制 495
32.5 柔顺控制应用 497
32.5.1 柔顺装配 497
32.5.2 柔顺抓取 500
32.5.3 碰撞检测与退让 502
32.6 重力补偿 505
32.6.1 简单重力补偿 505
32.6.2 多关节重力补偿 506
32.7 人机协作 508
32.7.1 拖动示教 508
32.7.2 安全人机交互 510
32.8 常见问题 511
32.8.1 力控不稳定 511
32.8.2 重力补偿不准 512
32.8.3 碰撞检测误报 512
32.9 总结 513
32.9.1 力控要点 513
32.9.2 应用总结 513
32.9.3 下一步 513

33 性能优化 513
33.1 OpenArmX 系统性能优化指南 513
33.2 性能优化概述 513
33.2.1 性能指标 513
33.2.2 性能瓶颈 514
33.3 控制频率优化 514
33.3.1 ros2_control 频率 514
33.3.2 CAN 通信优化 515
33.4 实时性优化 515
33.4.1 Linux 实时补丁 515
33.4.2 CPU 隔离 516
33.4.3 进程优先级 516
33.5 代码优化 517
33.5.1 Python 性能优化 517
33.5.2 C++ 性能 518
33.6 网络与通信优化 519
33.6.1 ROS2 DDS 配置 519
33.6.2 话题优化 520
33.7 系统调优 521
33.7.1 关闭不必要的服务 521
33.7.2 内存优化 521
33.7.3 磁盘 I/O 优化 522
33.8 性能监控 522
33.8.1 系统监控 522
33.8.2 ROS2 性能分析 523
33.8.3 自定义性能监控 523
33.9 性能测试 525
33.9.1 控制延迟测试 525
33.9.2 同步性能测试 527
33.10 优化检查清单 528
33.10.1 软件优化 528
33.10.2 硬件优化 528

33.10.3配置优化 528
33.11常见问题 529
33.11.1控制频率达不到目标 529
33.11.2系统卡顿 529
33.11.3CAN 总线拥塞 530
33.12总结 530
33.12.1优化策略 530
33.12.2性能目标 530
33.12.3下一步 531

34 CAN 通信问题 532
34.1 CAN 总线通信故障诊断与解决 532
34.2 CAN 通信概述 532
34.2.1 CAN 总线基础 532
34.2.2 常见 CAN 问题 532
34.3 问题诊断 532
34.3.1 检查 CAN 接口状态 532
34.3.2 查看 CAN 统计信息 533
34.3.3 监听 CAN 消息 533
34.4 常见问题及解决 534
34.4.1 CAN 接口未启动 534
34.4.2 设备未识别 535
34.4.3 通信超时 536
34.4.4 数据丢失/丢帧 537
34.4.5 总线拥塞 538
34.4.6 硬件故障 539
34.5 诊断工具 540
34.5.1 candump (监听) 540
34.5.2 cansend (发送测试) 540
34.5.3 cangen (生成测试流量) 540
34.5.4 cansequence (测试序列) 541
34.5.5 Python 诊断脚本 541
34.6 高级故障排查 542
34.6.1 波形分析 542
34.6.2 负载测试 543
34.6.3 电机逐一测试 544
34.7 预防措施 545
34.7.1 日常检查 545
34.7.2 最佳实践 545
34.7.3 监控脚本 545
34.8 故障速查表 547
34.9 总结 547
34.9.1 诊断流程 547
34.9.2 关键命令 548
34.9.3 下一步 548

35 电机异常处理 548
35.1 电机故障诊断与异常处理 548
35.2 电机异常概述 548
35.2.1 常见电机问题 548
35.2.2 电机状态检查 549
35.3 电机无响应 549

35.3.1 症状 549
35.3.2 诊断步骤 549
35.3.3 解决方法 550
35.4 电机异响 551
35.4.1 症状类型 551
35.4.2 诊断方法 551
35.4.3 解决方法 552
35.5 震荡/抖动 553
35.5.1 症状 553
35.5.2 原因分析 553
35.5.3 解决方法 554
35.6 过热保护 555
35.6.1 症状 555
35.6.2 温度监控 555
35.6.3 降温措施 557
35.7 位置偏差 557
35.7.1 症状 557
35.7.2 诊断 557
35.7.3 原因与解决 558
35.8 失控运动 559
35.8.1 症状 559
35.8.2 紧急处理 560
35.8.3 原因分析 560
35.8.4 预防措施 560
35.9 错误码处理 561
35.9.1 错误码表 561
35.9.2 错误码读取 562
35.9.3 错误恢复 563
35.10 诊断流程图 564
35.11 总结 564
35.11.1 诊断要点 564
35.11.2 安全提示 565
35.11.3 下一步 565

36 控制器问题 565
36.1 ros2_control 控制器故障诊断 565
36.2 控制器问题概述 565
36.2.1 常见问题 565
36.2.2 基本检查 566
36.3 控制器无法加载 566
36.3.1 症状 566
36.3.2 原因与解决 566
36.4 控制器无法启动 567
36.4.1 症状 567
36.4.2 原因与解决 567
36.5 轨迹执行失败 569
36.5.1 症状 569
36.5.2 诊断 569
36.5.3 常见失败原因 569
36.6 性能问题 571
36.6.1 症状 571
36.6.2 诊断工具 571

36.6.3 性能优化 . . . 571
36.7 控制器崩溃 . . . 572
36.7.1 症状 . . . 572
36.7.2 查看崩溃日志 . . . 572
36.7.3 常见崩溃原因 . . . 573
36.8 诊断流程 . . . 573
36.8.1 系统化诊断 . . . 573
36.8.2 诊断脚本 . . . 574
36.9 常见错误信息 . . . 576
36.9.1 错误信息速查 . . . 576
36.10 预防措施 . . . 577
36.10.1 配置验证 . . . 577
36.10.2 监控脚本 . . . 577
36.11 总结 . . . 579
36.11.1 诊断要点 . . . 579
36.11.2 常用命令 . . . 579
36.11.3 下一步 . . . 579

37 常用诊断命令 579
37.1 故障诊断命令速查手册 . . . 579
37.2 系统状态检查 . . . 580
37.2.1 操作系统 . . . 580
37.2.2 ROS2 环境 . . . 580
37.3 CAN 总线诊断 . . . 581
37.3.1 CAN 接口管理 . . . 581
37.3.2 CAN 通信测试 . . . 581
37.3.3 USB-CAN 设备 . . . 582
37.4 ROS2 诊断 . . . 583
37.4.1 节点与话题 . . . 583
37.4.2 服务与参数 . . . 584
37.4.3 Action . . . 584
37.5 ros2_control 诊断 . . . 585
37.5.1 控制器管理 . . . 585
37.5.2 控制器状态监控 . . . 586
37.6 电机诊断 . . . 586
37.6.1 OpenArmX 脚本 . . . 586
37.6.2 Python 快速诊断 . . . 587
37.7 系统性能 . . . 587
37.7.1 进程监控 . . . 587
37.7.2 网络监控 . . . 588
37.7.3 磁盘 I/O . . . 589
37.8 日志查看 . . . 589
37.8.1 ROS2 日志 . . . 589
37.8.2 系统日志 . . . 590
37.9 网络诊断 . . . 590
37.9.1 连接测试 . . . 590
37.9.2 ROS2 DDS . . . 591
37.10 编译与构建 . . . 591
37.10.1 colcon 命令 . . . 591
37.10.2 依赖管理 . . . 592
37.11 快速诊断脚本 . . . 592
37.11.1 完整系统检查 . . . 592

37.11.2 故障速查 . . . 593
37.12 命令速查表 . . . 594
37.12.1 基础操作 . . . 594
37.12.2 故障诊断 . . . 595
37.12.3 性能监控 . . . 595
37.13 总结 . . . 595
37.13.1 常用命令组合 . . . 595
37.13.2 诊断清单 . . . 596
37.13.3 下一步 . . . 596

38 包结构说明 597
38.1 OpenArmX ROS2 软件包结构详解 . . . 597
38.2 工作空间概览 . . . 597
38.2.1 工作空间目录结构 . . . 597
38.2.2 核心功能包 . . . 597
38.3 各功能包详解 . . . 597
38.3.1 openarm_description . . . 597
38.3.2 openarm_hardware . . . 599
38.3.3 openarm_controllers . . . 603
38.3.4 openarm_bringup . . . 605
38.3.5 openarm_moveit_config . . . 608
38.3.6 openarm_msgs . . . 610
38.4 motor_tests_openarmx_com . . . 612
38.5 依赖关系图 . . . 614
38.5.1 包依赖 . . . 614
38.5.2 ROS2 依赖 . . . 614
38.6 编译系统 . . . 615
38.6.1 编译顺序 . . . 615
38.6.2 编译选项 . . . 616
38.7 配置文件管理 . . . 616
38.7.1 参数文件层次 . . . 616
38.7.2 参数覆盖机制 . . . 617
38.8 开发工作流 . . . 617
38.8.1 典型开发流程 . . . 617
38.8.2 快速测试 . . . 617
38.9 代码规范 . . . 618
38.9.1 C++ 代码规范 . . . 618
38.9.2 Python 代码规范 . . . 619
38.10 插件机制 . . . 620
38.10.1 硬件接口插件 . . . 620
38.10.2 控制器插件 . . . 621
38.11 总结 . . . 621
38.11.1 包结构要点 . . . 621
38.11.2 开发入口点 . . . 621
38.11.3 下一步 . . . 622

39 扩展开发 622
39.1 OpenArmX 系统扩展开发指南 . . . 622
39.2 扩展开发概述 . . . 622
39.2.1 常见扩展场景 . . . 622
39.2.2 开发环境要求 . . . 622
39.3 添加新传感器 . . . 623

39.3.1 场景: 集成 Intel RealSense 相机 . . . 623
39.4 集成末端执行器 . . . 631
39.4.1 场景: 添加 Robotiq 2F-85 夹爪 . . . 631
39.5 自定义控制器 . . . 639
39.5.1 场景: 实现阻抗控制器 . . . 639
39.6 添加高级功能 . . . 645
39.6.1 场景: 实现视觉伺服 . . . 645
39.7 测试与调试 . . . 648
39.7.1 单元测试 . . . 648
39.7.2 集成测试 . . . 649
39.8 文档与发布 . . . 650
39.8.1 编写 README . . . 650
39.8.2 编译 . . . 651
39.9 使用 . . . 651
39.10 参数 . . . 651
39.11 话题 . . . 651
39.11.1 发布 . . . 651
39.12 许可 . . . 651
39.13 总结 . . . 652
39.13.1 扩展开发要点 . . . 652
39.13.2 开发检查清单 . . . 652
39.13.3 下一步 . . . 653

40 API 参考 653
40.1 OpenArmX API 完整参考文档 . . . 653
40.2 C++ API . . . 653
40.2.1 openarm_hardware::OpenArmSystem . . . 653
40.2.2 openarm_hardware::RobstrideCAN . . . 656
40.2.3 控制器接口 . . . 659
40.3 Python API . . . 661
40.3.1 robstride 模块 . . . 661
40.3.2 MoveIt Python API . . . 664
40.3.3 ROS2 Python API (rclpy) . . . 668
40.4 命令行 API . . . 673
40.4.1 ros2 control . . . 673
40.4.2 ros2 topic . . . 674
40.4.3 ros2 service . . . 675
40.4.4 ros2 param . . . 675
40.5 配置文件 API . . . 675
40.5.1 URDF/Xacro 标签 . . . 675
40.5.2 控制器配置 YAML . . . 676
40.6 总结 . . . 677
40.6.1 API 参考要点 . . . 677
40.6.2 常用 API 速查 . . . 678
40.6.3 下一步 . . . 678

41 贡献指南 678
41.1 OpenArmX 项目贡献指南 . . . 678
41.2 贡献方式 . . . 678
41.2.1 可以贡献的内容 . . . 678
41.2.2 贡献流程 . . . 679
41.3 开始之前 . . . 679

41.3.1 环境搭建 679
41.3.2 开发工具安装 680
41.4 代码规范 680
41.4.1 C++ 代码规范 680
41.4.2 Python 代码规范 684
41.4.3 YAML 配置规范 688
41.5 Git 工作流 689
41.5.1 分支管理 689
41.5.2 提交规范 690
41.5.3 Pull Request 流程 692
41.6 测试要求 694
41.6.1 单元测试 694
41.6.2 集成测试 698
41.7 文档要求 699
41.7.1 代码文档 699
41.7.2 README 文档 700
41.7.3 编译 700
41.8 使用 701
41.9 配置 701
41.10 API 701
41.11 许可 701
41.12 作者 701
41.13 社区规范 702
41.13.1 行为准则 702
41.13.2 沟通渠道 702
41.13.3 获取帮助 703
41.14 常见问题 703
41.14.1 Q1: 如何选择分支名称? 703
41.14.2 Q2: 提交前要做什么? 703
41.14.3 Q3: PR 被拒绝了怎么办? 704
41.14.4 Q4: 如何保持分支同步? 704
41.15 总结 704
41.15.1 贡献要点 704
41.15.2 资源链接 704
41.15.3 联系方式 705

42 参数速查表 706
42.1 OpenArmX 常用参数快速参考 706
42.2 电机参数 706
42.2.1 电机型号参数 706
42.2.2 电机 ID 映射 706
42.3 控制参数 706
42.3.1 推荐 KP/KD 参数 706
42.3.2 速度和加速度限制 707
42.3.3 MIT 模式参数范围 707
42.4 CAN 总线参数 707
42.4.1 CAN 接口配置 707
42.4.2 CAN 命令 707
42.5 关节限位 708
42.5.1 位置限制 (rad) 708
42.6 电源参数 708
42.6.1 电源规格 708

42.6.2 功耗估算 708
42.7 温度限制 709
42.8 ROS2 参数 709
42.8.1 控制频率 709
42.8.2 控制器名称 709
42.9 文件路径 709
42.9.1 重要配置文件 709
42.9.2 常用脚本 710
42.10 网络端口 711
42.10.1 ROS2 DDS 711
42.10.2 RViz 711
42.11 错误码 711
42.11.1 电机错误码 711
42.12 安全参数 711
42.12.1 安全距离 711
42.12.2 环境要求 711
42.13 版本信息 712
42.13.1 软件版本 712
42.13.2 硬件版本 712
42.14 联系信息 712

43 常用命令速查 712
43.1 OpenArmX 常用命令快速参考 712
43.2 系统启动 712
43.2.1 基础启动 712
43.2.2 分步启动 713
43.3 CAN 总线命令 713
43.3.1 CAN 接口管理 713
43.3.2 CAN 通信测试 714
43.4 ROS2 命令 715
43.4.1 节点管理 715
43.4.2 话题操作 715
43.4.3 服务调用 716
43.4.4 参数管理 716
43.4.5 Action 操作 717
43.5 ros2_control 命令 717
43.5.1 控制器管理 717
43.5.2 控制器切换 718
43.6 编译与构建 718
43.6.1 colcon 命令 718
43.6.2 依赖管理 719
43.7 系统诊断 720
43.7.1 系统状态 720
43.7.2 进程监控 720
43.7.3 网络诊断 721
43.8 日志查看 721
43.8.1 ROS2 日志 721
43.8.2 系统日志 722
43.9 电机测试 722
43.9.1 低层级测试 722
43.9.2 Python 快速测试 723
43.10 轨迹录制与回放 723

43.10.1 录制轨迹 723
43.10.2 回放轨迹 724
43.11 MoveIt 命令 724
43.11.1 启动 MoveIt 724
43.11.2 规划参数 725
43.12 快速诊断脚本 725
43.12.1 系统检查 725
43.12.2 性能检查 726
43.13 快捷命令组合 726
43.13.1 一键启动 726
43.13.2 一键检查 727
43.13.3 一键关闭 727
43.13.4 一键重启 727
43.14 环境配置 727
43.14.1 工作空间设置 727
43.14.2 Alias 设置 728
43.15 故障排查速查 728
43.16 常用文件路径 728
43.17 键盘快捷键 729
43.17.1 终端快捷键 729
43.17.2 RViz 快捷键 729
43.18 总结 730
43.18.1 最常用命令 TOP 10 730
43.18.2 紧急命令 730

44 术语表 730
44.1 OpenArmX 技术术语表 730
44.2 A 731
44.2.1 Action (动作) 731
44.2.2 Activation (激活) 731
44.2.3 Actuator (执行器) 731
44.2.4 AMENT (Ament Build System) 731
44.3 B 731
44.3.1 Base Link (基座链接) 731
44.3.2 Bitrate (比特率) 731
44.3.3 Broadcaster (广播器) 731
44.4 C 731
44.4.1 CAN (Controller Area Network) 731
44.4.2 CAN ID 732
44.4.3 Cartesian Space (笛卡尔空间) 732
44.4.4 Claim (声明/占用) 732
44.4.5 Collision Detection (碰撞检测) 732
44.4.6 colcon 732
44.4.7 Command Interface (命令接口) 732
44.4.8 Configuration (配置) 732
44.4.9 Controller (控制器) 732
44.4.10 Controller Manager (控制器管理器) 732
44.4.11 CSP Mode (Cyclic Synchronous Position Mode) 733
44.4.12 CycloneDDS 733
44.5 D 733
44.5.1 DDS (Data Distribution Service) 733
44.5.2 Deadband (死区) 733

44.5.3 Degrees of Freedom (DOF, 自由度) . . . 733
44.5.4 Direct Drive (直接驱动) . . . 733
44.5.5 DKin (Direct Kinematics, 正运动学) . . . 733
44.5.6 Domain ID (域 ID) . . . 733
44.6 E . . . 734
44.6.1 Effort (力/力矩) . . . 734
44.6.2 EGM (Externally Guided Motion) . . . 734
44.6.3 End Effector (末端执行器) . . . 734
44.6.4 Error Code (错误码) . . . 734
44.7 F . . . 734
44.7.1 FastDDS . . . 734
44.7.2 Feedback (反馈) . . . 734
44.7.3 FK (Forward Kinematics, 正运动学) . . . 734
44.7.4 Follow Joint Trajectory (跟随关节轨迹) . . . 734
44.7.5 Frame (坐标系/帧) . . . 734
44.8 G . . . 735
44.8.1 Gazebo . . . 735
44.8.2 Goal (目标) . . . 735
44.8.3 Goal Tolerance (目标容差) . . . 735
44.8.4 Gravity Compensation (重力补偿) . . . 735
44.9 H . . . 735
44.9.1 Hardware Interface (硬件接口) . . . 735
44.9.2 Homing (回零) . . . 735
44.9.3 Hz (赫兹) . . . 735
44.10 I . . . 735
44.10.1 IK (Inverse Kinematics, 逆运动学) . . . 735
44.10.2 Impedance Control (阻抗控制) . . . 736
44.10.3 Interface (接口) . . . 736
44.11 J . . . 736
44.11.1 Joint (关节) . . . 736
44.11.2 Joint Limits (关节限位) . . . 736
44.11.3 Joint Space (关节空间) . . . 736
44.11.4 Joint State (关节状态) . . . 736
44.11.5 Joint Trajectory (关节轨迹) . . . 736
44.12 K . . . 736
44.12.1 KD (Derivative Gain, 微分增益) . . . 736
44.12.2 Kinematics (运动学) . . . 737
44.12.3 KP (Proportional Gain, 比例增益) . . . 737
44.13 L . . . 737
44.13.1 Launch File (启动文件) . . . 737
44.13.2 Lifecycle (生命周期) . . . 737
44.13.3 Limit (限位) . . . 737
44.14 M . . . 737
44.14.1 Manipulator (机械臂) . . . 737
44.14.2 MIT Mode . . . 737
44.14.3 Motion Planning (运动规划) . . . 738
44.14.4 Move Group (运动组) . . . 738
44.14.5 MoveIt . . . 738
44.15 N . . . 738
44.15.1 Namespace (命名空间) . . . 738
44.15.2 Node (节点) . . . 738

44.16 O 738
44.16.1 OMPL (Open Motion Planning Library) 738
44.16.2 Orientation (方向/姿态) 738
44.17 P 739
44.17.1 Package (包) 739
44.17.2 package.xml 739
44.17.3 Path Planning (路径规划) 739
44.17.4 PD Control (比例-微分控制) 739
44.17.5 Pipeline (管道) 739
44.17.6 Planner (规划器) 739
44.17.7 Planning Group (规划组) 739
44.17.8 Planning Scene (规划场景) 739
44.17.9 Plugin (插件) 740
44.17.10 Pose (位姿) 740
44.17.11 Position (位置) 740
44.17.12 Publisher (发布器) 740
44.18 Q 740
44.18.1 QoS (Quality of Service, 服务质量) 740
44.18.2 Quasi-Direct Drive (准直驱) 740
44.18.3 Quaternion (四元数) 740
44.19 R 741
44.19.1 Rad (弧度) 741
44.19.2 Real-time (实时) 741
44.19.3 Reduction Ratio (减速比) 741
44.19.4 Reference Frame (参考坐标系) 741
44.19.5 Remapping (重映射) 741
44.19.6 RMW (ROS Middleware) 741
44.19.7 Robot State Publisher (机器人状态发布器) 741
44.19.8 Robstride 741
44.19.9 ros2_control 741
44.19.10 ROS2 (Robot Operating System 2) 742
44.19.11 rosdep 742
44.19.12 RViz 742
44.20 S 742
44.20.1 Scene (场景) 742
44.20.2 Semantic Description (语义描述) 742
44.20.3 Service (服务) 742
44.20.4 SRDF (Semantic Robot Description Format) 742
44.20.5 State Interface (状态接口) 742
44.20.6 Subscriber (订阅器) 742
44.20.7 System Interface (系统接口) 743
44.21 T 743
44.21.1 TF2 (Transform Library 2) 743
44.21.2 Time from Start (起始时间) 743
44.21.3 Topic (话题) 743
44.21.4 Tolerance (容差) 743
44.21.5 Torque (力矩) 743
44.21.6 Trajectory (轨迹) 743
44.21.7 Transform (变换) 743
44.22 U 744
44.22.1 Update Rate (更新频率) 744

44.22.2 URDF (Unified Robot Description Format) . . . 744
44.22.3 USB-CAN . . . 744
44.23 V . . . 744
44.23.1 Velocity (速度) . . . 744
44.23.2 Velocity Scaling (速度缩放) . . . 744
44.23.3 Visual Servoing (视觉伺服) . . . 744
44.24 W . . . 744
44.24.1 Workspace (工作空间) . . . 744
44.25 X . . . 745
44.25.1 Xacro (XML Macro) . . . 745
44.26 Y . . . 745
44.26.1 YAML (YAML Ain't Markup Language) . . . 745
44.26.2 Yaw (偏航) . . . 745
44.27 Z . . . 745
44.27.1 Zero Position (零位) . . . 745
44.28 符号和缩写 . . . 745
44.28.1 Δ (Delta) . . . 745
44.28.2 θ (Theta) . . . 745
44.28.3 ω (Omega) . . . 745
44.28.4 τ (Tau) . . . 746
44.28.5 DOF . . . 746
44.28.6 EEF . . . 746
44.28.7 FT . . . 746
44.28.8 HW . . . 746
44.28.9 ID . . . 746
44.28.10 IK . . . 746
44.28.11 FK . . . 746
44.28.12 PID . . . 746
44.28.13 ROS . . . 746
44.28.14 RT . . . 747
44.28.15 SF . . . 747
44.29 单位换算 . . . 747
44.29.1 角度 . . . 747
44.29.2 速度 . . . 747
44.29.3 力矩 . . . 747
44.30 参考标准 . . . 747
44.30.1 ROS2 标准 . . . 747
44.30.2 通信协议 . . . 747
44.31 常见命令缩写 . . . 748
44.32 总结 . . . 748

45 参考资料 748
45.1 OpenArmX 相关参考资料汇总 . . . 748
45.2 官方文档 . . . 748
45.2.1 ROS2 文档 . . . 748
45.2.2 MoveIt 文档 . . . 749
45.2.3 ros2_control 文档 . . . 749
45.3 书籍推荐 . . . 749
45.3.1 ROS 书籍 . . . 749
45.3.2 机器人学书籍 . . . 749
45.3.3 控制理论书籍 . . . 750
45.4 在线教程 . . . 750

45.4.1 ROS2 教程 . . . 750
45.4.2 MoveIt 教程 . . . 750
45.4.3 ros2_control 教程 . . . 750
45.5 视频课程 . . . 751
45.5.1 YouTube 频道 . . . 751
45.5.2 在线课程平台 . . . 751
45.6 开源项目 . . . 751
45.6.1 参考机器人项目 . . . 751
45.6.2 控制器示例 . . . 751
45.7 技术论文 . . . 752
45.7.1 运动规划 . . . 752
45.7.2 控制理论 . . . 752
45.8 社区与论坛 . . . 752
45.8.1 ROS 社区 . . . 752
45.8.2 GitHub 讨论 . . . 752
45.8.3 社交媒体 . . . 753
45.9 工具与软件 . . . 753
45.9.1 开发工具 . . . 753
45.9.2 仿真工具 . . . 753
45.9.3 可视化工具 . . . 753
45.10 标准与规范 . . . 754
45.10.1 ROS REPs (ROS Enhancement Proposals) . . . 754
45.10.2 通信协议 . . . 754
45.11 硬件相关 . . . 754
45.11.1 Robstride 电机 . . . 754
45.11.2 CAN 总线 . . . 754
45.12 学习路径建议 . . . 754
45.12.1 初学者路径 (0-3 个月) . . . 754
45.12.2 进阶路径 (3-6 个月) . . . 755
45.12.3 开发者路径 (6 个月 +) . . . 756
45.13 推荐博客 . . . 756
45.13.1 英文博客 . . . 756
45.13.2 中文博客 . . . 756
45.14 会议与活动 . . . 757
45.14.1 国际会议 . . . 757
45.14.2 国内活动 . . . 757
45.15 推荐实验室和研究组 . . . 757
45.15.1 国际 . . . 757
45.15.2 国内 . . . 757
45.16 技术支持 . . . 758
45.16.1 OpenArmX 官方支持 . . . 758
45.16.2 社区支持 . . . 758
45.17 持续学习资源 . . . 758
45.17.1 订阅推荐 . . . 758
45.17.2 GitHub 关注 . . . 758
45.18 总结 . . . 758
45.18.1 推荐优先级 . . . 758
45.18.2 学习建议 . . . 759
45.18.3 获取最新资源 . . . 759

1 安全警告

1.1 [警告] 重要安全警告

请在使用 OpenArmX 机器人系统前，务必仔细阅读本安全警告！

1.2 总体安全事项

1.2.1 适用对象

- 本设备仅供**具备相应技术背景和培训的专业人员**使用
- 使用前必须完整阅读本使用说明书
- 未经培训的人员严禁操作本设备

1.2.2 使用环境要求

- [√] 在干燥、无腐蚀性气体的室内环境使用
- [√] 环境温度：5°C ~ 40°C
- [√] 相对湿度：20% ~ 80%（无冷凝）
- [×] 严禁在易燃易爆环境中使用
- [×] 严禁在有水或潮湿的环境中使用

1.2.3 电源安全

- [√] 使用符合规格的电源（48V DC，推荐电流 ≥ 20A）
- [√] 确保电源有良好的接地保护
- [×] 严禁使用不符合规格的电源
- [×] 严禁在运行时插拔电源

1.3 机械危险

1.3.1 运动危险

[警告] **机器人运动可能造成人身伤害**

- 机器人臂展范围内可能产生**挤压、碰撞、剪切**危险
- 电机扭矩较大，可能造成**骨折、挫伤**等严重伤害
- 高速运动时存在**甩出物体**的风险

防护措施： 1. 在机器人工作空间设置**安全围栏或警示标识** 2. 确保工作范围内**无人员停留** 3. 初次测试时使用**较小的速度和力矩参数**（≤ 最大值的 10%）4. 操作人员必须**时刻保持警惕**，随时

准备按下急停按钮

1.3.2 夹持危险

- 夹爪关闭时可能造成**挤压伤害**
- 不得将手指或身体任何部位伸入夹爪工作范围

1.3.3 碰撞危险

- 机器人可能与周围物体发生**意外碰撞**
- 确保工作空间内无易碎、贵重或危险物品

1.4 电气危险

1.4.1 触电危险

[警告] **设备内部存在高压电，可能导致触电**

- [×] **严禁在通电状态下打开设备外壳**
- [×] **严禁触摸裸露的电气接线端子**
- [√] 维护检修前必须**断开所有电源**
- [√] 只有**专业维修人员**才能进行内部检修

1.4.2 短路危险

- 确保所有电气连接**牢固可靠**
- 避免导电物体接触电路板或接线端子
- 使用正确规格的**保险丝和断路器**

1.4.3 CAN 总线安全

- CAN 连接器连接时**确保设备断电**
- 使用**屏蔽良好的 CAN 线缆**
- 不得在 CAN 总线上连接**超过推荐数量的设备**

1.5 软件与控制危险

1.5.1 误操作风险

[警告] **错误的参数设置可能导致设备失控**

高风险参数： - **KP（位置增益）**：过大会导致震荡、噪声、甚至电机损坏 - **KD（速度增益）**：过大会导致电机噪声和不稳定 - **速度和加速度**：过大会导致失控 - **力矩限制**：设置不当可能导致过载或无力

安全建议： 1. 首次调试时使用**保守参数** 2. **逐步增加参数**，观察机器人行为 3. 发现异常立即按下**急停按钮** 4. 详细记录每次参数修改

1.5.2 控制软件故障

- 软件 BUG 可能导致**意外运动**
- 通信故障可能导致**失控或突然停止**
- **不依赖软件作为唯一安全措施**，必须配备硬件急停

1.5.3 网络安全

- 确保控制计算机**网络安全**
- 不要在公共网络上运行控制软件
- 定期更新**系统和依赖库**

1.6 紧急停止

1.6.1 急停按钮

[警告] **必须配备硬件急停按钮**

要求： - 急停按钮必须**易于触及** - 按下后能**立即切断所有电机电源** - 使用**机械自锁式**急停按钮（不会自动复位）

1.6.2 紧急停止操作

设备启动时： 1. 操作人员须将手**悬停在急停按钮上方** 2. 随时准备在出现异常时**立即按下** 3. 不得在没有人员监护的情况下运行

遇到以下情况立即按下急停： - [!] 机器人运动异常或失控 - [!] 发出异常声音或震动 - [!] 电机冒烟或有焦味 - [!] 有人员进入工作范围 - [!] 任何可能导致危险的情况

1.6.3 急停后处理

1. **保持急停状态**，不要立即复位
2. **检查设备状态**，确认异常原因
3. **排除故障**后才能解除急停

4. 解除急停前确认**人员已撤离**工作区域

1.7 特定操作警告

1.7.1 首次通电

[警告] **首次通电存在较高风险**

安全流程： 1. 确保所有人员已**撤离工作区域** 2. 检查所有**电气连接正确** 3. 确认**急停按钮功能正常** 4. 操作人员**手悬停在急停按钮上** 5. 逐步上电，先 CAN 接口，再电机使能 6. 观察有无异常响声、震动、冒烟

1.7.2 电机零位设置

[警告] **零位设置不当可能导致启动时突然运动**

安全流程： 1. 手动将机械臂**调整到零位姿态** 2. 确保所有关节在**安全位置** 3. 使用软件**设置零位** 4. **验证零位正确性**后再进行其他操作

1.7.3 轨迹执行

[警告] **轨迹执行可能产生高速运动**

安全流程： 1. 首次执行前**仔细检查轨迹** 2. 使用**较低速度**进行测试 3. 确认轨迹安全后才能**提高速度** 4. 执行过程中**持续监控**

1.7.4 示教模式

[警告] **示教模式下电机力矩较小，可能突然失力**

安全事项： 1. 进入示教模式前确保机械臂**处于安全姿态** 2. 手动拖动时**缓慢平稳** 3. 退出示教模式时注意电机**可能突然加力**

1.8 维护与检修安全

1.8.1 日常维护

- [√] 维护前**断开所有电源**
- [√] 等待至少 **5 分钟**让电容放电
- [√] 使用**绝缘工具**
- [×] 不得在通电状态下维护

1.8.2 部件更换

- 只能使用**原厂或认证的替换部件**
- 电机更换后必须**重新设置零位和参数**
- 更换后必须进行**完整的功能测试**

1.8.3 软件更新

- 更新前**备份重要数据和配置**
- 在**测试环境**验证后再应用到生产系统
- 更新后重新进行**安全检查**

1.9 禁止事项

1.9.1 [×] 严格禁止以下行为:

1. **在没有急停装置的情况下运行设备**
2. **擅自改装、拆卸设备**
3. **超过电机的额定负载运行**
4. **在有人员或障碍物的情况下高速运行**
5. **使用不合格的电源或线缆**
6. **绕过或禁用安全保护装置**
7. **在设备故障时继续运行**
8. **未经培训的人员操作设备**
9. **在设备运行时伸手进入工作区域**
10. **让儿童或未授权人员接近设备**

1.10 风险等级说明

本手册使用以下标识表示风险等级:

标识	含义	后果
[警告] **危险**	极高风险，如不避免将导致严重伤亡	死亡或重伤
[警告] **警告**	高风险，如不避免可能导致严重伤亡	重伤或中度伤害
[警告] **注意**	中等风险，如不避免可能导致伤害或财产损失	轻伤或设备损坏
i **提示**	重要信息或最佳实践	影响使用效果

1.11 用户责任

使用本设备即表示您已：

[√] **完整阅读并理解**本安全警告 [√] **同意遵守**所有安全规范和操作要求 [√] **承担相应风险**并对自身安全负责 [√] **确保操作环境安全**，配备必要的安全设施 [√] **对他人安全负责**，防止未授权人员接触设备

1.12 紧急联系方式

遇到严重问题或事故，请立即联系：

- **技术支持**：openarmrobot@gmail.com
- **联系人**：王先生
- **电话/微信**：+86-17746530375
- **网站**：https://openarmx.com/

最后提醒：安全第一！任何操作都不应以牺牲安全为代价。如有疑问，请停止操作并寻求专业帮助。

本文档版本：v1.0 最后更新：2025 年 10 月 19 日 成都长数机器人有限公司

2 操作规范

2.1 正确操作流程与规范

本文档规定了 OpenArmX 机器人系统的标准操作流程，所有操作人员必须严格遵守。

2.2 操作人员要求

2.2.1 资质要求

- [√] 具备**机器人或自动化系统**相关知识
- [√] 熟悉 **ROS2 和 Linux 系统**基本操作
- [√] 了解**电气安全**和机械安全知识
- [√] 完成 **OpenArmX 系统培训**
- [√] 阅读并理解**完整使用说明书**

2.2.2 健康要求

- [√] 身体健康，反应灵敏

- [√] 无色盲，视力良好
- [×] 不得在疲劳、饮酒或服用影响判断力的药物后操作

2.2.3 着装要求

- [√] 穿戴**防护服**和**防护眼镜**（推荐）
- [√] 穿**防滑、合脚的鞋子**
- [×] **不得**穿宽松衣物，避免被机器人夹住
- [×] **不得**佩戴长项链、手环等饰品
- [×] **不得**留长发散披，需束起

2.3 开机前检查

2.3.1 环境检查清单

☐ 工作区域整洁，无杂物

☐ 工作空间内无人员

☐ 照明充足

☐ 通风良好

☐ 地面干燥，无水渍

☐ 周围无易燃易爆物品

☐ 安全围栏或警示标识到位

2.3.2 设备检查清单

☐ 电源线连接牢固，无破损

☐ CAN总线连接正确（can0/can1）

☐ 电机连接牢固，无松动

☐ 机械臂无明显损坏或变形

☐ 夹爪动作顺畅

☐ 急停按钮功能正常

☐ 控制计算机正常启动

2.3.3 软件检查清单

☐ ROS2 环境正确配置

☐ 工作空间编译成功

☐ 网络连接正常（如需要）

☐ 相关节点启动脚本可用

2.4 标准开机流程

2.4.1 步骤 1：系统准备

```
# 启动控制计算机
# 打开终端，进入工作空间
cd ~/openarmx_robotstride_ws-cc

# 加载 ROS2 环境
source /opt/ros/humble/setup.bash
source install/setup.bash
```

2.4.2 步骤 2：启动 CAN 接口

```
# 使用 Python 脚本启动 CAN（推荐）
python3 src/motor_tests_openarmx_com/en_all_can.py

# 或使用手动命令
sudo ip link set can0 type can bitrate 1000000
sudo ip link set can0 up
sudo ip link set can1 type can bitrate 1000000
sudo ip link set can1 up
```

检查点： - CAN 接口 LED 指示灯应常亮 - 执行 `ip link show can0` 应显示状态为 UP

2.4.3 步骤 3：上电

1. **确认人员已撤离工作区域**
2. **手悬停在急停按钮上方**
3. 打开 48V 电源开关
4. 观察电机状态，确认无异常

2.4.4 步骤 4：使能电机

```
# 使用脚本使能所有电机
python3 src/motor_tests_openarmx_com/en_all_motors.py
```

注意事项： - [警告] 使能瞬间电机可能有**轻微抖动**，属正常现象 - [警告] 如发现**剧烈抖动或异响**，立即按下急停 - [警告] 使能后电机**保持当前位置**，小心触碰

2.4.5 步骤 5：检查电机状态

```
# 检查所有电机状态
python3 src/motor_tests_openarmx_com/check_motor_status.py
```

正常状态： - 所有电机响应正常 - 温度在合理范围（< 50°C）- 无错误代码

2.4.6 步骤 6：回零（首次启动或零位丢失时）

```
# 自动回零
python3 src/motor_tests_openarmx_com/control_motor_gohome.py
```

[警告] 回零注意事项： - 确保机械臂**初始位置接近零位**（±30 度以内）- 回零过程中**保持监控** - 发现异常立即**按下急停**

2.5 日常操作规范

2.5.1 单电机测试

使用场景：调试单个电机参数

操作流程：

```
#  启动单电机测试工具
python3 src/motor_tests_openarmx_com/test_motor_one.py

#  按提示输入参数
CAN 通道: 0 或 1
电机 ID: 1-8
位置: -12.5 ~ 12.5 (rad)
KP: 建议从 5.0 开始
```

```
KD: 建议从 0.5 开始
```

安全要求： - [√] 首次测试使用**小幅度运动**（±0.5 rad）- [√] 使用**保守的 KP/KD 参数** - [√] **持续监控**电机行为 - [×] 不得超过**电机角度限制**

2.5.2 MoveIt 运动规划

使用场景：通过可视化界面控制机械臂

操作流程：

```
# 启动 MoveIt demo
ros2 launch openarm_moveit_config demo.launch.py

# 在 RViz 中操作
# - 选择 Planning Group（left_arm/right_arm）
# - 拖动交互标记到目标位置
# - 点击 "Plan" 生成轨迹
# - 检查轨迹安全性
# - 点击 "Execute" 执行
```

安全要求： - [√] 首次执行前**降低速度系数**（0.1-0.3）- [√] **仔细检查轨迹**，确认无碰撞风险 - [√] 执行时**持续监控** - [√] 准备随时**按下急停** - [×] 不得在有障碍物时执行

2.5.3 轨迹录制

使用场景：记录并回放运动轨迹

操作流程：

```
# 连续录制
ros2 run openarm_record record_joint_states_always

# 回放轨迹
ros2 run openarm_record play_joint_trajectory --file trajectory.yaml
```

安全要求： - [√] 录制前确保**轨迹路径安全** - [√] 回放前**预览轨迹文件** - [√] 首次回放使用**低速度** - [×] 不得回放未经验证的轨迹

2.5.4 示教模式

使用场景：手动拖动机械臂录制轨迹

操作流程：

```
# 设置低 KP/KD 进入示教模式
# 关节 1-4: KP=1.0, KD=0.1
# 关节 5-7: KP=0.1, KD=0.05
# 夹爪: KP=0.1, KD=0.05

# 手动拖动机械臂
# 录制关键点
# 退出示教模式（恢复常规运动参数）
#   关节 1-4: KP=50.0, KD=2.5
#   关节 5-7: KP=10.0, KD=0.5
```

安全要求： - [√] 进入示教模式前确保**姿态安全** - [√] 拖动时**缓慢平稳** - [警告] 注意退出示教模式时电机**会突然加力**

2.6 参数设置规范

2.6.1 KP/KD 参数设置

关节 1-4 (RS04/RS03 电机):

应用场景	推荐 KP 值	推荐 KD 值	说明
示教模式	1.0	0.1	极低刚性，易于拖动
常规运动 (默认)	50.0	2.5	平衡精度和稳定性
高刚性模式	500.0	10.0	高精度，但可能产生噪声
危险范围	> 5000	> 100	可能导致振动、噪声、电机损坏

关节 5-7 (RS00 电机):

应用场景	推荐 KP 值	推荐 KD 值	说明
示教模式	0.1	0.05	极低刚性，易于拖动
常规运动 (默认)	10.0	0.5	平衡精度和稳定性
高刚性模式	50.0	1.0	高精度，但可能产生噪声

应用场景	推荐 KP 值	推荐 KD 值	说明
危险范围	> 500	> 5	可能导致振动、噪声、电机损坏

夹爪 (RS00 电机):

应用场景	推荐 KP 值	推荐 KD 值
默认参数	5.0	0.5
示教模式	0.1	0.05
高刚性模式	50.0	1.0

参数说明: - 以上参数来自代码:`openarm_hardware/include/openarm_hardware/v10_simple_hardware.hpp` - **切勿超过危险范围**，否则可能损坏电机

2.6.2 速度和加速度限制

```yaml
# 推荐的速度/加速度限制
joint_limits:
  velocity: 2.0      # rad/s, 初次测试建议 0.5
  acceleration: 1.0  # rad/s², 初次测试建议 0.3
```

2.6.3 参数调整原则

1. **从保守参数开始**
2. **逐步小幅增加**（每次增加 10-20%）
3. **观察机器人行为**（声音、震动、稳定性）
4. **记录每次修改**和效果
5. **发现异常立即回退**

2.7 异常情况处理规范

2.7.1 立即急停的情况

遇到以下情况**立即按下急停按钮**:

- [!] 机器人运动**超出预期范围**
- [!] 电机发出**异常声音**（尖叫、噪声）

- [!] 机械臂**剧烈震动**
- [!] 电机或驱动器**冒烟、有焦味**
- [!] 有人员**进入工作范围**
- [!] CAN 通信**频繁中断**
- [!] 控制软件**失去响应**
- [!] 任何**可能导致危险**的异常

2.7.2 急停后检查流程

1. 保持急停状态
2. 记录异常现象
3. 检查硬件状态:
   - ☐ 电气连接是否松动
   - ☐ 机械结构是否损坏
   - ☐ 电机是否过热
4. 检查软件日志:
   - ☐ 查看错误信息
   - ☐ 检查参数配置
5. 排除故障后才能解除急停
6. 解除急停后从头开始检查流程

2.7.3 常见异常处理

电机无响应:

```
# 检查 CAN 连接
ip link show can0

# 重启 CAN 接口
sudo ip link set down can0
sudo ip link set up can0

# 重启 PCAN 连接（左右手反向时可重启 PCAN 或拔插一下 PCAN 接口）
sudo rmmod peak_usb
sudo modprobe peak_usb
```

```
# 检查电机状态
python3 check_motor_status.py
```

电机抖动或噪声：

1. 降低 KP 值
2. 增加 KD 值
3. 降低运动速度
4. 检查机械连接是否松动

轨迹执行失败：

1. 检查控制器状态
2. 检查目标位置是否超限
3. 降低速度和加速度
4. 检查 CAN 通信质量

2.8 关机规范

2.8.1 标准关机流程

```
# 步骤 1: 停止所有运动
# 确保机械臂处于安全姿态

# 步骤 2: 禁用电机
python3 src/motor_tests_openarmx_com/dis_all_motors.py

# 步骤 3: 关闭 CAN 接口
python3 src/motor_tests_openarmx_com/dis_all_can.py

# 步骤 4: 断开电源
# 关闭 48V 电源开关

# 步骤 5: 关闭控制计算机
```

2.8.2 紧急关机

遇到严重故障时： 1. 按下**急停按钮** 2. 直接断开**电源总开关** 3. **不要**尝试恢复，等待专业人员检查

2.9 维护规范

2.9.1 日常维护（每次使用后）

- ☐ 清洁机械臂表面
- ☐ 检查电缆连接
- ☐ 检查机械磨损
- ☐ 记录运行时间

2.9.2 定期维护（每月）

- ☐ 紧固所有螺丝
- ☐ 检查电机温度历史
- ☐ 清理CAN接口连接器
- ☐ 检查急停按钮功能
- ☐ 更新软件（如有）

2.9.3 专业维护（每季度）

- ☐ 全面电气检查
- ☐ 机械磨损评估
- ☐ 零位精度校准
- ☐ 性能测试

2.10 记录与文档

2.10.1 操作日志

每次使用应记录：- 日期和时间 - 操作人员 - 执行的任务 - 参数设置 - 异常情况 - 维护记录

2.10.2 参数变更记录

修改参数时应记录：- 修改日期 - 修改内容 - 修改原因 - 修改效果 - 责任人

2.11 禁止操作清单

2.11.1 [×] 以下操作严格禁止：

1. 在**未经检查**的情况下直接开机
2. **跳过**标准开机流程
3. 在**没有急停按钮**的情况下运行
4. 在工作区域**有人员**时高速运行
5. **未经测试**直接执行高速轨迹
6. 使用**未知来源**的配置文件
7. **私自修改**安全参数
8. **绕过**安全保护措施
9. **疲劳**或**注意力不集中**时操作
10. **未经培训**的人员独立操作

2.12 培训要求

所有操作人员必须完成以下培训:

2.12.1 理论培训

- x 安全知识培训
- x 系统原理培训
- x 操作流程培训
- x 故障处理培训

2.12.2 实操培训

- x 开关机操作
- x 单电机测试
- x MoveIt 操作
- x 轨迹录制回放
- x 异常处理演练

2.12.3 考核要求

- x 理论考试（≥80 分）
- x 实操考核（合格）

- x 安全意识评估（合格）

操作规范是保证安全和效率的基础，所有人员必须严格遵守！

本文档版本：v1.0 最后更新：2025 年 10 月 19 日 成都长数机器人有限公司

3 紧急处理

3.1 紧急情况应急处理指南

本文档提供 OpenArmX 机器人系统紧急情况的应急处理流程，所有操作人员必须熟记。

3.2 紧急停止

3.2.1 急停按钮位置与使用

急停按钮要求： - [红] **红色蘑菇头按钮** - [红] 位置**易于触及**（手臂伸展范围内）- [红] **机械自锁式**（按下后保持锁定状态）- [红] 能够**立即切断所有电机电源**

使用方法：

按下：用手掌快速压下红色蘑菇头

解除：逆时针旋转蘑菇头使其弹起

3.2.2 需要立即急停的情况

[警告] 遇到以下情况立即按下急停按钮：

情况	现象	危险程度
失控运动	机械臂运动超出预期，速度过快	[红] 极高
剧烈震动	电机或机械臂剧烈抖动	[红] 极高
异常声音	尖叫、卡顿、金属摩擦声	[红] 极高
冒烟起火	电机、驱动器或线缆冒烟	[红] 极高
人员接近	有人员进入工作范围	[红] 极高
碰撞风险	即将碰撞障碍物或人员	[橙] 高
通信中断	CAN 通信频繁断开	[橙] 高
电机过热	电机温度 >80°C	[橙] 高
软件崩溃	控制软件失去响应	[橙] 高
电源异常	电压波动、频繁断电	[橙] 高

3.2.3 急停后立即行动

按下急停后：

1. **保持急停状态** [手]
   - 不要立即解除急停
   - 防止设备重启后再次出现危险
2. **切断电源**
   - 如果急停后仍有异常（冒烟、异味）
   - 立即断开 24V 电源总开关
3. **撤离人员** [人]
   - 确保所有人员撤离到安全区域
   - 建立警戒范围
4. **评估现场** [眼]
   - 观察是否有明显损坏
   - 检查是否有人员受伤
   - 确认火灾风险

3.3 火灾应急

3.3.1 预防措施

[√] 配备 CO2 或干粉灭火器（禁用水基灭火器）

[√] 保持工作区域整洁，无易燃物

[√] 定期检查电气连接

[√] 不得超负荷运行电机

3.3.2 火灾处理流程

小型电气火灾（初期）：

1. 立即按下急停按钮
2. 断开电源总开关
3. 使用 CO2 或干粉灭火器
4. [×] 严禁使用水灭火
5. 疏散人员，拨打 119

大型火灾：

1. 立即疏散所有人员

2. 拨打 119 报警
3. 不要尝试灭火
4. 等待消防人员到达

3.4 触电应急

3.4.1 触电预防

- 维护时必须**断开电源**
- 不得触碰**裸露的电气端子**
- 使用**绝缘工具**
- 穿**绝缘鞋**

3.4.2 触电救援

发现有人触电：

[警告] 错误做法： - [×] 直接用手拉触电者 - [×] 用湿手或导电物体接触

[√] 正确做法： 1. **立即切断电源** - 按下急停按钮 - 断开电源总开关

2. **使用绝缘物体**
   - 使用干燥的木棍、塑料凳等
   - 将触电者与电源分离
3. **急救措施**
   - 检查意识和呼吸
   - 如无呼吸，立即进行心肺复苏（CPR）
   - 拨打 120 急救电话
4. **伤口处理**
   - 不要移动伤者（除非有二次危险）
   - 保持伤者温暖
   - 等待医护人员

3.5 机械伤害应急

3.5.1 夹伤/挤压伤

发生夹伤时：

1. 立即按下急停

2. 不要强行拉拽被夹部位
3. 断开电源
4. 手动移开机械臂
5. 检查伤情
6. 严重情况拨打 120

轻度夹伤： - 用碘伏消毒 - 无菌敷料包扎 - 观察 24 小时

严重夹伤（骨折、出血）： - 不要移动伤者 - 止血（压迫止血法）- 固定伤处 - 立即拨打 120

3.5.2 碰撞伤

头部碰撞：

[警告] 极度危险，即使看似轻微也要就医

1. 让伤者平躺
2. 不要移动头部
3. 观察意识状态
4. 立即拨打 120

四肢碰撞：

1. 检查是否骨折（疼痛、肿胀、变形）
2. 如有骨折，固定患处
3. 冷敷减轻肿胀
4. 必要时就医

3.6 设备故障应急

3.6.1 电机失控

现象：电机高速旋转、无法停止

处理流程：

1. 立即按下急停 [警告]
2. 断开电源
3. 等待电机完全停止（至少30秒）
4. 检查CAN连接
5. 检查控制软件日志
6. 重新校准零位

7. 低速测试

3.6.2 CAN 通信故障

现象：频繁报错、电机无响应

处理流程：

```
# 检查 CAN 接口状态
ip link show can0
ip link show can1

# 检查错误计数
ip -details -statistics link show can0

# 重启 CAN 接口
sudo ip link set down can0
sudo ip link set up can0

# 检查物理连接
# - 检查线缆是否松动
# - 检查是否有断路
# - 检查终端电阻（120Ω）

# 降低波特率测试
sudo ip link set can0 type can bitrate 500000
```

3.6.3 电机过热

现象：电机温度 > 80°C

[警告] 危险等级： - 60-70°C：[黄] 正常（高负载）- 70-80°C：[橙] 警告（需降低负载）- 80-90°C：[红] 危险（立即停止）- >90°C：[红][红] 极度危险（可能损坏）

处理流程：

1. 立即停止运动
2. 禁用电机
3. 自然冷却（不要用水冷却）

4. 检查原因:
   - ☐ 负载是否过大
   - ☐ KP/KD 参数是否过高
   - ☐ 运动速度是否过快
   - ☐ 散热是否良好
5. 冷却后重新测试

3.6.4 零位丢失

现象: 机械臂位置与软件显示不一致

[警告] 危险: 可能导致突然运动

处理流程:

1. 立即禁用电机
2. 断电
3. 手动将机械臂移至安全姿态
4. 重新上电
5. 重新设置零位
6. 低速验证

3.7 软件故障应急

3.7.1 控制软件崩溃

现象: ROS 节点崩溃、无响应

处理流程:

```
# 立即按下急停或禁用电机
python3 src/motor_tests_openarmx_com/dis_all_motors.py

# 检查进程状态
ps aux | grep ros

# 强制终止所有 ROS 进程
killall -9 ros2

# 检查日志
```

```
ros2 log

# 重启系统（如需要）
```

3.7.2 参数配置错误

现象：机械臂行为异常

处理流程：

```
# 立即停止运动
# 恢复默认参数
joint_limits:
  velocity: 0.5
  acceleration: 0.3

kp_default: [10, 10, 10, 10, 5, 5, 5, 5]
kd_default: [0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5]

# 重新启动系统
# 逐步调整参数
```

3.8 应急联系方式

3.8.1 紧急救援

火警：119

急救：120

报警：110

3.8.2 技术支持

公司：成都长数机器人有限公司

联系人：王先生

电话/微信：+86-17746530375

邮箱：openarmrobot@gmail.com

网站：https://openarmx.com/

3.8.3 报告内容模板

事故时间：____年__月__日 __:__

事故地点：_____

操作人员：_____

事故类型：☐ 机械伤害 ☐ 电气事故 ☐ 设备故障

简要描述：_____

已采取措施：_____

人员伤情：_____

设备损坏：_____

3.9 应急物资清单

3.9.1 必备物资

☐ 急救箱（碘伏、绷带、止血带等）

☐ 灭火器（CO2 或干粉，≥2kg）

☐ 绝缘手套

☐ 绝缘工具

☐ 警示标识

☐ 应急照明

3.9.2 推荐物资

☐ AED 自动除颤器

☐ 担架

☐ 安全帽

☐ 防护眼镜

☐ 绝缘鞋

3.10 应急演练

3.10.1 演练频率

- 首次培训：必须进行完整演练
- 定期演练：**每季度一次**
- 新员工入职：必须参与演练

3.10.2 演练项目

☐ 急停按钮使用

☐ 电源断开操作

☐ 灭火器使用

☐ 触电救援模拟

☐ 伤员急救模拟

☐ 应急疏散演练

3.10.3 演练评估

☐ 响应速度（< 5秒）

☐ 操作正确性

☐ 团队协作

☐ 改进建议

3.11 事故后处理

3.11.1 事故调查

1. 保护现场
2. 记录现场情况（拍照）
3. 收集证据和日志
4. 访谈相关人员
5. 分析事故原因
6. 提出改进措施

3.11.2 设备恢复

[警告] 不得在未经检查的情况下重启设备

检查清单:

☐ 电气系统完好

☐ 机械结构无损

☐ 安全装置功能正常

☐ 零位校准正确

☐ 参数配置合理

☐ 进行低速测试

3.11.3 预防措施

1. 更新操作规程
2. 加强人员培训
3. 改进安全设施
4. 定期检查维护
5. 记录经验教训

3.12 应急响应流程图

```
                事故发生
                   ↓
        ┌──────────┴──────────┐
        │                     │
   生命危险?              设备故障?
        │                     │
     是 │                     │ 是
        ↓                     ↓
  1.急停/断电           1.按下急停
  2.急救/疏散           2.切断电源
  3.拨打120/119         3.记录现象
  4.报告管理层           4.初步排查
        │                     │
        └──────────┬──────────┘
                   ↓
              保护现场
                   ↓
              事故调查
                   ↓
              原因分析
                   ↓
              改进措施
                   ↓
              恢复运行
```

牢记：人员安全永远是第一位的！遇到紧急情况，沉着冷静，按流程处理。

本文档版本：v1.0 最后更新：2025 年 10 月 19 日 成都长数机器人有限公司

4 责任声明

4.1 使用责任与免责条款

生效日期： 2025 年 **适用产品：** OpenArmX 双臂机器人系统

4.2 总则

本责任声明是 OpenArmX 机器人系统使用的法律文件，详细说明了用户和制造商的权利与责任。**使用本产品即表示您已阅读、理解并同意本声明的所有条款**。

如您不同意本声明的任何内容，请勿使用本产品，并立即联系销售商办理退货。

4.3 产品定义

4.3.1 产品范围

本声明适用于以下产品及其组件：

- OpenArmX 双臂机器人硬件
- Robstride 系列电机及驱动器
- CAN 通信系统及相关配件
- OpenArmX 控制软件及 ROS2 软件包
- 使用说明书及技术文档
- 官方提供的配置文件和示例代码

4.3.2 产品性质

本产品是**工业级机器人开发平台**，具有以下特性：

- **开发用途**：用于研发、教学、实验
- [警告] **存在风险**：包含高速运动部件和电气系统
- **需要专业知识**：需要相关技术背景
- **可定制化**：允许用户修改参数和程序

本产品不是：- [×] 面向普通消费者的成品 - [×] 即插即用的傻瓜式设备 - [×] 经过全面安全认证的工业产品 - [×] 适合儿童或无专业背景人员使用

4.4 用户责任

4.4.1 使用前责任

用户在使用产品前必须：

[√] **完整阅读文档** - 阅读并理解完整使用说明书 - 特别是安全警告和操作规范章节 - 理解产品的能力和局限性

[√] **具备相应能力** - 具有机器人或自动化系统相关知识 - 熟悉 ROS2、Linux 和 CAN 通信 - 具备基本的电气和机械安全知识

[√] **完成必要培训** - 参加产品操作培训（如提供）- 进行安全操作演练 - 通过操作能力评估

[√] **准备工作环境** - 设置符合要求的工作空间 - 配备必要的安全设施（急停按钮、围栏等）- 准备应急物资（灭火器、急救箱等）

4.4.2 使用中责任

用户在使用产品时必须：

[√] **遵守安全规范** - 严格按照使用说明书操作 - 遵守所有安全警告和注意事项 - 不得绕过或禁用安全装置

[√] **持续监控** - 在设备运行时保持警惕 - 随时准备按下急停按钮 - 及时发现和处理异常情况

[√] **正确维护** - 定期进行维护检查 - 及时更换损坏部件 - 保持设备清洁和良好状态

[√] **记录管理** - 记录操作日志 - 记录参数变更 - 记录故障和维修情况

4.4.3 安全责任

用户对以下安全事项负全部责任：

人员安全 - 确保操作人员经过培训 - 防止未授权人员接触设备 - 保护他人不受伤害

环境安全 - 确保工作环境符合要求 - 设置安全警示标识 - 控制人员进出工作区域

设备安全 - 正确安装和固定设备 - 使用符合规格的电源和配件 - 防止设备损坏

法律合规 - 遵守当地安全法规 - 获得必要的许可和审批 - 承担法律责任

4.5 制造商责任

4.5.1 责任限制

成都长数机器人有限公司（以下简称 “制造商”）

制造商不对以下情况承担责任：

[×] **非产品缺陷导致的问题** - 用户操作不当 - 未按说明书使用 - 擅自改装或修理 - 使用非原厂配件

[×] **不可控因素** - 自然灾害（地震、雷电、洪水等）- 电力异常（电压波动、停电等）- 网络或第三方软件问题

[×] **超出设计用途** - 在不适合的环境使用 - 超负荷运行 - 用于未经批准的应用

4.6 质量保证

4.6.1 质保范围

质保期： 自交付之日起 12 个月

质保内容： - [√] 硬件制造缺陷 - [√] 电机和驱动器故障（非人为）- [√] 软件明显 BUG（在标准配置下）

质保方式： - 免费维修或更换有缺陷的部件 - 提供远程技术支持 - 必要时上门服务（需商议费用）

4.6.2 质保排除

以下情况不在质保范围内：

[×] **人为损坏** - 摔落、碰撞、进水 - 错误连接导致烧毁 - 过载运行导致损坏

[×] **不当使用** - 未按说明书操作 - 使用非规定电源 - 在不适合环境使用

[×] **未授权维修** - 擅自拆解或改装 - 使用非原厂配件 - 非制造商授权的维修

[×] **自然损耗** - 正常磨损 - 消耗品（如线缆）- 外观划伤、褪色

4.6.3 质保流程

申请质保服务：

1. **联系技术支持**
   - 邮箱：openarmrobot@gmail.com
   - 电话/微信：+86-17746530375

2. **提供信息**
   - 产品型号和序列号
   - 购买凭证
   - 故障详细描述
   - 照片或视频（如需要）
3. **技术诊断**
   - 远程协助排查
   - 确认是否属于质保范围
4. **维修方案**
   - 寄回维修
   - 寄送备件
   - 上门服务（特殊情况）

4.7 免责条款

4.7.1 人身伤害免责

[警告] 重要声明：

制造商**不对**因使用本产品导致的人身伤害承担责任，包括但不限于：

- 电击、烧伤
- 挤压、碰撞伤害
- 机械伤害
- 其他任何人身损害

原因：本产品是工业级开发平台，**本身具有危险性**，需要专业人员在安全环境下使用。用户应自行采取所有必要的安全措施。

用户同意： - 自行承担使用风险 - 实施适当的安全保护 - 为操作人员购买保险（推荐）- 不向制造商追究人身伤害责任

4.7.2 财产损失免责

制造商不对以下财产损失承担责任：

直接损失 - 产品本身之外的设备损坏 - 工作场所财产损失 - 原材料或半成品损失

间接损失 - 停工损失 - 利润损失 - 商誉损失 - 数据丢失

4.7.3 软件免责

开源软件声明：

本产品使用的软件包含开源组件（ROS2、MoveIt 等），这些软件**按“原样”提供，不含任何明示或暗示的担保**。

软件免责： - [×] 不保证软件无 BUG - [×] 不保证与所有系统兼容 - [×] 不保证持续更新 - [×] 不对软件故障导致的损失负责

用户责任： - [√] 在测试环境验证软件 - [√] 备份重要数据 - [√] 自行评估软件适用性

4.7.4 第三方组件

本产品包含第三方组件： - Robstride 电机（原厂质保）- CAN 转换器（原厂质保）- 其他标准件

对第三方组件： - 制造商仅提供集成服务 - 质保由原厂商提供 - 超出原厂质保范围的不承担责任

4.8 使用限制

4.8.1 禁止用途

严禁将本产品用于：

[×] **危及生命安全的应用** - 医疗手术或诊断 - 核设施 - 航空航天 - 自动驾驶（载人）

[×] **军事用途** - 武器系统 - 军事侦察 - 战场应用

[×] **违法用途** - 违反法律法规的任何用途

如用于上述用途： - 用户自行承担全部后果 - 制造商不承担任何责任 - 制造商保留法律追诉权

4.8.2 出口限制

本产品可能受到出口管制： - 遵守中国出口管制法规 - 遵守目的地国家/地区法规 - 不得出口到受制裁国家/地区

用户责任： - 确认出口合法性 - 获得必要的出口许可 - 承担违反出口管制的后果

4.9 知识产权

4.9.1 所有权

制造商拥有以下知识产权： - 产品设计和外观 - 专有软件和算法 - 技术文档和手册 - 商标和品牌

用户权利： - [√] 使用产品及软件 - [√] 修改配置文件 - [√] 基于产品进行开发 - [×] **不得**商业性复制或分发

4.9.2 开源许可

开源组件：本产品部分软件采用开源许可（Apache 2.0、BSD 等）：

- 用户可根据相应许可证使用
- 必须保留许可证和版权声明
- 详见各组件的 LICENSE 文件

4.10 责任限额

4.10.1 赔偿上限

在任何情况下，制造商的赔偿责任总额不超过：

产品购买价格的 100%

此限额适用于： - 所有类型的损失 - 累计赔偿总额 - 无论索赔次数

4.10.2 诉讼时效

法律诉讼时效： - 自交付之日起 1 年内 - 超过时效不再受理

4.11 法律适用

4.11.1 适用法律

本声明受**中华人民共和国法律**管辖。

4.11.2 争议解决

因本产品引起的任何争议，应通过以下方式解决：

1. **友好协商**（优先）
2. **第三方调解**
3. **仲裁**（在成都仲裁委员会）
4. **诉讼**（在制造商所在地人民法院）

4.12 声明变更

4.12.1 修订权利

制造商保留修订本声明的权利。

4.12.2 通知方式

修订后的声明将通过以下方式通知：- 官方网站公告 - 邮件通知（如有登记）- 产品更新说明

4.12.3 生效方式

修订后的声明自公布之日起 30 天后生效。

4.13 用户确认

使用本产品即表示您确认：

[√] 已完整阅读本责任声明 [√] 理解并接受所有条款 [√] 同意承担使用风险 [√] 不会因本声明中已免责的事项追究制造商责任

4.14 联系方式

如对本声明有任何疑问，请联系：

成都长数机器人有限公司 邮箱：openarmrobot@gmail.com 电话/微信：+86-17746530375 网站：https://openarmx.com/ 地址：天津经济技术开发区西区新业八街 11 号华诚机械厂

最后提醒：

本产品是专业开发平台，**使用存在固有风险**。请确保：1. [√] 您具备必要的技术能力 2. [√] 您已采取充分的安全措施 3. [√] 您理解并接受相关风险 4. [√] 您同意本责任声明的所有条款

如有疑虑，请勿使用本产品！

本文档版本：v1.0 生效日期：2025 年 10 月 19 日 制造商：成都长数机器人有限公司

签署确认（可选）：

我已阅读并理解本责任声明的所有内容，同意遵守相关条款。

姓名：_____ 日期：_____ 签名：_____

5 产品介绍

5.1 OpenArmX 双臂机器人系统产品介绍

5.2 产品概述

5.2.1 什么是 OpenArmX

OpenArmX 是一款开源的**双臂协作机器人开发平台**, 由成都长数机器人有限公司研发。它基于高性能的 Robstride 准直驱电机, 采用 ROS2 Humble 架构, 提供完整的硬件和软件解决方案。

核心特点: - **双臂设计**: 配备左右两条 7 自由度机械臂 - **高性能电机**: 采用 Robstride RS 系列准直驱电机 - **完全开源**: 硬件设计、软件代码完全开放 - **易于开发**: 基于 ROS2 生态, 支持 MoveIt 运动规划 - **高度可定制**: 支持参数调整、二次开发

5.2.2 产品定位

OpenArmX 是一款**面向研发、教学和实验的专业开发平台**, 而非即插即用的消费级产品。

适用场景: - 机器人研究与教学 - 算法开发与验证 - 工业自动化原型开发 - 创新应用探索

适用人群: - 机器人研究人员 - 高校师生 - 自动化工程师 - 创客和开发者

5.3 技术规格

5.3.1 机械参数

机械臂规格: | 参数 | 规格 | |——|——| | 臂数量 | 2 条 (左臂、右臂) | | 单臂自由度 | 7 DOF | | 总自由度 | 14 DOF(不含夹爪) | | 臂展 | 约 800mm(单臂) | | 有效载荷 | 约 10kg(单臂末端，前平举持续运行 1 分钟) |

夹爪规格: | 参数 | 规格 | |——|——| | 夹爪类型 | 平行夹爪 | | 夹爪数量 | 2 个 (每臂 1 个) | | 最大开口 | 约 100mm | | 夹持力 | 可调 (通过 KP 参数) |

5.3.2 电机配置

OpenArmX 使用 Robstride 系列准直驱电机:

电机型号分布: | 关节位置 | 电机型号 | 数量 | 特性 | |———-|———|——|——| | 肩部 (1-2) | RS04 | 4 个 (每臂 2 个) | 大扭矩, 适合肩部负载 | | 大臂 (3-4) | RS03 | 4 个 (每臂 2 个) | 紧凑设计, 灵活运动 | | 小臂-夹爪 (5-8) | RS00 | 8 个 (每臂 4 个) | 小型高精度电机 |

电机关键参数: | 型号 | 功率 | 额定扭矩 | 峰值扭矩 | 最大速度 | 通信接口 | |——|——|———|——|———|———-| | RS04 | 700W | 40Nm | 120Nm | 15rad/s | CAN 2.0 | | RS03 | 380W | 20Nm | 60Nm | 20rad/s | CAN 2.0 | | RS00 | 140W | 5Nm | 14Nm | 33rad/s | CAN 2.0 |

注: 具体参数以实际电机规格书为准

5.3.3 控制系统

硬件平台: - 主控计算机: x86_64 架构 PC(推荐 Ubuntu 22.04) - CAN 接口: 双通道 CAN-USB 转换器 (can0/can1) - 电源: 48V DC, 推荐 ≥20A

软件架构: - 操作系统: Ubuntu 22.04 LTS - 机器人中间件: ROS2 Humble - 运动规划: MoveIt 2 - 控制框架: ros2_control - 通信协议: CAN 2.0(1Mbps)

5.4 功能特性

5.4.1 核心功能

5.4.1.1 双臂独立控制

- [√] 左右臂可独立规划和执行轨迹
- [√] 支持单臂或双臂同时运动
- [√] 独立的 CAN 总线 (can0/can1) 避免干扰

5.4.1.2 运动规划

- [√] 基于 MoveIt 2 的路径规划
- [√] 可视化操作界面 (RViz)
- [√] 碰撞检测和避障
- [√] 逆运动学求解

5.4.1.3 轨迹录制与回放

- [√] 分步录制关节状态
- [√] 连续录制运动轨迹
- [√] YAML 格式轨迹文件
- [√] 轨迹回放和重复执行

5.4.1.4 示教编程

- [√] 拖动示教模式 (降低 KP/KD)

- [√] 记录关键点
- [√] 生成可复现轨迹

5.4.1.5 力控与柔顺控制

- [√] 通过 KP/KD 调节刚性
- [√] 力矩反馈
- [√] 阻抗控制 (需二次开发)

5.4.2 开发友好性

5.4.2.1 完整的软件包

```
openarmx_robotstride_ws-cc/
├── openarm_ros2/              # ROS2核心功能包
├── openarm_moveit_config/     # MoveIt配置
├── openarm_record/            # 轨迹录制
├── openarm_can_openarmx_com/  # CAN通信库
├── motor_tests_openarmx_com/  # 电机测试脚本
└── motor_manage_openarmx/     # 图形化管理界面
```

5.4.2.2 丰富的测试工具

- 单电机测试工具
- 电机状态监控
- 零位设置工具
- 图形化管理界面

5.4.2.3 详细的文档

- 完整的使用说明书
- API 参考文档
- 示例代码和教程

5.5 应用场景

5.5.1 研究与教学

机器人研究: - 双臂协调算法研究 - 人机交互研究 - 强化学习训练平台 - 视觉伺服研究

教学实验: - 机器人学课程实验 - 运动规划教学 - 控制理论验证 - ROS2 框架学习

5.5.2 工业应用原型

装配任务: - 小零件装配 - 螺丝拧紧 - 插件组装

物料搬运: - 分拣和码放 - 包装作业 - 上下料

精密操作: - 检测和测量 - 打磨和抛光 (需工具扩展) - 涂胶和点胶

5.5.3 创新应用

服务机器人: - 咖啡制作 - 简单烹饪 - 物品整理

娱乐与展示: - 绘画和书法 - 互动展示 - 游戏对战

5.6 系统优势

5.6.1 技术优势

5.6.1.1 准直驱电机 [√] **高扭矩密度**: 相比传统减速器方案更紧凑 [√] **低摩擦**: 减少机械传动损失 [√] **高响应**: 控制带宽更高 [√] **高精度**: 直接驱动, 无齿隙

5.6.1.2 ROS2 生态 [√] **标准化接口**: 符合 ROS-Industrial 规范 [√] **丰富的工具**: MoveIt、RViz 等成熟工具 [√] **社区支持**: 庞大的 ROS 开发者社区 [√] **易于集成**: 可与其他 ROS 包无缝协作

5.6.1.3 模块化设计 [√] **硬件模块化**: 电机可更换和升级 [√] **软件模块化**: 各功能包独立, 易于扩展 [√] **接口标准化**: 符合 ros2_control 规范

5.6.2 开源优势

5.6.2.1 完全开放

- 硬件设计文件开放
- 软件源代码开放
- 配置文件可修改
- 通信协议公开

5.6.2.2 可定制化

- 参数可自由调整 (KP/KD/速度/加速度)
- 控制策略可修改
- 硬件可扩展 (添加传感器/工具)
- 软件功能可扩展

5.6.2.3 低成本

- 相比商业机器人成本更低
- 维护成本低 (开源社区支持)
- 升级成本低 (软件免费更新)

5.7 系统局限性

5.7.1 技术限制

[警告] **有效载荷有限**: 单臂载荷约 2kg, 不适合重载应用 [警告] **精度限制**: 重复定位精度 ±1mm, 不适合超高精度作业 [警告] **速度限制**: 为保证安全, 运动速度受限 [警告] **环境限制**: 室内使用, 不防水不防尘

5.7.2 使用限制

[警告] **需要专业知识**: 需要 ROS2、机器人学、Linux 等背景 [警告] **非即插即用**: 需要配置、调试和开发 [警告] **安全责任**: 用户需自行采取安全措施 [警告] **商业化限制**: 需要大量工程化工作才能用于生产

5.7.3 应用限制

[×] **禁止用于生命关键应用**: 医疗手术、核设施等 [×] **禁止用于军事用途**: 武器系统等 [×] **不适合高负载**: 超过额定载荷会损坏电机 [×] **不适合高速应用**: 安全限制速度

5.8 产品组成

5.8.1 硬件清单

标准配置:

```
□ OpenArmX 双臂机器人本体 × 1
  ├── RS04 电机 × 4(肩部)
  ├── RS03 电机 × 4(大臂)
```

```
├── RS00 电机 × 8(小臂-夹爪)
└── 机械结构件和连接件
```

- ☐ CAN-USB 转换器 × 1(双通道)
- ☐ CAN 总线线缆
- ☐ 电源适配器(48V)
- ☐ 紧急停止按钮(推荐)

可选配件:

- ☐ 额外夹爪
- ☐ 力矩传感器
- ☐ 视觉传感器
- ☐ 安装底座

5.8.2 软件清单

预装软件:

- ☐ ROS2 Humble
- ☐ MoveIt 2
- ☐ RViz 可视化工具
- ☐ OpenArmX 控制软件包
- ☐ 电机测试工具
- ☐ 图形化管理界面

开发工具:

- ☐ Python 3.10+
- ☐ C++ 编译工具链
- ☐ colcon 构建工具
- ☐ Git 版本控制

5.9 与同类产品对比

5.9.1 对比商业机器人

特性	OpenArmX	商业协作机器人
价格	低	高

特性	OpenArmX	商业协作机器人
开源性	完全开源	闭源
可定制性		
安全认证	[×] 无	[√] CE/ISO
技术支持	社区 + 有限官方	完善的技术支持
易用性	需要专业知识	即插即用
应用场景	研发/教学	生产/服务

5.9.2 对比开源机器人

特性	OpenArmX	其他开源机器人
双臂设计	[√]	多为单臂
电机类型	准直驱 (高性能)	步进/舵机 (低成本)
ROS2 支持	[√] Humble	部分仅 ROS1
文档完整性		
商业支持	[√] 有	社区驱动
载荷能力	~2kg	0.5-3kg

5.10 发展路线

5.10.1 当前版本 (V1.0)

已实现功能: - [√] 双臂独立控制 - [√] MoveIt 运动规划 - [√] 轨迹录制和回放 - [√] 基本力控功能 - [√] 图形化管理界面

5.10.2 计划中的功能

近期规划 (6 个月内): - 双臂协调运动 - 视觉伺服集成 - 力/位混合控制 - 更多示例应用

远期规划 (1 年内): - 机器学习集成 (模仿学习) - 虚拟仿真环境 - 更强大的 GUI 工具 - 移动底座集成

5.11 获取与支持

5.11.1 获取产品

购买渠道: - 官方网站: https://openarmx.com/ - 邮箱咨询: openarmrobot@gmail.com - 电话/微信: +86-17746530375

开源代码: - GitHub: [待提供链接] - Gitee: [待提供链接]

5.11.2 技术支持

官方支持: - 技术支持邮箱: openarmrobot@gmail.com - 技术热线: +86-17746530375 - 官方网站文档: https://openarmx.com/docs

社区支持: - 用户论坛: [待建立] - 微信用户群: [联系获取] - GitHub Issues: [代码仓库]

5.11.3 培训服务

线上培训: - 视频教程 (免费) - 文档教程 (免费) - 在线答疑

线下培训: - 定制培训服务 (需商议) - 高校合作课程

5.12 安全与责任

5.12.1 安全声明

[警告] **重要提醒:**

OpenArmX 是**工业级开发平台**, 不是消费级产品。使用前必须: - [√] 完整阅读安全警告和操作规范 - [√] 具备必要的技术背景和培训 - [√] 配备急停按钮等安全设施 - [√] 采取适当的安全防护措施

5.12.2 免责条款

使用本产品即表示您: - [√] 理解并接受产品的固有风险 - [√] 同意自行承担使用责任 - [√] 不会用于禁止的应用场景 - [√] 遵守当地法律法规

详细内容请参阅《00_ 安全声明/04_ 责任声明.md》

5.13 总结

5.13.1 适合您吗?

OpenArmX 适合您, 如果您: - [√] 是机器人研究人员或工程师 - [√] 需要双臂机器人开发平台 - [√] 重视开源和可定制性 - [√] 具备 ROS2 和 Linux 基础 - [√] 接受需要调试和开发的现实

OpenArmX 不适合您, 如果您: - [×] 需要即插即用的产品 - [×] 缺乏技术背景 - [×] 用于生产环境 (未经工程化) - [×] 需要官方安全认证 - [×] 需要 24/7 技术支持

5.13.2 核心价值

为什么选择 OpenArmX:

1. **开放性**
   - 完全开源, 无黑盒
   - 自由修改和扩展
2. **专业性**
   - 高性能准直驱电机
   - 基于 ROS2 标准架构
3. **经济性**
   - 相比商业方案成本更低
   - 维护和升级成本低
4. **教育性**
   - 完整的学习资源
   - 丰富的示例代码
5. **创新性**
   - 快速验证创新想法
   - 灵活的开发平台

欢迎加入 OpenArmX 开源机器人社区!

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

6 系统架构

6.1 OpenArmX 系统架构详解

本文档详细介绍 OpenArmX 双臂机器人的软硬件架构、技术栈和控制流程。

6.2 总体架构

6.2.1 系统分层架构

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
│                 应用层 (Application Layer)                    │
│   ┌──────────────────────────────┐   ┌──────────────────────────────┐   ┌──────────────────────────────┐
│   │ MoveIt Demo   │   │   轨迹录制回放   │   │   自定义应用     │   │
│   └──────────────────────────────┘   └──────────────────────────────┘   └──────────────────────────────┘
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────

                          ↕

┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
│              规划层 (Planning Layer)                          │
│   ┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
│   │            MoveIt 2 运动规划框架                      │   │
│   │   (OMPL/逆运动学/碰撞检测/轨迹优化)                    │   │
│   └──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────

                          ↕

┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
│               控制层 (Control Layer)                          │
│   ┌──────────────────────────────────────────┐   ┌──────────────────────────────────────────┐       │
│   │ JointTrajectory     │   │ ForwardPosition     │         │
│   │ Controller          │   │ Controller          │         │
│   └──────────────────────────────────────────┘   └──────────────────────────────────────────┘       │
│                  ros2_control 框架                            │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────

                          ↕

┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
│            硬件接口层 (Hardware Interface Layer)              │
│   ┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
│   │        OpenArm V10 Hardware Interface                 │   │
│   │       (读取关节状态/写入关节命令)                        │   │
│   └──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────

                          ↕
```

```
┌──────────────────────────────────────────────────────────────────────────────
│            驱动层 (Driver Layer)                          │
│   ┌──────────────────────────────────────────────────────────────────────────
│   │      Robstride Motor Control Library              │   │
│   │     (MIT模式控制/参数读取/零位管理)                    │   │
│   └──────────────────────────────────────────────────────────────────────────
└──────────────────────────────────────────────────────────────────────────────
                          ↕
┌──────────────────────────────────────────────────────────────────────────────
│           通信层 (Communication Layer)                    │
│   ┌──────────────────────────┐              ┌──────────────────────────┐          │
│   │    can0       │              │    can1       │           │
│   │   (左臂+左夹爪)│              │  (右臂+右夹爪) │           │
│   └──────────────────────────┘              └──────────────────────────┘          │
└──────────────────────────────────────────────────────────────────────────────
                          ↕
┌──────────────────────────────────────────────────────────────────────────────
│            硬件层 (Hardware Layer)                        │
│   ┌──────────────────────────────┐          ┌──────────────────────────────┐      │
│   │   左臂电机 × 7   │              │   右臂电机 × 7   │        │
│   │   左夹爪 × 1     │              │   右夹爪 × 1     │        │
│   └──────────────────────────────┘          └──────────────────────────────┘      │
└──────────────────────────────────────────────────────────────────────────────
```

6.3 硬件架构

6.3.1 机械结构

双臂拓扑结构:

```
              ┌──────────────┐
              │   基座     │
              └──────────────┘
                     │
        ┌────────────────────────┴────────────────────────┐
        │                                                 │
   ┌────────▼────────┐                   ┌────────▼────────┐
```

```
│ 左肩关节1│          │ 右肩关节1│
└──────┬──────┘          └──────┬──────┘
┌──────▼──────┐          ┌──────▼──────┐
│ 左肩关节2│          │ 右肩关节2│
└──────┬──────┘          └──────┬──────┘
┌──────▼──────┐          ┌──────▼──────┐
│左大臂关节3│          │右大臂关节3│
└──────┬──────┘          └──────┬──────┘
┌──────▼──────┐          ┌──────▼──────┐
│左大臂关节4│          │右大臂关节4│
└──────┬──────┘          └──────┬──────┘
┌──────▼──────┐          ┌──────▼──────┐
│左小臂关节5│          │右小臂关节5│
└──────┬──────┘          └──────┬──────┘
┌──────▼──────┐          ┌──────▼──────┐
│左小臂关节6│          │右小臂关节6│
└──────┬──────┘          └──────┬──────┘
┌──────▼──────┐          ┌──────▼──────┐
│左小臂关节7│          │右小臂关节7│
└──────┬──────┘          └──────┬──────┘
┌──────▼──────┐          ┌──────▼──────┐
│   左夹爪   │          │   右夹爪   │
└─────────────┘          └─────────────┘
```

自由度 (DOF) 配置: - 左臂: 7 DOF(关节 1-7) - 右臂: 7 DOF(关节 1-7) - 左夹爪: 1 DOF - 右夹爪: 1 DOF - **总计: 16 DOF**

6.3.2 电机拓扑

CAN 总线拓扑:

```
┌──────────────────────────┐
│ 控制计算机       │
└──────┬──────────┬──────┘
       │          │
       │          └──────────────────────────┐
```

```
    │                          │
┌───────▼────────┐             ┌──────────▼──────┐
│   can0   │              │   can1    │
└────────┬───────┘             └──────────┬──────┘
    │                          │
    ├── ID 1: 左肩关节1(RS04)   ├── ID 1: 右肩关节1(RS04)
    ├── ID 2: 左肩关节2(RS04)   ├── ID 2: 右肩关节2(RS04)
    ├── ID 3: 左大臂关节3(RS03) ├── ID 3: 右大臂关节3(RS03)
    ├── ID 4: 左大臂关节4(RS03) ├── ID 4: 右大臂关节4(RS03)
    ├── ID 5: 左小臂关节5(RS00) ├── ID 5: 右小臂关节5(RS00)
    ├── ID 6: 左小臂关节6(RS00) ├── ID 6: 右小臂关节6(RS00)
    ├── ID 7: 左小臂关节7(RS00) ├── ID 7: 右小臂关节7(RS00)
    └── ID 8: 左夹爪(RS00)      └── ID 8: 右夹爪(RS00)
```

总线参数: - 波特率: 1 Mbps - 协议: CAN 2.0 - 终端电阻: 120Ω(总线两端)

6.3.3 电气系统

供电架构:

```
┌──────────────────────────┐
│ 48V 电源       │
│ (≥20A)         │
└──────────────┬───────────┘
        │
        ├──────────────── 电机驱动器(集成在电机内)
        │              │
        │              ├── 左臂电机 × 8
        │              └── 右臂电机 × 8
        │
        └──────────────── 控制计算机(如需要)
                       │
                       ├── CAN-USB转换器(USB供电)
                       └── 其他外设
```

安全电路:

```
急停按钮 ────┬──── 48V电源开关
```

```
            │
            └──── 控制信号(可选)
```

6.4 软件架构

6.4.1 ROS2 节点架构

核心节点图:

```
┌──────────────
```

```
│         │  can0      │ │  can1     │                  │
│         │  driver    │ │  driver   │                  │
│         └────────────────────────┘  └────────────────────────┘                  │
└──────────────────────────
```

```
├── 发布者: move_group
└── 订阅者: left_arm_controller

/right_arm_controller/joint_trajectory (trajectory_msgs/JointTrajectory)
├── 发布者: move_group
└── 订阅者: right_arm_controller

/left_gripper_controller/commands (std_msgs/Float64MultiArray)
├── 发布者: 用户应用
└── 订阅者: left_gripper_controller

/right_gripper_controller/commands (std_msgs/Float64MultiArray)
├── 发布者: 用户应用
└── 订阅者: right_gripper_controller
```

6.4.3 控制器架构

控制器配置:

```yaml
controller_manager:
  ros__parameters:
    update_rate: 100  # Hz

    # 控制器列表
    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster

    left_arm_controller:
      type: joint_trajectory_controller/JointTrajectoryController

    right_arm_controller:
      type: joint_trajectory_controller/JointTrajectoryController

    left_gripper_controller:
      type: forward_command_controller/ForwardCommandController
```

```yaml
  right_gripper_controller:
    type: forward_command_controller/ForwardCommandController
```

控制器层级:

```
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│          Controller Manager                  │
│  ┌──────────────────────────────────────────────────────────────────────────────────────────┐ │
│  │ joint_state_broadcaster (Always Active)│  │
│  └──────────────────────────────────────────────────────────────────────────────────────────┘ │
│                                              │
│  ┌──────────────────────────────┐  ┌──────────────────────────────┐      │
│  │ left_arm        │  │ right_arm       │      │
│  │ _controller     │  │ _controller     │      │
│  │                 │  │                 │      │
│  │ Type: Joint     │  │ Type: Joint     │      │
│  │ Trajectory      │  │ Trajectory      │      │
│  └──────────────────────────────┘  └──────────────────────────────┘      │
│                                              │
│  ┌──────────────────────────────┐  ┌──────────────────────────────┐      │
│  │ left_gripper    │  │ right_gripper   │      │
│  │ _controller     │  │ _controller     │      │
│  │                 │  │                 │      │
│  │ Type: Forward   │  │ Type: Forward   │      │
│  │ Command         │  │ Command         │      │
│  └──────────────────────────────┘  └──────────────────────────────┘      │
└──────────────────────────────────────────────────────────────────────────────────────────────┘
```

6.5 数据流

6.5.1 读取流程 (从硬件到软件)

```
┌──────────────────────┐
│ 电机          │
│ (实际位置)    │
└──────────┬───────────┘
           │ CAN帧
```

```
    ▼
┌──────────────────────────────────┐
│ CAN驱动(SocketCAN)  │
└──────────┬──────────────────────────┘
    │ can_frame
    ▼
┌──────────────────────────────────────────┐
│ RobstrideMotor::read()  │
│ (解码位置/速度/力矩)     │
└──────────┬──────────────────────────────────┘
    │ motor_state
    ▼
┌──────────────────────────────────────────────────────────┐
│ OpenArmV10Hardware::read()      │
│ (更新 hw_states_position_)      │
└──────────┬──────────────────────────────────────────────────┘
    │ joint_states
    ▼
┌──────────────────────────────────────────────┐
│ ros2_control Framework    │
│ (更新控制器状态)          │
└──────────┬──────────────────────────────────────┘
    │ /joint_states 话题
    ▼
┌──────────────────────────────────────┐
│ robot_state_publisher│
│ (发布TF树)            │
└──────────┬──────────────────────────────┘
    │ /tf 话题
    ▼
┌──────────────────────────────────┐
│ MoveIt / RViz     │
│ (可视化和规划)     │
└──────────────────────────────────┘
```

更新频率: - CAN 读取: ~1000 Hz(电机内部) - Hardware Interface read(): 100 Hz(配置值) - joint_states 发布: 100 Hz - TF 发布: 100 Hz

6.5.2 写入流程 (从软件到硬件)

轨迹执行流程:

```
┌──────────────────────────────┐
│ MoveIt              │
│ (规划轨迹)          │
└──────────┬───────────────────┘
      │ FollowJointTrajectory Action
      ▼
┌──────────────────────────────────────────────────────┐
│ JointTrajectoryController      │
│  (插值生成关节命令)              │
└──────────┬───────────────────────────────────────────┘
      │ joint_position_command (100Hz)
      ▼
┌──────────────────────────────────────────────────────────┐
│ OpenArmV10Hardware::write()      │
│ (读取 hw_commands_position_)     │
└──────────┬───────────────────────────────────────────────┘
      │ target_position
      ▼
┌──────────────────────────────────────────────────────────────┐
│ RobstrideMotor::setCommand()      │
│  (编码MIT模式控制帧)                 │
└──────────┬───────────────────────────────────────────────────┘
      │ CAN帧
      ▼
┌──────────────────────────────┐
│ CAN驱动(SocketCAN) │
└──────────┬───────────────────┘
      │
      ▼
```

```
┌──────────────────────┐
│ 电机          │
│ (执行运动)    │
└──────────────────────┘
```

6.5.3 完整控制循环

100Hz 控制循环:

```
    ┌──────────────────────────────────────────────────────────────────┐
    │                                          │
    │     每10ms执行一次                        │
    │                                          │
    ▼                                          │
┌──────────────────┐        ┌──────────────────┐        ┌──────────────────┐
│   Read    │────────────│  Update   │────────────│   Write    │
│ (读传感器)│        │ (控制器) │        │ (写命令)   │
└──────────────────┘        └──────────────────┘        └──────────────────┘
    │                  │                  │
    │                  │                  │
    ▼                  ▼                  ▼
  获取电机           计算控制律           发送电机
  位置/速度           (PID/轨迹)           目标位置
  力矩反馈           插值等               KP/KD等
    │                  │                  │
    └──────────────────────────────────┴──────────────────────────────────┘
                        │
                        │ 10ms后
                        └──────────────────────────────┐
                                        │
                                        ▼
                                    下一循环
```

6.6 技术栈

6.6.1 软件技术栈

操作系统层:

```
Ubuntu 22.04 LTS (Jammy Jellyfish)
├── Linux Kernel 5.15+
├── SocketCAN驱动
└── 实时性优化(可选: PREEMPT_RT补丁)
```

中间件层:

```
ROS2 Humble Hawksbill
├── DDS: FastDDS (默认)
├── rcl (ROS Client Library)
├── rclcpp (C++ API)
└── rclpy (Python API)
```

机器人框架层:

```
ros2_control
├── controller_interface
├── hardware_interface
├── controller_manager
└── 各类控制器插件


MoveIt 2
├── move_group
├── OMPL (路径规划)
├── KDL (运动学)
└── FCL (碰撞检测)
```

应用层:

```
OpenArmX 软件包
├── openarm_ros2 (核心功能)
├── openarm_moveit_config (MoveIt配置)
├── openarm_record (轨迹录制)
├── openarm_can_openarmx_com (CAN驱动)
```

```
└── motor_manage_openarmx (GUI)
```

6.6.2 开发语言

语言	用途	典型文件
C++ (17)	性能关键部分	硬件接口, 电机驱动
Python (3.10+)	工具脚本, GUI	测试脚本, 管理界面
YAML	配置文件	launch 参数, 控制器配置
URDF/Xacro	机器人描述	运动学模型
XML	ROS 配置	package.xml
CMake	构建系统	CMakeLists.txt

6.6.3 关键库依赖

C++ 库:

```
- rclcpp (ROS2 C++ API)
- hardware_interface (ros2_control)
- controller_interface
- moveit_core
- Eigen (线性代数)
- Boost (工具库)
```

Python 库:

```
- rclpy (ROS2 Python API)
- python-can (CAN通信)
- numpy (数值计算)
- PySide6 (GUI)
- pyyaml (配置解析)
```

6.7 文件系统结构

6.7.1 工作空间组织

```
~/openarmx_robotstride_ws-cc/
├── src/                          # 源代码目录
│   ├── openarm_ros2/              # ROS2核心包
```

```
│   │   ├── openarm_description/  # 机器人URDF模型
│   │   ├── openarm_hardware/     # 硬件接口实现
│   │   ├── openarm_bringup/      # 启动文件
│   │   └── openarm_controllers/  # 控制器配置
│   │
│   ├── openarm_moveit_config/    # MoveIt配置
│   │   ├── config/               # 运动学、规划器配置
│   │   └── launch/               # MoveIt启动文件
│   │
│   ├── openarm_record/           # 轨迹录制功能
│   │   ├── scripts/              # 录制/回放脚本
│   │   └── data/                 # 轨迹数据
│   │
│   ├── openarm_can_openarmx_com/ # CAN通信库
│   │   ├── include/              # 头文件
│   │   └── src/                  # 电机驱动实现
│   │
│   ├── motor_tests_openarmx_com/ # 电机测试工具
│   │   └── *.py                  # Python测试脚本
│   │
│   ├── motor_manage_openarmx/    # 图形化管理界面
│   │   ├── ui/                   # PySide6界面
│   │   └── resources/            # 资源文件
│   │
│   └── 使用说明书/               # 文档
│
├── build/                        # 编译生成目录
├── install/                      # 安装目录
└── log/                          # 日志目录
```

6.7.2 配置文件位置

URDF/Xacro:

```
src/openarm_ros2/openarm_description/
├── urdf/
```

```
│   ├── openarm_v10.urdf.xacro    # 主模型文件
│   ├── left_arm.xacro            # 左臂定义
│   ├── right_arm.xacro           # 右臂定义
│   └── materials.xacro           # 材质和颜色
└── meshes/                       # 3D网格文件(STL/DAE)
```

ros2_control 配置:

```
src/openarm_ros2/openarm_controllers/
├── config/
│   ├── openarm_controllers.yaml  # 控制器参数
│   └── hardware.yaml             # 硬件接口参数
```

MoveIt 配置:

```
src/openarm_moveit_config/
├── config/
│   ├── openarm_v10.srdf          # 语义机器人描述
│   ├── kinematics.yaml           # 逆运动学配置
│   ├── joint_limits.yaml         # 关节限制
│   ├── ompl_planning.yaml        # OMPL规划器参数
│   └── moveit_controllers.yaml   # MoveIt控制器配置
```

6.8 通信协议

6.8.1 CAN 协议

Robstride MIT 模式控制帧:

```
CAN ID: 0x000 - 0x01F (电机ID: 1-16)
```

数据帧格式(8字节):

Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7
Pos_H	Pos_L	Vel_H	Vel_L	KP_H	KP_L	KD_H	KD_L

```
Pos: 位置 (-12.5 ~ 12.5 rad, 16bit)
```

Vel: 速度 (-65 ~ 65 rad/s, 16bit)

KP: 位置增益 (0 ~ 500, 16bit)

KD: 速度增益 (0 ~ 5, 16bit)

电机反馈帧:

数据帧格式(8字节):

Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7
Pos_H	Pos_L	Vel_H	Vel_L	Tor_H	Tor_L	Temp	Error

Pos: 当前位置 (rad)

Vel: 当前速度 (rad/s)

Tor: 当前力矩 (Nm)

Temp: 温度 (°C)

Error: 错误代码

6.8.2 ROS2 消息类型

关节状态:

```
// sensor_msgs/msg/JointState
std_msgs/Header header
string[] name                    // 关节名称
float64[] position               // 位置 (rad)
float64[] velocity               // 速度 (rad/s)
float64[] effort                 // 力矩 (Nm)
```

轨迹点:

```
// trajectory_msgs/msg/JointTrajectory
std_msgs/Header header
string[] joint_names
JointTrajectoryPoint[] points
  float64[] positions
  float64[] velocities
```

```
float64[] accelerations
float64[] effort
builtin_interfaces/Duration time_from_start
```

6.9 性能参数

6.9.1 控制性能

指标	参数	说明
控制频率	100 Hz	ros2_control 更新率
CAN 波特率	1 Mbps	总线通信速度
延迟 (端到端)	~15-20ms	MoveIt 命令到电机执行
位置精度	±1mm	末端重复定位精度
轨迹跟踪误差	<5°	关节角度跟踪

6.9.2 通信性能

指标	can0	can1
电机数量	8	8
消息频率	100 Hz/电机	100 Hz/电机
总线利用率	~10%	~10%
最大延迟	<1ms	<1ms

6.9.3 计算资源

推荐配置: | 资源 | 最低要求 | 推荐配置 | |——|———|———| | CPU | 4 核 2.0GHz | 8 核 3.0GHz+ | | 内存 | 8GB | 16GB+ | | 磁盘 | 50GB | 100GB+ SSD | | 操作系统 | Ubuntu 22.04 | Ubuntu 22.04 + RT 补丁 |

典型 CPU 占用: - ros2_control_node: ~15% - move_group: ~20%(规划时) - rviz2: ~10% - 其他节点: ~5%

6.10 扩展性设计

6.10.1 硬件扩展

传感器集成:

```
├── 力/力矩传感器
│   └── 通过模拟输入或CAN总线
│
├── 视觉传感器
│   ├── USB相机
│   ├── RealSense深度相机
│   └── 发布 sensor_msgs/Image
│
└── 触觉传感器
    └── 集成到夹爪
```

末端工具扩展:

```
├── 不同夹爪
│   └── 修改URDF和控制器配置
│
├── 工具(吸盘、笔等)
│   └── 添加到tool_frame
│
└── 力控工具
    └── 实现力控控制器
```

6.10.2 软件扩展

自定义控制器:

```
// 继承 controller_interface::ControllerInterface
class MyCustomController : public controller_interface::ControllerInterface
{
  controller_interface::CallbackReturn on_init() override;
  controller_interface::InterfaceConfiguration command_interface_configuration() const override;
  controller_interface::return_type update(const rclcpp::Time & time, const rclcpp::Duration & period)
};
```

自定义规划器:

```
// 继承 MoveIt planning_interface
class MyPlanner : public planning_interface::PlannerManager
```

```
{
  bool solve(planning_interface::MotionPlanResponse &res) override;
};
```

6.10.3 多机器人协作

未来扩展方向:

```
┌──────────────────────┐          ┌──────────────────────┐
│ OpenArmX_1 │──────────────│ OpenArmX_2 │
└──────────┬───────────┘          └──────────┬───────────┘
      │                        │
      └────────────────────────┬────────────────────────┘
                  │
            ┌──────────────▼──────────────┐
            │ 协调节点          │
            │ (多机器人调度)    │
            └──────────────────────────────┘
```

6.11 安全架构

6.11.1 多层安全机制

```
┌──────────────────────────────────────────────────────────────────────────┐
│          软件安全层 (Software Safety)      │
│   ┌──────────────────────────────────────────────────────────────┐      │
│   │ - 关节限位检查                    │      │
│   │ - 速度/加速度限制                 │      │
│   │ - 碰撞检测(MoveIt)                │      │
│   │ - 奇异点回避                      │      │
│   └──────────────────────────────────────────────────────────────┘      │
└──────────────────────────────────────────────────────────────────────────┘
                    ↕
┌──────────────────────────────────────────────────────────────────────────┐
│        固件安全层 (Firmware Safety)        │
│   ┌──────────────────────────────────────────────────────────────┐      │
│   │ - 电机过载保护                    │      │
```

```
│   │ - 温度保护                    │     │
│   │ - 通信超时保护                │     │
│   └──────────────────────────────┘     │
└──────────────────────────────────────────┘
                ↕
┌──────────────────────────────────────────┐
│      硬件安全层 (Hardware Safety)        │
│   ┌──────────────────────────────┐     │
│   │ - 急停按钮(物理断电)          │     │
│   │ - 电源保护电路                │     │
│   │ - 机械限位(可选)              │     │
│   └──────────────────────────────┘     │
└──────────────────────────────────────────┘
```

6.11.2 故障处理流程

```
    检测到异常
        │
        ▼
    ┌──────────┐
    │日志记录  │
    └──────┬───┘
        │
        ▼
    ┌──────────────┐
    │ 判断严重程度 │
    └──────┬───────┘
        │
    ┌──────┴──────┐
    │             │
    ▼             ▼
  轻微          严重
    │             │
    │             ▼
    │        停止运动
```

```
  │          │
  │          ▼
  │     禁用电机
  │          │
  ▼          ▼
告警     紧急停止
  │          │
  └──────────┬──────────┘
             │
             ▼
  等待人工干预
```

6.12 总结

OpenArmX 采用**分层模块化架构**, 各层职责清晰:

1. **硬件层**: Robstride 电机提供高性能执行
2. **驱动层**: CAN 通信库封装底层协议
3. **接口层**: ros2_control 标准化硬件接口
4. **控制层**: 多种控制器支持不同应用
5. **规划层**: MoveIt 提供智能运动规划
6. **应用层**: 录制回放、示教等高级功能

这种架构具有以下优势: - [√] **标准化**: 符合 ROS-Industrial 规范 - [√] **可扩展**: 易于添加新功能 - [√] **可维护**: 模块独立, 便于调试 - [√] **高性能**: 100Hz 控制频率, 低延迟

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

7 电机配置说明

7.1 Robstride 电机配置详解

本文档详细说明 OpenArmX 系统中使用的 Robstride 电机型号、参数、ID 分配和映射关系。

7.2 电机型号概述

7.2.1 Robstride 电机系列

OpenArmX 使用三种 Robstride 准直驱电机:

型号	用途	数量	位置
RS04	肩部关节	4	关节 1-2(左右臂各 2 个)
RS03	大臂关节	4	关节 3-4(左右臂各 2 个)
RS00	小臂和夹爪	8	关节 5-8(左右臂各 4 个)
总计	-	**16**	-

7.2.2 电机技术参数

7.2.2.1 RS04 电机 (肩部关节)

参数	规格
功率	700W
额定扭矩	40 Nm
峰值扭矩	120 Nm
额定电流	14.6 A
最大速度	15 rad/s
位置范围	±12.57 rad
KP 范围	0-5000
KD 范围	0-100
编码器分辨率	16bit
通信接口	CAN 2.0
供电电压	48V DC
电缆直径	1.3-1.5 mm

适用关节: - 左臂关节 1, 2 - 右臂关节 1, 2

特点: - [√] 大扭矩输出, 适合承载机械臂重量 - [√] 较大的外形尺寸 - [√] 用于肩部关节

7.2.2.2 RS03 电机 (大臂关节)

参数	规格
功率	380W
额定扭矩	20 Nm
峰值扭矩	60 Nm

参数	规格
额定电流	7.9 A
最大速度	20 rad/s
位置范围	±12.57 rad
KP 范围	0-5000
KD 范围	0-100
编码器分辨率	16bit
通信接口	CAN 2.0
供电电压	48V DC

适用关节: - 左臂关节 3, 4 - 右臂关节 3, 4

特点: - [√] 中等扭矩, 适合大臂关节 - [√] 紧凑设计 - [√] 灵活的运动能力

7.2.2.3 RS00 电机 (小臂和夹爪)

参数	规格
功率	140W
额定扭矩	5 Nm
峰值扭矩	14 Nm
额定电流	2.9 A
最大速度	33 rad/s
位置范围	±12.57 rad
KP 范围	0-500
KD 范围	0-5
编码器分辨率	16bit
通信接口	CAN 2.0
供电电压	48V DC

适用关节: - 左臂关节 5, 6, 7, 夹爪 - 右臂关节 5, 6, 7, 夹爪

特点: - [√] 小型高精度 - [√] 适合小臂和末端执行器 - [√] 可调节夹持力

7.3 CAN 总线配置

7.3.1 CAN 通道分配

OpenArmX 使用**双 CAN 总线架构**, 左右臂独立控制:

```
┌──────────────────────────┐
│ 控制计算机        │
└──────┬──────────────┬────┘
    │          │
┌───▼────┐   ┌─▼──────┐
│ can0  │ │ can1  │
└──────┬─────┘   └──────┬────┘
    │           │
┌───▼──────────┐   ┌───▼──────────┐
│ 左臂+左夹爪│ │ 右臂+右夹爪│
│ (8个电机)  │ │ (8个电机)  │
└──────────────┘   └──────────────┘
```

优势: - [√] 总线负载分离, 降低通信冲突 - [√] 独立故障隔离 - [√] 更高的通信可靠性

7.3.2 电机 ID 映射表

7.3.2.1 can0 (左臂 + 左夹爪)

CAN ID	关节名称	电机型号	URDF 关节名	位置描述
1	左肩关节 1	RS04	left_joint_1	肩部关节 1
2	左肩关节 2	RS04	left_joint_2	肩部关节 2
3	左大臂关节 3	RS03	left_joint_3	大臂关节 1
4	左大臂关节 4	RS03	left_joint_4	大臂关节 2
5	左小臂关节 5	RS00	left_joint_5	小臂关节 1
6	左小臂关节 6	RS00	left_joint_6	小臂关节 2
7	左小臂关节 7	RS00	left_joint_7	小臂关节 3
8	左夹爪	RS00	left_gripper_joint	夹爪开合

7.3.2.2 can1 (右臂 + 右夹爪)

CAN ID	关节名称	电机型号	URDF 关节名	位置描述
1	右肩关节 1	RS04	right_joint_1	肩部关节 1
2	右肩关节 2	RS04	right_joint_2	肩部关节 2
3	右大臂关节 3	RS03	right_joint_3	大臂关节 1
4	右大臂关节 4	RS03	right_joint_4	大臂关节 2
5	右小臂关节 5	RS00	right_joint_5	小臂关节 1
6	右小臂关节 6	RS00	right_joint_6	小臂关节 2
7	右小臂关节 7	RS00	right_joint_7	小臂关节 3
8	右夹爪	RS00	right_gripper_joint	夹爪开合

[警告] 重要提示: - 每条 CAN 总线上的电机 ID 必须**唯一** (1-8) - 左右臂可以使用**相同的 ID**(因为在不同总线上) - **不得**随意更改电机 ID, 否则会导致控制混乱

7.3.3 CAN 总线参数

参数	can0	can1
波特率	1 Mbps	1 Mbps
协议	CAN 2.0	CAN 2.0
终端电阻	120Ω	120Ω
设备数量	8	8
总线长度	<5m	<5m

7.4 电机控制参数

7.4.1 MIT 模式参数

Robstride 电机使用 **MIT 模式**进行运动控制:

控制命令格式:

```
位置(pos):   ±12.57 rad (所有型号)
速度(vel):   RS04: ±15 rad/s, RS03: ±20 rad/s, RS00: ±33 rad/s
扭矩(tau):   根据电机型号
KP(kp):      RS04/RS03: 0~5000, RS00: 0~500
KD(kd):      RS04/RS03: 0~100, RS00: 0~5
```

各型号参数范围:

型号	峰值扭矩	额定扭矩	最大速度	KP 范围	KD 范围
RS04	±120 Nm	40 Nm	±15 rad/s	0-5000	0-100
RS03	±60 Nm	20 Nm	±20 rad/s	0-5000	0-100
RS00	±14 Nm	5 Nm	±33 rad/s	0-500	0-5

7.4.2 推荐 KP/KD 参数

7.4.2.1 按应用场景推荐

场景	关节 1-4 (RS04/RS03)	关节 5-7 (RS00)	夹爪 (RS00)
示教模式			
KP	1.0	0.1	0.1
KD	0.1	0.05	0.05
常规运动 (默认)			
KP	50.0	10.0	5.0
KD	2.5	0.5	0.5
高刚性模式			
KP	500.0	50.0	50.0
KD	10.0	1.0	1.0

[警告] 注意: - 以上参数来自代码文件:`openarm_hardware/include/openarm_hardware/v10_simple_hardware.hpp` - KP 过大会导致**震荡和噪声**，甚至电机会有振动 - KD 过大会导致**阻尼过强，响应慢** - 不同负载需要**重新调整参数** - **示教模式**：极低刚性，方便手动拖动示教 - **常规运动**：系统默认参数，平衡性能和稳定性 - **高刚性模式**：高精度定位，但可能产生噪声

7.4.2.2 默认参数配置 **文件位置:** `src/openarm_ros2/openarm_hardware/include/openarm_hardware/v10_simpl`

```
// 常规运动模式（默认）
const std::vector<double> DEFAULT_KP = {50.0, 50.0, 50.0, 50.0, 10, 10, 10, 10};
const std::vector<double> DEFAULT_KD = {2.5, 2.5, 2.5, 2.5, 0.5, 0.5, 0.5, 0.5};

// 夹爪参数
const double GRIPPER_DEFAULT_KP = 5.0;
```

```
const double GRIPPER_DEFAULT_KD = 0.5;

// 其他模式（在代码中注释，需要时修改）
// 示教录制模式
// const std::vector<double> DEFAULT_KP = {1.0, 1.0, 1.0, 1.0, 0.1, 0.1, 0.1, 0.1};
// const std::vector<double> DEFAULT_KD = {0.1, 0.1, 0.1, 0.1, 0.05, 0.05, 0.05, 0.05};

// 高刚性模式
// const std::vector<double> DEFAULT_KP = {500.0, 500.0, 500.0, 500.0, 50, 50, 50, 50};
// const std::vector<double> DEFAULT_KD = {10.0, 10.0, 10.0, 10.0, 1.0, 1.0, 1.0, 1.0};
```

参数说明: - 前 4 个值对应关节 1-4 (RS04/RS03 电机，肩部和大臂) - 后 4 个值对应关节 5-8 (RS00 电机，小臂和夹爪)

7.5 关节限位

7.5.1 软件限位配置

文件位置: src/openarm_moveit_config/config/joint_limits.yaml

```
joint_limits:
  left_joint_1:  # 肩部关节 1 (RS04)
    has_position_limits: true
    min_position: -3.14   # -180°
    max_position: 3.14    # +180°
    max_velocity: 2.0     # rad/s
    max_acceleration: 1.0 # rad/s²

  left_joint_2:  # 肩部关节 2 (RS04)
    has_position_limits: true
    min_position: -1.57   # -90°
    max_position: 1.57    # +90°
    max_velocity: 2.0
    max_acceleration: 1.0
```

```yaml
left_joint_3:  # 大臂关节 1 (RS03)
  has_position_limits: true
  min_position: -3.14
  max_position: 3.14
  max_velocity: 2.0
  max_acceleration: 1.0

left_joint_4:  # 大臂关节 2 (RS03)
  has_position_limits: true
  min_position: -2.79   # -160°
  max_position: 0.0     # 0°
  max_velocity: 2.0
  max_acceleration: 1.0

left_joint_5:  # 小臂关节 1 (RS00)
  has_position_limits: true
  min_position: -1.57
  max_position: 1.57
  max_velocity: 2.0
  max_acceleration: 1.0

left_joint_6:  # 小臂关节 2 (RS00)
  has_position_limits: true
  min_position: -1.57
  max_position: 1.57
  max_velocity: 2.0
  max_acceleration: 1.0

left_joint_7:  # 小臂关节 3 (RS00)
  has_position_limits: true
  min_position: -3.14
  max_position: 3.14
  max_velocity: 2.0
  max_acceleration: 1.0
```

```yaml
left_gripper_joint:  # 夹爪 (RS00)
  has_position_limits: true
  min_position: 0.0     # 完全张开
  max_position: 1.2     # 完全闭合 (约 69°)
  max_velocity: 1.0
  max_acceleration: 0.5
```

7.5.1.1 左臂关节限位 右臂关节限位与左臂相同

7.5.2 机械限位

[警告] **重要:** OpenArmX 默认**没有硬件机械限位开关**!

安全建议: 1. 严格遵守软件限位 2. 不要运行超限位的轨迹 3. 在关节限位附近降低速度 4. 如有需要, 可自行添加机械限位保护

7.6 零位定义

7.6.1 零位姿态

零位定义: 机械臂处于标准初始姿态时的关节角度。

零位姿态描述:

```
        Z(向上)
         |
         |
    ┌────┴────┐
    │  基座   │
    └─────────┘
         |
    ┌────▼────┐
    │ 左臂/右臂 │
    │ (垂直下垂) │
    |    |    |
    |    |    |
    |    |    |
    |    ▼    |
```

```
│   夹爪    │
│ (张开)   │
└──────────────┘
```

各关节零位角度:

关节	零位角度	姿态描述
joint_1	0 rad	肩部水平
joint_2	0 rad	肩部中立
joint_3	0 rad	无旋转
joint_4	0 rad	肘部伸直
joint_5	0 rad	腕部中立
joint_6	0 rad	腕部中立
joint_7	0 rad	无旋转
gripper	0 rad	完全张开

7.6.2 设置零位

方法一: 使用 GUI 工具

```
# 启动电机管理界面
cd ~/openarmx_robotstride_ws-cc/src/motor_manage_openarmx
python3 main.py

# 在界面中:
#  点击" 启动 CAN"
#  点击" 启动电机"
#  手动调整机械臂到零位姿态
#  点击" 设置零位"
```

方法二: 使用命令行脚本

```
cd ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com

#  手动调整机械臂到零位姿态 (±30 度以内)

#  执行零位设置脚本
```

```
python3 set_motor_zeros_setMIT.py

# 脚本会:
# - 读取当前电机位置
# - 检查是否在 ±30 度范围内
# - 设置当前位置为零位
# - 保存到电机 Flash
```

[警告] 零位设置注意事项: 1. 设置零位前确保机械臂处于**安全姿态** 2. 零位偏差不得超过 ±**30 度** 3. 零位设置后需要**重启电机**生效 4. 定期检查零位准确性 (推荐每月一次)

7.6.3 验证零位

```
# 回零测试
python3 control_motor_gohome.py

# 观察:
# - 机械臂是否平稳回到零位姿态
# - 各关节角度是否接近 0
# - 是否有明显偏差
```

7.7 电机状态监控

7.7.1 状态检查工具

```
cd ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com
python3 check_motor_status.py
```

显示信息:

```
CAN0 电机状态:
────────────────────────────────────────────────────────────
ID   型号   位置(rad)   速度(rad/s)   力矩(Nm)   温度(°C)   错误码
1    RS04    0.123       0.0          0.5        45         0x00
2    RS04   -0.056       0.0          0.3        43         0x00
3    RS04    0.234       0.0          0.7        46         0x00
4    RS03    0.012       0.0          0.1        42         0x00
```

```
5   RS03  -0.089     0.0        0.2      41       0x00
6   RS03   0.045     0.0        0.1      40       0x00
7   RS03  -0.023     0.0        0.0      39       0x00
8   RS00   0.000     0.0        0.0      38       0x00

CAN1 电机状态:
──────────────────────────────────────────────────────────────
...
```

7.7.2 关键状态参数

参数	正常范围	警告阈值	危险阈值
温度	<60°C	60-80°C	>80°C
位置误差	<0.1 rad	0.1-0.5 rad	>0.5 rad
力矩	根据负载	>80% 额定	> 峰值
错误码	0x00	非零	通信超时

温度监控建议: - [√] 每次使用前检查温度 - [√] 连续运行时每 15 分钟检查一次 - [警告] 温度 >70°C 时降低负载或速度 - [红] 温度 >80°C 时**立即停止运动**

7.7.3 错误码说明

错误码	含义	处理方法
0x00	正常	-
0x01	过压	检查电源电压
0x02	欠压	检查电源供应
0x04	过流	降低负载/KP
0x08	过温	停止运动, 等待冷却
0x10	编码器故障	重启电机, 仍故障则更换
0x20	通信超时	检查 CAN 连接

7.8 电机参数配置文件

7.8.1 配置文件位置

`src/openarm_can_openarmx_com/`

```
└── config/
    ├── motor_types.yaml      # 电机型号参数
    └── motor_mapping.yaml    # 电机ID映射
```

7.8.2 motor_types.yaml

```yaml
# Robstride 电机型号参数定义
motor_types:
  RS04:
    power: 700              # W
    rated_torque: 40.0      # Nm
    peak_torque: 120.0      # Nm
    rated_current: 14.6     # A
    max_velocity: 15.0      # rad/s
    position_limit: 12.57   # rad
    kp_range: [0, 5000]
    kd_range: [0, 100]

  RS03:
    power: 380              # W
    rated_torque: 20.0      # Nm
    peak_torque: 60.0       # Nm
    rated_current: 7.9      # A
    max_velocity: 20.0      # rad/s
    position_limit: 12.57   # rad
    kp_range: [0, 5000]
    kd_range: [0, 100]

  RS00:
    power: 140              # W
    rated_torque: 5.0       # Nm
    peak_torque: 14.0       # Nm
    rated_current: 2.9      # A
    max_velocity: 33.0      # rad/s
    position_limit: 12.57   # rad
```

```
    kp_range: [0, 500]
    kd_range: [0, 5]
```

7.8.3 motor_mapping.yaml

```yaml
# CAN 总线和电机 ID 映射
can_buses:
  can0:
    name: "left_arm_and_gripper"
    bitrate: 1000000
    motors:
      1: {type: RS04, joint: left_joint_1, location: " 肩部关节 1"}
      2: {type: RS04, joint: left_joint_2, location: " 肩部关节 2"}
      3: {type: RS03, joint: left_joint_3, location: " 大臂关节 1"}
      4: {type: RS03, joint: left_joint_4, location: " 大臂关节 2"}
      5: {type: RS00, joint: left_joint_5, location: " 小臂关节 1"}
      6: {type: RS00, joint: left_joint_6, location: " 小臂关节 2"}
      7: {type: RS00, joint: left_joint_7, location: " 小臂关节 3"}
      8: {type: RS00, joint: left_gripper_joint, location: " 夹爪"}

  can1:
    name: "right_arm_and_gripper"
    bitrate: 1000000
    motors:
      1: {type: RS04, joint: right_joint_1, location: " 肩部关节 1"}
      2: {type: RS04, joint: right_joint_2, location: " 肩部关节 2"}
      3: {type: RS03, joint: right_joint_3, location: " 大臂关节 1"}
      4: {type: RS03, joint: right_joint_4, location: " 大臂关节 2"}
      5: {type: RS00, joint: right_joint_5, location: " 小臂关节 1"}
      6: {type: RS00, joint: right_joint_6, location: " 小臂关节 2"}
      7: {type: RS00, joint: right_joint_7, location: " 小臂关节 3"}
      8: {type: RS00, joint: right_gripper_joint, location: " 夹爪"}
```

7.9 常见问题

7.9.1 电机 ID 冲突

问题: 同一 CAN 总线上有两个相同 ID 的电机

现象: - 电机行为混乱 - 部分电机无响应 - CAN 总线错误增加

解决: 1. 使用电机配置工具查看 ID 2. 修改冲突电机的 ID 3. 更新 motor_mapping.yaml

7.9.2 电机无响应

可能原因: 1. CAN 总线未启动 2. 电机未使能 3. ID 配置错误 4. 线缆连接问题

排查步骤:

```
# 检查 CAN 接口
ip link show can0

# 检查 CAN 统计
ip -details -statistics link show can0

# 监听 CAN 总线
candump can0

# 检查电机状态
python3 check_motor_status.py
```

7.9.3 电机抖动

可能原因: - KP 参数过大 - KD 参数过小 - 机械松动 - 负载过大

解决方法: 1. 降低 KP 值 (减小 20-30%) 2. 增加 KD 值 (增加 20-30%) 3. 检查机械连接 4. 降低运动速度

7.9.4 电机过热

处理流程: 1. 立即停止运动 2. 让电机自然冷却 (不要用水) 3. 检查负载是否过大 4. 检查 KP/KD 是否过高 5. 降低运动速度和加速度 6. 冷却后重新测试

7.10 维护与保养

7.10.1 日常检查 (每次使用前)

- ☐ 检查电机外观无损坏
- ☐ 检查连接线缆牢固
- ☐ 检查电机温度正常(<40°C)
- ☐ 检查零位准确性

7.10.2 定期维护 (每月)

- ☐ 清洁电机表面灰尘
- ☐ 检查固定螺丝紧固
- ☐ 校验零位精度
- ☐ 检查温度历史记录
- ☐ 更新固件(如有)

7.10.3 专业维护 (每季度)

- ☐ 全面电气检查
- ☐ 编码器精度测试
- ☐ 机械磨损评估
- ☐ 性能基准测试
- ☐ 必要时更换部件

7.11 电机更换

7.11.1 更换流程

准备工作: 1. 断开所有电源 2. 准备新电机 (相同型号) 3. 准备工具 (内六角扳手等)

拆卸步骤: 1. 记录当前电机位置 2. 断开 CAN 线和电源线 3. 拆卸固定螺丝 4. 小心取出电机

安装步骤: 1. 安装新电机到相同位置 2. 紧固螺丝 (不要过紧) 3. 连接 CAN 线和电源线 4. 确认连接正确

配置新电机:

```
# 设置电机 ID(如果不正确)
# 使用电机配置工具或脚本
```

```
# 设置零位
python3 set_motor_zeros_setMIT.py

# 验证电机
python3 check_motor_status.py

# 低速测试
python3 test_motor_one.py
```

7.12 总结

7.12.1 快速参考

电机分布: - RS04 × 4: 肩部关节 (关节 1-2) - RS03 × 4: 大臂关节 (关节 3-4) - RS00 × 8: 小臂和夹爪 (关节 5-8)

CAN 总线: - can0: 左臂 + 左夹爪 (ID 1-8) - can1: 右臂 + 右夹爪 (ID 1-8)

推荐参数: - 常规运动: RS04/RS03 KP=500-1500, KD=10-30; RS00 KP=50-150, KD=1-3 - 示教模式: RS04/RS03 KP=50-200, KD=5-10; RS00 KP=5-20, KD=0.5-1 - 夹爪抓取: RS00 KP=20-80, KD=0.5-2.0

安全限值: - 温度上限: 80°C - 速度限制: 2.0 rad/s(MoveIt) - 加速度限制: 1.0 rad/s^2

7.12.2 最佳实践

1. [√] 定期检查电机状态和温度
2. [√] 从保守参数开始, 逐步优化
3. [√] 记录参数变更和效果
4. [√] 每月校验零位准确性
5. [√] 出现异常立即停止并排查

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

8 硬件要求

8.1 OpenArmX 系统硬件要求

本文档详细说明运行 OpenArmX 双臂机器人系统所需的硬件配置和推荐设备。

8.2 控制计算机要求

8.2.1 最低配置

组件	最低要求	说明
CPU	Intel Core i5 / AMD Ryzen 5	4 核心, 2.0 GHz 以上
内存	8 GB RAM	DDR4 或更高
存储	50 GB 可用空间	HDD 可用,SSD 推荐
操作系统	Ubuntu 22.04 LTS	64 位 (amd64)
USB 接口	USB 2.0 × 2	用于 CAN 转换器
网络	以太网接口	可选, 用于远程访问

[警告] 注意: 最低配置可以运行系统, 但可能存在以下限制: - MoveIt 规划速度较慢 - RViz 可视化可能卡顿 - 不适合复杂的多任务场景

8.2.2 推荐配置

组件	推荐配置	说明
CPU	Intel Core i7 / AMD Ryzen 7	8 核心, 3.0 GHz 以上
内存	16 GB RAM 或更多	用于 MoveIt 和 RViz
存储	100 GB SSD	提升系统响应速度
显卡	独立显卡 (可选)	改善 RViz 渲染性能
操作系统	Ubuntu 22.04 + RT 补丁	实时性能优化
USB 接口	USB 3.0 × 2+	更快的数据传输

优势: - [√] 流畅的可视化体验 - [√] 快速的轨迹规划 - [√] 支持多任务并行 - [√] 更好的实时性能

8.2.3 专业配置

用于复杂应用 (视觉伺服、机器学习等):

组件	专业配置
CPU	Intel Core i9 / AMD Ryzen 9 (12 核心 +)
内存	32 GB RAM 或更多
显卡	NVIDIA RTX 系列 (CUDA 支持)
存储	500 GB NVMe SSD
操作系统	Ubuntu 22.04 + RT 补丁 + CUDA

8.3 CAN 通信设备

8.3.1 CAN-USB 转换器

OpenArmX 需要**双通道 CAN-USB 转换器**来控制左右臂:

推荐型号: 1. **USBCAN-II 双通道** - 支持 CAN 2.0 - 波特率最高 1 Mbps - Linux SocketCAN 驱动支持

2. **Peak PCAN-USB FD**
   - 高性能
   - 良好的 Linux 支持
   - 支持 CAN FD(未来扩展)
3. **Kvaser Leaf Light HS v2**
   - 工业级可靠性
   - 优秀的驱动支持

必需规格:

☐ 通道数量：至少2个独立CAN通道

☐ 波特率：支持1 Mbps

☐ 接口：USB(推荐USB 3.0)

☐ 驱动：Linux SocketCAN驱动支持

☐ 隔离：推荐带电气隔离

[警告] 重要提示: - 不推荐使用单通道转换器 (需要两个设备) - 确认 Linux 驱动兼容性 - 购买时确认支持 SocketCAN

8.3.2 CAN 线缆

规格要求:

- ☐ 类型：双绞线屏蔽线缆
- ☐ 特性阻抗：120Ω
- ☐ 线径：≥ 0.25 mm^2
- ☐ 长度：尽量短(<5m)
- ☐ 连接器：DB9或端子(根据设备)

推荐配置: - CAN_H, CAN_L 双绞 - 铝箔 + 编织网双重屏蔽 - 两端配 120Ω 终端电阻

布线建议: - [√] 远离强电线缆 (>30cm) - [√] 避免与电源线平行走线 - [√] 最小化环路面积 - [√] 两端良好接地

8.3.3 CAN 接口接线

典型接线图:

```
CAN转换器(DB9)          OpenArmX电机

┌──────────────────┐            ┌──────────────────┐
│  DB9     │            │ CAN端子  │
│  ┌──────────┐  │            │          │
│  │ 7 2 │  │  CAN_H   │  CAN_H  │
│  │6 1  │  │──────────────────┤          │
│  │ 8 3 │  │  CAN_L   │  CAN_L  │
│  │   9 4│  │──────────────────┤          │
│  │    5 │  │  GND     │  GND    │
│  └──────────┘  │──────────────────┤          │
└──────────────────┘            └──────────────────┘


DB9针脚定义(常见):
Pin 2: CAN_L
Pin 3: GND
Pin 7: CAN_H
```

终端电阻连接:

```
总线起点                                  总线末端
  120Ω                                      120Ω
    │                                         │
CAN_H ──────────────────────────────────────────────────────── CAN_H
```

CAN_L ── CAN_L

8.4 电源系统

8.4.1 主电源要求

规格:

输入电压: AC 100-240V (通用)

输出电压: DC 48V

输出电流: ≥ 20A (推荐 25A+)

输出功率: ≥ 1000W

推荐型号: - 明纬 (MEAN WELL) RSP-1500-48 - 台达 (DELTA) PMT-48V20W2BA - TDK-Lambda HWS1000-48

安全特性:

☐ 过压保护 (OVP)

☐ 过流保护 (OCP)

☐ 过温保护 (OTP)

☐ 短路保护 (SCP)

☐ EMI滤波

8.4.2 电源容量计算

最大功耗估算:

单个RS04电机: ~100W (峰值)

单个RS03电机: ~60W (峰值)

单个RS00电机: ~30W (峰值)

总计:

RS04 × 4 (关节1-2) = 400W

RS03 × 4 (关节3-4) = 240W

RS00 × 8 (关节5-8) = 240W

──────────────────────

合计: 880W (峰值)

推荐电源: 880W × 1.5 = 1320W

[警告] 注意: - 上述为理论峰值, 实际很少同时达到 - 典型运行功耗约 400-600W - 仍建议 1000W+ 电源保证稳定性

8.4.3 配电建议

推荐配电方案:

```
AC 220V ── 电源开关 ── 48V电源 ──┬── 机械臂电机
                                 │
                                 ├── 急停继电器 ── 电机使能
                                 │
                                 └── 控制计算机(可选)
```

安全措施: 1. **急停按钮** - 红色蘑菇头按钮 - 机械自锁式 - 直接切断 48V 供电

2. **保险丝**
   - 主电源输入: 10A 保险丝 (AC 端)
   - 电机供电: 25A 断路器 (DC 端)
3. **接地**
   - 电源外壳必须接地
   - 机器人基座接地
   - 控制计算机接地

8.5 安全设备

8.5.1 急停系统

必需设备:

急停按钮:

类型: 红色蘑菇头按钮

触点: 常闭触点(NC)

规格: ≥ 10A @ 48V DC

安装: 手臂伸展可及范围内

动作: 机械自锁,旋转解锁

推荐型号: - 施耐德 (Schneider) XB4BS542 - OMRON A22E-M - ABB CE4T

接线方式:

```
48V+ ─── 急停按钮NC ─── 电机供电
```

|

按下时断开

8.5.2 防护设施

推荐安全设施:

1. **物理围栏** (推荐)
   - 高度: ≥ 1.2m
   - 材质: 铝型材 + 透明亚克力板
   - 开口: 带安全门锁
2. **警示标识**
   - 黄色警示标志
   - “机器人工作区域, 请勿进入”
   - LED 警示灯 (可选)
3. **地面标线**
   - 黄黑警示地胶
   - 标注安全距离 (≥1m)
4. **应急设备**
   - 灭火器 (CO2 或干粉)
   - 急救箱
   - 应急照明

8.5.3 个人防护装备 (PPE)

推荐穿戴:

☐ 防护眼镜(必需)

☐ 实验服/工作服(推荐)

☐ 防滑鞋(推荐)

☐ 绝缘手套(维护时必需)

禁止穿戴:

宽松衣物

长项链、手链

散披长发

8.6 工作空间要求

8.6.1 物理空间

最小空间:

长度: 2.5m

宽度: 2.0m

高度: 2.5m

推荐空间:

长度: 3.5m

宽度: 3.0m

高度: 2.8m

布局建议:

```
┌──────────────────────────────────────────────────────────────────────────┐
│                                                                          │
│    安全围栏(可选)                                                        │
│    ┌──────────────────────────────────────────────────────────────┐      │
│    │                                                              │      │
│    │     OpenArmX机器人                                           │      │
│    │         ┌─┴─┐                                                │      │
│    │        ┌┴───┴┐                                               │      │
│    │        │ 基座 │                                               │      │
│    │        └─────┘                                               │      │
│    │                                                              │      │
│    │   工作半径: 1.6m(双臂)                                       │      │
│    │                                                              │      │
│    └──────────────────────────────────────────────────────────────┘      │
│                                                                          │
│   控制计算机                                                             │
│    ┌──────────┐                                                          │
│    │ PC       │   距离: 1-3m                                             │
│    └──────────┘                                                          │
│                                                                          │
│   急停按钮                                                               │
```

```
│     [红]                                    │
│                                             │
└─────────────────────────────────────────────┘
```

8.6.2 环境要求

温度:

工作温度: 5°C ~ 40°C

最佳温度: 15°C ~ 30°C

存储温度: -10°C ~ 60°C

湿度:

工作湿度: 20% ~ 80% RH(无冷凝)

最佳湿度: 40% ~ 60% RH

其他要求:

- ☐ 室内使用
- ☐ 无腐蚀性气体
- ☐ 无导电粉尘
- ☐ 无强磁场干扰
- ☐ 无剧烈振动
- ☐ 照明充足(≥300 lux)
- ☐ 通风良好

8.6.3 工作台面

推荐规格:

尺寸: ≥ 1.5m × 1.0m

高度: 0.7m - 0.9m(可调节更佳)

承重: ≥ 100kg

稳定性: 坚固,无晃动

表面: 平整,防滑

安装固定: - 机器人基座用螺栓固定在台面上 - 台面固定在地面或墙壁上 - 防止运动时整体移动

8.7 可选设备

8.7.1 显示设备

用于 RViz 可视化:

显示器: 24"+ 1080p或更高

接口: HDMI/DP

数量: 推荐双显示器(一个RViz,一个终端)

8.7.2 输入设备

键盘鼠标: - 标准 USB 键盘 - USB 鼠标 - 或无线键鼠套装

3D 鼠标 (可选): - 3Dconnexion SpaceMouse - 用于 RViz 中的 3D 视角控制

8.7.3 网络设备

远程访问:

路由器/交换机: 用于远程SSH访问

WiFi模块: 用于无线连接(不推荐实时控制)

8.7.4 扩展传感器

视觉系统: - Intel RealSense D435/D455 - USB 相机 - 用于视觉伺服和物体识别

力/力矩传感器: - 6 轴力/力矩传感器 - 安装在末端法兰 - 用于力控应用

8.8 硬件检查清单

8.8.1 购买清单

必需设备:

- ☐ 控制计算机(符合最低配置)
- ☐ 双通道CAN-USB转换器
- ☐ CAN线缆和连接器
- ☐ 48V DC电源(≥1000W)
- ☐ 电源线缆
- ☐ 急停按钮
- ☐ USB线缆(连接CAN转换器)

推荐设备:

- ☐ 显示器
- ☐ 键盘鼠标
- ☐ 安全围栏/警示标识
- ☐ 灭火器
- ☐ 工作台

可选设备:

- ☐ 独立显卡
- ☐ 3D鼠标
- ☐ 摄像头
- ☐ 力/力矩传感器

8.8.2 收货检查

OpenArmX 机器人:

- ☐ 机械臂本体完好
- ☐ 电机无损坏
- ☐ 线缆齐全
- ☐ 固定螺丝紧固
- ☐ 配件齐全

电气设备:

- ☐ CAN转换器功能正常
- ☐ 电源输出电压正确(48V)
- ☐ 急停按钮动作正常
- ☐ 线缆无破损

文档资料:

- ☐ 使用说明书
- ☐ 电机规格书
- ☐ 接线图
- ☐ 软件安装指南

8.9 硬件兼容性

8.9.1 已测试配置

[√] 确认兼容:

组件类型	型号/规格	备注
计算机	Dell OptiPlex 7080	i7-10700, 16GB RAM
计算机	Lenovo ThinkCentre M920t	i7-9700, 16GB RAM
计算机	自组装 PC (Intel/AMD)	Ubuntu 22.04
CAN 转换器	USBCAN-II	双通道
CAN 转换器	Peak PCAN-USB	Linux 驱动良好
电源	MEAN WELL RSP-1500-48	1500W, 48V

8.9.2 不推荐设备

[×] 已知问题:

设备类型	问题描述
某些 ARM 单板计算机	性能不足,RViz 卡顿
虚拟机	USB 透传不稳定, 实时性差
Windows WSL	CAN 驱动不完善
单通道 CAN 转换器	需要两个设备, 不方便

8.10 硬件连接示意图

8.10.1 完整系统连接

```
                ┌──────────────────────────┐
                │ 控制计算机               │
                │ Ubuntu 22.04             │
                └──────────┬───────────────┘
                           │          │
USB                        │          │ USB
 ┌─────────────────────────┘          └──────────────────────┐
 │                                                           │
 ┌──────────────▼──────────────┐        ┌──────────────▼──────────────┐
```

```
│ CAN转换器(can0)│          │ CAN转换器(can1)│
└──────────┬──────────┘          └──────────┬──────────┘
           │CAN线                  CAN线│
           │                            │
┌──────────▼──────────┐          ┌──────────▼──────────┐
│  左臂电机 × 7  │          │  右臂电机 × 7  │
│  左夹爪 × 1    │          │  右夹爪 × 1    │
└──────────┬──────────┘          └──────────┬──────────┘
           │48V供电                48V供电│
           │                            │
           └──────────────────────────┬──────────────────────────┘
                                      │
                          ┌──────────▼──────────┐
                          │ 急停继电器    │
                          └──────────┬──────────┘
                                      │
                          ┌──────────▼──────────┐
                          │ 48V DC电源   │
                          └──────────┬──────────┘
                                      │
                          ┌──────────▼──────────┐
                          │ AC 220V市电  │
                          └──────────────────────┘
```

8.10.2 急停电路连接

```
AC 220V ── [电源开关] ── 48V电源 ──┬── 控制电路(常开)
                                  │
                                  └── 急停按钮(NC) ── 电机供电
                                        │
                                      按下时断开
```

8.11 总结

8.11.1 核心硬件要求

必需: 1. [√] 控制计算机 (Ubuntu 22.04, ≥8GB RAM) 2. [√] 双通道 CAN-USB 转换器 3. [√] 48V ≥1000W 电源 4. [√] 急停按钮

推荐: 1. 16GB RAM + SSD 2. 独立显卡 3. 安全围栏 4. 双显示器

8.11.2 采购建议

预算分配:

```
控制计算机:   5000 - 10000元
CAN转换器:    500 - 1500元
电源系统:     500 - 1000元
安全设备:     200 - 500元
工作台:       500 - 2000元
其他配件:     300 - 1000元
──────────────────────────────
总计:         7000 - 16000元
```

购买顺序: 1. 首先: 控制计算机, CAN 转换器 2. 其次: 电源, 急停按钮 3. 然后: 工作台, 安全设施 4. 最后: 可选传感器和配件

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

9 系统依赖安装

9.1 系统环境和依赖库安装指南

本文档详细说明如何安装 OpenArmX 所需的操作系统、ROS2、CAN 工具和其他依赖库。

9.2 操作系统安装

9.2.1 Ubuntu 22.04 LTS 安装

下载 Ubuntu:

```
官方网站: https://ubuntu.com/download/desktop
版本: Ubuntu 22.04.3 LTS (Jammy Jellyfish)
架构: 64-bit (amd64)
```

文件: ubuntu-22.04.3-desktop-amd64.iso

创建启动 U 盘:

Windows 下:

1. 下载 Rufus: https://rufus.ie/
2. 插入 ≥4GB U盘
3. 打开 Rufus
4. 选择 Ubuntu ISO 文件
5. 分区方案: GPT
6. 目标系统: UEFI
7. 点击"开始"

Linux 下:

```
# 假设 U 盘设备为 /dev/sdb (请确认!!)
sudo dd bs=4M if=ubuntu-22.04.3-desktop-amd64.iso of=/dev/sdb status=progress oflag=sync
```

安装步骤:

1. 插入启动U盘,重启电脑
2. 进入BIOS,设置U盘启动
3. 选择"Install Ubuntu"
4. 语言选择: 简体中文 或 English
5. 键盘布局: Chinese 或 English (US)
6. 更新和其他软件:
   [√] 正常安装
   [√] 安装Ubuntu时下载更新
   [√] 为图形和Wi-Fi硬件安装第三方软件
7. 安装类型: 清除整个磁盘并安装Ubuntu (或自定义分区)
8. 设置用户名和密码
9. 等待安装完成
10. 重启,拔出U盘

9.2.2 系统初始化

更新系统:

```
# 更新软件源
sudo apt update

# 升级所有软件包
sudo apt upgrade -y

# 安装基础工具
sudo apt install -y \
    curl \
    wget \
    git \
    vim \
    net-tools \
    build-essential \
    cmake \
    python3-pip
```

设置软件源 (可选, 加速下载):

使用阿里云镜像:

```
# 备份原始 sources.list
sudo cp /etc/apt/sources.list /etc/apt/sources.list.backup

# 替换为阿里云镜像
sudo sed -i 's/archive.ubuntu.com/mirrors.aliyun.com/g' /etc/apt/sources.list
sudo sed -i 's/security.ubuntu.com/mirrors.aliyun.com/g' /etc/apt/sources.list

# 更新软件源
sudo apt update
```

9.3 ROS2 Humble 安装

9.3.1 设置 ROS2 软件源

设置 locale:

```
sudo apt update && sudo apt install locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8
```

添加 ROS2 apt 仓库:

```
# 安装 curl
sudo apt install -y curl

# 添加 ROS2 GPG 密钥
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
    -o /usr/share/keyrings/ros-archive-keyring.gpg

# 添加软件源
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \
    http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME) main" | \
    sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null

# 更新软件源
sudo apt update
```

中国用户加速 (可选):

```
# 使用清华镜像
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \
    https://mirrors.tuna.tsinghua.edu.cn/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME) mai
    sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null

sudo apt update
```

9.3.2 安装 ROS2 Humble

安装完整桌面版:

```
# 安装 ROS2 Humble 桌面完整版 (包含 RViz, Demo 等)
sudo apt install -y ros-humble-desktop-full
```

```
# 安装开发工具
sudo apt install -y \
    ros-dev-tools \
    python3-colcon-common-extensions \
    python3-rosdep
```

初始化 rosdep:

```
# 初始化 rosdep
sudo rosdep init
rosdep update
```

[警告] rosdep init 失败的解决方法:

```
# 如果 rosdep init 报错"ERROR: cannot download default sources list"
# 使用代理或手动创建文件

# 方法 1: 手动创建
sudo mkdir -p /etc/ros/rosdep/sources.list.d/
sudo curl -o /etc/ros/rosdep/sources.list.d/20-default.list \
    https://mirrors.tuna.tsinghua.edu.cn/rosdistro/rosdep/sources.list.d/20-default.list

# 方法 2: 修改 rosdep 源 (国内用户)
export ROSDISTRO_INDEX_URL=https://mirrors.tuna.tsinghua.edu.cn/rosdistro/index-v4.yaml
rosdep update
```

9.3.3 配置 ROS2 环境

添加到 bashrc:

```
# 编辑 .bashrc
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc

# 立即生效
source ~/.bashrc
```

验证安装:

```
# 检查 ROS2 版本
ros2 --version

# 应该显示: ros2 cli version: 0.XX.X

# 测试 ROS2 节点
ros2 run demo_nodes_cpp talker
# 在另一个终端:
ros2 run demo_nodes_cpp listener

# 如果能看到消息传递，说明安装成功
```

9.4 MoveIt 2 安装

9.4.1 安装 MoveIt 2

```
# 安装 MoveIt 2 核心包
sudo apt install -y ros-humble-moveit

# 安装 MoveIt 相关工具
sudo apt install -y \
    ros-humble-moveit-visual-tools \
    ros-humble-moveit-servo \
    ros-humble-pilz-industrial-motion-planner
```

验证安装:

```
# 检查 MoveIt 包
ros2 pkg list | grep moveit

# 应该看到大量 moveit 相关包
```

9.5 ros2_control 安装

9.5.1 安装 ros2_control 框架

```
# 安装 ros2_control 核心
sudo apt install -y \
    ros-humble-ros2-control \
    ros-humble-ros2-controllers \
    ros-humble-controller-manager \
    ros-humble-hardware-interface

# 安装常用控制器
sudo apt install -y \
    ros-humble-joint-state-broadcaster \
    ros-humble-joint-trajectory-controller \
    ros-humble-forward-command-controller \
    ros-humble-position-controllers
```

9.6 CAN 总线工具安装

9.6.1 安装 SocketCAN 工具

```
# 安装 CAN 工具包
sudo apt install -y \
    can-utils \
    iproute2

# can-utils 包含:
# - candump: 监听 CAN 总线
# - cansend: 发送 CAN 帧
# - cangen: 生成 CAN 流量
# - canconfig: 配置 CAN 接口 (旧版)
```

验证安装:

```
# 查看 can-utils 版本
candump --version
```

```
# 列出 CAN 接口 (此时可能为空)
ip link show
```

9.6.2 加载 CAN 内核模块

```
# 加载 CAN 模块
sudo modprobe can
sudo modprobe can_raw
sudo modprobe can_dev
sudo modprobe vcan  # 虚拟 CAN, 测试用

# 查看已加载模块
lsmod | grep can

# 开机自动加载
echo "can" | sudo tee -a /etc/modules
echo "can_raw" | sudo tee -a /etc/modules
echo "can_dev" | sudo tee -a /etc/modules
```

9.6.3 安装 Python CAN 库

```
# 安装 python-can
pip3 install python-can

# 或使用系统包管理器
sudo apt install -y python3-can
```

9.7 Python 依赖安装

9.7.1 基础 Python 包

```
# 确保使用 Python 3.10+
python3 --version
```

```
# 安装 pip
sudo apt install -y python3-pip

# 升级 pip
pip3 install --upgrade pip
```

9.7.2 OpenArmX 所需 Python 包

```
# ROS2 Python 库 (通常已随 ROS2 安装)
pip3 install \
    rclpy \
    transforms3d \
    numpy \
    pyyaml

# CAN 通信
pip3 install python-can

# GUI 相关 (电机管理界面)
pip3 install \
    PySide6 \
    PySide6-Addons

# 数据处理
pip3 install \
    numpy \
    scipy \
    matplotlib
```

使用 requirements.txt(推荐):

```
# 创建 requirements.txt
cat > ~/openarm_requirements.txt << EOF
python-can>=4.0.0
PySide6>=6.5.0
```

```
PySide6-Addons>=6.5.0
numpy>=1.24.0
scipy>=1.10.0
matplotlib>=3.7.0
pyyaml>=6.0
EOF

# 安装
pip3 install -r ~/openarm_requirements.txt
```

9.8 编译工具链

9.8.1 C++ 编译环境

```
# 安装 GCC/G++ (Ubuntu 22.04 默认 GCC 11)
sudo apt install -y \
    build-essential \
    gcc \
    g++ \
    gdb

# 检查版本
gcc --version   # 应为 gcc (Ubuntu 11.x.x)
g++ --version   # 应为 g++ (Ubuntu 11.x.x)
```

9.8.2 CMake

```
# 安装 CMake (≥3.16)
sudo apt install -y cmake

# 检查版本
cmake --version  # 应为 cmake version 3.22+
```

9.8.3 colcon 构建工具

```
# colcon 已在安装 ros-dev-tools 时安装
# 如果没有，手动安装:
sudo apt install -y python3-colcon-common-extensions

# 验证
colcon version-check

# 安装 colcon 自动补全（可不安）
echo "source /usr/share/colcon_argcomplete/hook/colcon-argcomplete.bash" >> ~/.bashrc
source ~/.bashrc
```

9.9 其他依赖库

9.9.1 Eigen (线性代数库)

```
# 安装 Eigen3
sudo apt install -y libeigen3-dev

# 检查版本
pkg-config --modversion eigen3
```

9.9.2 Boost

```
# 安装 Boost 库
sudo apt install -y libboost-all-dev
```

9.9.3 URDF/Xacro 相关

```
# 安装 URDF 和 Xacro 工具
sudo apt install -y \
    ros-humble-urdf \
    ros-humble-xacro \
    ros-humble-joint-state-publisher \
    ros-humble-joint-state-publisher-gui \
```

```
    ros-humble-robot-state-publisher
```

9.10 可选软件

9.10.1 开发工具

VSCode (推荐):

```
# 方法 1: 使用 snap 安装
sudo snap install --classic code

# 方法 2: 官网下载.deb 安装
wget -O ~/Downloads/code.deb https://code.visualstudio.com/sha/download?build=stable&os=linux-deb-x64
sudo dpkg -i ~/Downloads/code.deb
sudo apt install -f  # 修复依赖

# 安装 ROS 扩展
code --install-extension ms-iot.vscode-ros
code --install-extension ms-python.python
code --install-extension ms-vscode.cpptools
```

Terminator (多终端管理):

```
sudo apt install -y terminator
```

9.10.2 可视化工具

PlotJuggler (数据可视化):

```
sudo apt install -y ros-humble-plotjuggler-ros
```

rqt 工具集:

```
sudo apt install -y \
    ros-humble-rqt \
    ros-humble-rqt-common-plugins \
    ros-humble-rqt-robot-plugins
```

9.11 Git 配置

9.11.1 配置 Git

```
# 设置用户信息
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"

# 设置默认编辑器
git config --global core.editor vim

# 查看配置
git config --list
```

9.11.2 配置 SSH 密钥 (可选)

```
# 生成 SSH 密钥
ssh-keygen -t ed25519 -C "your.email@example.com"

# 查看公钥
cat ~/.ssh/id_ed25519.pub

# 复制公钥，添加到 GitHub/Gitee
```

9.12 系统优化 (可选)

9.12.1 实时性能优化

调整 GRUB 参数:

```
# 编辑 GRUB 配置
sudo vim /etc/default/grub

# 修改这一行 (添加参数):
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash isolcpus=2,3 nohz_full=2,3"
# isolcpus=2,3: 隔离 CPU 核心 2,3 用于实时任务
# nohz_full=2,3: 禁用这些核心的定时器中断
```

```
# 更新 GRUB
sudo update-grub

# 重启生效
sudo reboot
```

设置 CPU 调度器:

```
# 安装 cpufrequtils
sudo apt install -y cpufrequtils

# 设置为性能模式
sudo cpufreq-set -g performance

# 查看当前模式
cpufreq-info
```

9.12.2 网络延迟优化

```
# 禁用 IPv6(可选, 减少延迟)
echo "net.ipv6.conf.all.disable_ipv6 = 1" | sudo tee -a /etc/sysctl.conf
echo "net.ipv6.conf.default.disable_ipv6 = 1" | sudo tee -a /etc/sysctl.conf
sudo sysctl -p
```

9.13 安装验证

9.13.1 验证清单

```
#!/bin/bash
# 保存为 check_dependencies.sh

echo "========== 系统环境检查 ==========="

#  操作系统
echo "OS: $(lsb_release -d | cut -f2)"
```

```
#  ROS2
echo "ROS2: $(ros2 --version 2>&1)"

#  Python
echo "Python: $(python3 --version)"

#  CAN 工具
echo "can-utils: $(candump --version 2>&1 | head -n1)"

#  colcon
echo "colcon: $(colcon version-check 2>&1 | head -n1)"

#  CMake
echo "CMake: $(cmake --version | head -n1)"

#  Git
echo "Git: $(git --version)"

echo "====================================="

# 检查关键 ROS2 包
echo " 检查 ROS2 包..."
packages=(
    "moveit"
    "ros2_control"
    "joint_trajectory_controller"
    "urdf"
    "xacro"
)

for pkg in "${packages[@]}"; do
    if ros2 pkg prefix $pkg &> /dev/null; then
        echo "✓ $pkg"
    else
```

```
        echo " $pkg - 未安装!"
    fi
done

# 检查 Python 模块
echo " 检查 Python 模块..."
python3 << EOF
import sys

modules = {
    'rclpy': 'ROS2 Python',
    'can': 'Python-CAN',
    'PySide6': 'PySide6',
    'numpy': 'NumPy',
    'yaml': 'PyYAML'
}

for module, name in modules.items():
    try:
        __import__(module)
        print(f"✓ {name}")
    except ImportError:
        print(f" {name} - 未安装!")
EOF

echo "===================================="
```

运行检查:

```
chmod +x check_dependencies.sh
./check_dependencies.sh
```

9.13.2 预期输出

```
========== 系统环境检查 ==========
OS: Ubuntu 22.04.3 LTS
```

```
ROS2: ros2 cli version: 0.XX.X
Python: Python 3.10.XX
can-utils: candump ...
colcon: colcon version-check ...
CMake: cmake version 3.22.1
Git: git version 2.34.1
====================================
检查ROS2包...
✓ moveit
✓ ros2_control
✓ joint_trajectory_controller
✓ urdf
✓ xacro
检查Python模块...
✓ ROS2 Python
✓ Python-CAN
✓ PySide6
✓ NumPy
✓ PyYAML
====================================
```

9.14 常见问题

9.14.1 rosdep init 失败

问题: `ERROR: cannot download default sources list`

解决方法:

```
# 使用国内镜像
sudo mkdir -p /etc/ros/rosdep/sources.list.d/
sudo curl -o /etc/ros/rosdep/sources.list.d/20-default.list \
    https://mirrors.tuna.tsinghua.edu.cn/rosdistro/rosdep/sources.list.d/20-default.list

# 设置环境变量
export ROSDISTRO_INDEX_URL=https://mirrors.tuna.tsinghua.edu.cn/rosdistro/index-v4.yaml
```

```
# 更新
rosdep update
```

9.14.2 pip 安装速度慢

解决方法:

```
# 使用清华镜像
pip3 install -i https://pypi.tuna.tsinghua.edu.cn/simple <package>

# 或设置为默认
pip3 config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
```

9.14.3 CAN 模块加载失败

问题: `modprobe: FATAL: Module can not found`

解决方法:

```
# 检查内核版本
uname -r

# 安装内核模块
sudo apt install -y linux-modules-extra-$(uname -r)

# 重启后再次尝试
sudo reboot
```

9.15 卸载与清理

9.15.1 卸载 ROS2

```
# 卸载 ROS2 Humble
sudo apt remove ros-humble-* -y
sudo apt autoremove -y

# 删除配置文件
rm -rf ~/.ros
```

9.15.2 清理系统

```
# 清理 apt 缓存
sudo apt clean
sudo apt autoclean
sudo apt autoremove -y

# 清理 pip 缓存
pip3 cache purge
```

9.16 总结

9.16.1 安装顺序总结

```
1. Ubuntu 22.04 LTS
   ↓
2. 系统更新和基础工具
   ↓
3. ROS2 Humble
   ↓
4. MoveIt 2
   ↓
5. ros2_control
   ↓
6. CAN工具(SocketCAN, python-can)
   ↓
7. Python依赖(PySide6等)
   ↓
8. 编译工具(colcon, cmake)
   ↓
9. 验证安装
```

9.16.2 安装时间估计

步骤	时间	说明
Ubuntu 安装	20-30 分钟	取决于硬件
系统更新	10-20 分钟	取决于网络
ROS2 安装	30-60 分钟	取决于网络
其他依赖	10-20 分钟	
总计	**70-130 分钟**	

9.16.3 下一步

完成本章节后, 请继续: - **03_ 工作空间构建.md** - 克隆代码并编译 OpenArmX 工作空间

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

10 工作空间构建

10.1 OpenArmX ROS2 工作空间构建指南

本文档详细说明如何克隆 OpenArmX 代码, 配置工作空间并编译所有软件包。

10.2 工作空间概念

10.2.1 ROS2 工作空间结构

ROS2 工作空间是一个包含源代码、编译输出和安装文件的目录结构:

```
~/openarmx_robotstride_ws-cc/     # 工作空间根目录
├── src/                          # 源代码目录
│   ├── openarm_ros2/              # 功能包1
│   ├── openarm_moveit_config/     # 功能包2
│   └── ...                        # 其他功能包
│
├── build/                        # 编译输出(colcon自动生成)
│   ├── openarm_ros2/
│   └── ...
│
├── install/                      # 安装目录(colcon自动生成)
│   ├── setup.bash                 # 环境设置脚本
```

```
│     ├──── openarm_ros2/
│     └──── ...
│
└──── log/                          # 编译日志(colcon自动生成)
    └──── build_YYYY-MM-DD_HH-MM-SS/
```

10.2.2 工作空间流程

```
下载源码 → 安装依赖 → 编译 → 加载环境 → 运行
  (git)    (rosdep)   (colcon)  (source)
```

10.3 创建工作空间

10.3.1 创建工作空间目录

```
# 进入家目录
cd ~

# 创建工作空间
mkdir -p ~/openarmx_robotstride_ws-cc/src

# 进入 src 目录
cd ~/openarmx_robotstride_ws-cc/src
```

[警告] 注意: - 工作空间名称可以自定义, 但本文档统一使用 `openarmx_robotstride_ws-cc` - `src` 目录必须存在,colcon 才能识别为工作空间

10.4 克隆源代码

10.4.1 获取 OpenArmX 代码

方法 1: 从 Git 仓库克隆 (推荐):

```
cd ~/openarmx_robotstride_ws-cc/src

# 克隆主仓库 (请替换为实际 URL)
git clone https://github.com/openarmx/openarmx_robotstride_ws.git .
```

```
# 或使用 Gitee 镜像 (国内用户)
git clone https://gitee.com/openarmx/openarmx_robotstride_ws.git .

# 查看已克隆的包
ls -l
```

方法 2: 从压缩包解压:

```
cd ~/openarmx_robotstride_ws-cc/src

# 假设已下载 openarmx_src.tar.gz
tar -xzf ~/Downloads/openarmx_src.tar.gz -C .

# 查看解压内容
ls -l
```

10.4.2 验证源代码结构

```
# 检查主要功能包
cd ~/openarmx_robotstride_ws-cc/src
ls -d */

# 应该看到:
# openarm_ros2/
# openarm_moveit_config/
# openarm_record/
# openarm_can_openarmx_com/
# motor_tests_openarmx_com/
# motor_manage_openarmx/
# 使用说明书/
```

10.4.3 检查 package.xml 文件

```
# 查找所有 ROS2 功能包
find ~/openarmx_robotstride_ws-cc/src -name "package.xml"
```

```
# 每个 ROS2 功能包应该包含 package.xml
```

10.5 安装依赖

10.5.1 使用 rosdep 安装系统依赖

```
# 回到工作空间根目录
cd ~/openarmx_robotstride_ws-cc

# 使用 rosdep 安装依赖
rosdep install --from-paths src --ignore-src -r -y
```

命令解释: - `--from-paths src`: 扫描 src 目录下所有 package.xml - `--ignore-src`: 忽略已在 src 中的包 - `-r`: 继续安装, 即使某些包失败 - `-y`: 自动确认安装

预期输出:

```
#All required rosdeps installed successfully
```

[警告] 如果遇到错误:

```
# 更新 rosdep 数据库
rosdep update

# 重试安装
rosdep install --from-paths src --ignore-src -r -y
```

10.5.2 安装 Python 依赖

某些 Python 依赖可能不在 rosdep 数据库中, 需手动安装:

```
# 进入工作空间
cd ~/openarmx_robotstride_ws-cc

# 安装 Python CAN 库
pip3 install python-can

# 安装 GUI 库 (电机管理界面)
pip3 install PySide6
```

```
# 其他可能需要的库
pip3 install numpy scipy matplotlib pyyaml
```

10.6 编译工作空间

10.6.1 首次编译

```
# 回到工作空间根目录
cd ~/openarmx_robotstride_ws-cc

# 编译所有包
colcon build

# 编译过程可能需要 5-15 分钟，取决于硬件性能
```

编译选项:

```
# 并行编译 (加速)
colcon build --parallel-workers 4

# 带详细输出
colcon build --event-handlers console_direct+

# 仅编译特定包
colcon build --packages-select openarm_ros2

# 编译某个包及其依赖
colcon build --packages-up-to openarm_moveit_config
```

10.6.2 预期编译输出

成功输出:

```
Starting >>> openarm_description
Starting >>> openarm_can_openarmx_com
...
Finished <<< openarm_description [12.3s]
```

```
Finished <<< openarm_can_openarmx_com [45.6s]
...
Summary: 8 packages finished [1min 23s]
```

文件变化:

工作空间目录下新增:

```
├── build/      # 编译中间文件
├── install/    # 安装文件
└── log/        # 编译日志
```

10.6.3 编译错误处理

常见编译错误:

错误 1: 缺少依赖

```
CMake Error: Could not find package "xxx"
```

解决:

```
# 安装缺失的 ROS2 包
sudo apt install ros-humble-xxx

# 或使用 rosdep
rosdep install --from-paths src --ignore-src -r -y
```

错误 2: Python 模块缺失

```
ModuleNotFoundError: No module named 'xxx'
```

解决:

```
pip3 install xxx
```

错误 3: 编译器版本问题

```
error: 'xxx' was not declared in this scope
```

解决:

```
# 确保使用 GCC 11+
gcc --version
```

```
# 如果版本过低，升级编译器
sudo apt install gcc-11 g++-11
```

10.7 配置环境变量

10.7.1 加载工作空间环境

临时加载 (当前终端):

```
cd ~/openarmx_robotstride_ws-cc
source install/setup.bash
```

永久加载 (推荐):

```
# 编辑.bashrc
vim ~/.bashrc

# 在文件末尾添加:
source /opt/ros/humble/setup.bash
source ~/openarmx_robotstride_ws-cc/install/setup.bash

# 保存并退出

# 重新加载.bashrc
source ~/.bashrc
```

[警告] 重要提示: - `source /opt/ros/humble/setup.bash` 必须在工作空间之前 - 每次编译后需要重新 source

10.7.2 环境变量说明

```
# 查看 ROS2 环境变量
printenv | grep ROS

# 关键变量:
# ROS_DISTRO=humble
# ROS_VERSION=2
# AMENT_PREFIX_PATH=.../install/...
```

```
# CMAKE_PREFIX_PATH=.../install/...
```

10.7.3 验证环境配置

```
# 查看 ROS2 包路径
ros2 pkg list | grep openarm

# 应该看到:
# openarm_bringup
# openarm_controllers
# openarm_description
# openarm_hardware
# openarm_moveit_config
# ...

# 检查包路径
ros2 pkg prefix openarm_description
# 输出: /home/xxx/openarmx_robotstride_ws-cc/install/openarm_description
```

10.8 测试编译结果

10.8.1 测试 URDF 模型

```
# 查看机器人模型
ros2 launch openarm_description view_robot.launch.py

# 应该打开 RViz, 显示 OpenArmX 机器人模型
```

预期结果: - RViz 窗口打开 - 显示双臂机器人 3D 模型 - 可以拖动滑块控制关节

10.8.2 测试节点启动

```
# 测试控制器配置加载
ros2 control list_controllers

# 测试 MoveIt 配置
```

```
ros2 launch openarm_moveit_config demo.launch.py
```

10.9 工作空间管理

10.9.1 清理编译文件

清理 build 和 install:

```
cd ~/openarmx_robotstride_ws-cc

# 删除编译输出
rm -rf build/ install/ log/

# 重新编译
colcon build
```

仅清理特定包:

```
# 删除特定包的编译文件
rm -rf build/openarm_ros2 install/openarm_ros2

# 重新编译该包
colcon build --packages-select openarm_ros2
```

10.9.2 增量编译

修改代码后,colcon 会自动增量编译:

```
# 修改了某个文件后
cd ~/openarmx_robotstride_ws-cc
colcon build

# colcon 会检测变化，只编译修改的包
```

强制重新编译:

```
# 强制全部重新编译
colcon build --cmake-clean-cache
```

```
# 强制特定包重新编译
colcon build --packages-select openarm_ros2 --cmake-clean-cache
```

10.9.3 并行编译优化

```
# 使用所有 CPU 核心
colcon build --parallel-workers $(nproc)

# 限制并发数 (避免内存不足)
colcon build --parallel-workers 2

# 设置为默认 (添加到.bashrc)
echo "export COLCON_PARALLEL_WORKERS=4" >> ~/.bashrc
```

10.10 更新代码

10.10.1 拉取最新代码

```
cd ~/openarmx_robotstride_ws-cc/src

# 拉取更新
git pull

# 如果有子模块
git submodule update --init --recursive
```

10.10.2 更新后重新编译

```
cd ~/openarmx_robotstride_ws-cc

# 更新依赖
rosdep install --from-paths src --ignore-src -r -y

# 重新编译
colcon build
```

```
# 重新加载环境
source install/setup.bash
```

10.11 多工作空间管理

10.11.1 Overlay 概念

ROS2 支持工作空间叠加 (Overlay):

```
系统ROS2 (/opt/ros/humble)
    ↓ underlay
工作空间1 (~/openarmx_robotstride_ws-cc)
    ↓ underlay
工作空间2 (~/my_custom_ws)  ← 最高优先级
```

10.11.2 创建 Overlay 工作空间

```
# 创建新工作空间
mkdir -p ~/my_custom_ws/src
cd ~/my_custom_ws/src

# 添加自定义包
git clone <your_custom_package>

# 回到新工作空间根目录
cd ~/my_custom_ws

# 先 source 基础工作空间
source ~/openarmx_robotstride_ws-cc/install/setup.bash

# 编译新工作空间
colcon build

# 加载新工作空间
source install/setup.bash
```

```
# 现在新工作空间的包会覆盖基础工作空间的同名包
```

10.12 常用 colcon 命令

10.12.1 编译命令

```
# 基本编译
colcon build

# 详细输出
colcon build --event-handlers console_direct+

# 仅编译一个包
colcon build --packages-select <package_name>

# 编译包及其依赖
colcon build --packages-up-to <package_name>

# 跳过某些包
colcon build --packages-skip <package_name>

# 从某个包开始编译
colcon build --packages-start <package_name>

# 并行编译
colcon build --parallel-workers 4

# CMake 编译类型 (Debug/Release)
colcon build --cmake-args -DCMAKE_BUILD_TYPE=Release
```

10.12.2 测试命令

```
# 运行测试
colcon test
```

```
# 查看测试结果
colcon test-result --all

# 详细测试输出
colcon test-result --verbose
```

10.12.3 信息查询

```
# 列出所有包
colcon list

# 列出包的依赖关系
colcon list --packages-select openarm_ros2 --base-paths src

# 列出包的拓扑顺序
colcon list --topological-order
```

10.13 工作空间备份与迁移

10.13.1 备份工作空间

方法 1: 仅备份源代码

```
cd ~
tar -czf openarmx_src_$(date +%Y%m%d).tar.gz \
    openarmx_robotstride_ws-cc/src
```

方法 2: 完整备份 (包含编译结果)

```
cd ~
tar -czf openarmx_full_$(date +%Y%m%d).tar.gz \
    openarmx_robotstride_ws-cc
```

10.13.2 迁移到其他计算机

在原计算机:

```
# 备份源代码
cd ~
tar -czf openarmx_src.tar.gz openarmx_robotstride_ws-cc/src

# 传输到新计算机
scp openarmx_src.tar.gz user@new-computer:~/
```

在新计算机:

```
# 解压
cd ~
tar -xzf openarmx_src.tar.gz

# 创建工作空间目录结构
mkdir -p ~/openarmx_robotstride_ws-cc

# 移动 src 到工作空间
mv openarmx_robotstride_ws-cc/src ~/openarmx_robotstride_ws-cc/

# 安装依赖
cd ~/openarmx_robotstride_ws-cc
rosdep install --from-paths src --ignore-src -r -y

# 编译
colcon build

# 配置环境
source install/setup.bash
```

10.14 故障排查

10.14.1 colcon build 失败

症状: 编译过程中断

排查步骤:

```
# 查看详细日志
cd ~/openarmx_robotstride_ws-cc
colcon build --event-handlers console_direct+

# 查看特定包的日志
cat log/latest_build/openarm_ros2/stdout.log
cat log/latest_build/openarm_ros2/stderr.log

# 检查依赖
rosdep check --from-paths src --ignore-src

# 清理后重试
rm -rf build/ install/ log/
colcon build
```

10.14.2 找不到包

症状: Package 'openarm_xxx' not found

解决:

```
# 确认已 source 环境
source ~/openarmx_robotstride_ws-cc/install/setup.bash

# 检查包是否存在
ros2 pkg list | grep openarm

# 重新编译
cd ~/openarmx_robotstride_ws-cc
colcon build --packages-select openarm_xxx
source install/setup.bash
```

10.14.3 环境变量冲突

症状: 同时安装了多个 ROS 版本

解决:

```
# 检查当前 ROS 环境
printenv | grep ROS

# 清除所有 ROS 环境变量
unset ROS_DISTRO
unset AMENT_PREFIX_PATH
unset CMAKE_PREFIX_PATH

# 重新 source 正确的环境
source /opt/ros/humble/setup.bash
source ~/openarmx_robotstride_ws-cc/install/setup.bash
```

10.15 开发工作流

10.15.1 典型开发流程

```
# 修改代码
vim ~/openarmx_robotstride_ws-cc/src/openarm_ros2/...

# 编译
cd ~/openarmx_robotstride_ws-cc
colcon build --packages-select openarm_ros2

# 重新加载环境
source install/setup.bash

# 测试
ros2 launch openarm_bringup ...

# 调试 (如有问题)
# 查看日志，修改代码，重复步骤 2-4
```

10.15.2 快速测试脚本

创建快速编译脚本:

```
# 创建 ~/rebuild.sh
cat > ~/rebuild.sh << 'EOF'
#!/bin/bash
cd ~/openarmx_robotstride_ws-cc
colcon build --symlink-install --cmake-args -DCMAKE_BUILD_TYPE=Release
source install/setup.bash
echo "Build completed and environment sourced!"
EOF

chmod +x ~/rebuild.sh

# 使用
~/rebuild.sh
```

10.16 性能优化

10.16.1 编译加速

使用 ccache:

```
# 安装 ccache
sudo apt install -y ccache

# 配置 colcon 使用 ccache
export CC="ccache gcc"
export CXX="ccache g++"

# 编译
colcon build
```

使用 RAM 磁盘 (适合大内存):

```
# 创建 RAM 磁盘
sudo mkdir -p /mnt/ramdisk
sudo mount -t tmpfs -o size=4G tmpfs /mnt/ramdisk

# 在 RAM 磁盘中编译
```

```
cd ~/openarmx_robotstride_ws-cc
colcon build --build-base /mnt/ramdisk/build
```

10.16.2 symlink-install

Python 包使用符号链接安装, 修改后无需重新编译:

```
colcon build --symlink-install

# 之后修改 Python 代码, 无需重新编译, 直接运行即可
```

[警告] 注意: C++ 代码仍需重新编译

10.17 总结

10.17.1 完整工作空间构建流程

```
# 创建工作空间
mkdir -p ~/openarmx_robotstride_ws-cc/src
cd ~/openarmx_robotstride_ws-cc/src

# 克隆代码
git clone <repository_url> .

# 安装依赖
cd ~/openarmx_robotstride_ws-cc
rosdep install --from-paths src --ignore-src -r -y
pip3 install python-can PySide6

# 编译
colcon build --symlink-install

# 配置环境
echo "source ~/openarmx_robotstride_ws-cc/install/setup.bash" >> ~/.bashrc
source ~/.bashrc

# 验证
```

```
ros2 pkg list | grep openarm
```

10.17.2 关键检查点

- ☐ 工作空间目录结构正确
- ☐ 所有功能包已克隆
- ☐ `rosdep`依赖已安装
- ☐ `Python`依赖已安装
- ☐ `colcon build`成功
- ☐ 环境变量已配置
- ☐ `ros2 pkg list`可见包
- ☐ 测试`launch`文件可启动

10.17.3 下一步

完成工作空间构建后, 请继续: - **03_ 硬件连接与配置** - 连接 CAN 总线和电机 - **04_ 电机测试与调试** - 测试电机通信和运动

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

11 CAN 总线连接

11.1 CAN 总线物理连接指南

本文档详细说明 OpenArmX 系统的 CAN 总线物理连接方法和验证步骤。

11.2 CAN 总线概述

11.2.1 为什么使用双 CAN 总线

OpenArmX 采用**双 CAN 总线架构**:

```
can0: 左臂(7个关节) + 左夹爪(1个) = 8个电机
can1: 右臂(7个关节) + 右夹爪(1个) = 8个电机
```

优势: - [√] **降低总线负载**: 每条总线 8 个设备, 通信更流畅 - [√] **故障隔离**: 一侧故障不影响另一侧 - [√] **提高可靠性**: 减少 CAN 帧冲突 - [√] **独立控制**: 左右臂可独立操作

11.2.2 CAN 总线基本原理

CAN 2.0 特性: - 差分信号传输 (CAN_H, CAN_L) - 最高波特率: 1 Mbps - 多主机通信 - 自动仲裁和错误检测

信号线:

```
CAN_H: 高电平线(显性3.5V, 隐性2.5V)
CAN_L: 低电平线(显性1.5V, 隐性2.5V)
GND:   信号地线
```

11.3 硬件准备

11.3.1 所需设备清单

必需设备:

- ☐ OpenArmX机器人本体(含16个电机)
- ☐ 双通道CAN-USB转换器 × 1
  (或单通道CAN-USB转换器 × 2)
- ☐ CAN线缆(双绞屏蔽线)
- ☐ 120Ω终端电阻 × 4(可能已集成在设备中)
- ☐ USB数据线 × 2(连接转换器到电脑)
- ☐ 48V电源

☐ 控制计算机(已安装Ubuntu 22.04 + ROS2)

可选设备:

☐ CAN总线分析仪(调试用)

☐ 万用表(检查线路)

☐ 示波器(检查信号质量)

11.3.2 CAN 转换器选择

推荐型号:

1. USBCAN-II 双通道

优点:

✓ 双通道集成在一个设备

✓ Linux SocketCAN支持良好

✓ 性价比高

接口:

- USB Type-B接口(连接电脑)
- DB9接口 × 2(can0, can1)

2. Peak PCAN-USB

优点:

✓ 工业级可靠性

✓ 优秀的Linux驱动支持

✓ 支持CAN FD(未来扩展)

注意:

- 单通道,需要购买2个
- 价格较高

3. Kvaser Leaf Light HS v2

优点:

✓ 高性能

✓ 稳定性好

✓ API丰富

注意:

- 单通道,需要2个
- 驱动需要单独安装

11.4 CAN 线缆连接

11.4.1 DB9 接口针脚定义

标准 CAN DB9 针脚 (CiA DS-102):

DB9母头(从前面看):

```
┌─────────────────┐
│ 5 4 3 2 1 │
│  9 8 7 6  │
└─────────────────┘
```

针脚定义:

```
Pin 1: (保留)
Pin 2: CAN_L      ← 主要使用
Pin 3: GND        ← 主要使用
Pin 4: (保留)
Pin 5: CAN_SHLD (屏蔽层,可选)
Pin 6: GND (可选)
Pin 7: CAN_H      ← 主要使用
Pin 8: (保留)
Pin 9: CAN_V+ (可选,外部供电)
```

[警告] 重要: 不同厂商可能有不同定义, **连接前务必查阅设备手册**!

11.4.2 电机端接线

Robstride 电机 CAN 接口:

OpenArmX 的 Robstride 电机通常使用以下接线方式:

电机端子(典型):

```
┌──────────────────┐
│ CAN_H    │ ← 黄线/橙线
```

```
│ CAN_L     │ ← 蓝线/白线
│ GND       │ ← 黑线
│ 48V+      │ ← 红线(电源)
└──────────────────┘
```

[警告] 注意: - 电机 CAN 接口可能是端子、航插或 DB9 - 具体以实际产品为准 - 电源线 (48V+, GND) 与 CAN 信号线分开

11.4.3 完整连接拓扑

左臂 (can0) 连接示意:

```
控制电脑                    CAN转换器                左臂电机链

┌──────────────┐              ┌──────────────────┐
│   USB    │              │   DB9      │   CAN_H ────┬── 电机1 (ID=1)
│   接口   ├──────USB线──────│   can0     │             │
└──────────────┘              │            │   CAN_L ────┼── 电机2 (ID=2)
                              └──────────────────┘             │
                                   │                   ├── 电机3 (ID=3)
                              120Ω终端                 │
                                                       ├── 电机4 (ID=4)
                                                       │
                                                       ├── 电机5 (ID=5)
                                                       │
                                                       ├── 电机6 (ID=6)
                                                       │
                                                       ├── 电机7 (ID=7)
                                                       │
                                                       └── 电机8 (ID=8)
                                                            │
                                                       120Ω终端
```

右臂 (can1) 连接方式相同

11.4.4 菊花链 vs 星型拓扑

推荐: 菊花链 (Daisy Chain)

```
转换器 ──┬── 电机1 ──┬── 电机2 ──┬── 电机3 ──┬── ... ── 电机8
```

```
        │          │          │          │
    120Ω       (无)       (无)       (无)       120Ω
```

优点: - [√] 布线简单 - [√] 电缆用量少 - [√] 符合 CAN 标准

[×] 不推荐: 星型拓扑

```
          ┌── 电机1
          ├── 电机2
转换器 ──┼── 电机3
          ├── 电机4
          └── ...
```

缺点: - 反射严重 - 可靠性差 - 不符合 CAN 规范

11.5 终端电阻配置

11.5.1 为什么需要终端电阻

CAN 总线是传输线, 需要在**两端**安装 120Ω 电阻消除信号反射。

```
起点              总线              终点
120Ω ──────────────────────────────────────── 120Ω
      CAN_H
      CAN_L
```

11.5.2 终端电阻位置

典型配置:

```
can0总线:
- 位置1: CAN转换器内部(通常已集成)
- 位置2: 最后一个电机(电机8)或外接电阻

can1总线:
- 位置1: CAN转换器内部
- 位置2: 最后一个电机(电机8)或外接电阻
```

11.5.3 检查终端电阻

方法 1: 万用表测量

```
# 断开所有设备电源
# 将万用表设置为电阻档
# 测量 CAN_H 和 CAN_L 之间电阻

预期结果:
- 两个 120Ω 电阻并联 ≈ 60Ω
- 如果测量值在 55-65Ω，说明正常
- 如果>100Ω，说明缺少终端电阻
- 如果<50Ω，说明电阻过多或短路
```

方法 2: 查看设备手册

检查 CAN 转换器和电机是否内置终端电阻, 以及如何启用/禁用。

11.5.4 外接终端电阻

如果设备没有内置终端电阻:

材料:

☐ 120Ω电阻(1/4W或更高功率)

☐ DB9连接器或端子

接法:

```
CAN_H ──┬── 120Ω ──┬── CAN_L
        │          │
     (Pin 7)    (Pin 2)
```

11.6 实际接线步骤

11.6.1 准备工作

1. 确认所有设备已断电
2. 准备好所有线缆和工具
3. 查阅设备接口定义
4. 规划线缆走向

11.6.2 连接 CAN 转换器到电脑

```
# 将 CAN-USB 转换器通过 USB 线连接到电脑

# 检查是否识别
lsusb

# 应该看到类似:
# Bus 001 Device 005: ID 0c72:000c ... PEAK PCAN-USB
# 或
# Bus 001 Device 006: ID ... ... USBCAN-II

# 检查内核日志
dmesg | tail -20

# 应该看到 CAN 设备加载信息
```

11.6.3 连接 CAN 线缆到电机

步骤:

```
1. 从CAN转换器引出CAN线缆
   - CAN_H连接到所有电机的CAN_H
   - CAN_L连接到所有电机的CAN_L
   - GND连接到GND(或共用电源地)

2. 按照菊花链方式连接各电机
   转换器 → 电机1 → 电机2 → ... → 电机8

3. 确保接线牢固
   - 螺丝端子拧紧
   - 航插完全插入并锁定
   - DB9连接器固定螺丝拧紧

4. 检查接线
   - CAN_H到CAN_H
```

- CAN_L到CAN_L
- 没有接反
- 没有短路

11.6.4 电源连接

[警告] 注意: 电源连接单独处理, 不与 CAN 信号线混淆

```
48V电源 ────┬──── 电机1 (48V+, GND)
            ├──── 电机2
            ├──── 电机3
            ├──── ...
            └──── 电机8
```

推荐使用:

- 足够粗的电源线(≥16AWG)
- 带保险丝的配电板
- 急停继电器

11.7 接线验证

11.7.1 视觉检查

☐ 所有连接器插紧

☐ 没有裸露的金属线

☐ 线缆走向整齐,无交叉缠绕

☐ 没有线缆拉扯过紧

☐ 屏蔽层正确接地(如使用屏蔽线)

☐ 终端电阻已正确安装

11.7.2 电阻测量

```
# 断电状态下测量 CAN_H 和 CAN_L 之间电阻
# 使用万用表

can0: CAN_H <-> CAN_L ≈ 60Ω
can1: CAN_H <-> CAN_L ≈ 60Ω
```

```
如果阻值不对:
- >100Ω: 检查终端电阻是否安装
- <50Ω: 检查是否短路或电阻过多
- ∞(开路): 检查线缆连接
```

11.7.3 上电测试

[警告] 首次上电前确认:

- ☐ CAN接线正确
- ☐ 电源接线正确(极性)
- ☐ 终端电阻已安装
- ☐ 急停按钮可用
- ☐ 人员已撤离

上电步骤:

1. 连接USB到电脑
2. 启动CAN接口(见下节)
3. 打开48V电源
4. 观察:
   - 电源指示灯亮起
   - 无异常声音
   - 无冒烟
   - CAN转换器指示灯正常

11.8 CAN 接口配置

11.8.1 加载 CAN 驱动

```
# 加载 SocketCAN 内核模块
sudo modprobe can
sudo modprobe can_raw

# 验证模块加载
lsmod | grep can
```

11.8.2 配置 can0 和 can1

使用脚本 (推荐):

```
cd ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com

# 启动 CAN 接口
python3 en_all_can.py
```

手动配置:

```
# 配置 can0
sudo ip link set can0 type can bitrate 1000000
sudo ip link set can0 up

# 配置 can1
sudo ip link set can1 type can bitrate 1000000
sudo ip link set can1 up

# 检查状态
ip link show can0
ip link show can1

# 应该看到:
# can0: <NOARP,UP,LOWER_UP,ECHO> ...
# can1: <NOARP,UP,LOWER_UP,ECHO> ...
```

[警告] PCAN 设备特殊处理:

如果遇到左右臂反向, 可以重启 PCAN 模块:

```
# 卸载 PCAN 模块
sudo rmmod peak_usb

# 重新加载
sudo modprobe peak_usb

# 或者拔插 USB 接口
```

```
# 重新配置 CAN
python3 en_all_can.py
```

11.8.3 验证 CAN 接口

```
# 检查 CAN 接口状态
ip -details link show can0
ip -details link show can1

# 关键信息:
# - state UP (接口已启动)
# - bitrate 1000000 (波特率正确)
# - RX/TX 错误计数器为 0 或很小
```

11.9 通信测试

11.9.1 监听 CAN 总线

使用 candump:

```
# 监听 can0
candump can0

# 监听 can1
candump can1

# 同时监听
candump any

# 带时间戳
candump -t a can0
```

预期现象: - 如果电机已上电, 应该看到 CAN 帧 - 帧 ID 范围: 0x00 ~ 0xFF - 持续有数据流动

11.9.2 发送测试帧

[警告] 注意: 仅用于测试, 不要随意发送命令给电机

```
# 发送测试帧到 can0（不会影响电机）
cansend can0 123#DEADBEEF

# 在另一终端监听
candump can0

# 应该能看到刚发送的帧
```

11.9.3 检查电机响应

使用电机状态检查脚本:

```
cd ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com

# 检查所有电机状态
python3 check_motor_status.py
```

预期输出:

```
CAN0 电机状态:
ID 1: RS04 - 位置: 0.123 rad, 温度: 45°C
ID 2: RS04 - 位置: -0.056 rad, 温度: 43°C
...
ID 8: RS00 - 位置: 0.000 rad, 温度: 38°C

CAN1 电机状态:
ID 1: RS04 - 位置: 0.089 rad, 温度: 44°C
...
```

如果某些电机无响应: - 检查该电机的 CAN 连接 - 检查电机 ID 配置 - 检查电机供电

11.10 常见问题排查

11.10.1 CAN 接口无法启动

症状:

```
$ sudo ip link set can0 up
Error: Cannot find device "can0"
```

解决:

```
# 检查 USB 设备
lsusb

# 检查内核日志
dmesg | grep -i can

# 重新插拔 USB
# 检查驱动是否加载
lsmod | grep can
```

11.10.2 CAN 总线错误率高

症状:

```
$ ip -s link show can0
RX: errors 150  # 接收错误多
TX: errors 80   # 发送错误多
```

可能原因和解决:

1. 终端电阻问题

```
# 检查阻值应为 60Ω
# 添加或移除电阻
```

2. 波特率不匹配

```
# 确认所有设备使用 1Mbps
sudo ip link set can0 type can bitrate 1000000
```

3. 线缆质量差

- 更换屏蔽双绞线
- 减短线缆长度(<5m)
- 远离强电磁干扰源

4. 信号质量差

```
# 降低波特率测试
sudo ip link set can0 down
sudo ip link set can0 type can bitrate 500000
sudo ip link set can0 up
```

11.10.3 部分电机无响应

排查步骤:

```
# 使用 candump 监听总线
candump can0

# 检查哪些 ID 有响应
# 无响应电机:
#   - 检查该电机的 CAN 接线
#   - 检查电机供电
#   - 检查电机 ID 设置

# 尝试单独连接该电机
#   - 移除其他电机，只接一个
#   - 如果能通信，说明电机正常
#   - 逐个添加其他电机，找出问题所在
```

11.10.4 CAN 总线风暴 (Bus-Off)

症状:

```
$ ip link show can0
can0: <NOARP,UP,LOWER_UP,ECHO> ... state ERROR-PASSIVE
# 或
can0: ... state BUS-OFF
```

原因: - 错误计数器超过阈值 - 可能存在短路或通信冲突

解决:

```
# 重启 CAN 接口
sudo ip link set can0 down
```

```
sudo ip link set can0 up

#  检查总线是否有持续错误帧
candump can0

#  检查物理连接
#    - 有无短路
#    - 终端电阻是否正确

#  如果频繁 Bus-Off
#    - 降低波特率
#    - 检查所有设备是否正常
```

11.11 安全注意事项

11.11.1 接线安全

[警告] 断电操作
- 接线时务必断开48V电源
- 避免带电插拔CAN连接器

[警告] 防静电
- 触摸电机前先放电(触摸接地金属)
- 避免静电损坏电机控制器

[警告] 绝缘检查
- 确保裸露导体无短路风险
- 使用绝缘胶带包裹临时接头

11.11.2 调试安全

[警告] 不要发送未知命令
- 不了解CAN协议时不要乱发命令
- 可能导致电机失控

[警告] 急停准备

- 调试时手放在急停按钮上
- 发现异常立即按下急停

[警告] 逐步测试
- 先测试通信,再测试运动
- 从小幅度、低速度开始

11.12 最佳实践

11.12.1 线缆管理

✓ 使用线卡固定线缆
✓ 避免线缆悬空拉扯
✓ 电源线和信号线分开走线
✓ 预留足够活动余量
✓ 使用标签标识can0/can1

11.12.2 维护建议

☐ 每月检查接线紧固度
☐ 每季度测量终端电阻
☐ 定期检查线缆磨损
☐ 记录CAN错误统计趋势
☐ 备用线缆和连接器

11.12.3 故障预防

✓ 使用高质量屏蔽线缆
✓ 正确安装终端电阻
✓ 避免线缆过长(推荐<3m)
✓ 远离强电和高频设备
✓ 良好接地

11.13 总结

11.13.1 连接检查清单

硬件连接:

- ☐ CAN-USB转换器已连接到电脑
- ☐ can0连接到左臂8个电机
- ☐ can1连接到右臂8个电机
- ☐ 终端电阻已正确安装(60Ω)
- ☐ 48V电源已连接
- ☐ 急停按钮已接入

软件配置:

- ☐ CAN驱动模块已加载
- ☐ can0和can1已配置为1Mbps
- ☐ 接口状态为UP
- ☐ candump能看到CAN帧
- ☐ check_motor_status.py能读取电机

验证测试:

- ☐ 所有16个电机都能响应
- ☐ CAN错误计数低(<10)
- ☐ 电机状态正常(温度、位置)

11.13.2 下一步

完成 CAN 总线连接后, 请继续: - **02_CAN 接口配置.md** - 深入了解 CAN 接口参数 - **03_ 电机 ID 配置.md** - 设置和验证电机 ID - **04_ 硬件故障排查.md** - 硬件问题诊断

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

12 CAN 接口配置

12.1 SocketCAN 接口配置详解

本文档详细说明如何在 Linux 下配置和管理 SocketCAN 接口。

12.2 SocketCAN 简介

12.2.1 什么是 SocketCAN

SocketCAN 是 Linux 内核中的 CAN 总线协议族, 将 CAN 设备抽象为网络设备。

优势: - [√] 统一的 Socket API - [√] 与网络工具兼容 (ip 命令) - [√] 内核空间处理, 高性能 - [√] 支持多种 CAN 硬件

12.2.2 工作原理

```
应用程序
    ↕ Socket API
SocketCAN内核模块
    ↕ 驱动接口
硬件驱动(PCAN, USBCAN等)
    ↕ USB/SPI/...
CAN硬件设备
```

12.3 CAN 模块加载

12.3.1 检查内核支持

```
# 检查内核是否支持 CAN
zcat /proc/config.gz | grep CAN

# 应该看到:
# CONFIG_CAN=m
# CONFIG_CAN_RAW=m
# CONFIG_CAN_BCM=m
# ...
```

12.3.2 加载 CAN 模块

```
# 加载 CAN 核心模块
sudo modprobe can

# 加载 CAN 原始协议
sudo modprobe can_raw

# 加载 CAN 设备驱动
sudo modprobe can_dev
```

```
# 加载虚拟 CAN(测试用)
sudo modprobe vcan

# 验证模块已加载
lsmod | grep can
```

输出示例:

```
can_raw                20480  0
can                    24576  1 can_raw
can_dev                28672  0
vcan                   16384  0
```

12.3.3 开机自动加载

```
# 编辑 modules 文件
sudo vim /etc/modules

# 添加以下行:
can
can_raw
can_dev

# 保存退出
# 重启后自动加载
```

12.4 CAN 接口配置命令

12.4.1 基本命令格式

设置 CAN 参数:

```
sudo ip link set <device> type can <options>
```

启动接口:

```
sudo ip link set <device> up
```

关闭接口:

```
sudo ip link set <device> down
```

12.4.2 常用参数

参数	说明	示例
bitrate	波特率 (bps)	bitrate 1000000
sample-point	采样点位置 (0.0-1.0)	sample-point 0.875
sjw	同步跳转宽度	sjw 1
restart-ms	自动重启延迟 (ms)	restart-ms 100
fd on	启用 CAN FD	fd on dbitrate 5000000

12.5 配置 can0 和 can1

12.5.1 基本配置

配置 can0:

```
# 设置参数
sudo ip link set can0 type can bitrate 1000000

# 启动接口
sudo ip link set can0 up

# 检查状态
ip link show can0
```

配置 can1:

```
sudo ip link set can1 type can bitrate 1000000
sudo ip link set can1 up
ip link show can1
```

12.5.2 使用 Python 脚本配置 (推荐)

OpenArmX 提供的配置脚本:

```
cd ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com

# 启动所有 CAN 接口
python3 en_all_can.py

# 查看脚本内容
cat en_all_can.py
```

脚本内容示例:

```
#!/usr/bin/env python3
import os

# 配置 can0
os.system('sudo ip link set can0 type can bitrate 1000000')
os.system('sudo ip link set can0 up')

# 配置 can1
os.system('sudo ip link set can1 type can bitrate 1000000')
os.system('sudo ip link set can1 up')

print("CAN 接口已启动")
print("can0: 1 Mbps")
print("can1: 1 Mbps")
```

12.5.3 高级配置选项

设置采样点:

```
sudo ip link set can0 type can bitrate 1000000 sample-point 0.875
```

启用自动重启:

```
# Bus-Off 后自动重启 (延迟 100ms)
sudo ip link set can0 type can bitrate 1000000 restart-ms 100
```

设置三次采样:

```
sudo ip link set can0 type can bitrate 1000000 triple-sampling on
```

12.6 查看 CAN 接口状态

12.6.1 基本状态查看

```
# 查看接口列表
ip link show | grep can

# 查看特定接口
ip link show can0

# 详细状态
ip -details link show can0
```

输出示例:

```
3: can0: <NOARP,UP,LOWER_UP,ECHO> mtu 16 qdisc pfifo_fast state UP mode DEFAULT group default qlen 10
    link/can
    can state ERROR-ACTIVE (berr-counter tx 0 rx 0) restart-ms 0
      bitrate 1000000 sample-point 0.875
      tq 62 prop-seg 6 phase-seg1 7 phase-seg2 2 sjw 1
      pcan_usb: tseg1 1..16 tseg2 1..8 sjw 1..4 brp 1..64 brp-inc 1
      clock 16000000
```

关键字段解释: - `<NOARP,UP,LOWER_UP,ECHO>`: 接口标志 - `UP`: 接口已启动 - `LOWER_UP`: 物理层连接正常 - `state ERROR-ACTIVE`: CAN 状态 (正常) - `berr-counter tx 0 rx 0`: 错误计数器 (应为 0 或很小) - `bitrate 1000000`: 波特率 1Mbps - `sample-point 0.875`: 采样点位置

12.6.2 统计信息查看

```
# 查看收发统计
ip -s link show can0

# 详细统计
ip -s -s link show can0
```

输出示例:

```
3: can0: <NOARP,UP,LOWER_UP,ECHO> mtu 16 qdisc pfifo_fast state UP mode DEFAULT group default qlen 10
    link/can
    RX: bytes  packets  errors  dropped overrun mcast
    1248       156      0       0       0       0
    TX: bytes  packets  errors  dropped carrier collsns
    1040       130      0       0       0       0
```

关键指标: - `RX packets`: 接收的 CAN 帧数 - `TX packets`: 发送的 CAN 帧数 - `errors`: 错误帧数 (应该很小) - `dropped`: 丢弃帧数

12.7 CAN 接口管理

12.7.1 重启 CAN 接口

```
# 方法 1: 先 down 再 up
sudo ip link set can0 down
sudo ip link set can0 up

# 方法 2: 修改参数会自动重启
sudo ip link set can0 type can bitrate 500000
sudo ip link set can0 up
```

12.7.2 手动重启 (Bus-Off 恢复)

```
# 当接口处于 Bus-Off 状态时
sudo ip link set can0 type can restart
```

12.7.3 关闭 CAN 接口

```
# 使用 Python 脚本 (推荐)
python3 ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com/dis_all_can.py

# 或手动关闭
sudo ip link set can0 down
sudo ip link set can1 down
```

12.8 波特率配置

12.8.1 标准波特率

OpenArmX 使用 **1 Mbps** 波特率:

```
sudo ip link set can0 type can bitrate 1000000
```

CAN 2.0 常用波特率: | 波特率 | 最大总线长度 | 适用场景 | |——–|———–|——| | 1 Mbps | <40m | 短距离高速 (OpenArmX 使用) | | 500 kbps | <100m | 中速应用 | | 250 kbps | <200m | 汽车 CAN | | 125 kbps | <500m | 长距离低速 |

12.8.2 降低波特率 (故障排查)

如果通信不稳定, 可尝试降低波特率:

```
# 关闭接口
sudo ip link set can0 down

# 设置为 500kbps
sudo ip link set can0 type can bitrate 500000

# 重新启动
sudo ip link set can0 up
```

[警告] 注意: 降低波特率需要电机也支持,Robstride 电机默认 1Mbps。

12.8.3 自定义位时序

高级用户:

```
# 手动设置位时序参数
sudo ip link set can0 type can \
    tq 62 \
    prop-seg 6 \
    phase-seg1 7 \
    phase-seg2 2 \
    sjw 1
```

一般不需要手动设置, 使用 bitrate 参数即可

12.9 错误处理

12.9.1 CAN 状态机

CAN 控制器有三种状态:

```
ERROR-ACTIVE (正常)
    ↓ 错误增加
ERROR-PASSIVE (被动错误)
    ↓ 错误继续增加
BUS-OFF (总线关闭)
```

查看当前状态:

```
ip -details link show can0 | grep state

# 正常: state ERROR-ACTIVE
# 警告: state ERROR-PASSIVE
# 严重: state BUS-OFF
```

12.9.2 错误计数器

```
# 查看错误计数器
ip -details link show can0 | grep berr

# 输出示例:
# berr-counter tx 0 rx 0

# tx: 发送错误计数
# rx: 接收错误计数
```

正常范围: - 0-95: ERROR-ACTIVE - 96-127: ERROR-PASSIVE - >127: BUS-OFF

12.9.3 Bus-Off 恢复

自动恢复 (推荐):

```
# 设置自动重启
sudo ip link set can0 type can bitrate 1000000 restart-ms 100
sudo ip link set can0 up
```

```
# 发生 Bus-Off 后,100ms 自动重启
```

手动恢复:

```
# 检测到 Bus-Off
ip link show can0
# can0: ... state BUS-OFF

# 手动重启
sudo ip link set can0 type can restart
```

12.10 虚拟 CAN(vcan)

12.10.1 创建虚拟 CAN 接口

用于测试和开发:

```
# 加载 vcan 模块
sudo modprobe vcan

# 创建 vcan0
sudo ip link add dev vcan0 type vcan
sudo ip link set vcan0 up

# 查看
ip link show vcan0
```

12.10.2 使用 vcan 测试

```
# 终端 1: 监听
candump vcan0

# 终端 2: 发送
cansend vcan0 123#DEADBEEF

# 终端 1 会显示接收到的帧
```

12.10.3 删除 vcan

```
sudo ip link delete vcan0
```

12.11 网络命名空间 (高级)

12.11.1 隔离 CAN 接口

```
# 创建网络命名空间
sudo ip netns add can_test

# 将 can0 移到命名空间
sudo ip link set can0 netns can_test

# 在命名空间中配置
sudo ip netns exec can_test ip link set can0 type can bitrate 1000000
sudo ip netns exec can_test ip link set can0 up

# 在命名空间中运行程序
sudo ip netns exec can_test candump can0
```

一般用户无需使用, 仅高级测试场景

12.12 性能优化

12.12.1 接收缓冲区大小

```
# 增加接收队列长度
sudo ip link set can0 txqueuelen 1000
```

12.12.2 CPU 亲和性

```
# 将 CAN 中断绑定到特定 CPU 核心
# 查看中断号
cat /proc/interrupts | grep can

# 设置 CPU 亲和性 (假设中断号为 45, 绑定到 CPU2)
```

```
echo 4 | sudo tee /proc/irq/45/smp_affinity
```

12.12.3 实时优先级

```
# 提升 CAN 处理线程优先级 (需要 RT 内核)
sudo chrt -f 80 -p $(pgrep irq/.*can)
```

12.13 故障排查

12.13.1 设备不存在

症状:

```
$ sudo ip link set can0 up
Cannot find device "can0"
```

解决:

```
#  检查 USB 设备
lsusb

#  检查 dmesg
dmesg | tail -50 | grep -i can

#  重新插拔 USB

#  检查驱动模块
lsmod | grep peak  # PCAN 设备
lsmod | grep gs_usb  # 其他设备
```

12.13.2 权限不足

症状:

```
$ ip link set can0 up
RTNETLINK answers: Operation not permitted
```

解决:

```
# 方法 1: 使用 sudo
sudo ip link set can0 up

# 方法 2: 添加用户到 netdev 组
sudo usermod -a -G netdev $USER
# 重新登录生效
```

12.13.3 波特率设置失败

症状:

```
$ sudo ip link set can0 type can bitrate 1000000
RTNETLINK answers: Invalid argument
```

解决:

```
#  检查接口是否已关闭
sudo ip link set can0 down

#  重新设置
sudo ip link set can0 type can bitrate 1000000
sudo ip link set can0 up

#  检查硬件是否支持该波特率
ip -details link show can0
```

12.14 监控和调试

12.14.1 实时监控脚本

```
#!/bin/bash
# 保存为 monitor_can.sh

while true; do
    clear
    echo "========== CAN 接口状态 ==========="
    date
```

```
    echo ""

    echo "can0:"
    ip -s link show can0 | grep -A 4 "can0:"
    echo ""

    echo "can1:"
    ip -s link show can1 | grep -A 4 "can1:"
    echo ""

    sleep 1
done
```

使用:

```
chmod +x monitor_can.sh
./monitor_can.sh
```

12.14.2 错误日志记录

```
# 记录 CAN 统计到文件
while true; do
    echo "$(date): $(ip -s link show can0)" >> can0_stats.log
    sleep 60
done &
```

12.15 自动化配置

12.15.1 创建 systemd 服务

```
# 创建服务文件
sudo vim /etc/systemd/system/openarmx-can.service
```

内容:

```
[Unit]
Description=OpenArmX CAN Interface Setup
```

```
After=network.target

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/bin/bash -c 'ip link set can0 type can bitrate 1000000 && ip link set can0 up'
ExecStart=/bin/bash -c 'ip link set can1 type can bitrate 1000000 && ip link set can1 up'
ExecStop=/bin/bash -c 'ip link set can0 down'
ExecStop=/bin/bash -c 'ip link set can1 down'

[Install]
WantedBy=multi-user.target
```

启用服务:

```
# 重新加载 systemd
sudo systemctl daemon-reload

# 启用服务 (开机自启)
sudo systemctl enable openarmx-can.service

# 启动服务
sudo systemctl start openarmx-can.service

# 查看状态
sudo systemctl status openarmx-can.service
```

12.15.2 udev 规则 (自动配置)

```
# 创建 udev 规则
sudo vim /etc/udev/rules.d/99-openarmx-can.rules
```

内容:

```
# PCAN-USB设备插入时自动配置
ACTION=="add", SUBSYSTEM=="net", KERNEL=="can*", RUN+="/bin/bash -c 'sleep 1 && ip link set $name type can
```

重新加载 udev:

```
sudo udevadm control --reload-rules
sudo udevadm trigger
```

12.16 总结

12.16.1 常用命令速查

```
# 配置 CAN 接口
sudo ip link set can0 type can bitrate 1000000
sudo ip link set can0 up

# 查看状态
ip link show can0
ip -s link show can0

# 重启接口
sudo ip link set can0 down
sudo ip link set can0 up

# 关闭接口
sudo ip link set can0 down

# 监听总线
candump can0

# 查看错误
ip -details link show can0 | grep -E 'state|berr'
```

12.16.2 推荐配置

OpenArmX 标准配置:

```
# can0 和 can1 都使用:
bitrate: 1000000 (1 Mbps)
sample-point: 0.875 (自动)
```

```
restart-ms: 100 (自动恢复)
```

12.16.3 下一步

完成 CAN 接口配置后, 请继续: - **03_ 电机 ID 配置.md** - 设置电机 ID 和零位 - **04_ 硬件故障排查.md** - 硬件问题诊断

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

13 电机 ID 配置

13.1 电机 ID 设置和零位校准指南

本文档说明如何查看、设置电机 ID, 以及如何校准零位。

13.2 电机 ID 概述

13.2.1 ID 分配方案

OpenArmX 的 16 个电机按以下规则分配 ID:

can0 (左臂 + 左夹爪):

```
ID 1: 左肩关节1 (RS04) - 肩部
ID 2: 左肩关节2 (RS04) - 肩部
ID 3: 左大臂关节3 (RS03) - 大臂
ID 4: 左大臂关节4 (RS03) - 大臂
ID 5: 左小臂关节5 (RS00) - 小臂
ID 6: 左小臂关节6 (RS00) - 小臂
ID 7: 左小臂关节7 (RS00) - 小臂
ID 8: 左夹爪 (RS00) - 夹爪
```

can1 (右臂 + 右夹爪):

```
ID 1: 右肩关节1 (RS04) - 肩部
ID 2: 右肩关节2 (RS04) - 肩部
ID 3: 右大臂关节3 (RS03) - 大臂
ID 4: 右大臂关节4 (RS03) - 大臂
ID 5: 右小臂关节5 (RS00) - 小臂
ID 6: 右小臂关节6 (RS00) - 小臂
```

ID 7: 右小臂关节7 (RS00) - 小臂

ID 8: 右夹爪 (RS00) - 夹爪

13.2.2 ID 的重要性

- [√] 每条总线上 ID 必须唯一
- [√] ID 决定了控制命令的目标电机
- [×] ID 冲突会导致控制混乱
- [×] ID 错误会导致电机无响应

13.3 查看电机 ID

13.3.1 使用电机状态检查脚本

```
cd ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com

# 检查所有电机状态
python3 check_motor_status.py
```

输出示例:

```
CAN0 电机状态:
────────────────────────────────────────────────────────────
ID   型号   位置(rad)   速度   力矩   温度   错误
1    RS04    0.123     0.0    0.5    45     0x00
2    RS04   -0.056     0.0    0.3    43     0x00
3    RS04    0.234     0.0    0.7    46     0x00
...

CAN1 电机状态:
────────────────────────────────────────────────────────────
ID   型号   位置(rad)   速度   力矩   温度   错误
1    RS04    0.089     0.0    0.4    44     0x00
...
```

13.3.2 使用 candump 监听

```
# 监听 can0
candump can0

# 观察 CAN ID
# 正常情况下应该看到 ID 1-8 的帧
```

13.4 零位概念

13.4.1 什么是零位

零位是机器人的**标准初始姿态**, 所有关节角度定义为 0。

OpenArmX 零位姿态:

- 双臂垂直下垂
- 肩部、肘部、腕部伸直
- 夹爪完全张开

13.4.2 为什么要设置零位

- [√] 统一关节角度参考点
- [√] 保证运动学计算正确
- [√] 避免启动时突然运动
- [√] 便于轨迹规划

13.4.3 零位丢失的影响

如果零位不准确: - [×] 机器人姿态与软件显示不符 - [×] 运动轨迹错误 - [×] 可能碰撞或超限 - [×] 末端位置偏差

13.5 零位设置步骤

13.5.1 准备工作

前提条件:

☐ CAN接口已配置并启动

☐ 48V电源已连接

☐ 急停按钮可用

☐ 了解安全操作规范

13.5.2 调整到零位姿态

手动调整机械臂:

1. 断开48V电源(或禁用电机)

2. 手动将机械臂调整到零位姿态:
   - 肩部关节: 垂直向下
   - 肘部关节: 伸直
   - 腕部关节: 中立位置
   - 夹爪: 完全张开

3. 确保各关节在±30度范围内
   (超过30度设置零位脚本会报错)

4. 检查左右臂姿态对称

13.5.3 使用 GUI 设置零位

```
# 启动电机管理界面
cd ~/openarmx_robotstride_ws-cc/src/motor_manage_openarmx
python3 main.py

# 界面操作:
#  点击" 启动 CAN"
#  点击" 启动电机"
#  确认机械臂已在零位姿态
#  点击" 设置零位"
#  等待设置完成
```

13.5.4 使用命令行脚本设置

```
cd ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com
```

```
# 启动 CAN 接口
python3 en_all_can.py

# 启动电机
python3 en_all_motors.py

# 设置零位
python3 set_motor_zeros_setMIT.py
```

脚本执行过程:

```
正在读取电机位置...
can0 ID 1: 当前位置 0.05 rad (OK)
can0 ID 2: 当前位置 -0.03 rad (OK)
...
can1 ID 8: 当前位置 0.00 rad (OK)

所有电机位置偏差 < 0.52 rad (30度)
正在设置零位...
can0 ID 1: 零位设置成功
can0 ID 2: 零位设置成功
...
完成! 所有电机零位已设置
```

[警告] 如果出现错误:

```
错误: can0 ID 1 位置偏差过大: 0.85 rad (> 0.52 rad)
请手动调整机械臂到零位姿态后重试
```

解决方法: - 手动调整该关节到零位附近 - 重新运行脚本

13.5.5 零位设置原理

MIT 模式设置零位命令:

Robstride 电机使用特定 CAN 命令设置零位:

```
CAN ID: 电机ID + 特定偏移
数据: 零位设置命令帧
```

功能: 将当前位置设为0点

保存: 写入电机Flash存储

重启后生效: 设置零位后需要**重启电机** (断电重启或禁用再使能)

13.6 验证零位

13.6.1 回零测试

```
cd ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com

# 执行回零
python3 control_motor_gohome.py
```

观察: - 机械臂是否平稳移动到零位姿态 - 各关节角度是否接近 0 - 姿态是否对称

13.6.2 检查关节角度

```
# 查看当前关节位置
python3 check_motor_status.py
```

在零位姿态下, 所有关节位置应接近 0:

```
ID 1: 位置 0.001 rad  (约0.06度) ✓
ID 2: 位置 -0.002 rad (约0.11度) ✓
...
```

允许偏差: ±0.05 rad (±3 度)

13.6.3 使用 RViz 可视化验证

```
# 启动 robot_state_publisher
ros2 launch openarm_description view_robot.launch.py

# 观察 RViz 中的模型
# - 应该显示双臂垂直下垂
# - 与实际机械臂姿态一致
```

13.7 常见问题

13.7.1 设置零位失败

问题: 脚本报错 “位置偏差过大”

原因: - 机械臂未调整到零位附近 - 某个关节卡住 - 电机未使能

解决:

```
# 检查电机是否使能
python3 check_motor_status.py

# 手动调整机械臂
#   - 确保各关节能自由活动
#   - 调整到零位姿态

# 重新设置零位
python3 set_motor_zeros_setMIT.py
```

13.7.2 零位设置后仍然偏差大

问题: 设置零位后, 回零姿态不正确

可能原因: 1. 设置零位时机械臂姿态不正确 2. 设置后未重启电机 3. 电机编码器故障

解决:

```
# 断电重启电机
# 关闭 48V 电源，等待 5 秒，重新上电

# 重新使能电机
python3 en_all_motors.py

# 检查位置
python3 check_motor_status.py

# 如果仍然偏差大，重新设置零位
```

13.7.3 部分电机零位正确, 部分错误

排查步骤:

```
# 检查哪些电机零位错误
python3 check_motor_status.py

# 仅针对错误电机重新设置
#   修改 set_motor_zeros_setMIT.py
#   或使用 GUI 单独设置

# 检查错误电机的机械连接
#   - 是否松动
#   - 是否有卡顿
```

13.8 定期维护

13.8.1 零位检查频率

- ☐ 每次使用前：快速检查
- ☐ 每周：回零测试
- ☐ 每月：详细校准
- ☐ 更换电机后：必须重新设置

13.8.2 零位漂移监控

```
# 创建监控脚本
cat > ~/check_zero.sh << 'EOF'
#!/bin/bash
cd ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com
python3 en_all_can.py
python3 en_all_motors.py
sleep 2
python3 control_motor_gohome.py
sleep 5
python3 check_motor_status.py > /tmp/zero_check_$(date +%Y%m%d_%H%M%S).log
EOF
```

```
chmod +x ~/check_zero.sh

# 定期运行
./check_zero.sh
```

13.8.3 记录零位历史

建议记录每次设置零位的时间和原因:

日期	操作员	原因	备注
2025-01-15	张三	初次安装	全部电机
2025-02-20	李四	电机7更换	仅ID 7
2025-03-10	张三	定期校准	全部电机

13.9 高级操作

13.9.1 单独设置某个电机零位

修改脚本:

```
# 示例: 仅设置 can0 ID 7 的零位
import can
from lib.motor_control import set_zero_position

bus = can.interface.Bus(channel='can0', bustype='socketcan')
motor_id = 7

# 设置零位
set_zero_position(bus, motor_id)
print(f"can0 ID {motor_id} 零位已设置")
```

13.9.2 批量设置零位

如果有多个机器人:

```
# 为每个机器人创建零位配置文件
# robot1_zeros.yaml
# robot2_zeros.yaml
```

```
# 使用配置文件设置零位
python3 set_zeros_from_config.py --config robot1_zeros.yaml
```

13.9.3 零位偏移补偿

如果机械安装有固定偏差:

```
# 在 URDF 或控制器中添加偏移
joint_offset = {
    'left_joint_1': 0.05,  # 5 度偏移
    'left_joint_2': -0.03,
    # ...
}

# 在读取位置时应用偏移
actual_position = encoder_position + joint_offset[joint_name]
```

13.10 安全注意事项

13.10.1 设置零位前

[警告] 确保人员撤离
- 机械臂可能轻微运动

[警告] 手放在急停按钮上
- 发现异常立即按下

[警告] 检查机械臂姿态
- 确保在安全位置
- 避免碰撞

13.10.2 设置零位中

[警告] 不要触摸机械臂
- 电机已使能,有力矩输出

[警告] 观察异常情况

- 异常声音
- 剧烈抖动
- 冒烟

13.10.3 设置零位后

✓ 回零测试

- 确认零位设置正确

✓ 低速测试

- 先低速运动测试
- 确认无异常后提高速度

13.11 故障排查

13.11.1 电机 ID 冲突

症状: - 某些电机行为异常 - 控制一个电机时另一个也动

解决:

```
# 断开所有电机，逐个连接
# 检查每个电机 ID
# 修改冲突的 ID(需要电机配置工具)
```

13.11.2 零位无法保存

症状: - 设置零位后, 重启又恢复原值

可能原因: - 电机 Flash 写入失败 - 电源不稳定

解决:

```
# 确保电源稳定 (48V±2V)
# 重新设置零位
# 设置后等待 5 秒再断电
# 如果仍然失败，联系技术支持
```

13.12 总结

13.12.1 零位设置流程总结

```
1. 手动调整机械臂到零位姿态
   ↓
2. 启动CAN接口和电机
   ↓
3. 运行set_motor_zeros_setMIT.py
   ↓
4. 断电重启电机
   ↓
5. 回零测试验证
   ↓
6. 完成
```

13.12.2 关键点

✓ 零位姿态必须准确(双臂垂直下垂)
✓ 设置前确保偏差<30度
✓ 设置后必须重启电机
✓ 定期检查零位准确性
✓ 记录零位设置历史

13.12.3 下一步

完成电机 ID 和零位配置后, 请继续: - **04_ 硬件故障排查.md** - 硬件问题诊断 - **04_ 电机测试与调试** - 开始电机运动测试

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

14 硬件故障排查

14.1 硬件问题诊断与解决指南

本文档提供 OpenArmX 系统常见硬件问题的诊断方法和解决方案。

14.2 故障排查流程

14.2.1 系统化排查方法

1. 观察现象
   ↓
2. 定位子系统(电源/CAN/电机)
   ↓
3. 逐步排查
   ↓
4. 隔离问题
   ↓
5. 验证解决方案

14.2.2 安全第一原则

在排查过程中:

✓ 断电后进行物理检查

✓ 手放在急停按钮上

✓ 从小范围开始测试

✓ 记录所有操作

✓ 不确定时寻求帮助

14.3 电源问题

14.3.1 完全无电

症状: - 所有电机无响应 - 电源指示灯不亮 - 无任何声音

排查步骤:

```
# 检查市电输入
# - 插座是否有电
# - 电源开关是否打开

# 检查电源适配器
# - 指示灯是否亮
# - 使用万用表测量输出电压
```

```
#  测量输出电压
万用表 DC 档，测量 48V 输出端
正常：48V ± 2V
异常：0V 或远低于 48V

#  检查保险丝
- 查看是否熔断
- 更换相同规格保险丝

#  检查线缆连接
- 接线端子是否松动
- 线缆是否断裂
```

常见原因: - [×] 市电未接通 - [×] 电源开关未打开 - [×] 保险丝熔断 - [×] 电源故障

14.3.2 电压不稳定

症状: - 电机间歇性掉线 - 电压波动大 - 电机运行时电压下降明显

排查:

```
# 使用万用表监控电压
# 空载：48V ± 1V
# 负载：应不低于 46V

# 如果电压下降 >2V：
#  电源功率不足
#  线缆压降过大（线太细或太长）
#  电源老化
```

解决方法: - 更换更大功率电源 (≥1500W) - 使用更粗的电源线 (≥16AWG) - 缩短电源线长度 - 检查接触电阻

14.3.3 急停无法解除

症状: - 急停按钮解除后仍无电 - 需要多次操作

检查:

```
# 检查急停按钮触点
万用表电阻档:
- 按下: 触点断开 (∞)
- 解除: 触点闭合 (<1Ω)

# 检查继电器
- 是否卡死
- 触点是否氧化

# 清洁触点
使用电气清洁剂喷洗
```

14.4 CAN 通信问题

14.4.1 CAN 接口无法启动

症状:

```
$ sudo ip link set can0 up
Error: Cannot find device "can0"
```

排查:

```
# 检查 USB 连接
lsusb | grep -i can
# 或
lsusb | grep -i peak

# 应该看到 CAN 设备

# 检查驱动加载
lsmod | grep can
lsmod | grep peak_usb

# 查看 dmesg
dmesg | tail -50 | grep -i can
```

```
#  重新插拔 USB
# 拔出等待 5 秒，重新插入

#  尝试不同 USB 端口
```

常见原因: - USB 线缆故障 - USB 端口供电不足 - 驱动未加载 - CAN 转换器硬件故障

14.4.2 CAN 通信间歇性中断

症状: - 通信时好时坏 - RX/TX 错误计数器增加 - 电机偶尔无响应

诊断:

```
#  监控错误计数器
watch -n 1 'ip -s link show can0'

# 观察 RX errors 和 TX errors
# 如果持续增加，说明通信质量差

#  监听总线
candump can0 -e

# -e 参数显示错误帧
# 如果出现 ERROR FRAME，说明总线有问题

#  检查总线利用率
# 使用 can-utils 的 canbusload
canbusload can0@1000000 -p

# 如果利用率 >80%，可能拥塞
```

可能原因和解决:

1. 终端电阻问题

```
# 测量 CAN_H 和 CAN_L 间电阻
# 应为 60Ω(两个 120Ω 并联)
```

```
解决:
- 添加缺失的终端电阻
- 移除多余的终端电阻
```

2. 线缆质量差

解决:

- 更换屏蔽双绞线
- 确保线缆长度<5m
- 远离强电线缆(>30cm)

3. 波特率过高

```
# 降低波特率测试
sudo ip link set can0 down
sudo ip link set can0 type can bitrate 500000
sudo ip link set can0 up
```

4. 接触不良

解决:

- 检查所有连接器
- 紧固螺丝端子
- 清洁触点

14.4.3 Bus-Off 状态

症状:

```
$ ip link show can0
can0: ... state BUS-OFF
```

恢复:

```
#  重启接口
sudo ip link set can0 down
sudo ip link set can0 up

#  手动重启
sudo ip link set can0 type can restart
```

```
# 如果频繁 Bus-Off:
# - 检查物理层 (终端电阻、线缆)
# - 降低波特率
# - 检查电机是否短路
```

14.4.4 左右臂反向 (PCAN 特有)

症状: - can0 控制了右臂 - can1 控制了左臂

解决 (PCAN 设备):

```
# 方法 1: 重启 PCAN 模块
sudo rmmod peak_usb
sudo modprobe peak_usb

# 方法 2: 拔插 USB 接口
# 拔出两个 PCAN, 等待 5 秒
# 先插入左臂 (can0), 再插入右臂 (can1)

# 方法 3: 在软件中交换 can0 和 can1
# 修改 motor_mapping.yaml
```

14.5 电机问题

14.5.1 单个电机无响应

症状: - 其他电机正常 - 某一个电机完全无响应

排查:

```
# 使用 check_motor_status.py
python3 check_motor_status.py

# 查看该电机是否列出

# 监听 CAN 总线
candump can0 | grep "ID 对应的帧"
```

```
# 如果看不到该 ID 的帧，说明电机未通信

#  检查物理连接
# 断电后:
# - CAN 线是否连接
# - 电源线是否连接
# - 连接器是否插紧

#  单独连接测试
# - 断开其他电机
# - 仅连接该电机
# - 查看是否能通信

# 如果单独连接能通信:
# - 可能是 ID 冲突
# - 或总线负载问题

# 如果单独也不通信:
# - 电机 CAN 接口故障
# - 电机未供电
# - 电机硬件故障
```

14.5.2 电机抖动

症状: - 电机持续抖动 - 发出噪声 - 无法稳定在目标位置

原因和解决:

1. KP 参数过大

```
# 降低 KP 值
# 原值: KP=50
# 调整: KP=20-30
```

2. KD 参数过小

```
# 增加 KD 值
# 原值: KD=0.5
# 调整: KD=1.0-1.5
```

3. 机械卡顿

检查:

- 关节是否有杂物
- 轴承是否损坏
- 齿轮是否磨损

4. 编码器故障

```
# 检查位置反馈是否跳变
python3 check_motor_status.py

# 观察位置值是否稳定
# 如果静止时位置大幅跳变，编码器故障
```

14.5.3 电机过热

症状: - 温度 >80°C - 电机发烫 - 性能下降

立即处理:

```
#  停止运动
python3 dis_all_motors.py

#  让电机自然冷却
# 不要用水或风扇快速冷却

#  等待温度 <50°C
```

原因分析:

可能原因:

1. 负载过大
2. KP/KD参数过高(持续力矩输出)
3. 运动速度过快
4. 散热不良

5. 电机故障

解决方法:

- ☐ 降低负载
- ☐ 降低KP/KD参数
- ☐ 降低运动速度和加速度
- ☐ 增加散热(风扇)
- ☐ 检查电机是否有短路
- ☐ 如仍过热,更换电机

14.5.4 电机失控

症状: - 电机高速旋转无法停止 - 不受控制运动 - 超出限位

紧急处理:

1. 立即按下急停按钮! [红]
2. 断开48V电源
3. 等待电机完全停止
4. 不要靠近机械臂

事后分析:

```
# 检查日志
ros2 log

# 检查参数配置
# - KP 是否异常大
# - 目标位置是否错误

# 检查零位
python3 check_motor_status.py

# 重新设置零位和参数后测试
```

14.5.5 电机无力

症状: - 电机可以运动但力矩小 - 无法承载负载 - 容易被手推动

检查:

```
# 检查 KP 参数
# KP 过小会导致无力
# 增加 KP 值: 10 → 20 → 30

# 检查电机是否正常使能
python3 check_motor_status.py

# 检查电源电压
# 电压过低会导致输出扭矩小
```

14.6 机械问题

14.6.1 关节卡顿

症状: - 某个关节运动不顺畅 - 有阻力感 - 运动过程中有停顿

检查:

1. 断电后手动转动关节
   - 应该顺滑
   - 阻力均匀

2. 检查机械连接
   - 螺丝是否松动
   - 零件是否错位

3. 检查轴承
   - 是否磨损
   - 是否缺润滑

4. 检查传动机构
   - 齿轮是否啮合良好
   - 有无杂物

14.6.2 机械松动

症状: - 运动时有异响 - 关节晃动 - 定位精度下降

解决:

```
# 断电后:

1. 检查所有固定螺丝
   - 使用扭力扳手按规定扭矩紧固
   - 特别是电机固定螺丝

2. 检查连接件
   - 法兰连接
   - 轴套固定

3. 使用螺纹锁固胶 (可选)
   - Loctite 243(中强度)
   - 防止螺丝松动
```

14.6.3 机械碰撞痕迹

症状: - 外壳有划痕或凹陷 - 零件变形

处理:

1. 检查受损程度
   - 表面划伤：可继续使用
   - 零件变形：需要更换

2. 检查内部是否受损
   - 电机是否受冲击
   - 传动机构是否正常

3. 如有严重碰撞:
   - 联系技术支持
   - 可能需要专业检修

14.7 传感器问题

14.7.1 编码器故障

症状: - 位置读数跳变 - 零位丢失 - 运动轨迹异常

诊断:

```
# 监控位置反馈
python3 << EOF
import can
import time
from lib.motor_control import *

bus = can.interface.Bus(channel='can0', bustype='socketcan')

for i in range(100):
    # 读取电机 1 位置
    pos = read_motor_position(bus, 1)
    print(f" 位置: {pos:.4f} rad")
    time.sleep(0.1)
EOF

# 观察位置是否稳定
# 静止时应该基本不变 (±0.001 rad)
```

如果位置跳变: - 编码器磁铁松动 - 编码器芯片故障 - 干扰问题

解决: - 检查磁铁固定 - 更换编码器 - 改善屏蔽

14.7.2 温度传感器异常

症状: - 温度读数异常 (负值或 >100°C) - 温度不随负载变化

检查:

```
python3 check_motor_status.py

# 观察温度值
# 正常: 环境温度 +10~50°C
```

```
# 异常: <0°C 或 >100°C
```

如果异常: - 传感器故障 (可继续使用, 但无温度保护) - 更换电机

14.8 电气干扰

14.8.1 现象

- CAN 通信不稳定
- 电机偶尔误动作
- 控制计算机死机/重启

14.8.2 减少干扰措施

1. 接地

☐ 电源外壳接大地
☐ 机器人基座接大地
☐ 控制计算机接大地
☐ 使用同一接地点

2. 屏蔽

☐ 使用屏蔽CAN线缆
☐ 屏蔽层单端接地(CAN转换器端)
☐ 电源线使用滤波器

3. 隔离

☐ 电源线和信号线分开走线(>30cm)
☐ 避免平行走线
☐ 交叉时垂直交叉

4. 滤波

☐ 电源输入加EMI滤波器
☐ 电机端加共模扼流圈(如有需要)

14.9 诊断工具

14.9.1 必备工具

☐ 万用表(电压、电阻、通断)

- ☐ CAN总线分析仪(可选,专业调试)
- ☐ 示波器(可选,查看信号质量)
- ☐ 内六角扳手套装
- ☐ 螺丝刀套装
- ☐ 剥线钳、压线钳

14.9.2 软件工具

```
# CAN 工具
candump can0              # 监听 CAN
cansend can0 123#data     # 发送测试帧
canbusload can0@1000000   # 总线负载

# 系统监控
dmesg | grep -i can       # 内核日志
lsusb                     # USB 设备
ip -s link show can0      # 接口统计

# OpenArmX 工具
python3 check_motor_status.py    # 电机状态
python3 test_motor_one.py        # 单电机测试
```

14.10 故障记录

14.10.1 故障日志模板

建议记录每次故障:

故障报告

日期: 2025-10-19
时间: 14:30
操作员: 张三

现象:
can0 ID 5电机无响应

排查过程:

1. 检查CAN接口 - 正常
2. 检查其他电机 - 正常
3. 检查ID 5物理连接 - 发现CAN线松动

解决方法:

重新插紧CAN连接器,问题解决

预防措施:

定期检查连接器,使用线卡固定

备注:

建议增加连接器锁定机构

14.11 常见故障速查表

故障现象	可能原因	快速检查	参考章节
完全无电	电源故障	万用表测 48V	2.1
CAN 接口无法启动	USB 连接问题	lsusb 检查	3.1
单个电机无响应	接线/ID 问题	candump 监听	4.1
电机抖动	KP 过大	降低 KP 值	4.2
电机过热	参数/负载	降 KP, 减负载	4.3
通信间歇中断	终端电阻/干扰	测量 60Ω	3.2
关节卡顿	机械问题	手动转动	5.1
左右臂反向	PCAN 设备	重启模块	3.4

14.12 何时联系技术支持

14.12.1 建议联系的情况

- ☐ 电机冒烟或异味
- ☐ 多次尝试仍无法解决
- ☐ 需要更换硬件
- ☐ 怀疑设计缺陷
- ☐ 不确定操作安全性

14.12.2 联系方式

成都长数机器人有限公司

电话/微信: +86-17746530375

邮箱: openarmrobot@gmail.com

网站: https://openarmx.com/

14.12.3 准备信息

联系技术支持前准备:

- ☐ 产品型号和序列号
- ☐ 购买日期和凭证
- ☐ 故障详细描述
- ☐ 已尝试的解决方法
- ☐ 照片或视频(如适用)
- ☐ 系统日志和错误信息

14.13 总结

14.13.1 故障排查原则

1. 安全第一
2. 系统化排查
3. 从简单到复杂
4. 隔离问题
5. 记录过程
6. 验证解决方案

14.13.2 预防性维护

- ☐ 定期检查连接
- ☐ 监控温度和错误
- ☐ 及时紧固螺丝
- ☐ 保持清洁
- ☐ 记录故障历史
- ☐ 备用关键部件

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

15 CAN 总线管理

15.1 CAN 总线启动和管理

本文档说明如何使用 OpenArmX 提供的脚本管理 CAN 总线。

15.2 CAN 管理脚本概述

15.2.1 脚本位置

```
cd ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com

ls -l *can.py
# en_all_can.py    - 启动 CAN 接口
# dis_all_can.py   - 关闭 CAN 接口
```

15.2.2 脚本功能

脚本	功能	使用场景
en_all_can.py	启动 can0 和 can1	系统启动时
dis_all_can.py	关闭 can0 和 can1	系统关闭时

15.3 启动 CAN 接口

15.3.1 使用 en_all_can.py

```
cd ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com

# 启动 CAN 接口
python3 en_all_can.py
```

输出示例:

```
启动CAN接口...
设置can0波特率为1Mbps...
启动can0...
设置can1波特率为1Mbps...
启动can1...
```

CAN接口启动成功!

15.3.2 脚本内部实现

```
#!/usr/bin/env python3
import os
import sys

def enable_can():
    """ 启动 CAN 接口"""

    # 配置 can0
    print(" 设置 can0...")
    os.system('sudo ip link set can0 type can bitrate 1000000')
    os.system('sudo ip link set can0 up')

    # 配置 can1
    print(" 设置 can1...")
    os.system('sudo ip link set can1 type can bitrate 1000000')
    os.system('sudo ip link set can1 up')

    print("CAN 接口启动成功!")

    # 显示状态
    os.system('ip link show can0 | grep can0')
    os.system('ip link show can1 | grep can1')

if __name__ == '__main__':
    enable_can()
```

15.3.3 验证启动成功

```
# 检查接口状态
ip link show can0 can1
```

```
# 应该看到:
# can0: <NOARP,UP,LOWER_UP,ECHO> ...
# can1: <NOARP,UP,LOWER_UP,ECHO> ...

# 关键字: UP, LOWER_UP
```

15.4 关闭 CAN 接口

15.4.1 使用 dis_all_can.py

```
cd ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com

# 关闭 CAN 接口
python3 dis_all_can.py
```

输出:

```
关闭can0...
关闭can1...
CAN接口已关闭
```

15.4.2 何时关闭 CAN

推荐关闭时机:

- ☐ 系统长时间不使用
- ☐ 进行硬件维护
- ☐ 重新配置CAN参数
- ☐ 系统关机前

不建议频繁开关:

- 每次测试后关闭(不必要)
- 电机测试中间关闭

15.5 自动化管理

15.5.1 开机自动启动

方法 1: 添加到.bashrc

```
# 编辑.bashrc
vim ~/.bashrc

# 添加到文件末尾（不推荐，每次终端都执行）
# alias start-can='python3 ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com/en_all_can.py'
```

方法 2: 创建 systemd 服务 (推荐)

```
# 创建服务文件
sudo vim /etc/systemd/system/openarmx-can.service

# 内容:
[Unit]
Description=OpenArmX CAN Interface
After=network.target

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/usr/bin/python3 /home/你的用户名/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com/e
ExecStop=/usr/bin/python3 /home/你的用户名/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com/di

[Install]
WantedBy=multi-user.target

# 启用服务
sudo systemctl enable openarmx-can.service
sudo systemctl start openarmx-can.service
```

15.5.2 快捷命令设置

```
# 添加 alias 到.bashrc
echo "alias can-on='python3 ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com/en_all_can.py'"
echo "alias can-off='python3 ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com/dis_all_can.py'
```

```
source ~/.bashrc

# 使用
can-on   # 启动 CAN
can-off  # 关闭 CAN
```

15.6 故障排查

15.6.1 启动失败

问题: `Error: Cannot find device "can0"`

解决:

```
#  检查 USB 设备
lsusb | grep -i can

#  检查驱动
lsmod | grep can

#  重新插拔 USB
#  重新运行脚本
```

15.6.2 权限问题

问题: `RTNETLINK answers: Operation not permitted`

解决:

```
# 方法 1: 使用 sudo
sudo python3 en_all_can.py

# 方法 2: 添加用户到 netdev 组 (一次性)
sudo usermod -a -G netdev $USER
# 重新登录后生效
```

15.6.3 PCAN 设备反向

症状: can0 和 can1 顺序错误

解决:

```
# 重启 PCAN 模块
sudo rmmod peak_usb
sudo modprobe peak_usb

# 重新运行启动脚本
python3 en_all_can.py
```

15.7 监控 CAN 状态

15.7.1 实时状态监控

```
# 创建监控脚本
cat > ~/monitor_can.sh << 'EOF'
#!/bin/bash
while true; do
    clear
    echo "=== CAN 接口状态 ==="
    date
    echo ""
    ip -s link show can0
    echo ""
    ip -s link show can1
    sleep 2
done
EOF

chmod +x ~/monitor_can.sh
./monitor_can.sh
```

15.7.2 检查通信

```
# 终端 1: 监听 can0
candump can0
```

```
# 终端 2: 启动电机
python3 en_all_motors.py

# 终端 1 应该看到 CAN 帧流动
```

15.8 高级配置

15.8.1 修改波特率

如果需要使用其他波特率:

```
# 修改 en_all_can.py
# 将 1000000 改为 500000(500kbps)
os.system('sudo ip link set can0 type can bitrate 500000')
os.system('sudo ip link set can1 type can bitrate 500000')
```

15.8.2 启用自动重启

```
# 在 en_all_can.py 中添加
os.system('sudo ip link set can0 type can bitrate 1000000 restart-ms 100')
os.system('sudo ip link set can1 type can bitrate 1000000 restart-ms 100')

# restart-ms 100: Bus-Off 后 100ms 自动重启
```

15.9 总结

15.9.1 常用命令

```
# 启动 CAN
python3 en_all_can.py

# 关闭 CAN
python3 dis_all_can.py

# 检查状态
ip link show can0 can1
```

```
# 查看统计
ip -s link show can0
```

15.9.2 最佳实践

✓ 使用脚本而非手动命令(一致性)

✓ 系统启动后首先启动CAN

✓ 关机前关闭CAN(可选)

✓ 定期检查错误计数器

✓ 记录异常情况

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

16 电机使能与初始化

16.1 电机使能、禁用和初始化操作

本文档说明如何使能和禁用电机, 以及零位设置。

16.2 电机使能概述

16.2.1 什么是使能

使能 (Enable) 是让电机进入可控状态: - 电机通电并保持位置 - 可以接收和执行控制命令 - 输出力矩抵抗外力

禁用 (Disable) 是让电机进入自由状态: - 电机断电, 无力矩输出 - 可以手动推动 - 不响应控制命令

16.2.2 使能管理脚本

```
cd ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com


ls -l *motors.py
# en_all_motors.py   - 使能所有电机
# dis_all_motors.py  - 禁用所有电机
```

16.3 使能电机

16.3.1 使用 en_all_motors.py

```
cd ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com

# 前提: CAN 接口已启动
python3 en_all_can.py

# 使能所有电机
python3 en_all_motors.py
```

输出示例:

使能电机...
can0 ID 1: 使能成功
can0 ID 2: 使能成功
...
can1 ID 8: 使能成功
所有电机已使能

16.3.2 使能时的现象

正常现象: - [√] 电机轻微抖动 (瞬间) - [√] 保持当前位置 - [√] 温度略有上升

异常现象 (需要急停): - [×] 剧烈抖动或高速旋转 - [×] 异常声音 - [×] 冒烟

16.3.3 安全注意事项

使能前:

☐ 确认CAN接口已启动
☐ 机械臂处于安全姿态
☐ 人员已撤离工作区域
☐ 手放在急停按钮上

使能后:

☐ 不要触碰机械臂
☐ 观察电机状态
☐ 检查温度和声音

16.4 禁用电机

16.4.1 使用 dis_all_motors.py

```
cd ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com

# 禁用所有电机
python3 dis_all_motors.py
```

输出:

```
禁用电机...
can0 ID 1: 禁用成功
can0 ID 2: 禁用成功
...
can1 ID 8: 禁用成功
所有电机已禁用
```

16.4.2 禁用后的现象

- [√] 电机可以手动推动
- [√] 机械臂可能下垂 (无支撑力)
- [√] 温度逐渐下降

16.4.3 何时禁用电机

推荐禁用时机:

☐ 测试完成后

☐ 长时间不使用

☐ 需要手动调整姿态

☐ 发生异常情况

☐ 系统关机前

紧急情况:

[警告] 发现电机失控

[警告] 异常高温

[警告] 碰撞风险

16.5 零位设置

16.5.1 使用 set_motor_zeros_setMIT.py

前提条件:

☐ CAN接口已启动

☐ 电机已使能

☐ 机械臂手动调整到零位姿态

☐ 各关节偏差<30度

执行步骤:

```
cd ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com

# 启动 CAN
python3 en_all_can.py

# 使能电机
python3 en_all_motors.py

# 手动调整机械臂到零位 (双臂垂直下垂)

# 设置零位
python3 set_motor_zeros_setMIT.py
```

输出示例:

```
读取当前电机位置...
can0 ID 1: 位置 0.05 rad (偏差: 2.9°) ✓
can0 ID 2: 位置 -0.03 rad (偏差: 1.7°) ✓
...
所有电机位置在允许范围内

设置零位...
can0 ID 1: 零位设置成功
can0 ID 2: 零位设置成功
...
完成! 请重启电机使零位生效
```

16.5.2 零位设置失败

错误示例:

```
错误: can0 ID 5 位置偏差过大
当前位置: 0.85 rad (48.7°)
允许偏差: ±0.52 rad (±30°)
请手动调整后重试
```

解决方法:

```
# 禁用电机
python3 dis_all_motors.py

# 手动调整 ID 5 电机到零位附近

# 重新使能
python3 en_all_motors.py

# 重新设置零位
python3 set_motor_zeros_setMIT.py
```

16.5.3 验证零位

```
# 重启电机使零位生效
python3 dis_all_motors.py
sleep 2
python3 en_all_motors.py

# 回零测试
python3 control_motor_gohome.py

# 观察机械臂是否回到正确的零位姿态
```

16.6 回零操作

16.6.1 使用 control_motor_gohome.py

```
cd ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com

# 执行回零
python3 control_motor_gohome.py
```

功能: - 将所有关节移动到 0 位置 - 验证零位设置是否正确 - 作为运动测试的起始姿态

参数说明:

```
# 脚本中的关键参数
target_position = 0.0  # 目标位置 (rad)
kp = 20.0              # 位置增益
kd = 1.0               # 速度增益
duration = 3.0         # 运动时间 (秒)
```

16.6.2 回零安全检查

执行前:

☐ 确保周围无障碍物

☐ 机械臂当前位置接近零位(避免大幅度运动)

☐ 手放在急停按钮上

执行中:

☐ 观察运动轨迹

☐ 监听异常声音

☐ 准备随时急停

执行后:

☐ 检查最终姿态是否正确

☐ 检查关节角度(应接近0)

16.7 启动和关闭流程

16.7.1 标准启动流程

```
# 启动 CAN 接口
python3 en_all_can.py

# 打开 48V 电源
# (手动操作)

# 使能电机
python3 en_all_motors.py

# 检查电机状态
python3 check_motor_status.py

# 回零 (可选)
python3 control_motor_gohome.py

# 开始测试或使用
```

16.7.2 标准关闭流程

```
# 停止所有运动

# 回零 (可选，便于下次启动)
python3 control_motor_gohome.py

# 禁用电机
python3 dis_all_motors.py

# 关闭 CAN 接口
python3 dis_all_can.py

# 关闭 48V 电源
```

```
# (手动操作)
```

16.7.3 紧急关闭流程

遇到紧急情况:

1. 立即按下急停按钮 [红]

2. 或运行:
   python3 dis_all_motors.py

3. 断开48V电源

4. 记录异常情况

5. 排查问题后再启动

16.8 批量脚本

16.8.1 创建一键启动脚本

```
cat > ~/openarmx_start.sh << 'EOF'
#!/bin/bash
cd ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com

echo "===== OpenArmX 启动序列 ======"

echo "[1/3] 启动 CAN 接口..."
python3 en_all_can.py
if [ $? -ne 0 ]; then
    echo " 错误: CAN 启动失败"
    exit 1
fi

echo "[2/3] 等待 5 秒, 请打开 48V 电源..."
```

```
sleep 5

echo "[3/3] 使能电机..."
python3 en_all_motors.py
if [ $? -ne 0 ]; then
    echo " 错误: 电机使能失败"
    exit 1
fi

echo "===== 启动完成 ====="
echo " 运行 'python3 check_motor_status.py' 检查电机状态"
EOF

chmod +x ~/openarmx_start.sh
```

使用:

```
~/openarmx_start.sh
```

16.8.2 创建一键关闭脚本

```
cat > ~/openarmx_stop.sh << 'EOF'
#!/bin/bash
cd ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com

echo "===== OpenArmX 关闭序列 ====="

echo "[1/2] 禁用电机..."
python3 dis_all_motors.py

echo "[2/2] 关闭 CAN 接口..."
python3 dis_all_can.py

echo "===== 关闭完成 ====="
echo " 可以关闭 48V 电源"
```

```
EOF

chmod +x ~/openarmx_stop.sh
```

16.9 常见问题

16.9.1 使能失败

症状: 某些电机无法使能

排查:

```
# 检查 CAN 通信
candump can0

# 检查该电机是否响应
python3 check_motor_status.py

# 检查电机供电

# 尝试单独使能该电机
```

16.9.2 使能后立即掉线

症状: 使能成功, 但几秒后失去响应

可能原因: - 电源功率不足 - CAN 总线质量差 - 电机过载保护

解决:

```
# 检查电源电压 (应稳定在 48V±2V)

# 检查 CAN 错误计数
ip -s link show can0

# 降低初始 KP/KD 值
```

16.9.3 零位设置后仍然偏差

症状: 回零姿态不正确

解决:

```
# 确认设置零位时姿态正确

# 重启电机
python3 dis_all_motors.py
sleep 5
python3 en_all_motors.py

# 重新测试回零
python3 control_motor_gohome.py

# 如仍然偏差，重新设置零位
```

16.10 图形化管理界面

16.10.1 使用启动 _ 关闭 _ 电机.py

```
cd ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com

# 启动 GUI
python3 启动 _ 关闭 _ 电机.py
```

GUI 功能: - 启动 CAN 按钮 - 启动电机按钮 - 关闭电机按钮 - 设置零位按钮 - 状态显示

适合: - 不熟悉命令行的用户 - 快速测试 - 演示用途

16.11 总结

16.11.1 关键命令

```
# 启动
python3 en_all_can.py       # 启动 CAN
python3 en_all_motors.py    # 使能电机

# 关闭
python3 dis_all_motors.py   # 禁用电机
python3 dis_all_can.py      # 关闭 CAN
```

```
# 零位
python3 set_motor_zeros_setMIT.py  # 设置零位
python3 control_motor_gohome.py    # 回零
```

16.11.2 最佳实践

✓ 按标准流程启动和关闭
✓ 使能前检查机械臂姿态
✓ 定期验证零位准确性
✓ 记录异常情况
✓ 紧急情况立即禁用电机

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

17 电机状态监控

17.1 电机状态检查和参数读取

本文档说明如何使用 check_motor_status.py 监控电机状态。

17.2 电机状态检查

17.2.1 使用 check_motor_status.py

```
cd ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com

# 检查所有电机状态
python3 check_motor_status.py
```

输出示例:

```
========== CAN0 电机状态 ==========
ID  型号  位置(rad)  速度(rad/s)  力矩(Nm)  温度(°C)  错误码
1   RS04   0.123      0.0         0.5       45       0x00
2   RS04  -0.056      0.0         0.3       43       0x00
3   RS04   0.234      0.0         0.7       46       0x00
4   RS03   0.012      0.0         0.1       42       0x00
```

```
5   RS03  -0.089      0.0         0.2       41       0x00
6   RS03   0.045      0.0         0.1       40       0x00
7   RS03  -0.023      0.0         0.0       39       0x00
8   RS00   0.000      0.0         0.0       38       0x00

========== CAN1 电机状态 ==========
ID  型号  位置(rad)  速度(rad/s)  力矩(Nm)  温度(°C)  错误码
...
```

17.3 状态参数解读

17.3.1 位置 (Position)

单位: rad(弧度)

正常范围: - 静止时应该稳定 (变化 <0.01 rad) - 运动时连续变化 - 零位姿态接近 0

异常情况: - 静止时大幅跳变 → 编码器故障 - 数值明显偏离预期 → 零位丢失

17.3.2 速度 (Velocity)

单位: rad/s

正常状态: - 静止时应为 0 或很小 (<0.1 rad/s) - 运动时为设定值

异常情况: - 静止时速度大 → 电机抖动或失控

17.3.3 力矩 (Torque)

单位: Nm(牛·米)

正常范围: - 空载: 0~2 Nm - 负载: 根据重量增加 - 峰值: 不超过额定扭矩

异常情况: - 静止时扭矩过大 → KP 设置过高或卡顿 - 持续峰值 → 过载

17.3.4 温度 (Temperature)

单位: °C

安全范围: | 温度 | 状态 | 处理 | |——|——|——| | <50°C | 正常 | 无需处理 | | 50-60°C | [黄] 温和 | 注意监控 | | 60-70°C | [黄] 偏高 | 降低负载 | | 70-80°C | [橙] 警告 | 停止高强度运动 | | >80°C | [红] 危险 | 立即停止并冷却 |

影响因素: - 环境温度 - 运动强度 - KP/KD 参数 - 散热条件

17.3.5 错误码 (Error Code)

常见错误码: | 错误码 | 含义 | 处理 | |——-|——|——| | 0x00 | 正常 | 无 | | 0x01 | 过压 | 检查电源 | | 0x02 | 欠压 | 检查电源 | | 0x04 | 过流 | 降低负载/KP | | 0x08 | 过温 | 冷却 | | 0x10 | 编码器故障 | 检查/更换 | | 0x20 | CAN 超时 | 检查通信 |

17.4 持续监控

17.4.1 实时监控脚本

```
# 创建实时监控
cat > ~/monitor_motors.sh << 'EOF'
#!/bin/bash
cd ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com

while true; do
    clear
    echo "===== OpenArmX 电机监控 ======"
    date
    echo ""
    python3 check_motor_status.py
    sleep 2
done
EOF

chmod +x ~/monitor_motors.sh

# 运行
./monitor_motors.sh
```

17.4.2 温度监控

```
# 仅监控温度
python3 check_motor_status.py | grep -E "ID| 温度"
```

17.4.3 记录到日志

```
# 每分钟记录一次状态
while true; do
    echo "$(date)" >> motor_log.txt
    python3 check_motor_status.py >> motor_log.txt
    echo "" >> motor_log.txt
    sleep 60
done &
```

17.5 异常诊断

17.5.1 温度异常

高温处理:

```
# 停止运动
python3 dis_all_motors.py

# 等待冷却 (<50°C)

# 检查原因:
# - 负载过大
# - KP/KD 过高
# - 散热不良
# - 电机故障

# 调整后重新测试
```

17.5.2 位置异常

零位丢失:

```
# 回零测试
python3 control_motor_gohome.py

# 如果姿态不正确，重新设置零位
python3 dis_all_motors.py
```

```
# 手动调整到零位
python3 en_all_motors.py
python3 set_motor_zeros_setMIT.py
```

17.5.3 通信异常

部分电机无响应:

```
# 检查哪些电机无响应
python3 check_motor_status.py

# 监听 CAN 总线
candump can0

# 检查物理连接

# 重启 CAN 和电机
```

17.6 总结

关键检查项:

- ☐ 所有电机都能读取状态
- ☐ 温度在安全范围(<70°C)
- ☐ 错误码为0x00
- ☐ 位置反馈稳定
- ☐ 速度和力矩合理

监控频率:

- ☐ 每次启动后检查一次
- ☐ 长时间运行每15分钟检查
- ☐ 出现异常立即检查

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

18 单电机测试

18.1 单个电机运动测试

测试单个电机的位置控制和参数调整。

18.2 test_motor_one.py 使用

```
cd ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com

# 运行单电机测试
python3 test_motor_one.py
```

交互式输入:

```
请输入CAN通道 (0或1): 0
请输入电机ID (1-8): 1
请输入目标位置 (rad, -12.5~12.5): 0.5
请输入KP (0-500): 20
请输入KD (0-5): 1.0
```

安全建议: - 首次测试使用小幅度 (±0.5 rad) - 使用保守的 KP/KD(KP=10-20, KD=0.5-1.0) - 手放在急停按钮上

18.3 参数调整

18.3.1 KP 参数影响

KP 过小 (KP=5): - 响应慢 - 位置误差大 - 容易被推动

KP 适中 (KP=20): - 响应快 - 精度高 - 稳定性好

KP 过大 (KP=100): - 震荡 - 噪声大 - 可能损坏电机

18.3.2 KD 参数影响

KD 过小 (KD=0.1): - 震荡 - 超调

KD 适中 (KD=1.0): - 平稳运动 - 阻尼适当

KD 过大 (KD=5.0): - 运动缓慢 - 噪声

18.4 常见测试场景

18.4.1 位置精度测试

```
# 测试不同位置
positions = [0.0, 0.5, -0.5, 1.0, -1.0, 0.0]

for pos in positions:
    # 移动到 pos
    # 等待稳定
    # 记录实际位置
```

18.4.2 重复定位测试

```
# 重复移动到同一位置
for i in range(10):
    # 移动到 0.0
    # 移动到 1.0
    # 记录误差
```

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

19 多电机测试

19.1 多电机协调运动测试

测试多个电机的协调运动和同步控制。

19.2 多电机测试概述

19.2.1 测试目的

- 验证多电机同步控制
- 测试关节协调运动
- 检查电机间干扰
- 评估系统稳定性

19.2.2 测试范围

测试场景:

- ☐ 同一手臂多关节运动(如肩部3个关节)
- ☐ 整臂运动(7个关节)
- ☐ 双臂同步运动(14个关节)
- ☐ 夹爪配合测试

19.3 单臂多关节测试

19.3.1 肩部三关节测试

目标: 测试 ID 1-3 电机 (肩部) 协调运动

测试脚本示例:

```
#!/usr/bin/env python3
import can
from robstride import Motor
import time

# 初始化 CAN 总线
bus0 = can.interface.Bus(channel='can0', bustype='socketcan')

# 创建 3 个电机对象
motor1 = Motor(bus0, 1)
motor2 = Motor(bus0, 2)
motor3 = Motor(bus0, 3)

# 使能电机
motor1.enable()
motor2.enable()
motor3.enable()

time.sleep(1)

# 同步运动到目标位置
```

```
kp = 20.0
kd = 1.0

# 目标位置 (rad)
pos1 = 0.5
pos2 = 0.3
pos3 = -0.4

# 发送控制命令
motor1.set_position(pos1, vel=0.0, kp=kp, kd=kd, torque=0.0)
motor2.set_position(pos2, vel=0.0, kp=kp, kd=kd, torque=0.0)
motor3.set_position(pos3, vel=0.0, kp=kp, kd=kd, torque=0.0)

# 等待运动完成
time.sleep(3)

# 读取反馈
print(f"Motor 1: {motor1.get_feedback()}")
print(f"Motor 2: {motor2.get_feedback()}")
print(f"Motor 3: {motor3.get_feedback()}")

# 回零
motor1.set_position(0.0, vel=0.0, kp=kp, kd=kd, torque=0.0)
motor2.set_position(0.0, vel=0.0, kp=kp, kd=kd, torque=0.0)
motor3.set_position(0.0, vel=0.0, kp=kp, kd=kd, torque=0.0)

time.sleep(3)

# 失能
motor1.disable()
motor2.disable()
motor3.disable()
```

运行:

```
python3 test_shoulder_joints.py
```

19.3.2 整臂 7 关节测试

完整手臂运动测试:

```
#!/usr/bin/env python3
import can
from robstride import Motor
import time
import numpy as np

bus0 = can.interface.Bus(channel='can0', bustype='socketcan')

# 创建 7 个电机对象 (左臂)
motors = [Motor(bus0, i) for i in range(1, 8)]

# 使能所有电机
for m in motors:
    m.enable()
    time.sleep(0.1)

print(" 所有电机已使能, 等待 2 秒...")
time.sleep(2)

# 定义一个简单的关节空间轨迹
# 从零位移动到预定义姿态
target_positions = [0.5, 0.3, -0.4, -0.8, 0.2, 0.3, -0.2]  # 7 个关节

# KP/KD 参数
kp_rs04 = 20.0  # ID 1-3
kd_rs04 = 1.0
kp_rs03 = 20.0  # ID 4-7
kd_rs03 = 1.0
```

```
# 发送目标位置
for i, motor in enumerate(motors):
    if i < 3:
        kp, kd = kp_rs04, kd_rs04
    else:
        kp, kd = kp_rs03, kd_rs03

    motor.set_position(target_positions[i], vel=0.0, kp=kp, kd=kd, torque=0.0)
    time.sleep(0.05)  # 短暂延迟避免 CAN 总线拥塞

print(" 运动到目标位置，等待 5 秒...")
time.sleep(5)

# 打印反馈
for i, motor in enumerate(motors):
    fb = motor.get_feedback()
    print(f"Joint {i+1}: pos={fb['position']:.3f}, vel={fb['velocity']:.3f}, torque={fb['torque']:.3f}

# 回零
print(" 回零...")
for i, motor in enumerate(motors):
    if i < 3:
        kp, kd = kp_rs04, kd_rs04
    else:
        kp, kd = kp_rs03, kd_rs03

    motor.set_position(0.0, vel=0.0, kp=kp, kd=kd, torque=0.0)
    time.sleep(0.05)

time.sleep(5)

# 失能
for m in motors:
    m.disable()
```

```
    time.sleep(0.1)

print(" 测试完成")
```

运行:

```
python3 test_full_arm.py
```

预期现象: - 7 个关节平滑协调运动 - 无明显抖动或异响 - 所有关节到达目标位置 - 回零过程平稳

19.4 双臂同步测试

19.4.1 双臂镜像运动

测试目标: 左右臂执行镜像对称运动

```
#!/usr/bin/env python3
import can
from robstride import Motor
import time

# 初始化双 CAN 总线
bus0 = can.interface.Bus(channel='can0', bustype='socketcan')
bus1 = can.interface.Bus(channel='can1', bustype='socketcan')

# 左臂电机 (can0)
left_motors = [Motor(bus0, i) for i in range(1, 8)]

# 右臂电机 (can1)
right_motors = [Motor(bus1, i) for i in range(1, 8)]

# 使能所有电机
print(" 使能左臂电机...")
for m in left_motors:
    m.enable()
    time.sleep(0.1)

print(" 使能右臂电机...")
```

```
for m in right_motors:
    m.enable()
    time.sleep(0.1)

time.sleep(2)

# 定义镜像运动姿态
# 左臂目标位置
left_targets = [0.5, 0.3, -0.4, -0.8, 0.2, 0.3, -0.2]

# 右臂目标位置 (镜像,joint_1 反向)
right_targets = [-left_targets[0]] + left_targets[1:]

kp, kd = 20.0, 1.0

# 同步发送命令
print(" 执行镜像运动...")
for i in range(7):
    left_motors[i].set_position(left_targets[i], vel=0.0, kp=kp, kd=kd, torque=0.0)
    right_motors[i].set_position(right_targets[i], vel=0.0, kp=kp, kd=kd, torque=0.0)
    time.sleep(0.05)

time.sleep(5)

# 验证同步性
print(" 检查同步误差...")
for i in range(7):
    left_pos = left_motors[i].get_feedback()['position']
    right_pos = right_motors[i].get_feedback()['position']

    # 考虑镜像关系
    if i == 0:
        sync_error = abs(left_pos + right_pos)
    else:
```

```
        sync_error = abs(left_pos - right_pos)

    print(f"Joint {i+1}: 同步误差 = {sync_error:.4f} rad")

# 回零
print(" 回零...")
for i in range(7):
    left_motors[i].set_position(0.0, vel=0.0, kp=kp, kd=kd, torque=0.0)
    right_motors[i].set_position(0.0, vel=0.0, kp=kp, kd=kd, torque=0.0)
    time.sleep(0.05)

time.sleep(5)

# 失能
for m in left_motors + right_motors:
    m.disable()
    time.sleep(0.05)

print(" 双臂镜像测试完成")
```

19.4.2 双臂协同抓取

测试场景: 模拟双臂协同抓取大物体

```
#!/usr/bin/env python3
# 双臂协同抓取测试

import can
from robstride import Motor
import time

bus0 = can.interface.Bus(channel='can0', bustype='socketcan')
bus1 = can.interface.Bus(channel='can1', bustype='socketcan')

# 左右臂电机
```

```
left_arm = [Motor(bus0, i) for i in range(1, 8)]
right_arm = [Motor(bus1, i) for i in range(1, 8)]

# 左右夹爪
left_gripper = Motor(bus0, 8)
right_gripper = Motor(bus1, 8)

# 使能
for m in left_arm + right_arm + [left_gripper, right_gripper]:
    m.enable()
    time.sleep(0.05)

time.sleep(2)

# 阶段 1: 移动到抓取准备位置
print(" 阶段 1: 移动到准备位置...")
left_pre_grasp = [0.3, 0.5, -0.3, -1.0, 0.0, 0.5, 0.0]
right_pre_grasp = [-0.3, 0.5, -0.3, -1.0, 0.0, 0.5, 0.0]

kp, kd = 20.0, 1.0

for i in range(7):
    left_arm[i].set_position(left_pre_grasp[i], vel=0.0, kp=kp, kd=kd, torque=0.0)
    right_arm[i].set_position(right_pre_grasp[i], vel=0.0, kp=kp, kd=kd, torque=0.0)
    time.sleep(0.05)

time.sleep(4)

# 阶段 2: 打开夹爪
print(" 阶段 2: 打开夹爪...")
left_gripper.set_position(0.0, vel=0.0, kp=10.0, kd=0.5, torque=0.0)
right_gripper.set_position(0.0, vel=0.0, kp=10.0, kd=0.5, torque=0.0)
time.sleep(2)
```

```
# 阶段 3: 闭合夹爪 (模拟抓取)
print(" 阶段 3: 闭合夹爪...")
left_gripper.set_position(0.8, vel=0.0, kp=10.0, kd=0.5, torque=0.0)
right_gripper.set_position(0.8, vel=0.0, kp=10.0, kd=0.5, torque=0.0)
time.sleep(2)

# 阶段 4: 抬起 (模拟搬运)
print(" 阶段 4: 抬起物体...")
left_lift = [0.3, 0.2, -0.2, -0.8, 0.0, 0.3, 0.0]
right_lift = [-0.3, 0.2, -0.2, -0.8, 0.0, 0.3, 0.0]

for i in range(7):
    left_arm[i].set_position(left_lift[i], vel=0.0, kp=kp, kd=kd, torque=0.0)
    right_arm[i].set_position(right_lift[i], vel=0.0, kp=kp, kd=kd, torque=0.0)
    time.sleep(0.05)

time.sleep(4)

# 阶段 5: 释放
print(" 阶段 5: 释放物体...")
left_gripper.set_position(0.0, vel=0.0, kp=10.0, kd=0.5, torque=0.0)
right_gripper.set_position(0.0, vel=0.0, kp=10.0, kd=0.5, torque=0.0)
time.sleep(2)

# 回零
print(" 回零...")
for i in range(7):
    left_arm[i].set_position(0.0, vel=0.0, kp=kp, kd=kd, torque=0.0)
    right_arm[i].set_position(0.0, vel=0.0, kp=kp, kd=kd, torque=0.0)
    time.sleep(0.05)

time.sleep(5)

# 失能
```

```
for m in left_arm + right_arm + [left_gripper, right_gripper]:
    m.disable()
    time.sleep(0.05)

print(" 双臂协同抓取测试完成")
```

19.5 性能测试

19.5.1 同步性测试

目标: 评估多电机运动的同步精度

```
#!/usr/bin/env python3
import can
from robstride import Motor
import time
import numpy as np

bus0 = can.interface.Bus(channel='can0', bustype='socketcan')

# 测试肩部 3 个电机
motors = [Motor(bus0, i) for i in range(1, 4)]

for m in motors:
    m.enable()
    time.sleep(0.1)

time.sleep(1)

# 正弦轨迹测试
duration = 10.0  # 10 秒
freq = 0.5  # 0.5 Hz
amplitude = 0.3  # 幅值
rate = 100  # 控制频率 100Hz

kp, kd = 20.0, 1.0
```

```
print(" 开始同步性测试...")
start_time = time.time()

positions = [[], [], []]  # 记录 3 个电机的实际位置

while (time.time() - start_time) < duration:
    t = time.time() - start_time

    # 生成正弦轨迹
    target_pos = amplitude * np.sin(2 * np.pi * freq * t)

    # 同步发送给 3 个电机
    for motor in motors:
        motor.set_position(target_pos, vel=0.0, kp=kp, kd=kd, torque=0.0)

    # 读取反馈
    for i, motor in enumerate(motors):
        fb = motor.get_feedback()
        positions[i].append(fb['position'])

    time.sleep(1.0 / rate)

# 分析同步误差
positions = np.array(positions)  # (3, N)

# 计算电机间的位置差异
sync_error_12 = np.abs(positions[0] - positions[1])
sync_error_13 = np.abs(positions[0] - positions[2])
sync_error_23 = np.abs(positions[1] - positions[2])

print(f" 同步误差统计:")
print(f"Motor 1-2: 平均 ={np.mean(sync_error_12):.4f}, 最大 ={np.max(sync_error_12):.4f} rad")
print(f"Motor 1-3: 平均 ={np.mean(sync_error_13):.4f}, 最大 ={np.max(sync_error_13):.4f} rad")
```

```
print(f"Motor 2-3: 平均 ={np.mean(sync_error_23):.4f}, 最大 ={np.max(sync_error_23):.4f} rad")

# 回零
for motor in motors:
    motor.set_position(0.0, vel=0.0, kp=kp, kd=kd, torque=0.0)

time.sleep(3)

for m in motors:
    m.disable()

print(" 同步性测试完成")
```

评估标准:

优秀: 平均同步误差 <0.01 rad, 最大 <0.05 rad

良好: 平均同步误差 <0.05 rad, 最大 <0.1 rad

一般: 平均同步误差 <0.1 rad, 最大 <0.2 rad

需调整: 同步误差超过上述范围

19.5.2 负载测试

目标: 测试多电机协同承载能力

测试步骤:

1. 准备标准负载(如1kg砝码)
2. 安装在末端执行器
3. 执行标准轨迹
4. 监控电机电流和温度
5. 评估性能衰减

监控脚本:

```
#!/usr/bin/env python3
import can
from robstride import Motor
import time
```

```
bus0 = can.interface.Bus(channel='can0', bustype='socketcan')

motors = [Motor(bus0, i) for i in range(1, 8)]

for m in motors:
    m.enable()
    time.sleep(0.1)

time.sleep(1)

# 执行标准轨迹 (同时监控电流)
target = 0.5
kp, kd = 30.0, 1.5

print(" 开始负载测试...")
for motor in motors:
    motor.set_position(target, vel=0.0, kp=kp, kd=kd, torque=0.0)
    time.sleep(0.05)

time.sleep(3)

# 监控电流和温度
for i in range(10):
    print(f"\n--- 第{i+1}次采样 ---")
    for j, motor in enumerate(motors):
        fb = motor.get_feedback()
        print(f"Joint {j+1}: torque={fb['torque']:.2f} Nm, temp={fb.get('temperature', 'N/A')}°C")

    time.sleep(2)

# 回零
for motor in motors:
    motor.set_position(0.0, vel=0.0, kp=kp, kd=kd, torque=0.0)
    time.sleep(0.05)
```

```
time.sleep(3)

for m in motors:
    m.disable()

print(" 负载测试完成")
```

19.6 测试最佳实践

19.6.1 测试前准备

☐ 检查所有电机零位设置
☐ 验证CAN通信正常
☐ 确认电源电压稳定(48V)
☐ 清空工作空间障碍物
☐ 准备急停措施

19.6.2 测试流程

1. 从单电机开始
   ↓
2. 逐步增加电机数量
   ↓
3. 从小幅度开始
   ↓
4. 逐步提高速度和KP/KD
   ↓
5. 最后测试双臂协同

19.6.3 数据记录

建议记录内容:

```
test_log = {
    "date": "2025-10-19",
    "test_type": "multi_motor_sync",
    "motors": [1, 2, 3],
```

```
    "kp": 20.0,
    "kd": 1.0,
    "trajectory": "sine_wave",
    "sync_error_avg": 0.015,
    "sync_error_max": 0.048,
    "max_temperature": 45.2,
    "notes": " 无异常"
}
```

19.7 常见问题

19.7.1 电机不同步

症状: 多个电机运动不一致

可能原因: - 零位设置不准确 - CAN 总线负载过高 - 电机参数不一致 - 机械阻力差异

解决:

```
#  重新设置零位
python3 set_motor_zeros_setMIT.py

#  降低控制频率
# 修改脚本中的 time.sleep() 增加间隔

#  统一 KP/KD 参数

#  检查机械装配
```

19.7.2 CAN 总线拥塞

症状: 部分电机命令丢失, 运动卡顿

原因: 同时发送过多 CAN 消息

解决:

```
# 方案 1: 增加发送间隔
for motor in motors:
    motor.set_position(...)
```

```
    time.sleep(0.05)  # 50ms 间隔

# 方案 2: 批量发送 (底层支持)
# 待硬件/驱动支持

# 方案 3: 降低控制频率
rate = 50  # 从 100Hz 降到 50Hz
```

19.7.3 某个电机异常

症状: 多电机测试中单个电机行为异常

排查:

```
#  单独测试该电机
python3 test_motor_one.py
# 输入该电机的 CAN 和 ID

#  检查状态
python3 check_motor_status.py

#  查看错误码

#  必要时重新使能或重启电机
```

19.8 进阶测试

19.8.1 插补轨迹测试

使用线性插补生成平滑轨迹:

```
import numpy as np

def linear_interpolation(start, end, duration, rate=100):
    """
    生成线性插补轨迹
    start: 起始位置列表 [pos1, pos2, ...]
    end: 目标位置列表
```

```
    duration: 持续时间 (秒)
    rate: 控制频率 (Hz)
    """
    n_steps = int(duration * rate)
    start = np.array(start)
    end = np.array(end)

    trajectory = []
    for i in range(n_steps):
        alpha = i / (n_steps - 1)
        pos = start + alpha * (end - start)
        trajectory.append(pos)

    return trajectory

# 使用示例
start_pos = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
end_pos = [0.5, 0.3, -0.4, -0.8, 0.2, 0.3, -0.2]

traj = linear_interpolation(start_pos, end_pos, duration=5.0, rate=100)

for point in traj:
    for i, motor in enumerate(motors):
        motor.set_position(point[i], vel=0.0, kp=20.0, kd=1.0, torque=0.0)
        time.sleep(0.01)

    time.sleep(1.0 / 100)  # 100Hz 控制
```

19.8.2 力控协同测试

多电机力矩配合测试:

```
# 测试多电机力矩配合
# 如抓持物体时的力分配
```

```python
kp = 5.0  # 低 KP 实现柔顺
kd = 0.5

target_torque_1 = 2.0  # Nm
target_torque_2 = 2.0
target_torque_3 = 1.5

for motor, torque in zip(motors[:3], [target_torque_1, target_torque_2, target_torque_3]):
    motor.set_position(current_pos, vel=0.0, kp=kp, kd=kd, torque=torque)
    time.sleep(0.05)

# 监控实际力矩
time.sleep(2)
for i, motor in enumerate(motors[:3]):
    fb = motor.get_feedback()
    print(f"Motor {i+1}: 实际力矩 ={fb['torque']:.2f} Nm")
```

19.9 总结

19.9.1 多电机测试要点

✓ 从简单到复杂(单臂→双臂)

✓ 从少到多(3关节→7关节→14关节)

✓ 验证同步性和协调性

✓ 监控温度和电流

✓ 记录测试数据

19.9.2 性能指标

指标	目标值
同步误差	<0.05 rad (平均)
位置精度	±0.02 rad
响应时间	<0.5 s (到达目标)
温度	<60°C (连续运行)

19.9.3 下一步

完成多电机测试后: - **06_ 回零操作** - 学习标准回零流程 - **05_MoveIt 运动规划** - 使用高层规划工具 - **06_ 轨迹录制与回放** - 记录复杂动作

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

20 回零操作

20.1 机器人回零操作详解

将机器人从任意姿态安全平滑地回到零位姿态。

20.2 回零概述

20.2.1 什么是回零

回零 (Homing): 将机器人的所有关节从当前位置运动到预定义的零位姿态。

零位姿态定义:

左臂/右臂:

```
- 肩部关节(1-3): 0 rad (手臂垂直向下)
- 肘部关节(4): 0 rad (肘部伸直)
- 腕部关节(5-7): 0 rad (腕部中立)
- 夹爪(8): 0 rad (打开状态)
```

物理描述:

双臂自然垂直下垂,腕部保持中立位置,夹爪打开

20.2.2 回零的作用

✓ 系统启动后的初始化

✓ 测试前的标准起始姿态

✓ 异常后的安全恢复

✓ 验证零位校准是否准确

✓ 日常操作的参考姿态

20.2.3 回零前提条件

- ☐ CAN接口已启动
- ☐ 电机已使能
- ☐ 零位已正确校准
- ☐ 周围无障碍物
- ☐ 准备急停措施

20.3 使用 control_motor_gohome.py

20.3.1 基本用法

```
cd ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com

# 执行回零
python3 control_motor_gohome.py
```

输出示例:

```
OpenArmX 回零操作
=================
初始化CAN总线...
- can0: OK
- can1: OK

检测到16个电机

开始回零序列...

[阶段1/3] 肩部关节回零 (ID 1-3)
- Left Motor 1: 0.543 → 0.000 rad
- Left Motor 2: 0.321 → 0.000 rad
- Left Motor 3: -0.412 → 0.000 rad
- Right Motor 1: -0.534 → 0.000 rad
- Right Motor 2: 0.315 → 0.000 rad
- Right Motor 3: -0.408 → 0.000 rad
进度: 100% [完成]
```

```
[阶段2/3] 肘腕关节回零 (ID 4-7)
- Left Motor 4: -0.856 → 0.000 rad
...
进度: 100% [完成]

[阶段3/3] 夹爪回零 (ID 8)
- Left Gripper: 0.523 → 0.000 rad
- Right Gripper: 0.518 → 0.000 rad
进度: 100% [完成]

回零完成!
总耗时: 12.5秒
位置误差: <0.01 rad
```

20.3.2 脚本工作流程

内部逻辑:

```
1. 初始化 CAN 总线 (can0, can1)
   ↓
2. 读取当前所有电机位置
   ↓
3. 分阶段回零:
   - 阶段1: 肩部 (ID 1-3)
   - 阶段2: 肘腕 (ID 4-7)
   - 阶段3: 夹爪 (ID 8)
   ↓
4. 每个阶段:
   - 发送目标位置0.0
   - 等待运动完成
   - 验证误差
   ↓
5. 全部完成后报告结果
```

20.3.3 参数说明

默认 KP/KD 参数 (脚本内部):

```
# 肩部 (RS04)
kp_shoulder = 20.0
kd_shoulder = 1.0

# 肘腕 (RS03)
kp_elbow_wrist = 20.0
kd_elbow_wrist = 1.0

# 夹爪 (RS00)
kp_gripper = 10.0
kd_gripper = 0.5
```

速度控制:

```
# 回零速度 (内置)
max_velocity = 0.0  # 由 KP/KD 自然控制

# 超时设置
timeout_per_stage = 10.0  # 每阶段超时 10 秒
```

20.4 分步回零逻辑

20.4.1 为什么分阶段

原因:

1. 避免关节间干涉
   - 先回肩部避免碰撞
   - 再回肘部避免奇异位形
   - 最后回夹爪

2. 降低CAN总线负载
   - 分批发送命令
   - 避免消息拥塞

3. 便于监控和调试
   - 逐步验证
   - 发现问题易定位

20.4.2 回零顺序

标准顺序:

```
阶段1: 肩部3关节 (ID 1-3)
        ↓
阶段2: 肘部1关节 + 腕部3关节 (ID 4-7)
        ↓
阶段3: 夹爪 (ID 8)
```

示意图:

```
初始姿态(任意)
    ↓
[阶段1] 肩部回正,手臂大致垂直
    ↓
[阶段2] 肘部伸直,腕部归中
    ↓
[阶段3] 夹爪打开
    ↓
零位姿态(标准)
```

20.4.3 平滑运动

脚本实现平滑性的方法:

```
#  使用适中的 KP/KD(不过大)
kp = 20.0  # 不是 100, 避免震荡

#  MIT 模式自然加减速
# 电机内部会平滑加减速

#  分阶段等待
```

```
time.sleep(stage_duration)
# 确保上一阶段完成再进行下一阶段
```

20.5 手动实现回零

20.5.1 单臂手动回零

如果脚本不可用，手动实现：

```
#!/usr/bin/env python3
"""
手动回零脚本示例
"""
import can
from robstride import Motor
import time

# 初始化 CAN 总线
bus0 = can.interface.Bus(channel='can0', bustype='socketcan')

# 创建左臂 8 个电机对象
left_motors = [Motor(bus0, i) for i in range(1, 9)]

# 确认电机已使能
print(" 确保电机已使能 (en_all_motors.py)")
input(" 按回车继续...")

# 读取当前位置
print("\n当前位置:")
for i, motor in enumerate(left_motors):
    fb = motor.get_feedback()
    print(f"Motor {i+1}: {fb['position']:.3f} rad")

input("\n按回车开始回零...")

# 阶段 1: 肩部回零
```

```
print("\n[阶段 1] 肩部回零...")
kp, kd = 20.0, 1.0
for motor in left_motors[:3]:
    motor.set_position(0.0, vel=0.0, kp=kp, kd=kd, torque=0.0)
    time.sleep(0.1)

time.sleep(5)

# 阶段 2: 肘腕回零
print("[阶段 2] 肘腕回零...")
for motor in left_motors[3:7]:
    motor.set_position(0.0, vel=0.0, kp=kp, kd=kd, torque=0.0)
    time.sleep(0.1)

time.sleep(5)

# 阶段 3: 夹爪回零
print("[阶段 3] 夹爪回零...")
kp_gripper, kd_gripper = 10.0, 0.5
left_motors[7].set_position(0.0, vel=0.0, kp=kp_gripper, kd=kd_gripper, torque=0.0)

time.sleep(3)

# 验证
print("\n回零后位置:")
max_error = 0.0
for i, motor in enumerate(left_motors):
    fb = motor.get_feedback()
    error = abs(fb['position'] - 0.0)
    max_error = max(max_error, error)
    print(f"Motor {i+1}: {fb['position']:.4f} rad (误差: {error:.4f})")

print(f"\n最大误差: {max_error:.4f} rad")
```

```
if max_error < 0.05:
    print("✓ 回零成功!")
else:
    print(" 误差较大，可能需要重新校准零位")

print("\n回零完成")
```

运行:

```
python3 manual_homing.py
```

20.5.2 双臂同步回零

```
#!/usr/bin/env python3
"""
双臂同步回零
"""
import can
from robstride import Motor
import time

# 双 CAN 总线
bus0 = can.interface.Bus(channel='can0', bustype='socketcan')
bus1 = can.interface.Bus(channel='can1', bustype='socketcan')

# 左右臂
left_motors = [Motor(bus0, i) for i in range(1, 9)]
right_motors = [Motor(bus1, i) for i in range(1, 9)]

print("OpenArmX 双臂同步回零\n")

# 阶段 1: 双臂肩部
print("[阶段 1] 双臂肩部回零...")
kp, kd = 20.0, 1.0
```

```
for motor in left_motors[:3] + right_motors[:3]:
    motor.set_position(0.0, vel=0.0, kp=kp, kd=kd, torque=0.0)
    time.sleep(0.05)

time.sleep(5)

# 阶段 2: 双臂肘腕
print("[阶段 2] 双臂肘腕回零...")
for motor in left_motors[3:7] + right_motors[3:7]:
    motor.set_position(0.0, vel=0.0, kp=kp, kd=kd, torque=0.0)
    time.sleep(0.05)

time.sleep(5)

# 阶段 3: 双夹爪
print("[阶段 3] 双夹爪回零...")
kp_g, kd_g = 10.0, 0.5
left_motors[7].set_position(0.0, vel=0.0, kp=kp_g, kd=kd_g, torque=0.0)
right_motors[7].set_position(0.0, vel=0.0, kp=kp_g, kd=kd_g, torque=0.0)

time.sleep(3)

# 验证双臂同步性
print("\n双臂位置验证:")
for i in range(8):
    left_fb = left_motors[i].get_feedback()
    right_fb = right_motors[i].get_feedback()

    left_pos = left_fb['position']
    right_pos = right_fb['position']

    sync_error = abs(left_pos - right_pos)

    print(f"Motor {i+1}: L={left_pos:.4f}, R={right_pos:.4f}, 差异 ={sync_error:.4f} rad")
```

```
print("\n双臂回零完成")
```

20.6 回零中的安全考虑

20.6.1 避免碰撞

潜在风险:

- ☐ 双臂运动时相互碰撞
- ☐ 机械臂碰到底座
- ☐ 与桌面或其他物体碰撞

预防措施:

```
# 检查初始位置
current_positions = [motor.get_feedback()['position'] for motor in motors]

# 如果某关节超出安全范围，先单独处理
for i, pos in enumerate(current_positions):
    if abs(pos) > 3.0:  # 超过安全限制
        print(f" 警告: Motor {i+1} 位置异常: {pos:.2f} rad")
        # 可以选择中止或小心回零

# 分阶段回零 (已实现)

# 使用保守的 KP/KD
kp = 15.0  # 比常规 20.0 更小
kd = 0.8
```

20.6.2 超时保护

```
import time

def safe_homing_with_timeout(motor, timeout=10.0):
    """
    带超时的回零
    """
```

```
    start_time = time.time()

    motor.set_position(0.0, vel=0.0, kp=20.0, kd=1.0, torque=0.0)

    while (time.time() - start_time) < timeout:
        fb = motor.get_feedback()
        error = abs(fb['position'] - 0.0)

        if error < 0.05:  # 到达目标
            print(f"Motor {motor.motor_id} 回零完成")
            return True

        time.sleep(0.1)

    # 超时
    print(f" 警告: Motor {motor.motor_id} 回零超时!")
    return False
```

20.6.3 紧急停止

回零过程中如何急停:

```
import signal
import sys

# 注册信号处理
def signal_handler(sig, frame):
    print("\n紧急停止!")

    # 立即失能所有电机
    for motor in all_motors:
        motor.disable()

    sys.exit(0)
```

```
signal.signal(signal.SIGINT, signal_handler)

# 开始回零
print(" 开始回零 (Ctrl+C 急停)")
# ... 回零代码 ...
```

操作员应对:

- ☐ 手放在键盘Ctrl+C
- ☐ 准备断开电源
- ☐ 观察运动轨迹
- ☐ 发现异常立即中断

20.7 验证回零结果

20.7.1 位置精度检查

```
# 回零后验证
print("\n验证回零精度...")

all_ok = True

for i, motor in enumerate(motors):
    fb = motor.get_feedback()
    pos = fb['position']
    error = abs(pos - 0.0)

    status = "✓" if error < 0.05 else ""
    print(f"{status} Motor {i+1}: {pos:.4f} rad (误差: {error:.4f})")

    if error >= 0.05:
        all_ok = False

if all_ok:
    print("\n✓ 所有电机回零成功, 误差 <0.05 rad")
else:
```

```
    print("\n 部分电机误差较大，建议:")
    print("  1. 检查零位校准")
    print("  2. 检查机械装配")
    print("  3. 重新执行 set_motor_zeros_setMIT.py")
```

20.7.2 视觉检查

标准零位姿态特征:

- ☐ 双臂自然垂直下垂(与重力方向一致)
- ☐ 肘部完全伸直(无弯曲)
- ☐ 腕部保持中立位置(不偏转)
- ☐ 夹爪处于打开状态
- ☐ 左右对称(镜像)

不正确的表现:

手臂向前或向后倾斜

肘部有明显弯曲

腕部扭曲

夹爪闭合

左右不对称

20.7.3 重复性测试

```
# 测试回零重复性
import numpy as np

def test_homing_repeatability(motors, n_trials=5):
    """
    测试多次回零的重复性
    """
    positions_history = []

    for trial in range(n_trials):
        print(f"\n第{trial+1}次回零...")
```

```
        # 先移动到随机位置
        for motor in motors:
            random_pos = np.random.uniform(-0.5, 0.5)
            motor.set_position(random_pos, vel=0.0, kp=20.0, kd=1.0, torque=0.0)
            time.sleep(0.1)

        time.sleep(3)

        # 执行回零
        for motor in motors:
            motor.set_position(0.0, vel=0.0, kp=20.0, kd=1.0, torque=0.0)
            time.sleep(0.1)

        time.sleep(5)

        # 记录位置
        positions = [motor.get_feedback()['position'] for motor in motors]
        positions_history.append(positions)

        print(f" 位置: {[f'{p:.4f}' for p in positions]}")

    # 分析重复性
    positions_history = np.array(positions_history)  # (n_trials, n_motors)

    print("\n重复性分析:")
    for i in range(len(motors)):
        std = np.std(positions_history[:, i])
        print(f"Motor {i+1}: 标准差 ={std:.5f} rad")

    overall_std = np.mean(np.std(positions_history, axis=0))
    print(f"\n总体重复性: {overall_std:.5f} rad")

    if overall_std < 0.01:
        print("✓ 重复性优秀")
```

```
    elif overall_std < 0.05:
        print("✓ 重复性良好")
    else:
        print(" 重复性较差，需检查")

# 运行测试
test_homing_repeatability(left_motors[:3], n_trials=5)
```

20.8 常见问题

20.8.1 回零后位置偏差大

症状: 回零完成后，实际位置误差 >0.1 rad

原因: - 零位校准不准确 - 机械装配松动 - 编码器漂移

解决:

```
#  重新校准零位
# 手动调整到标准零位姿态
python3 set_motor_zeros_setMIT.py

#  断电重启电机

#  重新回零验证
python3 control_motor_gohome.py
```

20.8.2 回零过程中卡住

症状: 某个关节回零时停滞不动

排查:

```
#  检查电机状态
python3 check_motor_status.py
# 查看错误码

#  检查机械阻力
# 手动推动关节，感受阻力
```

```
#  检查电源
# 确认 48V 稳定

#  单独测试该电机
python3 test_motor_one.py
```

20.8.3 回零速度过快/过慢

调整 KP/KD:

```
# 速度过快 (震荡, 冲击)
kp = 10.0  # 降低 KP
kd = 0.5   # 降低 KD

# 速度过慢 (拖沓)
kp = 30.0  # 提高 KP
kd = 1.5   # 提高 KD

# 推荐平衡值
kp = 20.0
kd = 1.0
```

修改脚本:

```
# 编辑 control_motor_gohome.py
# 找到 KP/KD 定义行, 修改数值

kp_shoulder = 20.0  # 修改为期望值
kd_shoulder = 1.0
```

20.9 回零的最佳实践

20.9.1 日常使用

开机流程:

1. 上电(48V)
   ↓

2. 启动CAN (en_all_can.py)
   ↓
3. 使能电机 (en_all_motors.py)
   ↓
4. 回零 (control_motor_gohome.py)
   ↓
5. 开始工作

关机流程:

1. 回零 (可选,推荐)
   ↓
2. 失能电机 (dis_all_motors.py)
   ↓
3. 关闭CAN (dis_all_can.py)
   ↓
4. 断电

20.9.2 定期验证

建议频率:

- ☐ 每天首次使用前回零一次
- ☐ 长时间运行后(如2小时)回零验证
- ☐ 发生异常后必须回零检查
- ☐ 更换零部件后重新校准并回零

20.9.3 记录与维护

```
# 回零记录模板
homing_log = {
    "date": "2025-10-19",
    "time": "14:30:25",
    "operator": " 张三",
    "max_error": 0.023,  # rad
    "duration": 12.5,    # 秒
    "status": "success",
```

```
    "notes": " 无异常"
}

# 保存到日志文件
import json
with open("homing_log.json", "a") as f:
    f.write(json.dumps(homing_log, ensure_ascii=False) + "\n")
```

20.10 总结

20.10.1 回零要点

✓ 分阶段执行(肩→肘腕→夹爪)
✓ 使用适中的KP/KD参数
✓ 每次启动后必须回零
✓ 回零后验证精度
✓ 保持回零记录

20.10.2 安全检查清单

回零前:
☐ CAN通信正常
☐ 电机已使能
☐ 周围无障碍物
☐ 准备急停

回零后:
☐ 位置误差<0.05 rad
☐ 双臂对称
☐ 视觉检查姿态正确
☐ 无异响或震动

20.10.3 下一步

完成回零操作学习后: - **05_MoveIt 运动规划** - 使用高级运动规划工具 - **06_ 轨迹录制与回放** - 录制和回放复杂轨迹 - **07_ 控制器配置** - 深入了解控制参数

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

21 启动 MoveIt

21.1 MoveIt 运动规划系统启动

本文档说明如何启动 MoveIt 进行可视化运动规划。

21.2 MoveIt 简介

21.2.1 什么是 MoveIt

MoveIt 是 ROS 生态中最流行的机器人运动规划框架, 提供: - [√] 可视化操作界面 (RViz) - [√] 逆运动学求解 - [√] 路径规划 (OMPL) - [√] 碰撞检测 - [√] 轨迹执行

21.2.2 OpenArmX 的 MoveIt 配置

```
openarm_moveit_config/
├── config/
│   ├── openarm_v10.srdf          # 语义机器人描述
│   ├── kinematics.yaml           # 运动学配置
│   ├── joint_limits.yaml         # 关节限制
│   ├── ompl_planning.yaml        # OMPL规划器
│   └── moveit_controllers.yaml   # 控制器配置
└── launch/
    └── demo.launch.py            # 演示启动文件
```

21.3 启动 MoveIt Demo

21.3.1 基本启动

```
# 确保已 source 工作空间
source ~/openarmx_robotstride_ws-cc/install/setup.bash

# 启动 MoveIt Demo
ros2 launch openarm_moveit_config demo.launch.py
```

预期现象: - RViz 窗口打开 - 显示 OpenArmX 双臂机器人模型 - 左侧有 MoveIt 控制面板

21.3.2 启动参数

```
# 使用 fake 硬件 (仅仿真, 不连接实际电机)
ros2 launch openarm_moveit_config demo.launch.py use_fake_hardware:=true

# 连接实际硬件
ros2 launch openarm_moveit_config demo.launch.py use_fake_hardware:=false
```

21.3.3 启动检查

```
# 检查 MoveIt 节点
ros2 node list | grep move_group

# 应该看到:
# /move_group

# 检查话题
ros2 topic list | grep -E "joint_states|trajectory"

# 检查服务
ros2 service list | grep move_group
```

21.4 RViz 界面布局

21.4.1 主要组件

```
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│  RViz - OpenArmX MoveIt                                                                      │
├──────────────────┬──────────────────────────────────────────────────────────────────────────┤
│                  │                                                                          │
│ MoveIt           │                    3D 显示区域                                           │
│ 控制面板          │                                                                          │
│                  │                [机器人模型]                                              │
│ Planning         │                                                                          │
│ Scene            │                                                                          │
│ Joints           │                                                                          │
```

```
│          │                                │
├──────────────────┴──────────────────────────────────────────────────────┤
│  Terminal / Console Output                    │
└──────────────────────────────────────────────────────────────────────────┘
```

21.4.2 MoveIt 控制面板

主要选项卡: - **Planning**: 运动规划 - **Scene**: 场景编辑 - **Joints**: 关节控制

21.4.3 3D 显示设置

显示项 (Displays):

```
☐ RobotModel        - 机器人模型
☐ MotionPlanning    - 运动规划
  ├── Planned Path   - 规划路径
  ├── Planning Request - 规划请求
  └── Robot Interaction - 交互标记
☐ TF                - 坐标系
☐ Grid              - 网格
```

21.5 Planning Group 选择

21.5.1 可用的 Planning Group

OpenArmX 定义了以下规划组:

Group 名称	包含关节	用途
`left_arm`	left_joint_1~7	左臂运动规划
`right_arm`	right_joint_1~7	右臂运动规划
`left_gripper`	left_gripper_joint	左夹爪控制
`right_gripper`	right_gripper_joint	右夹爪控制

21.5.2 选择 Planning Group

在 MoveIt 面板中:

```
Planning -> Planning Group: [下拉菜单]
选择: left_arm 或 right_arm
```

21.6 基本运动规划

21.6.1 使用交互标记

步骤:

1. 在Planning面板选择Planning Group (如left_arm)

2. 在3D视图中会出现交互球和箭头标记

3. 拖动标记到目标位置:
   - 拖动球: 移动末端
   - 拖动箭头: 沿轴移动
   - 拖动环: 旋转

4. 点击"Plan"按钮生成轨迹

5. 检查规划的路径(橙色显示)

6. 如果满意,点击"Execute"执行

21.6.2 使用随机目标

1. 点击"Planning"标签下的"Select Random Valid Goal"

2. 系统自动生成一个有效的随机目标

3. 点击"Plan"规划

4. 点击"Execute"执行

21.6.3 使用预定义姿态

1. 在"Goal State"下拉菜单选择预定义姿态
   - <current>: 当前位置
   - home: 零位姿态
   - (其他自定义姿态)

2. 点击"Update"更新目标

3. 点击"Plan"规划轨迹

21.7 参数调整

21.7.1 规划器选择

```
# 在 Planning 面板:
Planning Library: OMPL

# 选择不同的规划算法:
- RRTConnect (默认, 快速)
- RRT
- PRM
- BKPIECE
- etc.
```

21.7.2 速度缩放

Planning -> Velocity Scaling Factor: [滑块]

范围: 0.0 ~ 1.0
- 0.1: 10%速度(安全测试)
- 0.5: 50%速度(常规)
- 1.0: 100%速度(最快)

[警告] 建议: - 首次测试使用 0.1-0.3 - 验证轨迹安全后提高速度

21.7.3 加速度缩放

Planning -> Acceleration Scaling Factor: [滑块]

范围: 0.0 ~ 1.0
建议与速度缩放同步

21.8 安全注意事项

21.8.1 规划前检查

- ☐ 确认Planning Group正确
- ☐ 检查当前机器人状态
- ☐ 观察目标位置是否合理
- ☐ 确认速度缩放设置保守
- ☐ 周围无障碍物

21.8.2 执行前检查

- ☐ 仔细查看规划的轨迹(橙色路径)
- ☐ 确认无碰撞风险
- ☐ 确认运动幅度合理
- ☐ 手放在急停按钮上
- ☐ 准备随时中断

21.8.3 执行中监控

- ☐ 观察实际运动轨迹
- ☐ 监听异常声音
- ☐ 注意温度变化
- ☐ 发现异常立即急停

21.9 常见问题

21.9.1 MoveIt 无法启动

症状: launch 失败

排查:

```
# 检查工作空间编译
cd ~/openarmx_robotstride_ws-cc
colcon build --packages-select openarm_moveit_config

# 重新 source
source install/setup.bash
```

```
# 检查依赖
rosdep install --from-paths src --ignore-src -y

# 查看错误日志
ros2 launch openarm_moveit_config demo.launch.py --debug
```

21.9.2 规划失败

症状: 点击 “Plan” 后显示失败

可能原因: - 目标位置超出关节限制 - 目标位置导致自碰撞 - 逆运动学无解

解决:

1. 调整目标位置(拖动标记)
2. 尝试不同的随机目标
3. 检查关节限制配置
4. 尝试不同的规划器

21.9.3 执行失败

症状: 规划成功但执行失败

排查:

```
# 检查控制器状态
ros2 control list_controllers

# 应该看到:
# left_arm_controller[active]
# right_arm_controller[active]

# 检查硬件连接
# - CAN 接口是否启动
# - 电机是否使能
# - 电源是否正常

# 查看日志
ros2 topic echo /move_group/result
```

21.10 Demo 模式 vs 实际硬件

21.10.1 Demo 模式 (仿真)

```
# 启动 Demo 模式
ros2 launch openarm_moveit_config demo.launch.py use_fake_hardware:=true
```

特点: - [√] 不需要实际硬件 - [√] 快速测试规划算法 - [√] 安全学习操作 - [×] 无法验证实际运动

21.10.2 实际硬件模式

```
# 启动实际硬件模式
ros2 launch openarm_moveit_config demo.launch.py use_fake_hardware:=false
```

前提条件:

☐ CAN接口已启动
☐ 电机已使能
☐ 零位已设置
☐ ros2_control已加载

启动流程:

```
# Terminal 1: 启动 CAN 和电机
cd ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com
python3 en_all_can.py
python3 en_all_motors.py

# Terminal 2: 启动 MoveIt
ros2 launch openarm_moveit_config demo.launch.py use_fake_hardware:=false
```

21.11 高级选项

21.11.1 添加场景对象

1. 切换到"Scene"标签

2. 点击"Scene Objects"

3. 添加碰撞对象:
   - Box: 长方体
   - Sphere: 球体
   - Cylinder: 圆柱体
   - Mesh: 网格模型

4. 设置位置和尺寸

5. MoveIt会自动避障规划

21.11.2 保存规划轨迹

```
# 在Terminal中记录话题
ros2 bag record /move_group/display_planned_path

# 回放
ros2 bag play <bag文件>
```

21.11.3 自定义配置

```
# 编辑运动学配置
vim ~/openarmx_robotstride_ws-cc/src/openarm_moveit_config/config/kinematics.yaml

# 编辑关节限制
vim ~/openarmx_robotstride_ws-cc/src/openarm_moveit_config/config/joint_limits.yaml

# 重新编译
colcon build --packages-select openarm_moveit_config
```

21.12 总结

21.12.1 快速启动流程

```
# Source 工作空间
source ~/openarmx_robotstride_ws-cc/install/setup.bash

# 启动 MoveIt (Demo 模式)
ros2 launch openarm_moveit_config demo.launch.py

# 选择 Planning Group

# 拖动交互标记

# Plan -> Execute
```

21.12.2 最佳实践

✓ 从Demo模式开始熟悉
✓ 首次使用低速度缩放(0.1-0.3)
✓ 仔细检查规划轨迹
✓ 逐步提高速度
✓ 手放在急停按钮上

21.12.3 下一步

学习完 MoveIt 启动后, 继续: - **02_RViz 界面操作** - 详细的界面使用 - **03_ 单臂运动规划** - 单臂轨迹规划技巧 - **04_ 双臂独立控制** - 协调左右臂运动

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

22 RViz 界面操作

22.1 RViz 可视化界面详细操作指南

深入了解 RViz 界面各组件的使用方法。

22.2 RViz 界面概览

22.2.1 主界面布局

```
│  File  Panels  View  Help                          │
├──────────────────────┬──────────────────────────────────────────────────────────
│            │                                        │
│  Displays  │                                        │
│  面板      │          3D 视图区域                    │
│            │                                        │
│  MotionPlanning                                    │
│  面板      │          [机器人模型]                   │
│            │                                        │
│  - Planning│                                        │
│  - Scene   │                                        │
│  - Joints  │                                        │
│            │                                        │
│            │                                        │
├──────────────────────┴──────────────────────────────────────────────────────────
│  Console: [log消息]                                │
└──────────────────────────────────────────────────────────────────────────────────
```

22.2.2 主要区域功能

区域	功能
Displays 面板	显示项管理 (开关、配置)
MotionPlanning 面板	运动规划控制
3D 视图	机器人和场景可视化
Console	日志和状态输出
工具栏	视图控制、交互工具

22.3 Displays 面板详解

22.3.1 显示项结构

```
Displays
├── Global Options
│   └── Fixed Frame: base_link
├── Grid
│   ├── Plane: XY
```

```
│   └── Cell Size: 0.5m
├── RobotModel
│   ├── Robot Description: robot_description
│   ├── Visual Enabled: ✓
│   └── Collision Enabled: □
├── TF
│   ├── Frames: [显示所有坐标系]
│   └── Show Names: ✓
└── MotionPlanning
    ├── Planning Request: ✓
    ├── Planned Path: ✓
    │   ├── Trail: ✓ (显示轨迹)
    │   └── Loop Animation: ✓
    ├── Robot Interaction: ✓
    └── Scene Robot
        ├── Show Robot Visual: ✓
        └── Alpha: 1.0
```

22.3.2 常用显示项配置

Grid (网格):

作用: 提供空间参考

配置:

- Plane Cell Count: 20
- Cell Size: 0.5 m
- Color: 浅灰色
- Line Style: Lines

RobotModel (机器人模型):

作用: 显示机器人当前状态

配置:

- Robot Description: robot_description
- Visual Enabled: ✓ (显示外观)
- Collision Enabled: □ (隐藏碰撞体)
- Alpha: 1.0 (不透明)

TF (坐标系):

作用: 显示坐标变换关系

配置:

- Show Axes: ✓ (显示坐标轴)
- Show Names: ✓ (显示名称)
- Marker Scale: 0.3 (轴长度)
- Update Interval: 0.05 s

重要坐标系:

- base_link: 机器人基座
- left_link_7: 左臂末端
- right_link_7: 右臂末端
- world: 世界坐标系

MotionPlanning:

作用: 运动规划可视化

重要子项:

- Planned Path: 显示规划的轨迹(橙色)
- Planning Request: 显示目标姿态
- Robot Interaction: 交互标记(球和箭头)

22.3.3 显示项开关

快速开关: - 点击显示项左侧的复选框 - ✓ 启用 - ☐ 禁用

推荐配置:

正常使用:

✓ Grid

✓ RobotModel

✓ MotionPlanning

☐ TF (初次使用可开启学习)

调试时:

✓ Grid

✓ RobotModel

✓ MotionPlanning

✓ TF (查看坐标系)

☐ Grid (可选关闭,减少干扰)

22.4 MotionPlanning 面板详解

22.4.1 Planning 标签

22.4.1.1 Planning Group

```
┌──────────────────────────────────────────────────┐
│ Planning Group:              │
│ [left_arm ▼]                 │
└──────────────────────────────────────────────────┘
```

可选项:

- left_arm: 左臂7关节
- right_arm: 右臂7关节
- left_gripper: 左夹爪
- right_gripper: 右夹爪

作用: 选择要规划的关节组

22.4.1.2 Query 区域

```
┌──────────────────────────────────────────────────────────────┐
│ Query                            │
│                                  │
│  ┌── Goal State ──────────────────────────────┐  │
│  │ <current> ▼                  │  │
│  │ [Update] [Random Valid Goal]│  │
│  └──────────────────────────────────────────────────┘  │
│                                  │
│  ┌── Start State ─────────────────────────────┐  │
│  │ <current>                    │  │
│  │ [Update]                     │  │
│  └──────────────────────────────────────────────────┘  │
└──────────────────────────────────────────────────────────────┘
```

Goal State: 目标状态 - 下拉菜单选择预定义姿态 - 或使用交互标记手动设置

Start State: 起始状态 - 通常使用 `<current>` (当前位置)

按钮: - `Update`: 更新状态 - `Random Valid Goal`: 生成随机有效目标

22.4.1.3 Planning Library

```
┌──────────────────────────────────────┐
│ Planning Library      │
│ [OMPL ▼]              │
│                       │
│ Planning Algorithm:   │
│ [RRTConnect ▼]        │
└──────────────────────────────────────┘
```

```
OMPL算法:
- RRTConnect (默认,快速)
- RRT
- RRTstar (优化路径)
- PRM
- BKPIECE
- LBKPIECE
- EST
- KPIECE
```

推荐: - **RRTConnect**: 快速, 适合大部分场景 - **RRTstar**: 路径更优, 但速度慢 - **PRM**: 多次规划, 环境不变

22.4.1.4 Planning 参数

```
┌──────────────────────────────────────────────────────────┐
│ Planning Time (s): 5.0          │
│ [滑块: 1.0 ──────────●──────── 30.0]     │
│                                 │
│ Planning Attempts: 10           │
│                                 │
│ Velocity Scaling: 0.1           │
```

```
│ [滑块: 0.0 ●──────────────── 1.0]    │
│                                      │
│ Acceleration Scaling: 0.1            │
│ [滑块: 0.0 ●──────────────── 1.0]    │
└──────────────────────────────────────┘
```

参数说明: - **Planning Time**: 规划超时 (秒) - 简单场景: 1-5 秒 - 复杂场景: 10-30 秒

- **Planning Attempts**: 规划尝试次数
  - 默认: 10
  - 增加可提高成功率, 但耗时更长
- **Velocity Scaling**: 速度缩放
  - 0.1: 10% 速度 (安全测试)
  - 0.5: 50% 速度 (常规)
  - 1.0: 100% 速度 (最大)
- **Acceleration Scaling**: 加速度缩放
  - 建议与 Velocity Scaling 同步

[警告] 安全建议:

首次使用: `Velocity=0.1, Acceleration=0.1`

熟悉后: `Velocity=0.3, Acceleration=0.3`

验证安全后: `Velocity=0.5-1.0`

22.4.1.5 控制按钮

```
┌──────────────────────────────────┐
│ [Plan]                           │
│ [Execute]                        │
│ [Plan & Execute]                 │
└──────────────────────────────────┘
```

按钮功能: - **Plan**: 仅规划轨迹, 不执行 - 显示橙色轨迹预览 - 可以检查是否合理

- **Execute**: 执行已规划的轨迹
  - 需要先 Plan 成功
- **Plan & Execute**: 规划并立即执行
  - 一键操作
  - [警告] 谨慎使用, 确保安全

使用流程:

```
1. 调整目标位置(拖动交互标记)
   ↓
2. 点击 [Plan]
   ↓
3. 观察橙色轨迹
   ↓
4. 确认无问题
   ↓
5. 点击 [Execute]
```

22.4.2 Scene 标签

场景编辑功能:

```
┌──────────────────────────────────────────────────────────┐
│ Scene Objects                  │
│                                │
│ [Add Object ▼]                 │
│   ├── Box                      │
│   ├── Sphere                   │
│   ├── Cylinder                 │
│   └── Mesh                     │
│                                │
│ 已添加对象:                     │
│ □ table (Box)                  │
│ □ obstacle_1 (Cylinder)        │
│                                │
│ [Publish Scene]                │
│ [Save Scene]                   │
│ [Load Scene]                   │
└──────────────────────────────────────────────────────────┘
```

添加障碍物示例:

1. **添加桌子:**

```
点击 [Add Object] → Box
Name: table
Size: X=1.0, Y=0.6, Z=0.05 m
Position: X=0.5, Y=0.0, Z=0.4 m
点击 [Add]
```

2. **添加圆柱障碍物:**

```
点击 [Add Object] → Cylinder
Name: obstacle_1
Radius: 0.1 m
Height: 0.5 m
Position: X=0.4, Y=0.3, Z=0.5 m
点击 [Add]
```

3. **发布场景:**

```
点击 [Publish Scene]
# MoveIt会自动避开障碍物规划
```

场景保存/加载:

```
# 保存场景
点击 [Save Scene]
# 保存为 .scene 文件

# 加载场景
点击 [Load Scene]
# 选择 .scene 文件
```

22.4.3 Joints 标签

关节滑块控制:

```
┌──────────────────────────────────────────────────────────────┐
│ left_joint_1                                                 │
│ [-3.14 ────●──────────────────────────── 3.14] rad           │
│                                                              │
│ left_joint_2                                                 │
│ [-1.57 ──●──────────────────────── 1.57] rad                 │
```

```
│                                    │
│  left_joint_3                      │
│  [-3.14 ──────────────●──────── 3.14] rad     │
│                                    │
│  ... (其他关节)                     │
│                                    │
│  [Plan to Target]                  │
└────────────────────────────────────────────────────────────────────┘
```

使用方法:

1. 拖动各关节滑块到期望位置
2. 点击 [Plan to Target]
3. 规划到该关节配置
4. 点击 [Execute] 执行

适用场景: - 精确关节位置控制 - 避免逆运动学奇异 - 学习关节空间规划

22.5 3D 视图操作

22.5.1 视角控制

鼠标操作:

旋转视角:
- 左键拖动: 绕中心旋转

平移视角:
- 中键(滚轮)拖动: 平移视图
- Shift + 左键拖动: 平移视图

缩放:
- 滚轮滚动: 缩放
- Ctrl + 左键拖动: 缩放

键盘快捷键:

F: Focus on Selection (聚焦到选中物体)
G: Grid开关
R: 重置视角

Z: 切换到俯视图

22.5.2 交互标记使用

标记类型:

```
┌──────────────────────────────────────┐
│                                      │
│      [球形标记]                      │
│    位置控制                          │
│                                      │
│    ↑ [箭头标记]                      │
│    轴向移动                          │
│                                      │
│      [环形标记]                      │
│    旋转控制                          │
│                                      │
└──────────────────────────────────────┘
```

操作方法:

1. **位置移动:**

拖动球形标记:
- 自由移动末端位置
- 逆运动学自动计算关节角度

2. **轴向移动:**

拖动红色箭头: X轴移动
拖动绿色箭头: Y轴移动
拖动蓝色箭头: Z轴移动

3. **旋转控制:**

拖动红色环: 绕X轴旋转
拖动绿色环: 绕Y轴旋转
拖动蓝色环: 绕Z轴旋转

技巧:

精确控制:

- 拖动时观察状态栏数值
- 小幅度拖动,逐步调整
- 配合不同视角操作

避免奇异位形:
- 不要将关节拖到极限位置
- 避免肘部完全伸直
- 保持冗余自由度

22.5.3 轨迹可视化

已规划轨迹显示:

颜色:
- 橙色: 规划的路径
- 绿色: 当前位置
- 半透明: 轨迹上的中间状态

轨迹播放:
- 自动循环播放(如果启用Loop Animation)
- 显示运动过程

配置:

```
Displays → MotionPlanning → Planned Path
   ├── Show Trail: ✓ (显示轨迹)
   ├── Trail Step Size: 1 (轨迹点间隔)
   ├── Loop Animation: ✓ (循环播放)
   └── Robot Alpha: 0.5 (透明度)
```

22.6 实用操作技巧

22.6.1 快速规划流程

标准流程:

1. 选择Planning Group (如left_arm)
2. 拖动交互标记到目标位置
3. 观察机器人实时预览(白色/半透明)

4. 点击 [Plan]
5. 检查橙色轨迹
6. 确认无碰撞无异常
7. 点击 [Execute]

快捷流程 (熟练后):

1. 选择Group
2. 拖动标记
3. 点击 [Plan & Execute]

22.6.2 避免规划失败

常见失败原因及对策:

失败原因	现象	解决方法
目标超出工作空间	“No IK solution”	将目标移近基座
关节超限	“Joint limits violated”	检查 Joint 标签限位
自碰撞	“State in collision”	调整姿态避免碰撞
逆运动学无解	“IK failed”	旋转末端姿态或改变接近角度
超时	“Timeout”	增加 Planning Time 或简化目标

提高成功率:

- ☐ 使用Random Valid Goal测试系统
- ☐ 从当前位置附近开始
- ☐ 避免极限位置
- ☐ 增加Planning Attempts
- ☐ 使用不同的规划算法

22.6.3 保存和恢复配置

保存 RViz 配置:

File → Save Config As...

保存为: openarmx_moveit.rviz

下次启动:

```
rviz -d openarmx_moveit.rviz
```

保存常用姿态:

```
MotionPlanning → Planning → Goal State
当前姿态 → [Save as named pose]
输入名称: "pick_pose"
下次可直接选择使用
```

22.7 高级功能

22.7.1 多规划组协同

场景: 同时规划左右臂

方法:

```
1. 规划左臂:
   - Planning Group: left_arm
   - 设置目标
   - Plan & Execute

2. 规划右臂:
   - Planning Group: right_arm
   - 设置目标
   - Plan & Execute

注意:
- 每次只能规划一个Group
- 需要分别执行
```

同步运动 (需编程):

```
# 需要通过 Python 或 C++ API 实现
# 参考开发者指南
```

22.7.2 路径优化

启用路径平滑:

```
Planning → Planning Library → OMPL
勾选: Simplify Solution
```

效果:

- 移除冗余路径点
- 优化轨迹平滑度
- 减少执行时间

使用优化算法:

```
Planning Algorithm: RRTstar
或: PRMstar
```

特点:

- 路径更短
- 更平滑
- 但规划时间更长

22.7.3 碰撞检查配置

调整碰撞检测:

```
Scene → 右键某对象 → Properties
- Collision Enabled: ✓/☐
- Collision Padding: 0.01 m (安全距离)
```

自碰撞:

```
Planning → Self-Collision Checking: ✓
```

22.8 常见问题

22.8.1 交互标记不出现

症状: 3D 视图中无交互球和箭头

排查:

☐ 检查Planning Group是否选择

☐ Displays → MotionPlanning → Robot Interaction: ✓

☐ 点击Query区域的 [Update]

☐ 重启RViz

22.8.2 规划的轨迹不显示

症状: Plan 后无橙色轨迹

排查:

☐ Displays → MotionPlanning → Planned Path: ✓

☐ Displays → MotionPlanning → Planned Path → Show Trail: ✓

☐ 检查Console是否有错误

☐ 重新Plan

22.8.3 Execute 无反应

症状: 点击 Execute 后机器人不动

排查:

```
#  检查控制器
ros2 control list_controllers

# 应该看到:
# left_arm_controller [active]
# right_arm_controller [active]

#  检查话题
ros2 topic echo /left_arm_controller/follow_joint_trajectory/goal

#  检查硬件连接
python3 check_motor_status.py

#  查看 MoveIt 日志
# Console 窗口中的错误信息
```

22.9 总结

22.9.1 RViz 关键操作

基本操作:

✓ 选择Planning Group

✓ 拖动交互标记设置目标

✓ 点击`Plan`规划轨迹

✓ 检查橙色轨迹

✓ 点击`Execute`执行

安全原则:

✓ 首次使用低速度缩放(`0.1`)

✓ 总是先`Plan`再`Execute`

✓ 仔细观察规划的轨迹

✓ 手放在急停上

22.9.2 界面导航速查

想要...	位置
选择机械臂	Planning → Planning Group
设置目标	拖动 3D 视图中的交互标记
规划轨迹	Planning → [Plan]
执行运动	Planning → [Execute]
调速度	Planning → Velocity Scaling
添加障碍物	Scene → Add Object
手动关节控制	Joints → 拖动滑块
保存姿态	Planning → Save as named pose

22.9.3 下一步

熟悉 RViz 界面后: - **03_ 单臂运动规划** - 单臂轨迹规划技巧 - **04_ 双臂独立控制** - 双臂协调运动 - **06_ 轨迹录制与回放** - 复杂动作录制

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

23 单臂运动规划

23.1 单臂轨迹规划技巧与应用

掌握单个机械臂的运动规划方法和实用技巧。

23.2 单臂规划概述

23.2.1 规划空间

OpenArmX 单臂有两种规划方式:

关节空间规划 (Joint Space):

直接指定7个关节的目标角度

优点:

✓ 无逆运动学计算

✓ 保证有解

✓ 关节轨迹可控

缺点:

末端轨迹难预测

不直观

笛卡尔空间规划 (Cartesian Space):

指定末端执行器的位置和姿态

优点:

✓ 直观(XYZ + 姿态)

✓ 易于任务描述

缺点:

可能无逆运动学解

可能存在多个解

23.2.2 单臂运动特点

左臂 (left_arm):

CAN接口: can0

关节: left_joint_1 ~ left_joint_7

末端坐标系: left_link_7

工作空间: 基座左侧

右臂 (right_arm):

CAN接口: can1

关节: right_joint_1 ~ right_joint_7

末端坐标系: right_link_7

工作空间: 基座右侧

23.3 使用 RViz 进行单臂规划

23.3.1 基本规划流程

步骤详解:

```
1. 启动MoveIt
   ↓
2. 选择Planning Group: left_arm
   ↓
3. 设置目标姿态(3种方式)
   ↓
4. 配置规划参数
   ↓
5. 执行规划
   ↓
6. 检查轨迹
   ↓
7. 执行运动
```

启动:

```
source ~/openarmx_robotstride_ws-cc/install/setup.bash
ros2 launch openarm_moveit_config demo.launch.py
```

在 RViz 中:

```
MotionPlanning面板:

┌──────────────────────────────────────────────┐
│ Planning Group:              │
│ [left_arm ▼]                 │
│                              │
│ Query:                       │
│ Goal State: <current> ▼      │
│ [Update] [Random Valid]      │
```

```
│                              │
│ Planning:                    │
│ Velocity Scaling: 0.3        │
│ [Plan] [Execute]             │
└──────────────────────────────┘
```

23.3.2 方式 1: 交互标记规划

最直观的方式:

1. **出现交互标记:**

选择left_arm后,3D视图中出现:
- 蓝色球形标记(末端位置)
- 红绿蓝箭头(X/Y/Z轴)
- 红绿蓝环(绕X/Y/Z轴旋转)

2. **拖动到目标:**

拖动蓝球：移动末端位置

拖动箭头：沿轴移动

拖动环：旋转末端姿态

3. **观察预览:**

拖动时,机器人实时预览目标姿态
- 白色/半透明：目标状态
- 绿色：当前状态

4. **规划轨迹:**

点击 [Plan]
- 等待几秒
- 橙色轨迹出现：成功
- 红色或无：失败

5. **执行:**

检查橙色轨迹合理

点击 [Execute]

观察机器人运动

示例任务: 抓取桌面物体

场景:

桌子位置: X=0.5, Y=-0.3, Z=0.4 m

物体位置: X=0.5, Y=-0.3, Z=0.45 m

步骤:

1. 将交互标记拖到物体上方
   X=0.5, Y=-0.3, Z=0.6 m (物体上方15cm)

2. 调整末端姿态
   - 拖动环,使夹爪朝下

3. Plan & Execute
   - 机械臂移动到上方

4. 继续拖到抓取位置
   X=0.5, Y=-0.3, Z=0.46 m

5. Plan & Execute
   - 下降到抓取高度

6. (单独控制夹爪闭合)

23.3.3 方式 2: 随机目标规划

用于系统测试和学习:

点击 [Select Random Valid Goal]

MoveIt自动生成:
- 有效的关节配置
- 满足限位
- 无碰撞
- 可达

然后:

点击 [Plan]

点击 [Execute]

多次测试:

```
# 自动化测试 (Python API)
for i in range(10):
    move_group.set_random_target()
    success = move_group.go(wait=True)
    print(f" 尝试{i+1}: {'成功' if success else '失败'}")
```

用途: - 验证 MoveIt 配置正确 - 探索机器人工作空间 - 学习关节运动范围 - 测试规划器性能

23.3.4 方式 3: 关节滑块规划

精确控制关节角度:

切换到 Joints 标签:

```
┌──────────────────────────────────────────────────────────┐
│ left_joint_1: [-3.14, 3.14]     │
│ [──────────────●──────────────] 0.523 rad   │
│                                 │
│ left_joint_2: [-1.57, 1.57]     │
│ [──●──────────────────────────] -0.342 rad  │
│                                 │
│ left_joint_3: [-3.14, 3.14]     │
│ [────────────────●────────────] 0.125 rad   │
│                                 │
│ ... (其余关节)                   │
│                                 │
│ [Plan to Target] [Execute]      │
└──────────────────────────────────────────────────────────┘
```

使用:

1. 拖动各滑块到期望角度
2. 观察3D视图中的预览

3. 点击 [Plan to Target]
4. 点击 [Execute]

应用场景: - 避免逆运动学奇异点 - 精确重现关节配置 - 调试特定关节问题 - 学习关节空间

23.4 使用 Python API 规划

23.4.1 基本脚本结构

```
#!/usr/bin/env python3
"""
单臂运动规划 Python 示例
"""
import rclpy
from rclpy.node import Node
from moveit_py import MoveGroupInterface
import geometry_msgs.msg
import moveit_msgs.msg

class SingleArmPlanner(Node):
    def __init__(self):
        super().__init__('single_arm_planner')

        # 初始化 MoveGroup
        self.left_arm = MoveGroupInterface(self, "left_arm")

        # 设置规划参数
        self.left_arm.set_planning_time(5.0)
        self.left_arm.set_num_planning_attempts(10)
        self.left_arm.set_max_velocity_scaling_factor(0.3)
        self.left_arm.set_max_acceleration_scaling_factor(0.3)

        self.get_logger().info(" 单臂规划器初始化完成")

    def plan_to_pose(self, x, y, z, roll, pitch, yaw):
        """
```

```
        规划到指定位姿
        """
        pose_target = geometry_msgs.msg.Pose()
        pose_target.position.x = x
        pose_target.position.y = y
        pose_target.position.z = z

        # 欧拉角转四元数
        from tf_transformations import quaternion_from_euler
        q = quaternion_from_euler(roll, pitch, yaw)
        pose_target.orientation.x = q[0]
        pose_target.orientation.y = q[1]
        pose_target.orientation.z = q[2]
        pose_target.orientation.w = q[3]

        self.left_arm.set_pose_target(pose_target)

        # 规划
        self.get_logger().info(f" 规划到: X={x}, Y={y}, Z={z}")
        success = self.left_arm.go(wait=True)

        # 清除目标
        self.left_arm.clear_pose_targets()

        if success:
            self.get_logger().info("✓ 规划并执行成功")
        else:
            self.get_logger().error(" 规划失败")

        return success

    def plan_to_joint_values(self, joint_values):
        """
        规划到指定关节配置
```

```
        joint_values: [j1, j2, j3, j4, j5, j6, j7]
        """
        self.left_arm.set_joint_value_target(joint_values)

        self.get_logger().info(f" 规划到关节配置: {joint_values}")
        success = self.left_arm.go(wait=True)

        if success:
            self.get_logger().info("✓ 关节空间规划成功")
        else:
            self.get_logger().error(" 关节空间规划失败")

        return success

    def get_current_pose(self):
        """
        获取当前末端位姿
        """
        pose = self.left_arm.get_current_pose().pose
        self.get_logger().info(f" 当前位置: X={pose.position.x:.3f}, "
                               f"Y={pose.position.y:.3f}, "
                               f"Z={pose.position.z:.3f}")
        return pose

    def get_current_joint_values(self):
        """
        获取当前关节角度
        """
        joints = self.left_arm.get_current_joint_values()
        self.get_logger().info(f" 当前关节: {[f'{j:.3f}' for j in joints]}")
        return joints

def main():
    rclpy.init()
```

```
    planner = SingleArmPlanner()

    # 示例 1: 获取当前状态
    planner.get_current_pose()
    planner.get_current_joint_values()

    # 示例 2: 规划到指定位姿
    import math
    success = planner.plan_to_pose(
        x=0.5,
        y=-0.3,
        z=0.5,
        roll=0.0,
        pitch=math.pi/2,  # 夹爪朝下
        yaw=0.0
    )

    # 示例 3: 规划到零位
    if success:
        planner.plan_to_joint_values([0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0])

    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

运行:

```
python3 single_arm_planner.py
```

23.4.2 笛卡尔路径规划

直线运动:

```python
def plan_cartesian_path(self, waypoints):
    """
    笛卡尔空间直线插补
    waypoints: 位姿列表
    """
    (plan, fraction) = self.left_arm.compute_cartesian_path(
        waypoints,   # 路径点
        0.01,        # eef_step: 末端步长 (m)
        0.0          # jump_threshold: 关节跳变阈值
    )

    self.get_logger().info(f" 笛卡尔路径规划完成: {fraction*100:.1f}%")

    if fraction > 0.95:  # 95% 以上路径可达
        self.left_arm.execute(plan, wait=True)
        return True
    else:
        self.get_logger().error(" 笛卡尔路径不完整")
        return False

# 使用示例: 垂直下降
current_pose = planner.left_arm.get_current_pose().pose

waypoints = []
waypoints.append(current_pose)

# 下降 10cm
down_pose = copy.deepcopy(current_pose)
down_pose.position.z -= 0.1
waypoints.append(down_pose)

planner.plan_cartesian_path(waypoints)
```

应用场景: - 抓取物体 (垂直下降) - 插入操作 (直线进入) - 涂覆轨迹 (沿表面移动)

23.4.3 完整抓取示例

```
#!/usr/bin/env python3
"""
左臂抓取物体完整示例
"""
import rclpy
from rclpy.node import Node
from moveit_py import MoveGroupInterface
import geometry_msgs.msg
import copy
import math

class PickAndPlace(Node):
    def __init__(self):
        super().__init__('pick_and_place')

        self.left_arm = MoveGroupInterface(self, "left_arm")
        self.left_gripper = MoveGroupInterface(self, "left_gripper")

        self.left_arm.set_max_velocity_scaling_factor(0.3)
        self.left_arm.set_max_acceleration_scaling_factor(0.3)

        self.get_logger().info(" 抓取节点初始化")

    def open_gripper(self):
        """ 打开夹爪"""
        self.left_gripper.set_joint_value_target([0.0])
        self.left_gripper.go(wait=True)
        self.get_logger().info(" 夹爪已打开")

    def close_gripper(self):
        """ 闭合夹爪"""
        self.left_gripper.set_joint_value_target([0.8])
        self.left_gripper.go(wait=True)
```

```
        self.get_logger().info(" 夹爪已闭合")

    def pick_object(self, x, y, z):
        """
        抓取物体
        x, y, z: 物体位置 (m)
        """
        self.get_logger().info(f" 开始抓取物体: ({x}, {y}, {z})")

        # 阶段 1: 移动到物体上方
        self.get_logger().info("[1/5] 移动到上方")
        pose_above = geometry_msgs.msg.Pose()
        pose_above.position.x = x
        pose_above.position.y = y
        pose_above.position.z = z + 0.15  # 上方 15cm

        # 夹爪朝下
        from tf_transformations import quaternion_from_euler
        q = quaternion_from_euler(0, math.pi/2, 0)
        pose_above.orientation.x = q[0]
        pose_above.orientation.y = q[1]
        pose_above.orientation.z = q[2]
        pose_above.orientation.w = q[3]

        self.left_arm.set_pose_target(pose_above)
        if not self.left_arm.go(wait=True):
            self.get_logger().error(" 移动到上方失败")
            return False

        # 阶段 2: 打开夹爪
        self.get_logger().info("[2/5] 打开夹爪")
        self.open_gripper()

        # 阶段 3: 下降到抓取位置 (笛卡尔路径)
```

```
        self.get_logger().info("[3/5] 下降到抓取位置")
        waypoints = []
        waypoints.append(self.left_arm.get_current_pose().pose)

        pose_grasp = copy.deepcopy(waypoints[0])
        pose_grasp.position.z = z + 0.02  # 物体上方 2cm
        waypoints.append(pose_grasp)

        (plan, fraction) = self.left_arm.compute_cartesian_path(
            waypoints, 0.01, 0.0
        )

        if fraction < 0.9:
            self.get_logger().error(" 笛卡尔路径规划失败")
            return False

        self.left_arm.execute(plan, wait=True)

        # 阶段 4: 闭合夹爪
        self.get_logger().info("[4/5] 闭合夹爪抓取")
        self.close_gripper()
        import time
        time.sleep(1.0)

        # 阶段 5: 提升
        self.get_logger().info("[5/5] 提升物体")
        waypoints_up = []
        waypoints_up.append(self.left_arm.get_current_pose().pose)

        pose_lift = copy.deepcopy(waypoints_up[0])
        pose_lift.position.z += 0.15
        waypoints_up.append(pose_lift)

        (plan, fraction) = self.left_arm.compute_cartesian_path(
```

```
            waypoints_up, 0.01, 0.0
        )

        self.left_arm.execute(plan, wait=True)

        self.get_logger().info("✓ 抓取完成")
        return True

    def place_object(self, x, y, z):
        """
        放置物体
        """
        self.get_logger().info(f" 放置物体到: ({x}, {y}, {z})")

        # 移动到目标上方
        pose_above = geometry_msgs.msg.Pose()
        pose_above.position.x = x
        pose_above.position.y = y
        pose_above.position.z = z + 0.15

        from tf_transformations import quaternion_from_euler
        q = quaternion_from_euler(0, math.pi/2, 0)
        pose_above.orientation.x = q[0]
        pose_above.orientation.y = q[1]
        pose_above.orientation.z = q[2]
        pose_above.orientation.w = q[3]

        self.left_arm.set_pose_target(pose_above)
        self.left_arm.go(wait=True)

        # 下降
        waypoints = []
        waypoints.append(self.left_arm.get_current_pose().pose)
        pose_down = copy.deepcopy(waypoints[0])
```

```
        pose_down.position.z = z + 0.05
        waypoints.append(pose_down)

        (plan, _) = self.left_arm.compute_cartesian_path(waypoints, 0.01, 0.0)
        self.left_arm.execute(plan, wait=True)

        # 打开夹爪
        self.open_gripper()

        # 提升
        waypoints_up = []
        waypoints_up.append(self.left_arm.get_current_pose().pose)
        pose_up = copy.deepcopy(waypoints_up[0])
        pose_up.position.z += 0.1
        waypoints_up.append(pose_up)

        (plan, _) = self.left_arm.compute_cartesian_path(waypoints_up, 0.01, 0.0)
        self.left_arm.execute(plan, wait=True)

        self.get_logger().info("✓ 放置完成")
        return True

def main():
    rclpy.init()

    pick_place = PickAndPlace()

    # 执行抓放任务
    success = pick_place.pick_object(x=0.5, y=-0.3, z=0.45)

    if success:
        pick_place.place_object(x=0.4, y=-0.4, z=0.45)

    rclpy.shutdown()
```

```
if __name__ == '__main__':
    main()
```

23.5 规划技巧

23.5.1 提高规划成功率

技巧 1: 合理的目标姿态

✓ 避免关节极限位置

✓ 保持肘部适度弯曲

✓ 末端姿态避免奇异

不要完全伸直

不要过度扭曲

技巧 2: 分步规划

```
# 不推荐：一步到位
arm.set_pose_target(final_pose)
arm.go()

# 推荐：分步规划
arm.set_pose_target(intermediate_pose)
arm.go()
arm.set_pose_target(final_pose)
arm.go()
```

技巧 3: 调整规划参数

```
# 增加规划时间
arm.set_planning_time(10.0)  # 默认 5 秒

# 增加尝试次数
arm.set_num_planning_attempts(20)  # 默认 10 次

# 尝试不同算法
arm.set_planner_id("RRTstar")  # 默认 RRTConnect
```

23.5.2 避免碰撞

添加场景物体:

```
# 添加桌子
box_pose = geometry_msgs.msg.PoseStamped()
box_pose.header.frame_id = "base_link"
box_pose.pose.position.x = 0.5
box_pose.pose.position.y = 0.0
box_pose.pose.position.z = 0.4
box_pose.pose.orientation.w = 1.0

scene = moveit_msgs.msg.PlanningSceneInterface()
scene.add_box("table", box_pose, size=(1.0, 0.6, 0.05))
```

自碰撞检查:

```
# 已默认开启
# 可以在 SRDF 中配置 disable_collisions
```

23.5.3 速度和加速度控制

```
# 谨慎起步
arm.set_max_velocity_scaling_factor(0.1)      # 10% 速度
arm.set_max_acceleration_scaling_factor(0.1)

# 逐步提高
arm.set_max_velocity_scaling_factor(0.3)      # 30% 速度
arm.set_max_acceleration_scaling_factor(0.3)

# 熟练后
arm.set_max_velocity_scaling_factor(0.8)      # 80% 速度
arm.set_max_acceleration_scaling_factor(0.8)
```

23.6 常见任务示例

23.6.1 扫描轨迹

```
def scan_trajectory(self, start_x, end_x, y, z, step=0.05):
    """
    水平扫描轨迹
    """
    waypoints = []

    # 起始位置
    pose = geometry_msgs.msg.Pose()
    pose.position.x = start_x
    pose.position.y = y
    pose.position.z = z
    # 设置姿态...
    waypoints.append(copy.deepcopy(pose))

    # 生成中间点
    x = start_x
    while x <= end_x:
        pose.position.x = x
        waypoints.append(copy.deepcopy(pose))
        x += step

    # 规划并执行
    (plan, fraction) = self.left_arm.compute_cartesian_path(
        waypoints, 0.01, 0.0
    )

    if fraction > 0.95:
        self.left_arm.execute(plan, wait=True)
        return True
    return False
```

23.6.2 圆形轨迹

```
import numpy as np

def circle_trajectory(self, center_x, center_y, center_z, radius, n_points=20):
    """
    圆形轨迹
    """
    waypoints = []

    for i in range(n_points + 1):
        angle = 2 * np.pi * i / n_points

        pose = geometry_msgs.msg.Pose()
        pose.position.x = center_x + radius * np.cos(angle)
        pose.position.y = center_y + radius * np.sin(angle)
        pose.position.z = center_z
        # 设置姿态...

        waypoints.append(copy.deepcopy(pose))

    (plan, fraction) = self.left_arm.compute_cartesian_path(
        waypoints, 0.01, 0.0
    )

    if fraction > 0.95:
        self.left_arm.execute(plan, wait=True)
        return True
    return False
```

23.7 总结

23.7.1 单臂规划要点

✓ 熟练使用三种规划方式(交互/随机/关节)

✓ 掌握Python API编程

✓ 笛卡尔路径用于直线/插入任务

✓ 分步规划提高成功率

✓ 合理设置速度和加速度

23.7.2 最佳实践

☐ 首次测试使用低速度(`0.1-0.3`)

☐ 规划前检查周围环境

☐ 使用笛卡尔路径进行精确运动

☐ 复杂任务分解为简单步骤

☐ 添加场景物体避免碰撞

☐ 监控规划成功率

23.7.3 下一步

- **04_ 双臂独立控制** - 左右臂协调运动
- **06_ 轨迹录制与回放** - 录制复杂示教轨迹
- **08_ 高级应用** - 力控和柔顺控制

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

24 双臂独立控制

24.1 双臂协调运动与独立控制

掌握 OpenArmX 双臂机器人的协同操作技巧。

24.2 双臂控制概述

24.2.1 控制模式

OpenArmX 支持以下双臂控制模式:

独立控制 (Independent Control):

左臂和右臂分别规划和执行

每个臂有独立的`Planning Group`

适用: 独立任务、测试、简单协作

协调控制 (Coordinated Control):

两臂同步规划和执行

需要编程实现时间同步

适用：抓持大物体、双臂装配

镜像控制 (Mirrored Control):

右臂镜像左臂的运动

常用于对称操作

适用：对称装配、双侧作业

24.2.2 Planning Groups

```
left_arm:
- 关节: left_joint_1 ~ left_joint_7
- 末端: left_link_7
- CAN: can0

right_arm:
- 关节: right_joint_1 ~ right_joint_7
- 末端: right_link_7
- CAN: can1

left_gripper:
- 关节: left_gripper_joint

right_gripper:
- 关节: right_gripper_joint
```

24.3 RViz 双臂独立控制

24.3.1 分别规划左右臂

步骤流程:

```
阶段1: 规划左臂
├── 选择Planning Group: left_arm
├── 拖动交互标记设置左臂目标
├── 点击 [Plan]
```

```
├── 检查轨迹
└── 点击 [Execute]

阶段2: 规划右臂
├── 选择Planning Group: right_arm
├── 拖动交互标记设置右臂目标
├── 点击 [Plan]
├── 检查轨迹
└── 点击 [Execute]
```

注意事项:

```
[警告] 左臂运动时,右臂保持当前位置
[警告] 右臂运动时,左臂保持当前位置
[警告] 需要手动确保两臂不会碰撞
[警告] 执行顺序影响最终姿态
```

24.3.2 示例: 对称摆放

任务: 左右臂同时移动到对称位置

```
1. 选择Planning Group: left_arm

2. 拖动左臂到目标位置
   X=0.4, Y=-0.3, Z=0.5

3. Plan & Execute

4. 选择Planning Group: right_arm

5. 拖动右臂到镜像位置
   X=0.4, Y=+0.3, Z=0.5

6. Plan & Execute
```

结果:

左右臂对称分布在基座两侧

24.3.3 夹爪控制

切换到夹爪 Planning Group:

```
Planning Group: [left_gripper ▼]


Joints标签:

┌──────────────────────────────────────────────┐
│ left_gripper_joint          │
│ [0.0 ──●──────────────── 1.2] rad   │
│                             │
│ [Plan to Target] [Execute] │
└──────────────────────────────────────────────┘


操作:
1. 拖动滑块
   - 0.0 rad: 打开
   - 0.8-1.2 rad: 闭合


2. Plan to Target


3. Execute
```

快捷方式:

```python
# Python API
left_gripper.set_joint_value_target([0.0])    # 打开
left_gripper.go(wait=True)

left_gripper.set_joint_value_target([0.8])    # 闭合
left_gripper.go(wait=True)
```

24.4 Python 双臂协调编程

24.4.1 基本双臂控制类

```python
#!/usr/bin/env python3
"""
```

```
双臂协调控制
"""
import rclpy
from rclpy.node import Node
from moveit_py import MoveGroupInterface
import geometry_msgs.msg
import threading

class DualArmController(Node):
    def __init__(self):
        super().__init__('dual_arm_controller')

        # 初始化 MoveGroup
        self.left_arm = MoveGroupInterface(self, "left_arm")
        self.right_arm = MoveGroupInterface(self, "right_arm")
        self.left_gripper = MoveGroupInterface(self, "left_gripper")
        self.right_gripper = MoveGroupInterface(self, "right_gripper")

        # 设置速度
        for arm in [self.left_arm, self.right_arm]:
            arm.set_max_velocity_scaling_factor(0.3)
            arm.set_max_acceleration_scaling_factor(0.3)

        self.get_logger().info(" 双臂控制器初始化完成")

    def move_left_arm(self, x, y, z, roll, pitch, yaw):
        """ 移动左臂"""
        pose = self.create_pose(x, y, z, roll, pitch, yaw)
        self.left_arm.set_pose_target(pose)
        success = self.left_arm.go(wait=True)
        self.left_arm.clear_pose_targets()
        return success

    def move_right_arm(self, x, y, z, roll, pitch, yaw):
```

```
        """ 移动右臂"""
        pose = self.create_pose(x, y, z, roll, pitch, yaw)
        self.right_arm.set_pose_target(pose)
        success = self.right_arm.go(wait=True)
        self.right_arm.clear_pose_targets()
        return success

    def create_pose(self, x, y, z, roll, pitch, yaw):
        """ 创建位姿"""
        pose = geometry_msgs.msg.Pose()
        pose.position.x = x
        pose.position.y = y
        pose.position.z = z

        from tf_transformations import quaternion_from_euler
        q = quaternion_from_euler(roll, pitch, yaw)
        pose.orientation.x = q[0]
        pose.orientation.y = q[1]
        pose.orientation.z = q[2]
        pose.orientation.w = q[3]

        return pose

    def open_left_gripper(self):
        """ 打开左夹爪"""
        self.left_gripper.set_joint_value_target([0.0])
        return self.left_gripper.go(wait=True)

    def close_left_gripper(self):
        """ 闭合左夹爪"""
        self.left_gripper.set_joint_value_target([0.8])
        return self.left_gripper.go(wait=True)

    def open_right_gripper(self):
```

```
        """ 打开右夹爪"""
        self.right_gripper.set_joint_value_target([0.0])
        return self.right_gripper.go(wait=True)

    def close_right_gripper(self):
        """ 闭合右夹爪"""
        self.right_gripper.set_joint_value_target([0.8])
        return self.right_gripper.go(wait=True)

    def move_both_arms_sequential(self, left_pose, right_pose):
        """
        顺序移动双臂
        先左后右
        """
        self.get_logger().info(" 顺序移动: 左臂 → 右臂")

        # 左臂
        self.left_arm.set_pose_target(left_pose)
        left_success = self.left_arm.go(wait=True)
        self.left_arm.clear_pose_targets()

        if not left_success:
            self.get_logger().error(" 左臂移动失败")
            return False

        # 右臂
        self.right_arm.set_pose_target(right_pose)
        right_success = self.right_arm.go(wait=True)
        self.right_arm.clear_pose_targets()

        if not right_success:
            self.get_logger().error(" 右臂移动失败")
            return False
```

```
        self.get_logger().info("✓ 双臂顺序移动完成")
        return True

    def move_both_arms_parallel(self, left_pose, right_pose):
        """
        并行移动双臂
        同时规划和执行
        """
        self.get_logger().info(" 并行移动双臂")

        # 使用线程并行执行
        left_success = [False]
        right_success = [False]

        def move_left():
            self.left_arm.set_pose_target(left_pose)
            left_success[0] = self.left_arm.go(wait=True)
            self.left_arm.clear_pose_targets()

        def move_right():
            self.right_arm.set_pose_target(right_pose)
            right_success[0] = self.right_arm.go(wait=True)
            self.right_arm.clear_pose_targets()

        # 创建线程
        left_thread = threading.Thread(target=move_left)
        right_thread = threading.Thread(target=move_right)

        # 启动线程
        left_thread.start()
        right_thread.start()

        # 等待完成
        left_thread.join()
```

```
        right_thread.join()

        if left_success[0] and right_success[0]:
            self.get_logger().info("✓ 双臂并行移动完成")
            return True
        else:
            self.get_logger().error(" 双臂并行移动部分失败")
            return False

def main():
    rclpy.init()

    controller = DualArmController()

    import math

    # 示例 1: 顺序控制
    left_pose = controller.create_pose(0.5, -0.3, 0.5, 0, math.pi/2, 0)
    right_pose = controller.create_pose(0.5, 0.3, 0.5, 0, math.pi/2, 0)

    controller.move_both_arms_sequential(left_pose, right_pose)

    # 示例 2: 并行控制
    # controller.move_both_arms_parallel(left_pose, right_pose)

    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

24.4.2 双臂协同抓取

任务: 双臂协作抓持大物体

```python
#!/usr/bin/env python3
"""
双臂协同抓取大物体
"""
import rclpy
from rclpy.node import Node
from moveit_py import MoveGroupInterface
import geometry_msgs.msg
import math
import time

class DualArmGrasp(Node):
    def __init__(self):
        super().__init__('dual_arm_grasp')

        self.left_arm = MoveGroupInterface(self, "left_arm")
        self.right_arm = MoveGroupInterface(self, "right_arm")
        self.left_gripper = MoveGroupInterface(self, "left_gripper")
        self.right_gripper = MoveGroupInterface(self, "right_gripper")

        for arm in [self.left_arm, self.right_arm]:
            arm.set_max_velocity_scaling_factor(0.2)
            arm.set_max_acceleration_scaling_factor(0.2)

        self.get_logger().info(" 双臂抓取控制器初始化")

    def create_pose(self, x, y, z, roll, pitch, yaw):
        pose = geometry_msgs.msg.Pose()
        pose.position.x = x
        pose.position.y = y
        pose.position.z = z

        from tf_transformations import quaternion_from_euler
        q = quaternion_from_euler(roll, pitch, yaw)
```

```
        pose.orientation.x = q[0]
        pose.orientation.y = q[1]
        pose.orientation.z = q[2]
        pose.orientation.w = q[3]

        return pose

    def grasp_large_object(self, object_x, object_y, object_z, object_width):
        """
        抓取大物体
        object_x, object_y, object_z: 物体中心位置
        object_width: 物体宽度 (Y 方向)
        """
        self.get_logger().info(f" 开始抓取大物体: 中心 ({object_x}, {object_y}, {object_z}), 宽度 ={o

        # 计算左右抓取点
        left_grasp_y = object_y - object_width / 2 - 0.05   # 左侧
        right_grasp_y = object_y + object_width / 2 + 0.05  # 右侧

        # 阶段 1: 移动到物体两侧上方
        self.get_logger().info("[1/6] 移动到物体两侧上方")

        left_above = self.create_pose(
            object_x, left_grasp_y, object_z + 0.15,
            0, math.pi/2, 0  # 夹爪朝下
        )

        right_above = self.create_pose(
            object_x, right_grasp_y, object_z + 0.15,
            0, math.pi/2, 0
        )

        # 顺序移动 (避免碰撞)
        self.left_arm.set_pose_target(left_above)
```

```
        if not self.left_arm.go(wait=True):
            self.get_logger().error(" 左臂移动失败")
            return False
        self.left_arm.clear_pose_targets()

        self.right_arm.set_pose_target(right_above)
        if not self.right_arm.go(wait=True):
            self.get_logger().error(" 右臂移动失败")
            return False
        self.right_arm.clear_pose_targets()

        # 阶段 2: 打开双夹爪
        self.get_logger().info("[2/6] 打开双夹爪")
        self.left_gripper.set_joint_value_target([0.0])
        self.left_gripper.go(wait=True)
        self.right_gripper.set_joint_value_target([0.0])
        self.right_gripper.go(wait=True)

        # 阶段 3: 下降到抓取高度
        self.get_logger().info("[3/6] 下降到抓取高度")

        import copy
        left_waypoints = []
        left_waypoints.append(self.left_arm.get_current_pose().pose)
        left_grasp = copy.deepcopy(left_waypoints[0])
        left_grasp.position.z = object_z
        left_waypoints.append(left_grasp)

        right_waypoints = []
        right_waypoints.append(self.right_arm.get_current_pose().pose)
        right_grasp = copy.deepcopy(right_waypoints[0])
        right_grasp.position.z = object_z
        right_waypoints.append(right_grasp)
```

```
        # 笛卡尔路径
        (left_plan, left_frac) = self.left_arm.compute_cartesian_path(
            left_waypoints, 0.01, 0.0
        )
        (right_plan, right_frac) = self.right_arm.compute_cartesian_path(
            right_waypoints, 0.01, 0.0
        )

        if left_frac < 0.9 or right_frac < 0.9:
            self.get_logger().error(" 笛卡尔路径规划失败")
            return False

        # 同步下降
        self.left_arm.execute(left_plan, wait=False)   # 非阻塞
        self.right_arm.execute(right_plan, wait=True)  # 阻塞等待

        # 阶段 4: 闭合双夹爪
        self.get_logger().info("[4/6] 闭合双夹爪抓取")
        self.left_gripper.set_joint_value_target([0.8])
        self.left_gripper.go(wait=True)
        self.right_gripper.set_joint_value_target([0.8])
        self.right_gripper.go(wait=True)

        time.sleep(1.0)  # 确保抓稳

        # 阶段 5: 同步提升
        self.get_logger().info("[5/6] 同步提升物体")

        left_up_waypoints = []
        left_up_waypoints.append(self.left_arm.get_current_pose().pose)
        left_up = copy.deepcopy(left_up_waypoints[0])
        left_up.position.z += 0.2
        left_up_waypoints.append(left_up)
```

```
        right_up_waypoints = []
        right_up_waypoints.append(self.right_arm.get_current_pose().pose)
        right_up = copy.deepcopy(right_up_waypoints[0])
        right_up.position.z += 0.2
        right_up_waypoints.append(right_up)

        (left_plan, _) = self.left_arm.compute_cartesian_path(
            left_up_waypoints, 0.01, 0.0
        )
        (right_plan, _) = self.right_arm.compute_cartesian_path(
            right_up_waypoints, 0.01, 0.0
        )

        # 同步提升
        self.left_arm.execute(left_plan, wait=False)
        self.right_arm.execute(right_plan, wait=True)

        # 阶段 6: 移动到目标位置 (可选)
        self.get_logger().info("[6/6] 保持抓取姿态")
        time.sleep(2.0)

        self.get_logger().info("✓ 双臂协同抓取完成")
        return True

    def release_object(self):
        """ 释放物体"""
        self.get_logger().info(" 释放物体")

        # 打开双夹爪
        self.left_gripper.set_joint_value_target([0.0])
        self.left_gripper.go(wait=True)
        self.right_gripper.set_joint_value_target([0.0])
        self.right_gripper.go(wait=True)
```

```
        self.get_logger().info("✓ 物体已释放")

def main():
    rclpy.init()

    grasp_controller = DualArmGrasp()

    # 抓取物体
    # 物体: 中心 (0.5, 0.0, 0.45), 宽度 0.3m
    success = grasp_controller.grasp_large_object(
        object_x=0.5,
        object_y=0.0,
        object_z=0.45,
        object_width=0.3
    )

    if success:
        time.sleep(3)
        grasp_controller.release_object()

    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

24.4.3 镜像运动

```
def mirror_motion(self, left_target_joints):
    """
    右臂镜像左臂运动
    left_target_joints: [j1, j2, j3, j4, j5, j6, j7]
    """
    # 镜像规则:
    # joint_1: 取反 (肩部旋转)
```

```
# 其他关节：相同

right_target_joints = [
    -left_target_joints[0],  # joint_1 反向
    left_target_joints[1],
    left_target_joints[2],
    left_target_joints[3],
    left_target_joints[4],
    left_target_joints[5],
    left_target_joints[6]
]

# 同时设置目标
self.left_arm.set_joint_value_target(left_target_joints)
self.right_arm.set_joint_value_target(right_target_joints)

# 并行执行
import threading

def move_left():
    self.left_arm.go(wait=True)

def move_right():
    self.right_arm.go(wait=True)

left_thread = threading.Thread(target=move_left)
right_thread = threading.Thread(target=move_right)

left_thread.start()
right_thread.start()

left_thread.join()
right_thread.join()
```

```
    self.get_logger().info(" 镜像运动完成")

# 使用
controller.mirror_motion([0.5, 0.3, -0.4, -0.8, 0.0, 0.3, 0.0])
```

24.5 碰撞避免

24.5.1 自碰撞检测

MoveIt 自动检查:

- ☐ 左臂与右臂之间
- ☐ 左臂与基座
- ☐ 右臂与基座
- ☐ 单臂自碰撞

配置: `openarm_v10.srdf`

查看禁用的碰撞对:

```
<!-- openarm_v10.srdf -->
<disable_collisions link1="left_link_1" link2="right_link_1" reason="Never"/>
<disable_collisions link1="left_link_2" link2="left_link_3" reason="Adjacent"/>
...
```

24.5.2 运动时避碰

策略:

1. 顺序运动(推荐)
   - 先移动一臂
   - 再移动另一臂
   - 保守但安全

2. 分区工作
   - 左臂负责左侧工作空间
   - 右臂负责右侧工作空间
   - 避免交叉

3. 高度分离
   - 一臂在高处作业
   - 另一臂在低处作业
   - 垂直方向分离

4. 添加虚拟障碍物
   - 在两臂间添加虚拟墙
   - 强制MoveIt避让

添加虚拟分隔:

```
# 添加中间分隔板
separator_pose = geometry_msgs.msg.PoseStamped()
separator_pose.header.frame_id = "base_link"
separator_pose.pose.position.x = 0.3
separator_pose.pose.position.y = 0.0
separator_pose.pose.position.z = 0.5
separator_pose.pose.orientation.w = 1.0

scene = moveit_msgs.msg.PlanningSceneInterface()
scene.add_box("separator", separator_pose, size=(0.6, 0.01, 1.0))
```

24.6 同步控制技巧

24.6.1 时间同步

方法 1: 速度缩放统一

```
# 确保两臂使用相同的速度缩放
for arm in [left_arm, right_arm]:
    arm.set_max_velocity_scaling_factor(0.3)
    arm.set_max_acceleration_scaling_factor(0.3)
```

方法 2: 轨迹时间对齐

```
# 规划后调整轨迹时间
left_plan = left_arm.plan()
```

```
right_plan = right_arm.plan()

# 获取最长时间
max_duration = max(
    left_plan.joint_trajectory.points[-1].time_from_start,
    right_plan.joint_trajectory.points[-1].time_from_start
)

# 重新规划，指定时间
# (需要底层 API 支持)
```

24.6.2 位置同步

检查同步误差:

```
def check_sync_error(self):
    """ 检查双臂同步误差"""
    left_pose = self.left_arm.get_current_pose().pose
    right_pose = self.right_arm.get_current_pose().pose

    # 假设对称任务，检查 Y 坐标
    sync_error_y = abs(left_pose.position.y + right_pose.position.y)

    self.get_logger().info(f"Y 方向同步误差: {sync_error_y:.4f} m")

    if sync_error_y > 0.05:  # 5cm
        self.get_logger().warn(" 同步误差较大!")
        return False
    return True
```

24.7 实用应用场景

24.7.1 双臂装配

```
def assembly_task(self):
    """
```

```
    双臂装配示例
    左臂持零件，右臂安装螺丝
    """
    # 左臂抓取零件
    self.move_left_arm(0.5, -0.3, 0.5, 0, math.pi/2, 0)
    self.close_left_gripper()

    # 左臂移动到装配位置
    self.move_left_arm(0.4, -0.2, 0.6, 0, math.pi/2, 0)

    # 右臂抓取工具
    self.move_right_arm(0.6, 0.4, 0.45, 0, math.pi/2, 0)
    self.close_right_gripper()

    # 右臂移动到装配位置
    self.move_right_arm(0.4, 0.0, 0.6, 0, math.pi/2, math.pi/4)

    # 执行装配动作...
```

24.7.2 双臂传递

```
def handover_object(self):
    """
    物体从左臂传递到右臂
    """
    # 左臂抓取物体
    self.move_left_arm(0.5, -0.3, 0.45, 0, math.pi/2, 0)
    self.close_left_gripper()

    # 左臂移动到中间交接区
    self.move_left_arm(0.3, 0.0, 0.6, 0, math.pi/2, 0)

    # 右臂打开夹爪靠近
    self.open_right_gripper()
```

```
    self.move_right_arm(0.3, 0.05, 0.6, 0, math.pi/2, 0)

    # 右臂闭合夹爪
    self.close_right_gripper()
    time.sleep(0.5)

    # 左臂松开
    self.open_left_gripper()

    # 右臂移走
    self.move_right_arm(0.5, 0.3, 0.5, 0, math.pi/2, 0)

    self.get_logger().info(" 物体传递完成")
```

24.8 常见问题

24.8.1 双臂运动不同步

症状: 并行执行时, 两臂到达时间差异大

解决:

```
# 使用相同的速度缩放
left_arm.set_max_velocity_scaling_factor(0.3)
right_arm.set_max_velocity_scaling_factor(0.3)

# 选择相似复杂度的路径
# 或分步执行，每步同步
```

24.8.2 碰撞检测过于敏感

症状: MoveIt 频繁报告碰撞, 阻止合理运动

解决:

```
# 调整碰撞检测 padding
# 编辑 openarm_v10.srdf
<disable_collisions link1="left_link_4" link2="right_link_4" reason="Never"/>
```

```
# 或在代码中临时调整
scene.allow_collision("left_link_4", "right_link_4")
```

24.8.3 规划失败率高

双臂同时规划时失败

原因: 组合搜索空间大 (14 自由度)

解决:

1. 分步规划(先左后右)
2. 使用关节空间目标
3. 增加规划时间
4. 添加中间路径点

24.9 总结

24.9.1 双臂控制要点

✓ 优先使用顺序控制(安全)
✓ 并行控制需同步时间和速度
✓ 分区工作避免碰撞
✓ 镜像运动适用对称任务
✓ 协同抓取需精确规划

24.9.2 最佳实践

☐ 从独立控制开始学习
☐ 逐步尝试协调控制
☐ 监控同步误差
☐ 添加虚拟障碍物提高安全性
☐ 测试时使用低速度(0.2-0.3)
☐ 复杂任务分解为简单步骤

24.9.3 下一步

- **06_ 轨迹录制与回放** - 录制双臂复杂动作
- **07_ 控制器配置** - 调整控制参数

- **08_ 高级应用** - 力控协同

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

25 连续录制

25.1 连续录制机器人运动轨迹

本文档说明如何录制机器人运动轨迹并保存。

25.2 轨迹录制概述

25.2.1 录制功能

OpenArmX 提供两种录制方式: - **连续录制**: 实时记录所有关节状态 - **分步录制**: 记录关键姿态点

25.2.2 录制脚本

```
cd ~/openarmx_robotstride_ws-cc

# 查看录制节点
ros2 pkg list | grep openarm_record

# 录制相关节点
ros2 run openarm_record record_joint_states_always    # 连续录制
```

25.3 连续录制使用

25.3.1 启动连续录制

```
# 前提: 系统已启动, 电机已使能
source ~/openarmx_robotstride_ws-cc/install/setup.bash

# 启动连续录制
ros2 run openarm_record record_joint_states_always
```

输出:

```
开始录制关节状态...
按Ctrl+C停止录制

[录制中] 采样: 100, 时长: 10.0s
```

[录制中] 采样: 200, 时长: 20.0s

...

25.3.2 停止录制

按 Ctrl+C 停止

^C

录制已停止

总采样数: 500

总时长: 50.0s

保存到: trajectory_20251019_143025.yaml

25.3.3 录制参数

采样频率:

```
# 默认: 10 Hz (每秒 10 次)
# 修改录制节点源码可调整
rate = 10.0  # Hz
```

保存格式:

```yaml
# trajectory_YYYYMMDD_HHMMSS.yaml
trajectory:
  points:
    - positions: [0.0, 0.0, 0.0, ...]
      time_from_start: 0.0
    - positions: [0.1, 0.2, 0.1, ...]
      time_from_start: 0.1
    ...
```

25.4 录制工作流

25.4.1 准备工作

```
#  启动系统
python3 en_all_can.py
```

```
python3 en_all_motors.py

# 回零 (可选)
python3 control_motor_gohome.py

# 启动 ros2_control (如果使用)
ros2 launch openarm_bringup openarm.launch.py
```

25.4.2 执行录制

方法 1: 手动移动 (示教)

```
# 降低 KP/KD 进入示教模式
# (修改参数或使用示教脚本)

# 启动录制
ros2 run openarm_record record_joint_states_always

# 手动拖动机械臂执行动作

# Ctrl+C 停止录制
```

方法 2: 通过 MoveIt 规划

```
# Terminal 1: 启动 MoveIt
ros2 launch openarm_moveit_config demo.launch.py

# Terminal 2: 启动录制
ros2 run openarm_record record_joint_states_always

# 在 MoveIt 中执行运动

# Ctrl+C 停止录制
```

25.4.3 验证录制

```
# 查看生成的文件
ls -lh trajectory_*.yaml

# 查看文件内容
cat trajectory_20251019_143025.yaml | head -20
```

25.5 轨迹文件格式

25.5.1 YAML 结构

```yaml
header:
  stamp:
    sec: 1729329025
    nanosec: 123456789
  frame_id: "base_link"

joint_names:
  - left_joint_1
  - left_joint_2
  - left_joint_3
  - left_joint_4
  - left_joint_5
  - left_joint_6
  - left_joint_7
  - right_joint_1
  - right_joint_2
  - right_joint_3
  - right_joint_4
  - right_joint_5
  - right_joint_6
  - right_joint_7

points:
```

```yaml
- positions: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
  velocities: []
  accelerations: []
  effort: []
  time_from_start:
    sec: 0
    nanosec: 0

- positions: [0.01, 0.02, 0.01, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
  velocities: []
  accelerations: []
  effort: []
  time_from_start:
    sec: 0
    nanosec: 100000000  # 0.1 秒

# ... 更多轨迹点
```

25.5.2 字段说明

字段	说明
joint_names	关节名称列表
positions	各关节位置 (rad)
time_from_start	相对起始时间
velocities	速度 (可选)
accelerations	加速度 (可选)
effort	力矩 (可选)

25.6 录制技巧

25.6.1 平滑轨迹录制

要点:

- ☐ 缓慢均匀地移动
- ☐ 避免突然加速/减速

- ☐ 保持稳定的速度
- ☐ 避免抖动

25.6.2 录制前准备

- ☐ 规划好运动路径
- ☐ 确认无障碍物
- ☐ 检查关节限位
- ☐ 预演动作

25.6.3 录制质量检查

```
# 查看轨迹点数量
grep "positions:" trajectory_*.yaml | wc -l

# 检查时间间隔是否均匀
cat trajectory_*.yaml | grep "time_from_start" -A 1

# 可视化检查 (使用 Python)
python3 visualize_trajectory.py trajectory_*.yaml
```

25.7 常见问题

25.7.1 录制文件过大

原因: 录制时间过长或采样频率过高

解决:

1. 缩短录制时间
2. 降低采样频率(修改代码)
3. 录制后下采样

25.7.2 轨迹不平滑

原因: 手动移动不稳定

解决:

1. 练习后重新录制

2. 使用MoveIt规划录制
3. 录制后进行平滑处理

25.7.3 录制中断

原因: 通信中断或程序崩溃

解决:

```
# 检查已保存的文件
ls -lt trajectory_*.yaml | head -1

# 验证文件完整性
cat <最新文件> | tail -10

# 如果文件不完整，重新录制
```

25.8 高级功能

25.8.1 指定保存路径

```
# 修改录制节点，指定输出目录
export TRAJECTORY_DIR=~/trajectories
mkdir -p $TRAJECTORY_DIR

ros2 run openarm_record record_joint_states_always \
    --ros-args -p output_dir:=$TRAJECTORY_DIR
```

25.8.2 添加标注信息

手动编辑 YAML 文件:

```
# 在文件开头添加
description: " 抓取演示轨迹"
author: " 张三"
date: "2025-10-19"
notes: " 从零位开始，抓取桌面物体"
```

```yaml
# 原有内容
header:
  ...
```

25.8.3 合并多个轨迹

```python
# 使用 Python 脚本合并
import yaml

traj1 = yaml.safe_load(open('traj1.yaml'))
traj2 = yaml.safe_load(open('traj2.yaml'))

# 合并 points
merged_points = traj1['points'] + traj2['points']

# 调整时间戳
# ...

# 保存
with open('merged_traj.yaml', 'w') as f:
    yaml.dump(merged_traj, f)
```

25.9 总结

25.9.1 录制流程

```
1. 准备系统(CAN, 电机, ros2_control)
   ↓
2. 启动录制节点
   ↓
3. 执行运动(手动或MoveIt)
   ↓
4. Ctrl+C停止
   ↓
5. 获得YAML轨迹文件
```

25.9.2 最佳实践

✓ 录制前规划好动作

✓ 保持平稳的运动

✓ 避免过长的录制时间(<1分钟)

✓ 及时备份重要轨迹

✓ 添加描述信息

25.9.3 下一步

完成轨迹录制后: - **02_ 轨迹回放** - 回放录制的轨迹 - **03_ 轨迹文件格式** - 深入了解文件结构

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

26 轨迹回放

26.1 录制轨迹的回放与执行

重现已录制的机器人运动轨迹。

26.2 轨迹回放概述

26.2.1 回放原理

工作流程:

```
1. 加载YAML轨迹文件
   ↓
2. 解析轨迹点(positions, time_from_start)
   ↓
3. 生成ros2_control轨迹消息
   ↓
4. 发送到控制器
   ↓
5. 控制器执行轨迹
   ↓
6. 机器人重现录制的运动
```

26.2.2 回放方式

OpenArmX 提供两种回放方式:

方式 1: 使用 playback 节点

```
ros2 run openarm_record playback_trajectory
```

方式 2: 使用 Python 脚本

```
自定义 Python 脚本加载和回放
```

26.3 使用 playback 节点回放

26.3.1 基本用法

```
cd ~/openarmx_robotstride_ws-cc

# 确保系统已启动
source install/setup.bash

# 启动 ros2_control
ros2 launch openarm_bringup openarm.launch.py

# 新终端: 回放轨迹
ros2 run openarm_record playback_trajectory
```

交互式输入:

```
OpenArmX 轨迹回放
==================
请输入轨迹文件路径: trajectory_20251019_143025.yaml

加载轨迹文件...
轨迹点数: 500
总时长: 50.0s
涉及关节: 14

Planning Group: [left_arm ▼]
```

```
选择回放组: left_arm

速度缩放 (0.1-1.0, 默认1.0): 0.5

开始回放? (y/n): y

发送轨迹...
回放中: 25% [====>           ] 12.5s/50.0s
回放中: 50% [========>       ] 25.0s/50.0s
回放中: 75% [============>   ] 37.5s/50.0s
回放中: 100% [===============] 50.0s/50.0s

✓ 回放完成!
```

26.3.2 参数说明

轨迹文件路径:

```
# 相对路径
trajectory_20251019_143025.yaml

# 绝对路径
/home/user/trajectories/my_traj.yaml

# 使用通配符
trajectory_*.yaml  # (如果支持)
```

Planning Group:

left_arm: 仅回放左臂轨迹

right_arm: 仅回放右臂轨迹

both_arms: 回放双臂轨迹(如果支持)

速度缩放:

0.1: 10%速度(慢速验证)

0.5: 50%速度(安全测试)

1.0: 100%速度(完整回放)

>1.0: 加速回放(谨慎!)

26.4 Python 脚本回放

26.4.1 基本回放脚本

```
#!/usr/bin/env python3
"""
轨迹回放脚本
"""
import rclpy
from rclpy.node import Node
from rclpy.action import ActionClient
from control_msgs.action import FollowJointTrajectory
from trajectory_msgs.msg import JointTrajectory, JointTrajectoryPoint
import yaml
from builtin_interfaces.msg import Duration

class TrajectoryPlayback(Node):
    def __init__(self):
        super().__init__('trajectory_playback')

        # 创建 Action Client
        self.left_arm_client = ActionClient(
            self,
            FollowJointTrajectory,
            '/left_arm_controller/follow_joint_trajectory'
        )

        self.right_arm_client = ActionClient(
            self,
            FollowJointTrajectory,
            '/right_arm_controller/follow_joint_trajectory'
        )
```

```
        self.get_logger().info(" 轨迹回放节点初始化")

    def load_trajectory(self, filename):
        """ 加载 YAML 轨迹文件"""
        with open(filename, 'r') as f:
            data = yaml.safe_load(f)

        self.get_logger().info(f" 加载轨迹: {filename}")
        self.get_logger().info(f" 轨迹点数: {len(data['points'])}")

        return data

    def playback_left_arm(self, trajectory_data, speed_scale=1.0):
        """ 回放左臂轨迹"""
        # 构建 JointTrajectory 消息
        traj_msg = JointTrajectory()

        # 设置关节名称 (仅左臂)
        traj_msg.joint_names = [
            'left_joint_1', 'left_joint_2', 'left_joint_3',
            'left_joint_4', 'left_joint_5', 'left_joint_6',
            'left_joint_7'
        ]

        # 提取左臂轨迹点
        for point_data in trajectory_data['points']:
            point = JointTrajectoryPoint()

            # 提取左臂关节位置 (前 7 个)
            point.positions = point_data['positions'][:7]

            # 时间缩放
            time_sec = point_data['time_from_start']['sec']
            time_nsec = point_data['time_from_start']['nanosec']
```

```
            total_sec = time_sec + time_nsec * 1e-9
            scaled_sec = total_sec / speed_scale

            point.time_from_start = Duration(
                sec=int(scaled_sec),
                nanosec=int((scaled_sec - int(scaled_sec)) * 1e9)
            )

            traj_msg.points.append(point)

        # 创建 Goal
        goal_msg = FollowJointTrajectory.Goal()
        goal_msg.trajectory = traj_msg

        # 等待 Action Server
        self.get_logger().info(" 等待左臂控制器...")
        self.left_arm_client.wait_for_server()

        # 发送 Goal
        self.get_logger().info(f" 发送左臂轨迹 (速度缩放: {speed_scale})")
        future = self.left_arm_client.send_goal_async(goal_msg)

        return future

    def playback_right_arm(self, trajectory_data, speed_scale=1.0):
        """ 回放右臂轨迹"""
        traj_msg = JointTrajectory()

        traj_msg.joint_names = [
            'right_joint_1', 'right_joint_2', 'right_joint_3',
            'right_joint_4', 'right_joint_5', 'right_joint_6',
            'right_joint_7'
        ]
```

```
        for point_data in trajectory_data['points']:
            point = JointTrajectoryPoint()

            # 提取右臂关节位置 (后 7 个, 索引 7-13)
            point.positions = point_data['positions'][7:14]

            time_sec = point_data['time_from_start']['sec']
            time_nsec = point_data['time_from_start']['nanosec']
            total_sec = time_sec + time_nsec * 1e-9
            scaled_sec = total_sec / speed_scale

            point.time_from_start = Duration(
                sec=int(scaled_sec),
                nanosec=int((scaled_sec - int(scaled_sec)) * 1e9)
            )

            traj_msg.points.append(point)

        goal_msg = FollowJointTrajectory.Goal()
        goal_msg.trajectory = traj_msg

        self.get_logger().info(" 等待右臂控制器...")
        self.right_arm_client.wait_for_server()

        self.get_logger().info(f" 发送右臂轨迹 (速度缩放: {speed_scale})")
        future = self.right_arm_client.send_goal_async(goal_msg)

        return future

def main():
    import sys

    if len(sys.argv) < 2:
        print(" 用法: python3 playback.py < 轨迹文件.yaml> [速度缩放]")
```

```
        sys.exit(1)

    filename = sys.argv[1]
    speed_scale = float(sys.argv[2]) if len(sys.argv) > 2 else 1.0

    rclpy.init()

    playback = TrajectoryPlayback()

    # 加载轨迹
    trajectory_data = playback.load_trajectory(filename)

    # 选择回放组
    choice = input(" 回放: (1) 左臂 (2) 右臂 (3) 双臂: ")

    if choice == '1':
        playback.playback_left_arm(trajectory_data, speed_scale)
    elif choice == '2':
        playback.playback_right_arm(trajectory_data, speed_scale)
    elif choice == '3':
        playback.playback_left_arm(trajectory_data, speed_scale)
        playback.playback_right_arm(trajectory_data, speed_scale)

    # 等待完成
    rclpy.spin_once(playback, timeout_sec=trajectory_data['points'][-1]['time_from_start']['sec'] + 5)

    playback.get_logger().info("✓ 回放完成")

    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

运行:

```
python3 playback.py trajectory_20251019_143025.yaml 0.5
```

26.4.2 循环回放

```
def loop_playback(self, trajectory_data, speed_scale=1.0, num_loops=3):
    """
    循环回放轨迹
    """
    for i in range(num_loops):
        self.get_logger().info(f" 第 {i+1}/{num_loops} 次回放")

        future = self.playback_left_arm(trajectory_data, speed_scale)

        # 等待完成
        rclpy.spin_until_future_complete(self, future)

        result = future.result()
        if result:
            self.get_logger().info(f" 第 {i+1} 次完成")
        else:
            self.get_logger().error(f" 第 {i+1} 次失败")
            break

        # 间隔
        import time
        time.sleep(2.0)

    self.get_logger().info(" 循环回放结束")

# 使用
playback.loop_playback(trajectory_data, speed_scale=0.5, num_loops=3)
```

26.4.3 带反馈的回放

```
from rclpy.action import ActionClient
from control_msgs.action import FollowJointTrajectory

def playback_with_feedback(self, trajectory_data, speed_scale=1.0):
    """
    回放并显示实时反馈
    """
    # 构建轨迹消息（省略，同上）
    traj_msg = self.build_trajectory_message(trajectory_data, speed_scale)

    goal_msg = FollowJointTrajectory.Goal()
    goal_msg.trajectory = traj_msg

    # 发送 Goal 并注册回调
    send_goal_future = self.left_arm_client.send_goal_async(
        goal_msg,
        feedback_callback=self.feedback_callback
    )

    rclpy.spin_until_future_complete(self, send_goal_future)

    goal_handle = send_goal_future.result()

    if not goal_handle.accepted:
        self.get_logger().error("Goal 被拒绝")
        return

    self.get_logger().info("Goal 已接受，开始执行...")

    # 等待结果
    result_future = goal_handle.get_result_async()
    rclpy.spin_until_future_complete(self, result_future)
```

```
    result = result_future.result().result
    self.get_logger().info(f" 回放完成, 错误码: {result.error_code}")

def feedback_callback(self, feedback_msg):
    """ 反馈回调"""
    feedback = feedback_msg.feedback

    # 计算进度
    if len(feedback.desired.positions) > 0:
        # 显示当前位置
        self.get_logger().info(
            f" 进度: desired={feedback.desired.positions[0]:.3f}, "
            f"actual={feedback.actual.positions[0]:.3f}"
        )
```

26.5 回放前准备

26.5.1 系统检查

```
#  CAN 接口启动
python3 en_all_can.py

#  电机使能
python3 en_all_motors.py

#  启动 ros2_control
ros2 launch openarm_bringup openarm.launch.py

#  检查控制器
ros2 control list_controllers

# 应该看到:
# left_arm_controller [active]
# right_arm_controller [active]
```

26.5.2 轨迹文件验证

```
# 检查文件存在
ls -lh trajectory_20251019_143025.yaml

# 查看文件内容
head -50 trajectory_20251019_143025.yaml

# 验证 YAML 格式
python3 -c "import yaml; yaml.safe_load(open('trajectory_20251019_143025.yaml'))"
```

检查项:

- ☐ 文件存在且可读
- ☐ YAML格式正确
- ☐ 包含joint_names字段
- ☐ 包含points列表
- ☐ 每个point有positions和time_from_start
- ☐ 轨迹点数量合理(>10)

26.5.3 安全准备

- ☐ 确认机器人在零位或安全位置
- ☐ 检查周围无障碍物
- ☐ 准备急停措施
- ☐ 首次回放使用低速(0.1-0.3)
- ☐ 观察人员就位

26.6 回放技巧

26.6.1 速度调整

渐进式测试:

第1次: 速度缩放 0.1 (10%速度)
　　验证轨迹正确性

第2次: 速度缩放 0.3 (30%速度)

验证平滑性

第3次: 速度缩放 0.5 (50%速度)

接近正常速度

第4次: 速度缩放 1.0 (100%速度)

完整回放

脚本中实现:

```
speed_scales = [0.1, 0.3, 0.5, 1.0]

for speed in speed_scales:
    print(f"\n以 {speed*100}% 速度回放...")
    playback.playback_left_arm(trajectory_data, speed_scale=speed)

    # 等待完成
    time.sleep(duration / speed + 2)

    confirm = input(" 继续下一速度? (y/n): ")
    if confirm.lower() != 'y':
        break
```

26.6.2 部分回放

回放轨迹的一部分:

```
def playback_segment(self, trajectory_data, start_idx, end_idx, speed_scale=1.0):
    """
    回放轨迹片段
    start_idx, end_idx: 轨迹点索引
    """
    # 提取片段
    segment_points = trajectory_data['points'][start_idx:end_idx]

    # 重新调整时间戳 (从 0 开始)
    first_time = segment_points[0]['time_from_start']['sec'] + \
```

```
                segment_points[0]['time_from_start']['nanosec'] * 1e-9

    for point in segment_points:
        original_time = point['time_from_start']['sec'] + \
                        point['time_from_start']['nanosec'] * 1e-9

        adjusted_time = original_time - first_time

        point['time_from_start']['sec'] = int(adjusted_time)
        point['time_from_start']['nanosec'] = int((adjusted_time - int(adjusted_time)) * 1e9)

    # 构建片段轨迹
    segment_traj = {
        'joint_names': trajectory_data['joint_names'],
        'points': segment_points
    }

    # 回放
    return self.playback_left_arm(segment_traj, speed_scale)

# 使用: 回放第 100-200 个点
playback.playback_segment(trajectory_data, start_idx=100, end_idx=200, speed_scale=0.5)
```

26.6.3 镜像回放

左臂轨迹在右臂回放:

```
def mirror_playback(self, left_trajectory_data, speed_scale=1.0):
    """
    左臂轨迹镜像到右臂回放
    """
    # 复制轨迹数据
    import copy
    right_traj = copy.deepcopy(left_trajectory_data)
```

```
    # 修改关节名称
    right_traj['joint_names'] = [
        'right_joint_1', 'right_joint_2', 'right_joint_3',
        'right_joint_4', 'right_joint_5', 'right_joint_6',
        'right_joint_7'
    ]

    # 镜像 positions (joint_1 取反)
    for point in right_traj['points']:
        positions = list(point['positions'][:7])  # 左臂 7 个关节
        positions[0] = -positions[0]  # joint_1 反向
        point['positions'] = positions

    # 回放右臂
    return self.playback_right_arm(right_traj, speed_scale)
```

26.7 回放监控

26.7.1 实时监控

监控话题:

```
# 期望轨迹
ros2 topic echo /left_arm_controller/follow_joint_trajectory/goal

# 实际状态
ros2 topic echo /joint_states

# 控制器状态
ros2 topic echo /left_arm_controller/state
```

可视化:

```
# RViz 中观察
ros2 run rviz2 rviz2 -d openarmx.rviz

# 绘制关节位置曲线 (如果有 rqt_plot)
```

```
ros2 run rqt_plot rqt_plot /joint_states/position[0]:position[1]
```

26.7.2 误差监控

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import JointState
import numpy as np

class TrajectoryMonitor(Node):
    def __init__(self):
        super().__init__('traj_monitor')

        self.subscription = self.create_subscription(
            JointState,
            '/joint_states',
            self.joint_state_callback,
            10
        )

        self.desired_positions = None  # 从轨迹文件加载
        self.trajectory_index = 0
        self.errors = []

    def joint_state_callback(self, msg):
        """ 监控关节误差"""
        if self.desired_positions is None:
            return

        # 获取左臂关节实际位置
        left_joint_names = [f'left_joint_{i}' for i in range(1, 8)]
        actual_positions = []

        for name in left_joint_names:
```

```
            if name in msg.name:
                idx = msg.name.index(name)
                actual_positions.append(msg.position[idx])

        # 计算误差
        if len(actual_positions) == 7 and self.trajectory_index < len(self.desired_positions):
            desired = self.desired_positions[self.trajectory_index]
            error = np.array(actual_positions) - np.array(desired)
            rms_error = np.sqrt(np.mean(error**2))

            self.errors.append(rms_error)

            self.get_logger().info(f"RMS 误差: {rms_error:.4f} rad")

            self.trajectory_index += 1

    def get_statistics(self):
        """ 获取误差统计"""
        if len(self.errors) > 0:
            print(f" 平均误差: {np.mean(self.errors):.4f} rad")
            print(f" 最大误差: {np.max(self.errors):.4f} rad")
            print(f" 最小误差: {np.min(self.errors):.4f} rad")
            print(f" 标准差: {np.std(self.errors):.4f} rad")
```

26.8 常见问题

26.8.1 回放失败

症状: Action Goal 被拒绝或执行失败

排查:

```
# 检查控制器状态
ros2 control list_controllers

# 检查关节名称匹配
```

```
# 轨迹文件中的 joint_names 必须与控制器配置一致

#  检查时间戳
# 确保 time_from_start 单调递增

#  检查关节限位
# 轨迹点不能超出关节限制

#  查看日志
ros2 topic echo /rosout
```

26.8.2 回放不平滑

症状: 机器人运动抖动、不流畅

可能原因: - 录制采样频率过低 - 速度缩放不合理 - 控制器参数不当

解决:

```
#  插值增加轨迹点
from scipy.interpolate import interp1d

def interpolate_trajectory(points, factor=2):
    """ 增加轨迹密度"""
    # 提取时间和位置
    times = [p['time_from_start'] for p in points]
    positions = [p['positions'] for p in points]

    # 插值
    # ... (scipy interp1d)

    return interpolated_points

#  调整控制器 KP/KD
# 参考 07_ 控制器配置
```

```
#  降低速度缩放
speed_scale = 0.5  # 慢速可能更平滑
```

26.8.3 轨迹与录制不符

症状: 回放的运动与录制时不同

可能原因: - 零位漂移 - 机械装配变化 - 负载不同

验证:

```
#  检查零位
python3 control_motor_gohome.py
# 观察是否到达标准零位

#  重新校准零位
python3 set_motor_zeros_setMIT.py

#  对比录制和回放环境
# 确保负载、温度等条件一致
```

26.9 高级功能

26.9.1 轨迹编辑后回放

修改轨迹文件:

```
import yaml

# 加载
with open('trajectory.yaml', 'r') as f:
    data = yaml.safe_load(f)

# 修改: 缩放所有位置
for point in data['points']:
    point['positions'] = [p * 0.8 for p in point['positions']]

# 保存
```

```
with open('trajectory_scaled.yaml', 'w') as f:
    yaml.dump(data, f)

# 回放修改后的轨迹
```

26.9.2 轨迹拼接

```
def concatenate_trajectories(traj1_data, traj2_data):
    """
    拼接两个轨迹
    """
    import copy

    merged = copy.deepcopy(traj1_data)

    # 获取 traj1 的结束时间
    last_time_1 = traj1_data['points'][-1]['time_from_start']['sec'] + \
                  traj1_data['points'][-1]['time_from_start']['nanosec'] * 1e-9

    # 将 traj2 的时间偏移
    for point in traj2_data['points']:
        adjusted_point = copy.deepcopy(point)

        original_time = point['time_from_start']['sec'] + \
                        point['time_from_start']['nanosec'] * 1e-9

        new_time = last_time_1 + original_time

        adjusted_point['time_from_start']['sec'] = int(new_time)
        adjusted_point['time_from_start']['nanosec'] = int((new_time - int(new_time)) * 1e9)

        merged['points'].append(adjusted_point)

    return merged
```

```
# 使用
traj1 = playback.load_trajectory('traj1.yaml')
traj2 = playback.load_trajectory('traj2.yaml')

merged_traj = concatenate_trajectories(traj1, traj2)

# 保存
with open('merged_traj.yaml', 'w') as f:
    yaml.dump(merged_traj, f)
```

26.10 总结

26.10.1 回放要点

✓ 回放前检查系统状态
✓ 首次使用低速缩放(0.1-0.3)
✓ 监控回放误差
✓ 验证轨迹文件格式正确
✓ 准备急停措施

26.10.2 最佳实践

☐ 测试环境与录制环境一致
☐ 定期检查零位校准
☐ 使用渐进式速度测试
☐ 监控关节误差
☐ 备份重要轨迹文件

26.10.3 下一步

- **03_ 轨迹文件格式** - 深入了解 YAML 格式
- **07_ 控制器配置** - 调整回放平滑度
- **08_ 高级应用** - 轨迹优化与编辑

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

27 轨迹文件格式

27.1 轨迹 YAML 文件结构详解

深入理解 OpenArmX 轨迹文件的格式和编辑方法。

27.2 文件格式概述

27.2.1 YAML 简介

YAML (YAML Ain’t Markup Language):

人类可读的数据序列化格式

常用于配置文件和数据交换

Python友好(易于解析和生成)

基本语法:

```yaml
# 注释以 # 开头

# 键值对
key: value

# 列表
list:
  - item1
  - item2
  - item3

# 嵌套
parent:
  child1: value1
  child2: value2

# 数字
integer: 42
float: 3.14
```

```yaml
# 布尔
boolean: true

# 字符串
string: "hello"
```

27.2.2 轨迹文件总体结构

```yaml
# OpenArmX 轨迹文件
header:
  stamp:
    sec: 时间戳（秒）
    nanosec: 时间戳（纳秒）
  frame_id: 坐标系

joint_names:
  - 关节名称列表

points:
  - 轨迹点 1
  - 轨迹点 2
  - ...
```

27.3 Header 字段

27.3.1 header 结构

```yaml
header:
  stamp:
    sec: 1729329025
    nanosec: 123456789
  frame_id: "base_link"
```

字段说明:

字段	类型	说明
stamp.sec	int	Unix 时间戳 (秒部分)
stamp.nanosec	int	时间戳纳秒部分
frame_id	string	参考坐标系 (通常为 “base_link”)

时间戳转换:

```
import time

# 获取当前时间戳
timestamp = time.time()  # 1729329025.123456789

sec = int(timestamp)
nanosec = int((timestamp - sec) * 1e9)

# 构建 header
header = {
    'stamp': {
        'sec': sec,
        'nanosec': nanosec
    },
    'frame_id': 'base_link'
}
```

作用:

- ☐ 记录录制时间
- ☐ 用于时间同步
- ☐ 调试和追溯

27.4 joint_names 字段

27.4.1 关节名称列表

```
joint_names:
  - left_joint_1
```

```
  - left_joint_2
  - left_joint_3
  - left_joint_4
  - left_joint_5
  - left_joint_6
  - left_joint_7
  - right_joint_1
  - right_joint_2
  - right_joint_3
  - right_joint_4
  - right_joint_5
  - right_joint_6
  - right_joint_7
```

顺序:

必须与positions数组对应

固定顺序:

- 左臂7个关节(ID 1-7)
- 右臂7个关节(ID 1-7)

总共14个关节(不包括夹爪)

仅左臂轨迹:

```
joint_names:
  - left_joint_1
  - left_joint_2
  - left_joint_3
  - left_joint_4
  - left_joint_5
  - left_joint_6
  - left_joint_7
```

仅右臂轨迹:

```yaml
joint_names:
  - right_joint_1
  - right_joint_2
  - right_joint_3
  - right_joint_4
  - right_joint_5
  - right_joint_6
  - right_joint_7
```

27.5 points 字段 (核心)

27.5.1 轨迹点结构

```yaml
points:
  - positions: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
    velocities: []
    accelerations: []
    effort: []
    time_from_start:
      sec: 0
      nanosec: 0

  - positions: [0.01, 0.02, 0.01, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
    velocities: []
    accelerations: []
    effort: []
    time_from_start:
      sec: 0
      nanosec: 100000000  # 0.1 秒

  - positions: [0.02, 0.04, 0.02, -0.01, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
    velocities: []
    accelerations: []
    effort: []
    time_from_start:
```

```
    sec: 0
    nanosec: 200000000  # 0.2 秒
```

27.5.2 字段详解

```
positions: [j1, j2, j3, j4, j5, j6, j7, j8, j9, j10, j11, j12, j13, j14]
```

27.5.2.1 positions (必需)

说明: - 类型: float 列表 - 单位: rad (弧度) - 长度: 与 joint_names 对应 - 必需字段

示例:

```
# 左臂单独
positions: [0.5, 0.3, -0.4, -0.8, 0.2, 0.3, -0.2]

# 双臂
positions: [0.5, 0.3, -0.4, -0.8, 0.2, 0.3, -0.2, -0.5, 0.3, -0.4, -0.8, 0.2, 0.3, -0.2]
```

```
velocities: [v1, v2, v3, v4, v5, v6, v7, ...]
```

27.5.2.2 velocities (可选)

说明: - 类型: float 列表 - 单位: rad/s - 长度: 与 positions 相同, 或为空 [] - 可选字段 (通常为空)

用途: - 指定关节速度 - 控制器可据此优化轨迹 - 连续录制通常不记录

```
accelerations: [a1, a2, a3, a4, a5, a6, a7, ...]
```

27.5.2.3 accelerations (可选)

说明: - 类型: float 列表 - 单位: rad/s^2 - 长度: 与 positions 相同, 或为空 [] - 可选字段 (通常为空)

```
effort: [e1, e2, e3, e4, e5, e6, e7, ...]
```

27.5.2.4 effort (可选)

说明: - 类型: float 列表 - 单位: Nm (牛·米) - 表示关节力矩 - 可选字段 (通常为空)

用途: - 力控应用 - 记录实际力矩 - 重现接触力

```yaml
time_from_start:
  sec: 0
  nanosec: 100000000  # 0.1 秒
```

27.5.2.5 time_from_start (必需)

说明: - 类型: Duration (sec + nanosec) - 相对于轨迹起始的时间 - 必须单调递增 - 决定轨迹速度

时间计算:

```python
# 总时间 (秒)
total_time = sec + nanosec * 1e-9

# 示例
sec = 1
nanosec = 500000000
total = 1 + 0.5 = 1.5秒
```

时间间隔:

```
第1点: 0.0秒
第2点: 0.1秒 (间隔0.1s)
第3点: 0.2秒 (间隔0.1s)
...

采样频率 = 1 / 间隔时间
如间隔0.1s → 10 Hz
```

27.6 完整示例文件

27.6.1 最小示例 (左臂 3 个点)

```yaml
header:
  stamp:
    sec: 1729329025
    nanosec: 0
  frame_id: "base_link"
```

```yaml
joint_names:
  - left_joint_1
  - left_joint_2
  - left_joint_3
  - left_joint_4
  - left_joint_5
  - left_joint_6
  - left_joint_7

points:
  - positions: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
    velocities: []
    accelerations: []
    effort: []
    time_from_start:
      sec: 0
      nanosec: 0

  - positions: [0.5, 0.3, -0.4, -0.8, 0.2, 0.3, -0.2]
    velocities: []
    accelerations: []
    effort: []
    time_from_start:
      sec: 2
      nanosec: 0

  - positions: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
    velocities: []
    accelerations: []
    effort: []
    time_from_start:
      sec: 4
      nanosec: 0
```

27.6.2 带速度和力矩的示例


```yaml
header:
  stamp:
    sec: 1729329025
    nanosec: 0
  frame_id: "base_link"

joint_names:
  - left_joint_1
  - left_joint_2
  - left_joint_3
  - left_joint_4
  - left_joint_5
  - left_joint_6
  - left_joint_7

points:
  - positions: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
    velocities: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
    accelerations: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
    effort: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
    time_from_start:
      sec: 0
      nanosec: 0

  - positions: [0.5, 0.3, -0.4, -0.8, 0.2, 0.3, -0.2]
    velocities: [0.25, 0.15, -0.2, -0.4, 0.1, 0.15, -0.1]
    accelerations: [0.1, 0.1, -0.1, -0.2, 0.05, 0.1, -0.05]
    effort: [5.0, 3.0, 2.5, 8.0, 1.5, 2.0, 1.0]
    time_from_start:
      sec: 2
      nanosec: 0

  - positions: [1.0, 0.6, -0.8, -1.2, 0.4, 0.6, -0.4]
```

```
    velocities: [0.25, 0.15, -0.2, -0.2, 0.1, 0.15, -0.1]
    accelerations: [0.0, 0.0, 0.0, 0.1, 0.0, 0.0, 0.0]
    effort: [6.0, 4.0, 3.0, 10.0, 2.0, 2.5, 1.5]
    time_from_start:
      sec: 4
      nanosec: 0
```

27.7 文件编辑

27.7.1 手动编辑

文本编辑器:

```
vim trajectory.yaml
nano trajectory.yaml
gedit trajectory.yaml
```

修改示例:

原始:

```
positions: [0.5, 0.3, -0.4, -0.8, 0.2, 0.3, -0.2]
```

缩放 50%:

```
positions: [0.25, 0.15, -0.2, -0.4, 0.1, 0.15, -0.1]
```

[警告] 注意事项:

- ☐ 保持YAML语法正确
- ☐ positions数组长度不变
- ☐ time_from_start单调递增
- ☐ 关节位置在限位内
- ☐ 备份原始文件

27.7.2 Python 编辑

```
#!/usr/bin/env python3
"""
轨迹文件编辑脚本
```

```
"""
import yaml

def load_trajectory(filename):
    """ 加载轨迹"""
    with open(filename, 'r') as f:
        return yaml.safe_load(f)

def save_trajectory(data, filename):
    """ 保存轨迹"""
    with open(filename, 'w') as f:
        yaml.dump(data, f, default_flow_style=False)

def scale_positions(traj_data, scale_factor):
    """ 缩放所有位置"""
    for point in traj_data['points']:
        point['positions'] = [p * scale_factor for p in point['positions']]
    return traj_data

def reverse_trajectory(traj_data):
    """ 反向轨迹"""
    # 反转点顺序
    traj_data['points'] = list(reversed(traj_data['points']))

    # 重新计算时间
    max_time_sec = traj_data['points'][0]['time_from_start']['sec']
    max_time_nsec = traj_data['points'][0]['time_from_start']['nanosec']
    max_time = max_time_sec + max_time_nsec * 1e-9

    for point in traj_data['points']:
        original_time = point['time_from_start']['sec'] + \
                        point['time_from_start']['nanosec'] * 1e-9

        new_time = max_time - original_time
```

```
        point['time_from_start']['sec'] = int(new_time)
        point['time_from_start']['nanosec'] = int((new_time - int(new_time)) * 1e9)

    return traj_data

def downsample_trajectory(traj_data, factor=2):
    """ 下采样 (保留每 N 个点)"""
    traj_data['points'] = traj_data['points'][::factor]

    # 重新调整时间从 0 开始
    first_time = traj_data['points'][0]['time_from_start']['sec'] + \
                 traj_data['points'][0]['time_from_start']['nanosec'] * 1e-9

    for point in traj_data['points']:
        original_time = point['time_from_start']['sec'] + \
                        point['time_from_start']['nanosec'] * 1e-9

        new_time = original_time - first_time

        point['time_from_start']['sec'] = int(new_time)
        point['time_from_start']['nanosec'] = int((new_time - int(new_time)) * 1e9)

    return traj_data

def add_velocities(traj_data):
    """
    添加速度字段 (数值微分)
    """
    for i in range(len(traj_data['points'])):
        if i == 0:
            # 第一个点速度为 0
            traj_data['points'][i]['velocities'] = [0.0] * len(traj_data['points'][i]['positions'])
        else:
```

```
            # 计算速度
            dt = (traj_data['points'][i]['time_from_start']['sec'] -
                  traj_data['points'][i-1]['time_from_start']['sec']) + \
                 (traj_data['points'][i]['time_from_start']['nanosec'] -
                  traj_data['points'][i-1]['time_from_start']['nanosec']) * 1e-9

            if dt > 0:
                pos_curr = traj_data['points'][i]['positions']
                pos_prev = traj_data['points'][i-1]['positions']

                velocities = [(pos_curr[j] - pos_prev[j]) / dt for j in range(len(pos_curr))]
                traj_data['points'][i]['velocities'] = velocities
            else:
                traj_data['points'][i]['velocities'] = [0.0] * len(traj_data['points'][i]['positions']

    return traj_data

# 使用示例
if __name__ == '__main__':
    # 加载
    traj = load_trajectory('trajectory.yaml')

    # 缩放 50%
    traj = scale_positions(traj, 0.5)

    # 下采样
    traj = downsample_trajectory(traj, factor=2)

    # 添加速度
    traj = add_velocities(traj)

    # 保存
    save_trajectory(traj, 'trajectory_edited.yaml')
```

```
    print(" 轨迹编辑完成")
```

运行:

```
python3 edit_trajectory.py
```

27.8 轨迹分析

27.8.1 基本统计

```
def analyze_trajectory(traj_data):
    """ 分析轨迹统计信息"""
    num_points = len(traj_data['points'])
    num_joints = len(traj_data['joint_names'])

    # 总时长
    last_point = traj_data['points'][-1]
    duration = last_point['time_from_start']['sec'] + \
               last_point['time_from_start']['nanosec'] * 1e-9

    # 采样频率
    if num_points > 1:
        avg_interval = duration / (num_points - 1)
        avg_frequency = 1.0 / avg_interval if avg_interval > 0 else 0
    else:
        avg_frequency = 0

    # 关节运动范围
    import numpy as np

    positions_array = np.array([point['positions'] for point in traj_data['points']])

    joint_ranges = []
    for j in range(num_joints):
        joint_pos = positions_array[:, j]
        joint_range = np.max(joint_pos) - np.min(joint_pos)
```

```
        joint_ranges.append(joint_range)

    # 输出
    print(f" 轨迹统计:")
    print(f"  轨迹点数: {num_points}")
    print(f"  关节数: {num_joints}")
    print(f"  总时长: {duration:.2f} s")
    print(f"  平均采样频率: {avg_frequency:.2f} Hz")
    print(f"\n各关节运动范围:")
    for j, name in enumerate(traj_data['joint_names']):
        print(f"  {name}: {joint_ranges[j]:.3f} rad ({np.rad2deg(joint_ranges[j]):.1f}°)")

# 使用
traj = load_trajectory('trajectory.yaml')
analyze_trajectory(traj)
```

输出示例:

```
轨迹统计:
  轨迹点数: 500
  关节数: 7
  总时长: 50.00 s
  平均采样频率: 10.00 Hz

各关节运动范围:
  left_joint_1: 1.047 rad (60.0°)
  left_joint_2: 0.524 rad (30.0°)
  left_joint_3: 0.873 rad (50.0°)
  left_joint_4: 1.571 rad (90.0°)
  left_joint_5: 0.349 rad (20.0°)
  left_joint_6: 0.524 rad (30.0°)
  left_joint_7: 0.436 rad (25.0°)
```

27.8.2 可视化

```
import matplotlib.pyplot as plt
import numpy as np

def visualize_trajectory(traj_data):
    """ 可视化轨迹"""
    # 提取时间和位置
    times = []
    positions = []

    for point in traj_data['points']:
        t = point['time_from_start']['sec'] + \
            point['time_from_start']['nanosec'] * 1e-9
        times.append(t)
        positions.append(point['positions'])

    times = np.array(times)
    positions = np.array(positions)  # (N, num_joints)

    # 绘图
    num_joints = positions.shape[1]
    fig, axes = plt.subplots(num_joints, 1, figsize=(10, 2*num_joints))

    for j in range(num_joints):
        axes[j].plot(times, np.rad2deg(positions[:, j]))
        axes[j].set_ylabel(f'{traj_data["joint_names"][j]} (deg)')
        axes[j].grid(True)

    axes[-1].set_xlabel('Time (s)')
    plt.suptitle('Joint Trajectories')
    plt.tight_layout()
    plt.show()

# 使用
```

```
traj = load_trajectory('trajectory.yaml')
visualize_trajectory(traj)
```

27.9 格式验证

27.9.1 验证脚本

```
def validate_trajectory(traj_data):
    """ 验证轨迹文件格式"""
    errors = []

    # 检查必需字段
    if 'joint_names' not in traj_data:
        errors.append(" 缺少 joint_names 字段")

    if 'points' not in traj_data:
        errors.append(" 缺少 points 字段")
        return errors

    # 检查 points
    num_joints = len(traj_data.get('joint_names', []))

    for i, point in enumerate(traj_data['points']):
        # 检查 positions
        if 'positions' not in point:
            errors.append(f" 点{i}: 缺少 positions")
        elif len(point['positions']) != num_joints:
            errors.append(f" 点{i}: positions 长度{len(point['positions'])}与 joint_names 长度{num_jo

        # 检查 time_from_start
        if 'time_from_start' not in point:
            errors.append(f" 点{i}: 缺少 time_from_start")
        elif 'sec' not in point['time_from_start'] or 'nanosec' not in point['time_from_start']:
            errors.append(f" 点{i}: time_from_start 格式错误")
```

```
    # 检查时间单调性
    for i in range(1, len(traj_data['points'])):
        t_prev = traj_data['points'][i-1]['time_from_start']['sec'] + \
                 traj_data['points'][i-1]['time_from_start']['nanosec'] * 1e-9

        t_curr = traj_data['points'][i]['time_from_start']['sec'] + \
                 traj_data['points'][i]['time_from_start']['nanosec'] * 1e-9

        if t_curr <= t_prev:
            errors.append(f" 点{i}: 时间戳{t_curr}不大于前一点{t_prev}")

    # 输出结果
    if len(errors) == 0:
        print("✓ 轨迹文件格式正确")
        return True
    else:
        print(f" 发现{len(errors)}个错误:")
        for error in errors:
            print(f"  - {error}")
        return False

# 使用
traj = load_trajectory('trajectory.yaml')
validate_trajectory(traj)
```

27.10 常见问题

27.10.1 YAML 解析错误

症状: `yaml.scanner.ScannerError`

原因: YAML 语法错误

检查:

```
# 使用 yamllint 检查
sudo apt install yamllint
```

```
yamllint trajectory.yaml

# 或用 Python
python3 -c "import yaml; yaml.safe_load(open('trajectory.yaml'))"
```

常见错误:

```
# 错误: 缩进不一致
points:
  - positions: [0.0]
   velocities: []  #  缩进错误

# 正确:
points:
  - positions: [0.0]
    velocities: []  # ✓ 对齐
```

27.10.2 文件过大

症状: 文件几十 MB, 打开缓慢

原因: 轨迹点过多或采样频率过高

解决:

```
# 下采样
traj = load_trajectory('large_traj.yaml')
traj = downsample_trajectory(traj, factor=5)  # 保留 1/5
save_trajectory(traj, 'small_traj.yaml')
```

27.10.3 关节限位超出

症状: 回放时报错 “joint limits violated”

检查:

```
# 检查是否超出限位
joint_limits = {
    'left_joint_1': (-3.14, 3.14),
```

```
    'left_joint_2': (-1.57, 1.57),
    'left_joint_3': (-3.14, 3.14),
    'left_joint_4': (-2.79, 0.0),
    'left_joint_5': (-1.57, 1.57),
    'left_joint_6': (-1.57, 1.57),
    'left_joint_7': (-3.14, 3.14),
}

for i, point in enumerate(traj['points']):
    for j, name in enumerate(traj['joint_names']):
        pos = point['positions'][j]
        limits = joint_limits.get(name, (-3.14, 3.14))

        if pos < limits[0] or pos > limits[1]:
            print(f" 警告: 点{i}, {name}={pos:.3f} 超出限位{limits}")
```

27.11 总结

27.11.1 文件格式要点

✓ YAML人类可读,易于编辑

✓ header记录元数据

✓ joint_names定义关节顺序

✓ points包含轨迹点列表

✓ positions必需,其他字段可选

✓ time_from_start必须单调递增

27.11.2 编辑最佳实践

☐ 编辑前备份原文件

☐ 使用Python脚本批量修改

☐ 编辑后验证格式正确

☐ 检查关节限位

☐ 可视化验证轨迹合理

27.11.3 下一步

- **07_ 控制器配置** - 调整回放参数
- **08_ 高级应用** - 轨迹优化算法
- **10_ 开发者指南** - 深入 API 使用

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

28 控制器类型

28.1 ros2_control 控制器类型介绍

了解 OpenArmX 支持的各种控制器类型及其特点。

28.2 控制器概述

28.2.1 什么是 ros2_control

ros2_control:

ROS2的标准化控制框架

提供硬件抽象层

支持多种控制器类型

实现实时控制循环

架构:

```
应用层 (MoveIt, 自定义节点)
    ↓
控制器管理器 (Controller Manager)
    ↓
控制器 (JointTrajectoryController, PositionController, etc.)
    ↓
硬件接口 (Hardware Interface)
    ↓
实际硬件 (Robstride Motors via CAN)
```

28.2.2 OpenArmX 控制器配置

位置:

```
~/openarmx_robotstride_ws-cc/src/openarm_ros2/openarm_controllers/config/
├── openarm_controllers.yaml
└── ros2_controllers.yaml
```

已配置的控制器:

1. joint_state_broadcaster
2. left_arm_controller

3. right_arm_controller
4. left_gripper_controller
5. right_gripper_controller

28.3 控制器类型

28.3.1 JointStateBroadcaster

类型: `joint_state_broadcaster/JointStateBroadcaster`

作用:

发布所有关节的状态信息

不执行控制,仅广播

必须始终激活

发布话题:

```
/joint_states (sensor_msgs/msg/JointState)
  - position: 关节位置
  - velocity: 关节速度
  - effort: 关节力矩
```

配置示例:

```yaml
joint_state_broadcaster:
  ros__parameters:
    type: joint_state_broadcaster/JointStateBroadcaster
```

用途:

✓ RViz可视化

✓ 状态监控

✓ TF变换发布

✓ 其他节点获取状态

28.3.2 JointTrajectoryController

类型: `joint_trajectory_controller/JointTrajectoryController`

作用:

执行轨迹跟踪控制

接收轨迹目标

实时跟随轨迹点

支持多关节协调

OpenArmX 配置:

```yaml
left_arm_controller:
  ros__parameters:
    type: joint_trajectory_controller/JointTrajectoryController

    joints:
      - left_joint_1
      - left_joint_2
      - left_joint_3
      - left_joint_4
      - left_joint_5
      - left_joint_6
      - left_joint_7

    command_interfaces:
      - position

    state_interfaces:
      - position
      - velocity

    allow_partial_joints_goal: false
    open_loop_control: true
    allow_integration_in_goal_trajectories: true
```

Action 接口:

`/left_arm_controller/follow_joint_trajectory`

`(control_msgs/action/FollowJointTrajectory)`

特点:

✓ 支持轨迹插值

✓ 路径容差检查

✓ 目标容差检查

✓ 轨迹抢占

适用场景:

- MoveIt运动规划
- 轨迹回放
- 复杂多关节运动

28.3.3 PositionController

类型: position_controllers/JointGroupPositionController

作用:

直接位置控制

发送单一目标位置

简单快速

适合点到点运动

配置示例:

```yaml
position_controller:
  ros__parameters:
    type: position_controllers/JointGroupPositionController

    joints:
      - left_joint_1
      - left_joint_2
      - left_joint_3
      - left_joint_4
      - left_joint_5
      - left_joint_6
      - left_joint_7
```

命令话题:

/position_controller/commands

(std_msgs/msg/Float64MultiArray)

使用示例:

```
# 发送位置命令
ros2 topic pub /position_controller/commands std_msgs/msg/Float64MultiArray \
  "data: [0.5, 0.3, -0.4, -0.8, 0.2, 0.3, -0.2]"
```

特点:

✓ 简单直接

✓ 低延迟

无轨迹插值

不保证平滑

28.3.4 VelocityController

类型: `velocity_controllers/JointGroupVelocityController`

作用:

速度控制模式

直接指定关节速度

实时速度控制

配置示例:

```yaml
velocity_controller:
  ros__parameters:
    type: velocity_controllers/JointGroupVelocityController

    joints:
      - left_joint_1
      - left_joint_2
      - left_joint_3
      - left_joint_4
      - left_joint_5
      - left_joint_6
      - left_joint_7
```

命令话题:

`/velocity_controller/commands`

(std_msgs/msg/Float64MultiArray)

使用示例:

```
# 发送速度命令 (rad/s)
ros2 topic pub /velocity_controller/commands std_msgs/msg/Float64MultiArray \
  "data: [0.1, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]"
```

适用场景:

- 遥操作
- 实时速度控制
- 力控配合

28.3.5 EffortController (力矩控制)

类型: effort_controllers/JointGroupEffortController

作用:

力矩控制模式

直接指定关节力矩

力控应用

柔顺控制

配置示例:

```yaml
effort_controller:
  ros__parameters:
    type: effort_controllers/JointGroupEffortController

    joints:
      - left_joint_1
      - left_joint_2
      - left_joint_3
      - left_joint_4
      - left_joint_5
      - left_joint_6
      - left_joint_7
```

命令话题:

/effort_controller/commands

(std_msgs/msg/Float64MultiArray)

使用示例:

```
# 发送力矩命令 (Nm)
ros2 topic pub /effort_controller/commands std_msgs/msg/Float64MultiArray \
  "data: [1.0, 0.5, 0.5, 2.0, 0.3, 0.3, 0.2]"
```

[警告] 注意:

需要硬件支持力矩控制

Robstride电机支持MIT模式力矩

需要仔细调参,避免失控

28.4 控制器对比

28.4.1 功能对比表

控制器类型	输入	特点	平滑性	实时性	复杂度	推荐场景
JointTrajectoryController	轨迹	插值、容差检查	★★★★★	★★★☆☆	★★★★☆	MoveIt、
PositionController	位置	简单快速	★★☆☆☆	★★★★★	★☆☆☆☆	简单点位
VelocityController	速度	实时速度	★★★☆☆	★★★★★	★★☆☆☆	遥操作、
EffortController	力矩	力控柔顺	★★★★☆	★★★★★	★★★★★	力控、柔

28.4.2 控制接口对比

命令接口 (Command Interfaces):

position: 位置控制

velocity: 速度控制

effort: 力矩控制

状态接口 (State Interfaces):

position: 读取位置

velocity: 读取速度

effort: 读取力矩

OpenArmX 硬件接口支持:

```yaml
# openarm_v10.ros2_control.xacro

command_interfaces:
  - position  # ✓ 支持
  - velocity  # ✓ 支持
  - effort    # ✓ 支持 (MIT 模式)

state_interfaces:
  - position  # ✓ 支持
  - velocity  # ✓ 支持
  - effort    # ✓ 支持
```

28.5 控制器选择指南

28.5.1 应用场景推荐

场景 1: MoveIt 运动规划

推荐: JointTrajectoryController

原因:

✓ MoveIt原生支持

✓ 轨迹平滑

✓ 容错检查

✓ 标准Action接口

场景 2: 示教录制回放

推荐: JointTrajectoryController

原因:

✓ 轨迹插值

✓ 时间同步

✓ 平滑回放

场景 3: 简单测试/调试

推荐: PositionController

原因:

✓ 简单直接

✓ 快速响应

✓ 易于调试

场景 4: 遥操作

推荐: `VelocityController`

原因:

✓ 实时响应

✓ 速度控制直观

✓ 安全停止

场景 5: 力控装配/抓取

推荐: `EffortController`

原因:

✓ 力矩控制

✓ 柔顺交互

✓ 接触检测

28.5.2 控制器组合

推荐组合 1: 标准配置 (默认)

```
controllers:
  - joint_state_broadcaster  # 状态发布
  - left_arm_controller      # 左臂轨迹控制
  - right_arm_controller     # 右臂轨迹控制
  - left_gripper_controller  # 左夹爪控制
  - right_gripper_controller # 右夹爪控制
```

推荐组合 2: 混合控制

```
controllers:
  - joint_state_broadcaster
  - left_arm_trajectory_controller   # 左臂轨迹控制
  - right_arm_position_controller    # 右臂位置控制
```

[警告] 注意:

同一组关节不能同时被多个控制器控制

需要切换控制器(load/unload)

28.6 控制器配置文件

28.6.1 配置文件结构

openarm_controllers.yaml:

```yaml
controller_manager:
  ros__parameters:
    update_rate: 100  # 控制频率 100Hz

    # 控制器列表
    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster

    left_arm_controller:
      type: joint_trajectory_controller/JointTrajectoryController

    right_arm_controller:
      type: joint_trajectory_controller/JointTrajectoryController

    left_gripper_controller:
      type: joint_trajectory_controller/JointTrajectoryController

    right_gripper_controller:
      type: joint_trajectory_controller/JointTrajectoryController

# 左臂控制器详细配置
left_arm_controller:
  ros__parameters:
    joints:
      - left_joint_1
```

```yaml
    - left_joint_2
    - left_joint_3
    - left_joint_4
    - left_joint_5
    - left_joint_6
    - left_joint_7

  command_interfaces:
    - position

  state_interfaces:
    - position
    - velocity

  state_publish_rate: 50.0
  action_monitor_rate: 20.0

  allow_partial_joints_goal: false
  allow_integration_in_goal_trajectories: true

  constraints:
    stopped_velocity_tolerance: 0.01
    goal_time: 0.0

    left_joint_1:
      trajectory: 0.1
      goal: 0.05
    left_joint_2:
      trajectory: 0.1
      goal: 0.05
    # ... 其他关节
```

28.6.2 关键参数说明

update_rate:

```
update_rate: 100  # Hz

说明: 控制器更新频率
范围: 10-1000 Hz
推荐:
  - 一般应用: 100 Hz
  - 高精度: 200-500 Hz
  - 低速应用: 50 Hz
```

state_publish_rate:

```
state_publish_rate: 50.0  # Hz

说明: 状态发布频率
推荐: update_rate 的一半
作用: 发布控制器状态到话题
```

action_monitor_rate:

```
action_monitor_rate: 20.0  # Hz

说明: Action 监控频率
作用: 检查轨迹执行状态
```

stopped_velocity_tolerance:

```
stopped_velocity_tolerance: 0.01  # rad/s

说明: 停止速度容差
作用: 判断关节是否已停止
```

trajectory / goal tolerance:

```
left_joint_1:
  trajectory: 0.1  # rad (路径容差)
  goal: 0.05       # rad (目标容差)

说明:
  trajectory: 执行过程中允许的偏差
```

```
  goal: 到达目标时允许的偏差
```

28.7 控制器管理

28.7.1 查看控制器状态

```
# 列出所有控制器
ros2 control list_controllers

# 输出示例:
# joint_state_broadcaster[joint_state_broadcaster/JointStateBroadcaster] active
# left_arm_controller[joint_trajectory_controller/JointTrajectoryController] active
# right_arm_controller[joint_trajectory_controller/JointTrajectoryController] active
# left_gripper_controller[joint_trajectory_controller/JointTrajectoryController] active
# right_gripper_controller[joint_trajectory_controller/JointTrajectoryController] active
```

状态说明:

active: 激活,正在运行

inactive: 已加载,未激活

unconfigured: 未配置

28.7.2 控制器生命周期

```
unconfigured (未配置)
    ↓ configure
inactive (已配置,未激活)
    ↓ activate
active (激活)
    ↓ deactivate
inactive
    ↓ cleanup
unconfigured
```

命令:

```
# 配置
ros2 control set_controller_state left_arm_controller configure

# 激活
ros2 control set_controller_state left_arm_controller start

# 停止
ros2 control set_controller_state left_arm_controller stop
```

28.7.3 加载/卸载控制器

```
# 加载控制器
ros2 control load_controller --set-state start left_arm_controller

# 卸载控制器
ros2 control unload_controller left_arm_controller

# 重新加载
ros2 control load_controller --set-state start left_arm_controller
```

28.8 控制器切换

28.8.1 切换控制器

场景: 从 JointTrajectoryController 切换到 PositionController

```
#  停止当前控制器
ros2 control set_controller_state left_arm_controller stop

#  启动新控制器
ros2 control load_controller --set-state start left_arm_position_controller
```

切换脚本:

```
#!/bin/bash

# switch_controller.sh
```

```
CONTROLLER_TO_STOP=$1
CONTROLLER_TO_START=$2

echo " 停止 $CONTROLLER_TO_STOP ..."
ros2 control set_controller_state $CONTROLLER_TO_STOP stop

echo " 启动 $CONTROLLER_TO_START ..."
ros2 control load_controller --set-state start $CONTROLLER_TO_START

echo " 切换完成"
```

使用:

```
chmod +x switch_controller.sh
./switch_controller.sh left_arm_controller left_arm_position_controller
```

28.8.2 Python 切换示例

```python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from controller_manager_msgs.srv import SwitchController

class ControllerSwitcher(Node):
    def __init__(self):
        super().__init__('controller_switcher')

        self.client = self.create_client(
            SwitchController,
            '/controller_manager/switch_controller'
        )

        while not self.client.wait_for_service(timeout_sec=1.0):
            self.get_logger().info('等待服务...')
```

```
    def switch(self, stop_controllers, start_controllers):
        """ 切换控制器"""
        request = SwitchController.Request()
        request.stop_controllers = stop_controllers
        request.start_controllers = start_controllers
        request.strictness = 2  # STRICT
        request.start_asap = False
        request.timeout = rclpy.duration.Duration(seconds=0.0).to_msg()

        future = self.client.call_async(request)
        rclpy.spin_until_future_complete(self, future)

        if future.result().ok:
            self.get_logger().info('切换成功')
            return True
        else:
            self.get_logger().error('切换失败')
            return False

def main():
    rclpy.init()

    switcher = ControllerSwitcher()

    # 切换控制器
    switcher.switch(
        stop_controllers=['left_arm_controller'],
        start_controllers=['left_arm_position_controller']
    )

    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

28.9 常见问题

28.9.1 控制器启动失败

症状: `ros2 control list_controllers` 显示 `inactive` 或 `unconfigured`

排查:

```
# 检查配置文件
cat ~/openarmx_robotstride_ws-cc/src/openarm_ros2/openarm_controllers/config/openarm_controllers.yaml

# 检查硬件接口
ros2 control list_hardware_interfaces

# 查看日志
ros2 topic echo /rosout
```

28.9.2 控制器冲突

症状: 同一关节被多个控制器控制

解决:

确保同一时间只有一个控制器active
使用控制器切换机制

28.9.3 控制频率过低

症状: 运动不流畅, 卡顿

解决:

```
# 提高 update_rate
controller_manager:
  ros__parameters:
    update_rate: 200  # 从 100 提高到 200
```

28.10 总结

28.10.1 控制器选择要点

✓ JointTrajectoryController: MoveIt和轨迹回放

✓ `PositionController`: 简单测试

✓ `VelocityController`: 遥操作

✓ `EffortController`: 力控应用

✓ 根据应用选择合适类型

28.10.2 配置最佳实践

☐ 合理设置`update_rate` (100-200 Hz)

☐ 调整容差参数

☐ 使用标准控制器类型

☐ 避免控制器冲突

☐ 定期检查控制器状态

28.10.3 下一步

- **02_ 参数调优** - KP/KD 调节技巧
- **03_ 控制模式说明** - MIT vs CSP 模式
- **04_ 控制器管理** - 高级管理技巧

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

29 参数调优

29.1 控制器 KP/KD 参数调优指南

掌握控制参数调节, 优化机器人运动性能。

29.2 参数调优概述

29.2.1 什么是 KP/KD

PD 控制器原理:

控制输出 = KP × 位置误差 + KD × 速度误差

```
KP (比例增益):
  - 响应位置误差
  - 值越大,响应越快
  - 过大会震荡
```

```
KD (微分增益):
  - 响应速度误差
  - 提供阻尼
  - 抑制震荡
```

物理意义:

```
KP: 类似弹簧刚度
  - 高KP = 硬弹簧 = 快速但可能震荡
  - 低KP = 软弹簧 = 慢速但稳定

KD: 类似阻尼器
  - 高KD = 强阻尼 = 平滑但慢
  - 低KD = 弱阻尼 = 快但可能震荡
```

29.2.2 OpenArmX 控制架构

```
ros2_control (位置目标)
    ↓
Hardware Interface (位置 → KP/KD/力矩)
    ↓
Robstride电机MIT模式 (位置+速度+KP+KD+力矩)
    ↓
电机内部控制器
    ↓
实际运动
```

MIT 模式命令:

```
motor.set_position(
    position=0.5,    # 目标位置 (rad)
    velocity=0.0,    # 前馈速度 (rad/s)
    kp=20.0,         # 位置增益
    kd=1.0,          # 速度增益
    torque=0.0       # 前馈力矩 (Nm)
)
```

29.3 参数范围与推荐值

29.3.1 电机类型对应参数

RS04/RS03 (关节 1-4: 肩部和大臂):

```
RS04 额定扭矩: 40 Nm,  峰值扭矩: 120 Nm
RS03 额定扭矩: 20 Nm,  峰值扭矩: 60 Nm

KP 范围: 0 ~ 5000
KD 范围: 0 ~ 100

推荐值:
  示教模式: KP=1.0,  KD=0.1
  常规运动: KP=50.0, KD=2.5
  高刚性:   KP=500.0, KD=10.0
```

RS00 (关节 5-7: 小臂):

```
额定扭矩: 5 Nm
峰值扭矩: 14 Nm

KP 范围: 0 ~ 500
KD 范围: 0 ~ 5

推荐值:
  示教模式: KP=0.1,  KD=0.05
  常规运动: KP=10.0, KD=0.5
  高刚性:   KP=50.0, KD=1.0
```

RS00 (夹爪):

```
额定扭矩: 5 Nm
峰值扭矩: 14 Nm

KP 范围: 0 ~ 500
KD 范围: 0 ~ 5
```

```
推荐值:
  打开/关闭: KP=5.0, KD=0.5
  精确抓取: KP=50.0, KD=1.0
  力控抓取: KP=0.1, KD=0.05
```

29.3.2 应用场景参数表

场景	关节 1-4 KP	关节 1-4 KD	关节 5-7 KP	关节 5-7 KD	夹爪 KP	夹爪 KD	特点
示教模式	1.0	0.1	0.1	0.05	0.1	0.05	极低刚性,
常规运动 (默认)	50.0	2.5	10.0	0.5	5.0	0.5	平衡性能和
高刚性模式	500.0	10.0	50.0	1.0	50.0	1.0	高精度但可

说明: - 参数来源: `openarm_hardware/include/openarm_hardware/v10_simple_hardware.hpp` - 关节 1-4: RS04/RS03 电机（肩部和大臂）- 关节 5-7: RS00 电机（小臂）- 夹爪: RS00 电机（关节 8）

29.4 参数调优方法

29.4.1 经验调参法

步骤:

1. 从保守值开始

   `KP = 10, KD = 0.5`

2. 逐步增加KP

   观察响应速度

   直到出现轻微震荡

3. 增加KD抑制震荡

   调整到运动平滑

4. 微调优化

   平衡速度和稳定性

5. 负载测试

   加载后可能需要增加KP

调参口诀:

KP调速度,KD调平滑

先调KP后调KD

震荡就加KD

慢了就加KP

29.4.2 Ziegler-Nichols 法

理论方法:

```
1. 设置KD = 0

2. 逐步增加KP直到系统震荡
   记录临界KP值: Ku
   记录震荡周期: Tu

3. 计算PD参数:
   KP = 0.8 × Ku
   KD = 0.1 × Ku × Tu
```

示例:

```
临界值: Ku = 100, Tu = 0.2s

计算:
KP = 0.8 × 100 = 80
KD = 0.1 × 100 × 0.2 = 2.0

起始值: KP=80, KD=2.0
微调优化
```

29.4.3 实验调参步骤

测试设置:

```python
#!/usr/bin/env python3
"""
参数调优测试脚本
```

```
"""
import can
from robstride import Motor
import time

bus0 = can.interface.Bus(channel='can0', bustype='socketcan')
motor = Motor(bus0, 1)  # 测试电机 1

motor.enable()
time.sleep(1)

# 参数范围
kp_values = [10, 20, 30, 50, 80]
kd_values = [0.5, 1.0, 1.5, 2.0, 3.0]

# 测试目标
target_position = 0.5  # rad

for kp in kp_values:
    for kd in kd_values:
        print(f"\n测试: KP={kp}, KD={kd}")

        # 回零
        motor.set_position(0.0, vel=0.0, kp=20, kd=1.0, torque=0.0)
        time.sleep(2)

        # 移动到目标
        start_time = time.time()
        motor.set_position(target_position, vel=0.0, kp=kp, kd=kd, torque=0.0)

        # 记录响应
        positions = []
        times = []
```

```
        for i in range(50):  # 5 秒,10Hz 采样
            fb = motor.get_feedback()
            positions.append(fb['position'])
            times.append(time.time() - start_time)
            time.sleep(0.1)

        # 分析性能
        # 超调量, 稳定时间, 震荡等
        overshoot = max(positions) - target_position
        settling_time = next((t for t, p in zip(times, positions)
                              if abs(p - target_position) < 0.02), times[-1])

        print(f"  超调: {overshoot:.4f} rad")
        print(f"  稳定时间: {settling_time:.2f} s")

        # 保存数据用于后续分析
        # ...

motor.disable()
```

性能指标:

- ☐ 上升时间: 从10%到90%所需时间
- ☐ 超调量: 超过目标的最大偏差
- ☐ 稳定时间: 到达±2%误差带所需时间
- ☐ 震荡次数: 震荡周期数

29.4.4 可视化调参

实时监控:

```
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 8))

positions = []
```

```
velocities = []
times = []

def update(frame):
    fb = motor.get_feedback()
    t = time.time() - start_time

    positions.append(fb['position'])
    velocities.append(fb['velocity'])
    times.append(t)

    ax1.clear()
    ax1.plot(times, positions, 'b-', label='Position')
    ax1.axhline(y=target_position, color='r', linestyle='--', label='Target')
    ax1.set_ylabel('Position (rad)')
    ax1.legend()
    ax1.grid(True)

    ax2.clear()
    ax2.plot(times, velocities, 'g-', label='Velocity')
    ax2.set_xlabel('Time (s)')
    ax2.set_ylabel('Velocity (rad/s)')
    ax2.legend()
    ax2.grid(True)

    plt.suptitle(f'KP={current_kp}, KD={current_kd}')

ani = FuncAnimation(fig, update, interval=100)
plt.show()
```

29.5 参数配置方式

29.5.1 硬件接口层配置

位置:

~/openarmx_robotstride_ws-cc/src/openarm_ros2/openarm_hardware/

修改源码:

```
// openarm_hardware_interface.cpp

// 在 write() 函数中设置 KP/KD
void OpenArmHardwareInterface::write(const rclcpp::Time & time, const rclcpp::Duration & period)
{
    for (size_t i = 0; i < hw_commands_positions_.size(); i++)
    {
        double target_position = hw_commands_positions_[i];

        // 根据关节设置不同的 KP/KD
        double kp, kd;

        if (i < 3) {
            // 肩部关节 (RS04)
            kp = 20.0;
            kd = 1.0;
        } else if (i < 7) {
            // 肘腕关节 (RS03)
            kp = 20.0;
            kd = 1.0;
        } else {
            // 夹爪 (RS00)
            kp = 10.0;
            kd = 0.5;
        }

        // 发送 MIT 命令
        motors_[i].set_position(target_position, 0.0, kp, kd, 0.0);
    }
}
```

使用参数服务器:

```
// 声明参数
this->declare_parameter("left_joint_1.kp", 20.0);
this->declare_parameter("left_joint_1.kd", 1.0);
// ...

// 读取参数
double kp = this->get_parameter("left_joint_1.kp").as_double();
double kd = this->get_parameter("left_joint_1.kd").as_double();
```

配置文件:

```
# openarm_controllers.yaml

hardware_interface:
  ros__parameters:
    left_joint_1:
      kp: 20.0
      kd: 1.0
    left_joint_2:
      kp: 20.0
      kd: 1.0
    # ...
```

29.5.2 运行时动态调整

使用 rqt_reconfigure:

```
# 安装
sudo apt install ros-humble-rqt-reconfigure

# 运行
ros2 run rqt_reconfigure rqt_reconfigure
```

Python 动态参数:

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
```

```python
from rcl_interfaces.srv import SetParameters
from rcl_interfaces.msg import Parameter, ParameterValue, ParameterType

class ParameterTuner(Node):
    def __init__(self):
        super().__init__('parameter_tuner')

        self.client = self.create_client(
            SetParameters,
            '/hardware_interface/set_parameters'
        )

    def set_kp_kd(self, joint_name, kp, kd):
        """ 动态设置 KP/KD"""
        request = SetParameters.Request()

        # KP 参数
        kp_param = Parameter()
        kp_param.name = f'{joint_name}.kp'
        kp_param.value.type = ParameterType.PARAMETER_DOUBLE
        kp_param.value.double_value = kp

        # KD 参数
        kd_param = Parameter()
        kd_param.name = f'{joint_name}.kd'
        kd_param.value.type = ParameterType.PARAMETER_DOUBLE
        kd_param.value.double_value = kd

        request.parameters = [kp_param, kd_param]

        future = self.client.call_async(request)
        rclpy.spin_until_future_complete(self, future)

        self.get_logger().info(f'设置 {joint_name}: KP={kp}, KD={kd}')
```

```python
def main():
    rclpy.init()

    tuner = ParameterTuner()

    # 动态调整 left_joint_1 的参数
    tuner.set_kp_kd('left_joint_1', kp=30.0, kd=1.5)

    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

29.6 调优案例

29.6.1 案例 1: 示教模式参数

需求: 手动拖动机械臂, 阻力小

调整前:

KP = 20, KD = 1.0

现象: 难以推动,阻力大

调整后:

KP = 2, KD = 0.2

现象: 易于推动,但会下垂

优化:

KP = 3, KD = 0.3

+ 重力补偿力矩

结果: 易推动且不下垂

29.6.2 案例 2: 高速运动震荡

需求: 快速运动, 出现震荡

现象:

KP = 50, KD = 1.0
到达目标后震荡5-6次才稳定

分析:

KP过高,KD不足以抑制震荡

调整:

方案1: 降低KP
  KP = 30, KD = 1.0
  结果: 震荡减少,但速度变慢

方案2: 增加KD
  KP = 50, KD = 2.5
  结果: 震荡消失,速度保持

选择方案2

29.6.3 案例 3: 负载运动下垂

需求: 抓取重物后, 手臂下垂

现象:

KP = 20, KD = 1.0
抓取1kg物体,手臂下降5cm

调整:

方案1: 增加KP
  KP = 40, KD = 1.0
  结果: 下垂减少,但震荡

方案2: KP+KD同时增加
  KP = 35, KD = 1.8
  结果: 下垂消失,运动平滑

方案3: 添加重力补偿

```
  KP = 25, KD = 1.2 + 重力补偿
  结果: 最优,能耗低
```

选择方案3

29.6.4 案例 4: 精确定位

需求: 插孔任务, 定位精度 ±0.5mm

调整前:

```
KP = 20, KD = 1.0
定位误差: ±2mm
```

调整过程:

```
尝试1: KP = 50, KD = 1.0
  误差: ±1.5mm, 有震荡

尝试2: KP = 50, KD = 2.0
  误差: ±1mm, 无震荡

尝试3: KP = 80, KD = 3.0
  误差: ±0.4mm, 无震荡
  ✓ 满足要求
```

29.7 不同控制器的参数调优

29.7.1 JointTrajectoryController

特点:

轨迹插值,连续控制

需要平衡响应速度和平滑性

推荐参数:

```
常规应用:
  KP: 20-30
  KD: 1.0-1.5
```

```
高速轨迹:
  KP: 15-25
  KD: 0.8-1.2

精确轨迹:
  KP: 30-50
  KD: 1.5-2.5
```

29.7.2 PositionController

特点:

点到点控制

可以使用较高增益

推荐参数:

```
快速定位:
  KP: 30-50
  KD: 1.5-2.5

精确定位:
  KP: 50-80
  KD: 2.0-3.5
```

29.7.3 EffortController (力控)

特点:

力矩控制为主

位置控制为辅

推荐参数:

```
柔顺控制:
  KP: 5-10
  KD: 0.3-0.8

混合控制:
```

```
  KP: 10-20
  KD: 0.5-1.2
```

29.8 调参工具

29.8.1 自动调参脚本

```
#!/usr/bin/env python3
"""
自动 KP/KD 调参工具
"""
import can
from robstride import Motor
import time
import numpy as np

class AutoTuner:
    def __init__(self, motor):
        self.motor = motor

    def evaluate_performance(self, kp, kd, target_pos):
        """
        评估性能
        返回分数 (越高越好)
        """
        # 回零
        self.motor.set_position(0.0, 0.0, 20, 1.0, 0.0)
        time.sleep(2)

        # 运动到目标
        start_time = time.time()
        self.motor.set_position(target_pos, 0.0, kp, kd, 0.0)

        # 记录响应
        positions = []
```

```
        times = []

        for _ in range(50):
            fb = self.motor.get_feedback()
            positions.append(fb['position'])
            times.append(time.time() - start_time)
            time.sleep(0.1)

        # 计算指标
        positions = np.array(positions)
        error = np.abs(positions - target_pos)

        # 稳定时间 (到达 ±2% 误差带)
        settling_threshold = 0.02
        settling_idx = np.where(error < settling_threshold)[0]
        settling_time = times[settling_idx[0]] if len(settling_idx) > 0 else times[-1]

        # 超调量
        overshoot = np.max(positions) - target_pos

        # 最终误差
        final_error = error[-10:].mean()

        # 综合评分 (越小越好, 转为越大越好)
        score = 1.0 / (settling_time + 10*abs(overshoot) + 100*final_error + 0.1)

        print(f"KP={kp:.1f}, KD={kd:.2f} → "
              f" 稳定时间 ={settling_time:.2f}s, "
              f" 超调 ={overshoot:.4f}, "
              f" 误差 ={final_error:.4f}, "
              f" 得分 ={score:.3f}")

        return score
```

```
    def grid_search(self, kp_range, kd_range, target_pos=0.5):
        """ 网格搜索最优参数"""
        best_score = -1
        best_kp = None
        best_kd = None

        for kp in kp_range:
            for kd in kd_range:
                score = self.evaluate_performance(kp, kd, target_pos)

                if score > best_score:
                    best_score = score
                    best_kp = kp
                    best_kd = kd

        print(f"\n最优参数: KP={best_kp}, KD={best_kd}, 得分 ={best_score:.3f}")
        return best_kp, best_kd

# 使用
bus0 = can.interface.Bus(channel='can0', bustype='socketcan')
motor = Motor(bus0, 1)
motor.enable()

tuner = AutoTuner(motor)

# 搜索范围
kp_range = [10, 15, 20, 25, 30, 40, 50]
kd_range = [0.5, 0.8, 1.0, 1.2, 1.5, 2.0, 2.5]

best_kp, best_kd = tuner.grid_search(kp_range, kd_range)

motor.disable()
```

29.8.2 实时调参界面

```python
#!/usr/bin/env python3
"""
实时调参 GUI (使用 tkinter)
"""
import tkinter as tk
from tkinter import Scale
import can
from robstride import Motor
import threading

class TunerGUI:
    def __init__(self):
        self.window = tk.Tk()
        self.window.title("KP/KD 调参工具")

        # 初始化电机
        bus0 = can.interface.Bus(channel='can0', bustype='socketcan')
        self.motor = Motor(bus0, 1)
        self.motor.enable()

        # KP 滑块
        tk.Label(self.window, text="KP:").pack()
        self.kp_scale = Scale(self.window, from_=0, to=100, resolution=1,
                              orient=tk.HORIZONTAL, length=400,
                              command=self.update_params)
        self.kp_scale.set(20)
        self.kp_scale.pack()

        # KD 滑块
        tk.Label(self.window, text="KD:").pack()
        self.kd_scale = Scale(self.window, from_=0, to=5, resolution=0.1,
                              orient=tk.HORIZONTAL, length=400,
                              command=self.update_params)
```

```
        self.kd_scale.set(1.0)
        self.kd_scale.pack()

        # 目标位置
        tk.Label(self.window, text=" 目标位置 (rad):").pack()
        self.target_scale = Scale(self.window, from_=-3.14, to=3.14, resolution=0.01,
                                  orient=tk.HORIZONTAL, length=400)
        self.target_scale.set(0.5)
        self.target_scale.pack()

        # 执行按钮
        tk.Button(self.window, text=" 执行运动", command=self.execute_motion).pack()

        # 回零按钮
        tk.Button(self.window, text=" 回零", command=self.go_home).pack()

        # 状态显示
        self.status_label = tk.Label(self.window, text=" 就绪")
        self.status_label.pack()

    def update_params(self, event=None):
        """ 更新参数显示"""
        kp = self.kp_scale.get()
        kd = self.kd_scale.get()
        self.status_label.config(text=f" 当前参数: KP={kp}, KD={kd}")

    def execute_motion(self):
        """ 执行运动"""
        kp = self.kp_scale.get()
        kd = self.kd_scale.get()
        target = self.target_scale.get()

        self.status_label.config(text=f" 执行: 目标 ={target:.2f}, KP={kp}, KD={kd}")
```

```
        # 在新线程中执行，避免阻塞 GUI
        threading.Thread(target=self._move, args=(target, kp, kd)).start()

    def _move(self, target, kp, kd):
        self.motor.set_position(target, 0.0, kp, kd, 0.0)

    def go_home(self):
        """ 回零"""
        self.motor.set_position(0.0, 0.0, 20, 1.0, 0.0)
        self.status_label.config(text=" 回零中...")

    def run(self):
        self.window.mainloop()
        self.motor.disable()

# 运行
gui = TunerGUI()
gui.run()
```

29.9 常见问题

29.9.1 震荡问题

症状: 到达目标后持续震荡

原因: KP 过高,KD 不足

解决:

1. 降低KP：减少20-30%
2. 增加KD：增加50-100%
3. 检查机械松动
4. 降低控制频率

29.9.2 响应慢

症状: 移动速度慢, 响应迟缓

原因: KP 过低

解决:

1. 增加KP: 增加50-100%
2. 适当增加KD (保持平滑)
3. 检查负载是否过重

29.9.3 稳态误差

症状: 无法准确到达目标位置

原因: KP 不足或有负载

解决:

1. 增加KP
2. 添加积分项 (KI)
3. 添加重力/摩擦力补偿

29.9.4 噪声/抖动

症状: 微小快速震动

原因: KD 过高或传感器噪声

解决:

1. 降低KD
2. 添加低通滤波
3. 降低控制频率

29.10 总结

29.10.1 调优要点

✓ 从保守值开始 (KP=10-20, KD=0.5-1.0)
✓ 先调KP后调KD
✓ 震荡加KD,慢了加KP
✓ 记录测试数据
✓ 不同应用使用不同参数

29.10.2 推荐流程

1. 确定应用场景

```
    ↓
2. 选择基础参数
    ↓
3. 实际测试
    ↓
4. 观察现象
    ↓
5. 调整参数
    ↓
6. 迭代优化
    ↓
7. 负载验证
```

29.10.3 下一步

- **03_ 控制模式说明** - MIT vs CSP 模式对比
- **04_ 控制器管理** - 高级管理技巧
- **08_ 高级应用** - 自适应控制

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

30 控制模式说明

30.1 MIT 模式与 CSP 模式对比

深入了解 Robstride 电机的控制模式及其特点。

30.2 控制模式概述

30.2.1 Robstride 支持的模式

Robstride 电机支持多种控制模式:

```
1. MIT模式 (Motion Control Mode)
   - 位置+速度+KP+KD+力矩混合控制
   - OpenArmX主要使用


2. CSP模式 (Cyclic Synchronous Position)
```

- 周期同步位置模式
- 纯位置控制

3. CSV模式 (Cyclic Synchronous Velocity)
   - 周期同步速度模式
   - 纯速度控制

4. CST模式 (Cyclic Synchronous Torque)
   - 周期同步力矩模式
   - 纯力矩控制

OpenArmX 使用:

默认: MIT模式

原因:

✓ 灵活性高

✓ 支持柔顺控制

✓ 可调KP/KD

✓ 力矩前馈

30.3 MIT 模式详解

30.3.1 MIT 模式原理

控制方程:

```
τ_command = KP × (θ_target - θ_actual)
          + KD × (ω_target - ω_actual)
          + τ_feedforward
```

τ_command: 命令力矩

θ_target: 目标位置

θ_actual: 实际位置

ω_target: 目标速度

ω_actual: 实际速度

τ_feedforward: 前馈力矩

命令格式:

```
motor.set_position(
    position=0.5,    # θ_target (rad)
    velocity=0.0,    # ω_target (rad/s)
    kp=20.0,         # 位置增益
    kd=1.0,          # 速度增益
    torque=0.0       # τ_feedforward (Nm)
)
```

30.3.2 MIT 模式特点

优点:

✓ 柔顺控制
- 低KP/KD实现柔性
- 示教模式易推动
- 人机协作安全

✓ 力矩控制
- 可叠加力矩
- 重力补偿
- 接触力控制

✓ 参数可调
- 实时调整KP/KD
- 适应不同任务
- 动态刚度调节

✓ 混合控制
- 位置+力矩
- 速度前馈
- 阻抗控制

缺点:

参数调试复杂
- 需要调KP/KD
- 不同负载需调整

精度依赖参数

- 高精度需高增益
- 可能引起震荡

30.3.3 MIT 模式应用

场景 1: 示教录制

```
# 低刚度，易推动
motor.set_position(
    position=current_position,  # 跟随当前位置
    velocity=0.0,
    kp=2.0,      # 低 KP
    kd=0.2,      # 低 KD
    torque=0.0
)
```

场景 2: 轨迹跟踪

```
# 中等刚度，平衡性能
motor.set_position(
    position=trajectory_point,
    velocity=0.0,
    kp=20.0,     # 中等 KP
    kd=1.0,      # 中等 KD
    torque=0.0
)
```

场景 3: 精确定位

```
# 高刚度，高精度
motor.set_position(
    position=target_position,
    velocity=0.0,
    kp=50.0,     # 高 KP
    kd=2.5,      # 高 KD
    torque=0.0
```

```
)
```

场景 4: 重力补偿

```
# 计算重力力矩
gravity_torque = calculate_gravity_torque(joint_positions)

motor.set_position(
    position=target_position,
    velocity=0.0,
    kp=15.0,
    kd=1.0,
    torque=gravity_torque  # 补偿重力
)
```

场景 5: 力控抓取

```
# 柔顺抓取
motor.set_position(
    position=grasp_position,
    velocity=0.0,
    kp=5.0,      # 低 KP, 柔顺
    kd=0.5,
    torque=1.0   # 小力矩保持
)
```

30.4 CSP 模式详解

30.4.1 CSP 模式原理

控制方式:

用户: 发送目标位置

电机: 内部PID跟踪位置

特点: 纯位置控制,内部闭环

命令格式:

```
# 伪代码 (Robstride SDK 可能不同)
motor.set_mode(CSP_MODE)
motor.set_target_position(0.5)  # rad
```

30.4.2 CSP 模式特点

优点:

✓ 简单易用
- 只需发送位置
- 无需调KP/KD
- 参数固定

✓ 精度稳定
- 出厂调优
- 一致性好
- 适合批量部署

✓ 标准化
- CANopen标准
- 兼容性好

缺点:

刚度固定
- 无法调节柔顺性
- 不适合示教
- 人机交互差

无力矩控制
- 无法叠加力矩
- 难以重力补偿
- 力控困难

灵活性低
- 参数不可调
- 难以优化特定任务

30.4.3 CSP 模式应用

适用场景:

- 固定任务,无需调节
- 高精度重复定位
- 标准化应用
- 无力控需求

不适用场景:

- 示教录制
- 人机协作
- 力控应用
- 动态调节刚度

30.5 MIT vs CSP 对比

30.5.1 功能对比表

特性	MIT 模式	CSP 模式
控制方式	位置 + 速度 +KP+KD+ 力矩	纯位置
参数调节	实时可调 KP/KD	固定
柔顺控制	√ 支持	不支持
力矩前馈	√ 支持	不支持
示教模式	√ 易实现	困难
精度	依赖参数	稳定
复杂度	★★★★☆	★★☆☆☆
灵活性	★★★★★	★★☆☆☆
人机协作	★★★★★	★☆☆☆☆

30.5.2 性能对比

响应速度:

```
MIT模式: 取决于KP/KD设置
  - 高KP → 快速响应
  - 低KP → 慢速响应
```

CSP模式：固定响应速度

- 出厂优化
- 中等速度

定位精度:

MIT模式：可调

- 高KP → 高精度
- 低KP → 低精度

CSP模式：固定高精度

- ±0.01 rad 典型值

能耗:

MIT模式：可优化

- 低KP → 低能耗
- 高KP → 高能耗

CSP模式：中等能耗

- 固定增益

30.5.3 应用场景对比

MIT 模式适用:

✓ 示教录制

✓ 人机协作

✓ 力控装配

✓ 柔顺抓取

✓ 动态负载

✓ 研发测试

CSP 模式适用:

✓ 固定自动化

✓ 高精度定位

✓ 批量生产

✓ 简单应用

✓ 标准设备集成

30.6 其他控制模式

30.6.1 CSV 模式 (速度控制)

原理:

用户: 发送目标速度

电机: 内部PID跟踪速度

应用:

- 传送带
- 连续旋转
- 速度同步

示例:

```
motor.set_mode(CSV_MODE)
motor.set_target_velocity(1.0)  # rad/s
```

30.6.2 CST 模式 (力矩控制)

原理:

用户: 发送目标力矩

电机: 直接输出力矩

应用:

- 纯力控
- 阻抗控制
- 力传感器标定

示例:

```
motor.set_mode(CST_MODE)
motor.set_target_torque(2.0)  # Nm
```

[警告] 注意:

纯力矩控制无位置限制

需要外部安全监控

避免超限和碰撞

30.6.3 混合模式

MIT 本质是混合模式:

位置控制 + 速度控制 + 力矩控制

可实现:
- 位置主导 (高KP, 低torque)
- 力矩主导 (低KP, 高torque)
- 阻抗控制 (中KP, 中torque)

30.7 OpenArmX 控制模式配置

30.7.1 当前配置

默认使用 MIT 模式:

```
// openarm_hardware_interface.cpp

void OpenArmHardwareInterface::write(...)
{
    // MIT 模式控制
    motors_[i].set_position(
        target_position,
        0.0,      // velocity
        kp,       // 可配置
        kd,       // 可配置
        0.0       // torque
    );
}
```

30.7.2 切换到 CSP 模式

修改硬件接口:

```
// 初始化时设置模式
void OpenArmHardwareInterface::on_activate(...)
{
    for (auto& motor : motors_)
```

```
    {
        // motor.set_mode(CSP_MODE);  // 如果 SDK 支持
    }
}

// 控制时仅发送位置
void OpenArmHardwareInterface::write(...)
{
    // motors_[i].set_target_position(target_position);
}
```

[警告] 注意:

切换模式可能需要:

1. SDK支持
2. 固件版本
3. 重新编译
4. 测试验证

30.8 模式选择指南

30.8.1 决策流程图

```
需要示教录制?
  ├── 是 → MIT模式
  └── 否
      ↓
需要力控?
  ├── 是 → MIT模式
  └── 否
      ↓
需要调节刚度?
  ├── 是 → MIT模式
  └── 否
      ↓
固定任务,高精度?
  ├── 是 → CSP模式
```

```
└── 否 → MIT模式 (更灵活)
```

30.8.2 推荐配置

OpenArmX 推荐:

```
MIT模式 (默认)

原因:
✓ 支持示教
✓ 柔顺控制
✓ 灵活性高
✓ 适合研发和多样化应用
```

特殊应用:

```
如果是固定生产线,无需示教:
  → 可考虑CSP模式

如果需要纯力控:
  → MIT模式 (低KP + 力矩)
  或 CST模式
```

30.9 实际应用案例

30.9.1 案例 1: 柔顺装配 (MIT)

```python
#!/usr/bin/env python3
"""
柔顺装配示例 - 使用 MIT 模式
"""
import can
from robstride import Motor
import time

bus0 = can.interface.Bus(channel='can0', bustype='socketcan')
motor = Motor(bus0, 4)  # 肘部关节
motor.enable()
```

```
# 阶段 1: 移动到插孔上方 (高刚度)
print(" 阶段 1: 快速定位")
motor.set_position(
    position=-0.8,
    velocity=0.0,
    kp=30.0,     # 高 KP, 快速
    kd=1.5,
    torque=0.0
)
time.sleep(3)

# 阶段 2: 接近插孔 (中刚度)
print(" 阶段 2: 接近目标")
motor.set_position(
    position=-1.0,
    velocity=0.0,
    kp=15.0,     # 中 KP
    kd=1.0,
    torque=0.0
)
time.sleep(2)

# 阶段 3: 插入 (低刚度 + 力控)
print(" 阶段 3: 柔顺插入")
for i in range(20):
    motor.set_position(
        position=-1.0,
        velocity=0.0,
        kp=5.0,      # 低 KP, 柔顺
        kd=0.5,
        torque=2.0   # 小力矩推入
    )
    time.sleep(0.1)
```

```
    # 检测接触力
    fb = motor.get_feedback()
    if abs(fb['torque']) > 5.0:
        print(" 检测到大阻力，停止")
        break

print(" 装配完成")
motor.disable()
```

30.9.2 案例 2: 示教录制 (MIT)

```
#!/usr/bin/env python3
"""
示教录制 - MIT 模式低刚度
"""
import can
from robstride import Motor
import time

bus0 = can.interface.Bus(channel='can0', bustype='socketcan')
motors = [Motor(bus0, i) for i in range(1, 8)]

for m in motors:
    m.enable()

print(" 进入示教模式，可以手动拖动机械臂")
print(" 按 Ctrl+C 退出")

trajectory = []

try:
    while True:
        # 读取当前位置
        positions = [m.get_feedback()['position'] for m in motors]
```

```
        # 记录轨迹
        trajectory.append(positions)

        # 低刚度跟随
        for i, motor in enumerate(motors):
            if i < 3:  # 肩部
                kp, kd = 3.0, 0.3
            else:      # 肘腕
                kp, kd = 2.0, 0.2

            motor.set_position(
                position=positions[i],  # 跟随当前位置
                velocity=0.0,
                kp=kp,    # 低刚度，易推动
                kd=kd,
                torque=0.0
            )

        time.sleep(0.1)

except KeyboardInterrupt:
    print(f"\n录制结束，共{len(trajectory)}个点")

    # 保存轨迹
    import yaml
    with open('taught_trajectory.yaml', 'w') as f:
        yaml.dump({'points': trajectory}, f)

    for m in motors:
        m.disable()
```

30.9.3 案例 3: 高精度定位 (CSP 或高增益 MIT)

```python
#!/usr/bin/env python3
"""
高精度定位 - 使用高增益 MIT
(如果使用 CSP, 则无需 KP/KD)
"""
import can
from robstride import Motor
import time

bus0 = can.interface.Bus(channel='can0', bustype='socketcan')
motor = Motor(bus0, 1)
motor.enable()

# 精确目标位置
target_positions = [0.0, 0.5, 1.0, 0.5, 0.0]

for target in target_positions:
    print(f" 移动到 {target:.3f} rad")

    # MIT 高增益模式
    motor.set_position(
        position=target,
        velocity=0.0,
        kp=80.0,     # 高 KP, 高精度
        kd=3.0,      # 高 KD, 无震荡
        torque=0.0
    )

    # 等待到达
    time.sleep(3)

    # 验证精度
    fb = motor.get_feedback()
```

```
    error = abs(fb['position'] - target)
    print(f"  实际: {fb['position']:.4f}, 误差: {error:.4f} rad")

motor.disable()
```

30.10 常见问题

30.10.1 MIT 模式调参困难

问题: 不同任务需要不同参数, 难以管理

解决:

```
# 使用参数字典
PARAM_PROFILES = {
    'teaching': {'kp': 3.0, 'kd': 0.3},
    'normal':   {'kp': 20.0, 'kd': 1.0},
    'precise':  {'kp': 50.0, 'kd': 2.5},
    'compliant':{'kp': 5.0, 'kd': 0.5}
}

# 切换配置
profile = PARAM_PROFILES['teaching']
motor.set_position(pos, 0, profile['kp'], profile['kd'], 0)
```

30.10.2 CSP 模式缺少灵活性

问题: CSP 模式无法调节, 太刚

解决:

```
使用MIT模式代替
设置固定的中等增益 (KP=20, KD=1.0)
获得CSP的简单性 + MIT的灵活性
```

30.10.3 力矩控制不稳定

问题: 使用 MIT 力矩前馈时不稳定

解决:

1. 降低KP/KD,避免与力矩冲突
2. 力矩值从小到大逐步增加
3. 添加力矩限幅
4. 监控实际力矩,避免超限

30.11 总结

30.11.1 控制模式选择

MIT模式 (推荐):
✓ 灵活性高
✓ 支持力控
✓ 适合研发
✓ OpenArmX默认

CSP模式:
✓ 简单易用
✓ 精度稳定
✓ 适合生产
不支持示教

30.11.2 MIT 模式要点

☐ 理解KP/KD作用
☐ 不同任务不同参数
☐ 善用力矩前馈
☐ 调参需耐心

30.11.3 下一步

- **04_ 控制器管理** - 控制器生命周期管理
- **08_ 高级应用** - 力控与柔顺控制深入
- **02_ 参数调优** - KP/KD 调参详解

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

31 控制器管理

31.1 ros2_control 控制器高级管理

掌握控制器的加载、卸载、切换和监控技巧。

31.2 控制器管理概述

31.2.1 Controller Manager

作用:

`ros2_control`的核心组件

管理所有控制器

协调硬件接口

提供`ROS2`服务接口

主要功能:

- ☐ 加载/卸载控制器
- ☐ 配置/激活/停止控制器
- ☐ 监控控制器状态
- ☐ 切换控制器
- ☐ 硬件接口管理

31.2.2 控制器生命周期

```
                ┌──────────────────────┐
                │ unconfigured │
                └──────────────┬───────┘
                       │ configure
                       ↓
                ┌──────────────────┐
            ┌───────┤ inactive ├───────┐
            │     └──────────┬───────┘     │
            │          │          │
    cleanup │    activate       │
            │          ↓          │
            │     ┌──────────────┐      │
```

```
              └──────→│  active  │       │
                      └──────────┬────────┘       │
                                 │          │
                    deactivate       │
                                 └──────────────────┘
```

unconfigured: 未配置

inactive: 已配置,未激活

active: 激活运行

状态转换:

unconfigured → configure → inactive

inactive → activate → active

active → deactivate → inactive

inactive → cleanup → unconfigured

31.3 控制器命令行管理

31.3.1 列出控制器

```
# 列出所有控制器
ros2 control list_controllers

# 输出示例:
# joint_state_broadcaster[joint_state_broadcaster/JointStateBroadcaster] active
# left_arm_controller[joint_trajectory_controller/JointTrajectoryController] active
# right_arm_controller[joint_trajectory_controller/JointTrajectoryController] active
```

输出格式:

控制器名称[类型] 状态

状态:

active: 激活

inactive: 未激活

unconfigured: 未配置

31.3.2 查看硬件接口

```
# 列出硬件接口
ros2 control list_hardware_interfaces

# 输出示例:
# command interfaces:
#   left_joint_1/position [available] [claimed]
#   left_joint_2/position [available] [claimed]
#   ...
# state interfaces:
#   left_joint_1/position [available]
#   left_joint_1/velocity [available]
#   ...
```

接口状态:

available: 硬件提供

claimed: 被控制器占用

unclaimed: 未被占用

31.3.3 加载控制器

```
# 加载控制器
ros2 control load_controller left_arm_controller

# 加载并启动
ros2 control load_controller --set-state start left_arm_controller

# 加载并配置 (不启动)
ros2 control load_controller --set-state configure left_arm_controller
```

参数:

--set-state start: 加载后激活

--set-state configure: 加载后配置(inactive)

(无参数): 仅加载(unconfigured)

31.3.4 卸载控制器

```
# 卸载控制器
ros2 control unload_controller left_arm_controller

# 注意：需要先停止控制器
```

完整流程:

```
#  停止
ros2 control set_controller_state left_arm_controller stop

#  卸载
ros2 control unload_controller left_arm_controller
```

31.3.5 切换控制器状态

```
# 配置
ros2 control set_controller_state left_arm_controller configure

# 启动
ros2 control set_controller_state left_arm_controller start

# 停止
ros2 control set_controller_state left_arm_controller stop
```

状态转换表:

```
当前状态        命令              目标状态
unconfigured → configure    → inactive
inactive     → start        → active
active       → stop         → inactive
inactive     → cleanup      → unconfigured (需先unload)
```

31.3.6 切换控制器

```
# 切换控制器 (原子操作)
ros2 control switch_controllers \
  --start-controllers left_arm_trajectory_controller \
  --stop-controllers left_arm_position_controller \
  --strictness 2 \
  --start-asap false
```

参数说明:

```
--start-controllers: 要启动的控制器(列表)
--stop-controllers: 要停止的控制器(列表)
--strictness: 严格程度
  1 = BEST_EFFORT: 尽力而为
  2 = STRICT: 严格,全部成功或全部失败
--start-asap: 是否立即启动
  true: 立即
  false: 等待当前循环结束
```

31.4 Python 管理接口

31.4.1 基本管理类

```
#!/usr/bin/env python3
"""
控制器管理 Python 接口
"""
import rclpy
from rclpy.node import Node
from controller_manager_msgs.srv import (
    ListControllers,
    LoadController,
    UnloadController,
    SwitchController,
    ConfigureController
)
```

```python
class ControllerManager(Node):
    def __init__(self):
        super().__init__('controller_manager_client')

        # 创建服务客户端
        self.list_client = self.create_client(
            ListControllers,
            '/controller_manager/list_controllers'
        )

        self.load_client = self.create_client(
            LoadController,
            '/controller_manager/load_controller'
        )

        self.unload_client = self.create_client(
            UnloadController,
            '/controller_manager/unload_controller'
        )

        self.switch_client = self.create_client(
            SwitchController,
            '/controller_manager/switch_controller'
        )

        self.configure_client = self.create_client(
            ConfigureController,
            '/controller_manager/configure_controller'
        )

        # 等待服务可用
        self.wait_for_services()

    def wait_for_services(self):
```

```
        """ 等待所有服务可用"""
        services = [
            self.list_client,
            self.load_client,
            self.unload_client,
            self.switch_client,
            self.configure_client
        ]

        for service in services:
            service.wait_for_service(timeout_sec=5.0)

        self.get_logger().info(" 所有服务已就绪")

    def list_controllers(self):
        """ 列出所有控制器"""
        request = ListControllers.Request()
        future = self.list_client.call_async(request)
        rclpy.spin_until_future_complete(self, future)

        result = future.result()
        self.get_logger().info(f" 找到 {len(result.controller)} 个控制器")

        for ctrl in result.controller:
            self.get_logger().info(
                f"  {ctrl.name} [{ctrl.type}] - {ctrl.state}"
            )

        return result.controller

    def load_controller(self, controller_name):
        """ 加载控制器"""
        request = LoadController.Request()
        request.name = controller_name
```

```
        future = self.load_client.call_async(request)
        rclpy.spin_until_future_complete(self, future)

        result = future.result()
        if result.ok:
            self.get_logger().info(f"✓ 加载成功: {controller_name}")
            return True
        else:
            self.get_logger().error(f" 加载失败: {controller_name}")
            return False

    def unload_controller(self, controller_name):
        """ 卸载控制器"""
        request = UnloadController.Request()
        request.name = controller_name

        future = self.unload_client.call_async(request)
        rclpy.spin_until_future_complete(self, future)

        result = future.result()
        if result.ok:
            self.get_logger().info(f"✓ 卸载成功: {controller_name}")
            return True
        else:
            self.get_logger().error(f" 卸载失败: {controller_name}")
            return False

    def configure_controller(self, controller_name):
        """ 配置控制器"""
        request = ConfigureController.Request()
        request.name = controller_name

        future = self.configure_client.call_async(request)
```

```
        rclpy.spin_until_future_complete(self, future)

        result = future.result()
        if result.ok:
            self.get_logger().info(f"✓ 配置成功: {controller_name}")
            return True
        else:
            self.get_logger().error(f" 配置失败: {controller_name}")
            return False

    def switch_controllers(self, start_controllers, stop_controllers):
        """ 切换控制器"""
        request = SwitchController.Request()
        request.start_controllers = start_controllers
        request.stop_controllers = stop_controllers
        request.strictness = 2  # STRICT
        request.start_asap = False
        request.timeout = rclpy.duration.Duration(seconds=0.0).to_msg()

        future = self.switch_client.call_async(request)
        rclpy.spin_until_future_complete(self, future)

        result = future.result()
        if result.ok:
            self.get_logger().info(f"✓ 切换成功")
            return True
        else:
            self.get_logger().error(f" 切换失败")
            return False

def main():
    rclpy.init()

    manager = ControllerManager()
```

```
    # 示例: 列出控制器
    controllers = manager.list_controllers()

    # 示例: 切换控制器
    manager.switch_controllers(
        start_controllers=['left_arm_controller'],
        stop_controllers=[]
    )

    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

31.4.2 控制器状态监控

```
#!/usr/bin/env python3
"""
实时监控控制器状态
"""
import rclpy
from rclpy.node import Node
from controller_manager_msgs.msg import ControllerState
from controller_manager_msgs.srv import ListControllers
import time

class ControllerMonitor(Node):
    def __init__(self):
        super().__init__('controller_monitor')

        self.list_client = self.create_client(
            ListControllers,
            '/controller_manager/list_controllers'
        )
```

```
        # 定期检查
        self.timer = self.create_timer(2.0, self.check_controllers)

    def check_controllers(self):
        """ 定期检查控制器状态"""
        if not self.list_client.wait_for_service(timeout_sec=1.0):
            self.get_logger().warn("Controller Manager 服务不可用")
            return

        request = ListControllers.Request()
        future = self.list_client.call_async(request)

        rclpy.spin_until_future_complete(self, future, timeout_sec=1.0)

        if future.result() is None:
            return

        result = future.result()

        print("\n========== 控制器状态 ==========")
        print(f" 时间: {time.strftime('%H:%M:%S')}")
        print(f" 总数: {len(result.controller)}")
        print("-" * 50)

        for ctrl in result.controller:
            status_icon = "" if ctrl.state == "active" else "[红]"
            print(f"{status_icon} {ctrl.name:<30} {ctrl.state}")

        print("=" * 50)

def main():
    rclpy.init()
```

```
    monitor = ControllerMonitor()

    try:
        rclpy.spin(monitor)
    except KeyboardInterrupt:
        pass

    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

31.5 控制器切换策略

31.5.1 零宕机切换

场景: 不中断运动, 平滑切换控制器

```
def smooth_switch(manager, from_controller, to_controller):
    """
    平滑切换控制器
    """
    # 1. 加载目标控制器 (如果未加载)
    manager.load_controller(to_controller)

    # 2. 配置目标控制器
    manager.configure_controller(to_controller)

    # 3. 原子切换
    success = manager.switch_controllers(
        start_controllers=[to_controller],
        stop_controllers=[from_controller]
    )

    if success:
        # 4. 卸载旧控制器 (可选)
```

```
        # manager.unload_controller(from_controller)
        print(f" 切换成功: {from_controller} → {to_controller}")
    else:
        print(" 切换失败")

    return success
```

31.5.2 停止-启动切换

场景: 完全停止后再启动, 确保安全

```
def safe_switch(manager, from_controller, to_controller):
    """
    安全切换 (先停止再启动)
    """
    import time

    # 1. 停止当前控制器
    manager.switch_controllers(
        start_controllers=[],
        stop_controllers=[from_controller]
    )

    # 2. 等待停稳
    time.sleep(1.0)

    # 3. 启动新控制器
    manager.switch_controllers(
        start_controllers=[to_controller],
        stop_controllers=[]
    )

    print(f" 安全切换完成: {from_controller} → {to_controller}")
```

31.5.3 备用控制器

场景: 主控制器失败时自动切换到备用

```
class FallbackController(Node):
    def __init__(self):
        super().__init__('fallback_controller')

        self.manager = ControllerManager()

        self.primary = 'left_arm_controller'
        self.fallback = 'left_arm_position_controller'

        # 监控主控制器
        self.timer = self.create_timer(1.0, self.monitor)

    def monitor(self):
        """ 监控主控制器，失败时切换"""
        controllers = self.manager.list_controllers()

        primary_state = None
        for ctrl in controllers:
            if ctrl.name == self.primary:
                primary_state = ctrl.state
                break

        if primary_state != 'active':
            self.get_logger().warn(f" 主控制器异常: {primary_state}，切换到备用")

            self.manager.switch_controllers(
                start_controllers=[self.fallback],
                stop_controllers=[self.primary]
            )
```

31.6 批量控制器管理

31.6.1 批量启动

```bash
#!/bin/bash
# start_all_controllers.sh

CONTROLLERS=(
    "joint_state_broadcaster"
    "left_arm_controller"
    "right_arm_controller"
    "left_gripper_controller"
    "right_gripper_controller"
)

for ctrl in "${CONTROLLERS[@]}"; do
    echo " 启动 $ctrl ..."
    ros2 control load_controller --set-state start $ctrl

    if [ $? -eq 0 ]; then
        echo "  ✓ $ctrl 启动成功"
    else
        echo "   $ctrl 启动失败"
        exit 1
    fi
done

echo " 所有控制器已启动"
```

31.6.2 批量停止

```bash
#!/bin/bash
# stop_all_controllers.sh

# 获取所有 active 控制器
```

```
ACTIVE_CONTROLLERS=$(ros2 control list_controllers | grep "active" | awk '{print $1}' | sed 's/\[.*//'

for ctrl in $ACTIVE_CONTROLLERS; do
    # 保留 joint_state_broadcaster
    if [ "$ctrl" != "joint_state_broadcaster" ]; then
        echo " 停止 $ctrl ..."
        ros2 control set_controller_state $ctrl stop
    fi
done

echo " 所有控制器已停止"
```

31.6.3 Python 批量管理

```
#!/usr/bin/env python3
"""
批量控制器管理
"""
import rclpy
from controller_manager import ControllerManager

def start_all_controllers(manager, controller_list):
    """ 批量启动控制器"""
    failed = []

    for ctrl_name in controller_list:
        print(f" 启动 {ctrl_name} ...")

        # 加载并启动
        if manager.load_controller(ctrl_name):
            if manager.configure_controller(ctrl_name):
                success = manager.switch_controllers(
                    start_controllers=[ctrl_name],
                    stop_controllers=[]
```

```
                )

                if not success:
                    failed.append(ctrl_name)
            else:
                failed.append(ctrl_name)
        else:
            failed.append(ctrl_name)

    if len(failed) == 0:
        print("✓ 所有控制器启动成功")
        return True
    else:
        print(f" 以下控制器启动失败: {failed}")
        return False

def main():
    rclpy.init()

    manager = ControllerManager()

    controllers = [
        'joint_state_broadcaster',
        'left_arm_controller',
        'right_arm_controller',
        'left_gripper_controller',
        'right_gripper_controller'
    ]

    start_all_controllers(manager, controllers)

    rclpy.shutdown()

if __name__ == '__main__':
```

```
    main()
```

31.7 控制器诊断

31.7.1 诊断检查

```
#!/usr/bin/env python3
"""
控制器健康检查
"""
import rclpy
from controller_manager import ControllerManager

def diagnose_controllers(manager):
    """ 诊断控制器问题"""
    controllers = manager.list_controllers()

    print("\n========== 控制器诊断 ==========\n")

    issues = []

    # 检查 1: 是否有控制器
    if len(controllers) == 0:
        issues.append(" 错误: 没有找到任何控制器")

    # 检查 2: joint_state_broadcaster 必须 active
    jsc_found = False
    for ctrl in controllers:
        if 'joint_state_broadcaster' in ctrl.name:
            jsc_found = True
            if ctrl.state != 'active':
                issues.append(f" 警告: joint_state_broadcaster 状态为{ctrl.state}, 应为 active")

    if not jsc_found:
        issues.append(" 错误: 未找到 joint_state_broadcaster")
```

```
    # 检查 3: 检查 claimed 接口
    # (需要调用 list_hardware_interfaces 服务)

    # 检查 4: 检查配置一致性
    for ctrl in controllers:
        if ctrl.state == 'unconfigured':
            issues.append(f" 提示: {ctrl.name}未配置")

    # 输出结果
    if len(issues) == 0:
        print("✓ 所有检查通过, 控制器正常")
    else:
        print(f" 发现 {len(issues)} 个问题:")
        for issue in issues:
            print(f"  - {issue}")

    print("\n" + "=" * 50 + "\n")

    return len(issues) == 0

def main():
    rclpy.init()

    manager = ControllerManager()

    diagnose_controllers(manager)

    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

31.7.2 日志监控

```
# 查看 Controller Manager 日志
ros2 topic echo /rosout | grep controller_manager

# 查看特定控制器日志
ros2 topic echo /rosout | grep left_arm_controller
```

31.8 高级管理技巧

31.8.1 控制器参数热更新

```
#!/usr/bin/env python3
"""
运行时更新控制器参数
"""
import rclpy
from rclpy.node import Node
from rcl_interfaces.srv import SetParameters
from rcl_interfaces.msg import Parameter, ParameterValue

class ParameterUpdater(Node):
    def __init__(self):
        super().__init__('param_updater')

        self.client = self.create_client(
            SetParameters,
            '/left_arm_controller/set_parameters'
        )

    def update_constraints(self, joint_name, trajectory_tolerance, goal_tolerance):
        """ 更新轨迹容差"""
        request = SetParameters.Request()

        # 轨迹容差
```

```
        traj_param = Parameter()
        traj_param.name = f'constraints.{joint_name}.trajectory'
        traj_param.value.type = 3  # DOUBLE
        traj_param.value.double_value = trajectory_tolerance

        # 目标容差
        goal_param = Parameter()
        goal_param.name = f'constraints.{joint_name}.goal'
        goal_param.value.type = 3
        goal_param.value.double_value = goal_tolerance

        request.parameters = [traj_param, goal_param]

        future = self.client.call_async(request)
        rclpy.spin_until_future_complete(self, future)

        self.get_logger().info(f" 更新 {joint_name} 容差")

# 使用
rclpy.init()
updater = ParameterUpdater()
updater.update_constraints('left_joint_1', trajectory_tolerance=0.2, goal_tolerance=0.05)
rclpy.shutdown()
```

31.8.2 控制器性能监控

```
#!/usr/bin/env python3
"""
监控控制器性能
"""
import rclpy
from rclpy.node import Node
from control_msgs.msg import JointTrajectoryControllerState
```

```python
class PerformanceMonitor(Node):
    def __init__(self):
        super().__init__('perf_monitor')

        self.subscription = self.create_subscription(
            JointTrajectoryControllerState,
            '/left_arm_controller/state',
            self.state_callback,
            10
        )

        self.error_history = []

    def state_callback(self, msg):
        """ 监控误差"""
        if len(msg.desired.positions) > 0:
            # 计算误差
            errors = [abs(d - a) for d, a in zip(
                msg.desired.positions,
                msg.actual.positions
            )]

            max_error = max(errors)
            avg_error = sum(errors) / len(errors)

            self.error_history.append(max_error)

            # 保持最近 100 个样本
            if len(self.error_history) > 100:
                self.error_history.pop(0)

            # 输出统计
            if len(self.error_history) % 10 == 0:
                import statistics
```

```
                print(f" 最大误差: {max_error:.4f}, "
                      f" 平均误差: {avg_error:.4f}, "
                      f" 标准差: {statistics.stdev(self.error_history):.4f}")

def main():
    rclpy.init()
    monitor = PerformanceMonitor()
    rclpy.spin(monitor)
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

31.9 常见问题

31.9.1 控制器加载失败

症状: `load_controller` 返回 False

排查:

```
# 检查配置文件
cat openarm_controllers.yaml

# 检查控制器类型是否安装
ros2 pkg list | grep controller

# 查看日志
ros2 topic echo /rosout | grep ERROR

# 检查硬件接口
ros2 control list_hardware_interfaces
```

31.9.2 控制器无法启动

症状: `start` 命令失败

原因: - 硬件接口不可用 - 接口被其他控制器占用 - 配置错误

解决:

```
#  检查硬件接口是否 claimed
ros2 control list_hardware_interfaces

#  停止冲突的控制器
ros2 control list_controllers
ros2 control set_controller_state <冲突控制器> stop

#  重新启动
ros2 control set_controller_state <目标控制器> start
```

31.9.3 切换控制器失败

症状: switch_controllers 失败

排查:

```
# 检查 strictness 设置
# strictness=2 (STRICT) 要求全部成功
# 改为 strictness=1 (BEST_EFFORT) 可能解决

manager.switch_controllers(
    start_controllers=[...],
    stop_controllers=[...],
    strictness=1  # 尝试 BEST_EFFORT
)
```

31.10 总结

31.10.1 管理要点

✓ 理解控制器生命周期

✓ 使用命令行和Python接口

✓ 平滑切换避免中断

✓ 定期监控状态

✓ 批量管理提高效率

31.10.2 最佳实践

- ☐ 启动时按顺序加载控制器
- ☐ 使用脚本自动化管理
- ☐ 监控控制器健康状态
- ☐ 切换前检查接口占用
- ☐ 记录控制器配置变更

31.10.3 下一步

- **08_ 高级应用** - 力控与自适应控制
- **09_ 故障排查** - 控制器问题诊断
- **10_ 开发者指南** - 自定义控制器开发

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

32 力控与柔顺控制

32.1 力控制与柔顺控制应用

掌握 OpenArmX 的力控技术, 实现人机协作和精密装配。

32.2 力控概述

32.2.1 什么是力控

力控制 (Force Control):

控制机器人与环境的接触力

而非仅控制位置

实现柔顺交互

柔顺控制 (Compliance Control):

机器人"软"的行为

受力时可以让步

类似人的手臂

阻抗控制 (Impedance Control):

控制位置-力之间的关系

调节"刚度"和"阻尼"

MIT模式的核心

32.2.2 OpenArmX 力控能力

硬件支持:

✓ Robstride电机

- MIT模式
- 力矩传感器
- 电流检测
- 实时力矩反馈

✓ 准直驱设计

- 低摩擦
- 高透明性

- 力矩传递好

软件支持:

✓ MIT模式控制
- 可调KP/KD
- 力矩前馈
- 位置+力矩混合

✓ ros2_control
- Effort接口
- 实时控制

32.3 阻抗控制原理

32.3.1 阻抗方程

基本方程:

F = M × ⌧ + D × x˙+ K × x

F: 外力
M: 惯性 (Mass)
D: 阻尼 (Damping) ~ KD
K: 刚度 (Stiffness) ~ KP
x: 位置偏差

在 MIT 模式中:

τ = KP × (θ_target - θ_actual) + KD × (ω_target - ω_actual) + τ_feedforward

KP: 控制刚度
KD: 控制阻尼
τ_feedforward: 额外力矩

32.3.2 刚度调节

高刚度 (High Stiffness):

KP: 高 (50-100)

KD: 高 (2-5)

特点:
- 硬,不易推动
- 精确定位
- 快速响应

低刚度 (Low Stiffness):

KP: 低 (1-10)
KD: 低 (0.1-1)

特点:
- 软,易推动
- 柔顺交互
- 安全人机协作

示例:

```
# 高刚度 - 精确定位
motor.set_position(target, 0, kp=80, kd=3, torque=0)

# 低刚度 - 示教模式
motor.set_position(current, 0, kp=3, kd=0.3, torque=0)

# 中等刚度 - 常规控制
motor.set_position(target, 0, kp=20, kd=1, torque=0)
```

32.4 力矩控制

32.4.1 纯力矩控制

原理:

直接指定输出力矩
不管位置
实现主动力输出

实现:

```
# 方法 1: 低 KP + 高力矩
motor.set_position(
    position=current_position,  # 跟随当前位置
    velocity=0.0,
    kp=1.0,      # 极低 KP, 几乎不抵抗
    kd=0.1,
    torque=5.0   # 主导力矩
)

# 方法 2: 使用 CST 模式 (如果支持)
# motor.set_torque(5.0)
```

应用场景:

- 抛光/打磨 (恒力)
- 按压操作
- 力传感器标定
- 重力补偿

32.4.2 力矩前馈

原理:

在位置控制基础上叠加力矩
实现混合控制

示例:

```
# 重力补偿
def calculate_gravity_torque(joint_positions, joint_index):
    """
    计算关节重力力矩
    简化模型
    """
    # 假设手臂质量 m=2kg, 臂长 l=0.5m, 重力 g=9.8m/s²
    m = 2.0
    l = 0.5
    g = 9.8
```

```
    # 重力力矩 τ = m*g*l*cos(θ)
    theta = joint_positions[joint_index]
    torque = m * g * l * np.cos(theta)

    return torque

# 带重力补偿的控制
joint_positions = get_current_positions()
gravity_torque = calculate_gravity_torque(joint_positions, joint_index=2)

motor.set_position(
    position=target_position,
    velocity=0.0,
    kp=20.0,
    kd=1.0,
    torque=gravity_torque  # 补偿重力
)
```

效果:

✓ 减少位置误差
✓ 降低能耗
✓ 减少电机发热
✓ 提高负载能力

32.4.3 恒力控制

目标: 保持恒定接触力

实现:

```
#!/usr/bin/env python3
"""
恒力控制示例 - 抛光应用
"""
import can
from robstride import Motor
```

```
import time

bus0 = can.interface.Bus(channel='can0', bustype='socketcan')
motor = Motor(bus0, 4)
motor.enable()

# 目标接触力
target_force = 10.0  # N

# 转换为力矩 (简化: 假设力臂 0.3m)
force_arm = 0.3  # m
target_torque = target_force * force_arm  # 3.0 Nm

# 初始位置
initial_position = motor.get_feedback()['position']

# 力控循环
try:
    while True:
        # 读取当前力矩
        fb = motor.get_feedback()
        current_torque = fb['torque']
        current_position = fb['position']

        # 力矩误差
        torque_error = target_torque - current_torque

        # 位置调整 (积分控制)
        position_adjustment = torque_error * 0.01  # 积分增益

        # 新目标位置
        new_target = current_position + position_adjustment

        # 发送控制命令 (低刚度 + 力矩前馈)
```

```
        motor.set_position(
            position=new_target,
            velocity=0.0,
            kp=5.0,              # 低刚度
            kd=0.5,
            torque=target_torque  # 力矩前馈
        )

        print(f" 力矩: {current_torque:.2f}/{target_torque:.2f} Nm, "
              f" 位置: {current_position:.3f} rad")

        time.sleep(0.01)  # 100Hz 控制

except KeyboardInterrupt:
    motor.disable()
```

32.5 柔顺控制应用

32.5.1 柔顺装配

场景: 插销孔装配, 允许误差

```
#!/usr/bin/env python3
"""
柔顺装配 - Peg-in-Hole
"""
import can
from robstride import Motor
import time
import numpy as np

class CompliantInsertion:
    def __init__(self, motors):
        self.motors = motors

    def insertion(self, approach_position, insert_depth):
```

```
        """
        柔顺插入
        approach_position: 接近位置
        insert_depth: 插入深度
        """
        print("[1/4] 快速接近...")
        # 阶段 1: 快速移动到接近位置 (高刚度)
        for motor in self.motors:
            motor.set_position(approach_position, 0, kp=30, kd=1.5, torque=0)

        time.sleep(2)

        print("[2/4] 接触搜索...")
        # 阶段 2: 慢速接触 (中等刚度, 监控力矩)
        contact_detected = False
        search_step = 0.01  # 1cm 步长

        for i in range(20):  # 最多搜索 20cm
            current_pos = self.motors[0].get_feedback()['position']
            new_pos = current_pos - search_step

            for motor in self.motors:
                motor.set_position(new_pos, 0, kp=15, kd=1.0, torque=0)

            time.sleep(0.2)

            # 检测接触 (力矩突增)
            fb = self.motors[0].get_feedback()
            if abs(fb['torque']) > 3.0:  # 检测到阻力
                contact_detected = True
                print(f"  检测到接触, 力矩: {fb['torque']:.2f} Nm")
                break

        if not contact_detected:
```

```
            print("  未检测到接触")
            return False

        print("[3/4] 柔顺插入...")
        # 阶段 3: 柔顺插入 (低刚度 + 力控)
        insert_force = 2.0  # 插入力 (Nm)
        insert_target = current_pos - insert_depth

        for i in range(100):  # 10 秒插入
            fb = self.motors[0].get_feedback()
            current_pos = fb['position']

            # 如果已到达深度
            if current_pos <= insert_target:
                print("  到达目标深度")
                break

            # 柔顺控制
            for motor in self.motors:
                motor.set_position(
                    position=current_pos - 0.005,  # 缓慢推进
                    velocity=0.0,
                    kp=5.0,       # 低刚度, 允许让步
                    kd=0.5,
                    torque=insert_force  # 恒力推入
                )

            time.sleep(0.1)

            # 监控力矩, 过大则停止
            if abs(fb['torque']) > 10.0:
                print(f"  力矩过大 ({fb['torque']:.2f} Nm), 停止插入")
                return False
```

```
        print("[4/4] 插入完成")
        return True

# 使用
bus0 = can.interface.Bus(channel='can0', bustype='socketcan')
motors = [Motor(bus0, i) for i in range(4, 7)]  # 肘腕关节

for m in motors:
    m.enable()

inserter = CompliantInsertion(motors)
success = inserter.insertion(approach_position=-0.5, insert_depth=0.1)

for m in motors:
    m.disable()
```

32.5.2 柔顺抓取

场景: 抓取易碎物品

```
#!/usr/bin/env python3
"""
柔顺抓取 - 软物体
"""
import can
from robstride import Motor
import time

bus0 = can.interface.Bus(channel='can0', bustype='socketcan')
gripper = Motor(bus0, 8)  # 夹爪
gripper.enable()

def compliant_grasp(target_force=1.0):
    """
    柔顺抓取
```

```
    target_force: 目标夹持力 (Nm)
    """
    print(" 开始柔顺抓取...")

    # 初始打开
    gripper.set_position(0.0, 0, kp=10, kd=0.5, torque=0)
    time.sleep(1)

    # 慢速闭合，监控力矩
    close_step = 0.05  # 每步 5 度
    max_close = 1.2    # 最大闭合角度

    for i in range(int(max_close / close_step)):
        target_position = i * close_step

        # 柔顺控制
        gripper.set_position(
            position=target_position,
            velocity=0.0,
            kp=8.0,          # 中低刚度
            kd=0.6,
            torque=0.0
        )

        time.sleep(0.2)

        # 检查力矩
        fb = gripper.get_feedback()
        current_torque = abs(fb['torque'])

        print(f" 位置: {fb['position']:.3f}, 力矩: {current_torque:.3f} Nm")

        # 达到目标力则停止
        if current_torque >= target_force:
```

```
            print(f" 达到目标夹持力: {current_torque:.3f} Nm")
            break

    # 保持夹持 (力控)
    final_position = gripper.get_feedback()['position']

    print(" 保持夹持...")
    for _ in range(50):  # 保持 5 秒
        gripper.set_position(
            position=final_position,
            velocity=0.0,
            kp=5.0,          # 低刚度
            kd=0.5,
            torque=target_force  # 恒力夹持
        )
        time.sleep(0.1)

    print(" 释放...")
    gripper.set_position(0.0, 0, kp=10, kd=0.5, torque=0)
    time.sleep(1)

# 执行
compliant_grasp(target_force=0.8)
gripper.disable()
```

32.5.3 碰撞检测与退让

场景: 运动中检测碰撞并退让

```
#!/usr/bin/env python3
"""
碰撞检测与自动退让
"""
import can
from robstride import Motor
```

```python
import time
import numpy as np

class CollisionDetection:
    def __init__(self, motor):
        self.motor = motor
        self.torque_history = []
        self.baseline_torque = 0.0

    def calibrate_baseline(self):
        """ 标定无负载力矩"""
        print(" 标定基准力矩...")
        torques = []

        for _ in range(50):
            fb = self.motor.get_feedback()
            torques.append(abs(fb['torque']))
            time.sleep(0.02)

        self.baseline_torque = np.mean(torques)
        print(f" 基准力矩: {self.baseline_torque:.3f} Nm")

    def move_with_collision_detection(self, target_position, threshold=5.0):
        """
        带碰撞检测的运动
        threshold: 碰撞检测阈值 (Nm)
        """
        print(f" 移动到 {target_position:.3f} rad...")

        collision_detected = False

        for i in range(100):  # 最多 10 秒
            # 发送控制命令
            self.motor.set_position(
```

```
            position=target_position,
            velocity=0.0,
            kp=20.0,
            kd=1.0,
            torque=0.0
        )

        time.sleep(0.1)

        # 读取力矩
        fb = self.motor.get_feedback()
        current_torque = abs(fb['torque'])

        # 碰撞检测
        torque_deviation = current_torque - self.baseline_torque

        if torque_deviation > threshold:
            collision_detected = True
            print(f"[警告] 检测到碰撞! 力矩: {current_torque:.2f} Nm")
            break

        # 到达目标
        if abs(fb['position'] - target_position) < 0.05:
            print("✓ 到达目标")
            break

    if collision_detected:
        # 退让
        print(" 退让中...")
        current_pos = self.motor.get_feedback()['position']
        retreat_pos = current_pos + 0.1  # 退让 10cm

        self.motor.set_position(
            position=retreat_pos,
```

```
                velocity=0.0,
                kp=15.0,
                kd=1.0,
                torque=0.0
            )

            time.sleep(1)
            print(" 退让完成")

            return False

        return True

# 使用
bus0 = can.interface.Bus(channel='can0', bustype='socketcan')
motor = Motor(bus0, 1)
motor.enable()

detector = CollisionDetection(motor)
detector.calibrate_baseline()

# 尝试移动
success = detector.move_with_collision_detection(target_position=1.0, threshold=5.0)

if not success:
    print(" 运动中断，检测到障碍物")

motor.disable()
```

32.6 重力补偿

32.6.1 简单重力补偿

单关节模型:

```python
def gravity_compensation(joint_angle, mass=2.0, length=0.5):
    """
    计算重力补偿力矩
    joint_angle: 关节角度 (rad)
    mass: 负载质量 (kg)
    length: 力臂长度 (m)
    """
    import numpy as np

    g = 9.8  # m/s²
    torque = mass * g * length * np.cos(joint_angle)

    return torque

# 使用
joint_angle = motor.get_feedback()['position']
gravity_torque = gravity_compensation(joint_angle)

motor.set_position(
    position=target,
    velocity=0,
    kp=15,
    kd=1,
    torque=gravity_torque  # 补偿重力
)
```

32.6.2 多关节重力补偿

完整手臂模型:

```python
#!/usr/bin/env python3
"""
完整手臂重力补偿
"""
import numpy as np
```

```python
class GravityCompensation:
    def __init__(self):
        # 手臂参数 (简化模型)
        self.link_masses = [1.0, 0.8, 0.5, 0.3, 0.2, 0.15, 0.1]  # kg
        self.link_lengths = [0.2, 0.25, 0.25, 0.2, 0.15, 0.1, 0.05]  # m
        self.link_coms = [0.1, 0.125, 0.125, 0.1, 0.075, 0.05, 0.025]  # 质心位置

        self.g = 9.8

    def compute_gravity_torques(self, joint_positions):
        """
        计算所有关节的重力力矩
        joint_positions: [θ1, θ2, ..., θ7]
        返回: [τ1, τ2, ..., τ7]
        """
        torques = np.zeros(7)

        # 简化计算: 每个关节受后续所有连杆重力影响
        for i in range(7):
            # 累加当前及后续连杆的重力力矩
            for j in range(i, 7):
                # 计算该连杆对关节 i 的力臂
                # (简化: 仅考虑关节 2, 3, 4 的重力效应)
                if i < 3:  # 肩部关节
                    # 重力力矩 ≈ m*g*l*cos(θ)
                    torques[i] += self.link_masses[j] * self.g * \
                                  self.link_coms[j] * np.cos(joint_positions[i])

        return torques

# 使用
comp = GravityCompensation()
```

```
# 在控制循环中
while True:
    joint_positions = get_current_joint_positions()
    gravity_torques = comp.compute_gravity_torques(joint_positions)

    # 应用补偿
    for i, motor in enumerate(motors):
        motor.set_position(
            position=target_positions[i],
            velocity=0,
            kp=20,
            kd=1,
            torque=gravity_torques[i]  # 重力补偿
        )

    time.sleep(0.01)
```

32.7 人机协作

32.7.1 拖动示教

全臂拖动示教:

```
#!/usr/bin/env python3
"""
拖动示教 - 记录轨迹
"""
import can
from robstride import Motor
import time
import yaml

bus0 = can.interface.Bus(channel='can0', bustype='socketcan')
motors = [Motor(bus0, i) for i in range(1, 8)]

for m in motors:
```

```
    m.enable()

print(" 进入拖动示教模式")
print(" 手动拖动机械臂, 按 Ctrl+C 结束")

trajectory = []

# 计算重力补偿 (简化)
def simple_gravity_comp(joint_idx, position):
    if joint_idx < 3:
        return 2.0 * 9.8 * 0.3 * np.cos(position)
    elif joint_idx < 6:
        return 1.0 * 9.8 * 0.2 * np.cos(position)
    else:
        return 0.0

try:
    while True:
        # 读取当前位置
        positions = []
        for motor in motors:
            fb = motor.get_feedback()
            positions.append(fb['position'])

        # 记录轨迹
        trajectory.append({
            'positions': positions,
            'timestamp': time.time()
        })

        # 低刚度跟随 + 重力补偿
        for i, motor in enumerate(motors):
            gravity_torque = simple_gravity_comp(i, positions[i])
```

```
            if i < 3:  # 肩部
                kp, kd = 3.0, 0.3
            else:      # 肘腕
                kp, kd = 2.0, 0.2

            motor.set_position(
                position=positions[i],  # 跟随
                velocity=0.0,
                kp=kp,
                kd=kd,
                torque=gravity_torque  # 补偿重力
            )

        time.sleep(0.1)  # 10Hz 采样

except KeyboardInterrupt:
    print(f"\n示教结束，记录{len(trajectory)}个点")

    # 保存轨迹
    with open('taught_trajectory.yaml', 'w') as f:
        yaml.dump({'trajectory': trajectory}, f)

    for m in motors:
        m.disable()
```

32.7.2 安全人机交互

力限制与碰撞检测:

```
class SafeHRI:
    def __init__(self, motors):
        self.motors = motors
        self.max_torque = 10.0  # 最大安全力矩 (Nm)
        self.emergency_stop = False
```

```python
    def safe_control_loop(self):
        """ 安全控制循环"""
        while not self.emergency_stop:
            for i, motor in enumerate(self.motors):
                fb = motor.get_feedback()

                # 力矩检查
                if abs(fb['torque']) > self.max_torque:
                    print(f"[警告] 关节{i+1}力矩超限: {fb['torque']:.2f} Nm")
                    self.emergency_stop = True
                    break

                # 正常控制
                motor.set_position(
                    position=target_positions[i],
                    velocity=0,
                    kp=15,   # 中等刚度
                    kd=1.0,
                    torque=0
                )

            time.sleep(0.01)

        # 急停
        print(" 触发急停!")
        for motor in self.motors:
            motor.disable()
```

32.8 常见问题

32.8.1 力控不稳定

症状: 力矩控制时抖动或震荡

原因: - KP/KD 设置不当 - 力矩变化过快 - 传感器噪声

解决:

```
# 降低刚度
kp, kd = 5.0, 0.5  # 从 20/1.0 降低

# 力矩平滑
def smooth_torque(target_torque, current_torque, alpha=0.1):
    return alpha * target_torque + (1 - alpha) * current_torque

# 添加死区
if abs(torque_error) < 0.5:
    torque_command = 0
```

32.8.2 重力补偿不准

症状: 手臂仍会下垂或上翘

原因: - 模型不准确 - 负载变化 - 摩擦力

解决:

1. 细化模型(多连杆,质心位置)
2. 实际测量参数
3. 在线学习/自适应
4. 添加摩擦力补偿

32.8.3 碰撞检测误报

症状: 正常运动时触发碰撞检测

原因: - 阈值过低 - 未标定基准 - 加速度大

解决:

```
# 提高阈值
threshold = 8.0  # 从 5.0 提高

# 动态阈值 (考虑加速度)
threshold = baseline + k * abs(acceleration)

# 滤波
```

```
torque_filtered = lowpass_filter(torque_raw)
```

32.9 总结

32.9.1 力控要点

✓ 理解阻抗控制原理

✓ 调节KP/KD实现刚度控制

✓ 善用力矩前馈

✓ 重力补偿提高性能

✓ 监控力矩,确保安全

32.9.2 应用总结

应用	KP	KD	力矩	特点
示教	低 (2-5)	低 (0.2-0.5)	重力补偿	易推动
柔顺装配	低 (5-10)	低 (0.5-1.0)	插入力	允许让步
恒力抛光	低 (3-8)	低 (0.3-0.8)	目标力	保持接触力
碰撞检测	中 (15-25)	中 (1.0-1.5)	0	监控力矩

32.9.3 下一步

- **02_ 性能优化** - 控制频率和实时性优化
- **09_ 故障排查** - 力控相关问题诊断
- **10_ 开发者指南** - 高级力控算法开发

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

33 性能优化

33.1 OpenArmX 系统性能优化指南

优化控制频率、实时性和整体性能。

33.2 性能优化概述

33.2.1 性能指标

关键性能指标:

控制频率: 100-500 Hz

控制延迟: <10 ms

定位精度: ±0.02 rad

同步误差: <0.05 rad

CPU使用率: <60%

33.2.2 性能瓶颈

常见瓶颈:

- ☐ CAN总线带宽
- ☐ 控制器更新频率
- ☐ 计算资源不足
- ☐ Python性能限制
- ☐ 系统调度延迟

33.3 控制频率优化

33.3.1 ros2_control 频率

配置文件:

```yaml
# openarm_controllers.yaml

controller_manager:
  ros__parameters:
    update_rate: 200  # 从 100 提升到 200 Hz
```

推荐值:

一般应用: 100 Hz

高性能: 200-300 Hz

极限: 500 Hz (需要实时系统)

验证频率:

```
# 监控实际更新频率
ros2 topic hz /joint_states

# 输出:
```

```
# average rate: 200.123
#   min: 0.004s max: 0.006s std dev: 0.00012s
```

33.3.2 CAN 通信优化

减少 CAN 消息数量:

```
# 不推荐: 每个关节单独发送
for motor in motors:
    motor.set_position(...)
    time.sleep(0.001)  # 7 个电机 = 7ms

# 推荐: 批量发送 (如果硬件支持)
# 或至少减少间隔
for motor in motors:
    motor.set_position(...)
# 无延迟, 让 CAN 队列处理
```

CAN 总线利用率:

```
# 计算 CAN 带宽使用
# 每个 MIT 命令 ≈ 8 字节数据 + 协议开销
# Mbps CAN, 理论最大约 10000 帧/秒

# 14 个电机 @ 100Hz = 1400 帧/秒 (14% 利用率) ✓
# 14 个电机 @ 200Hz = 2800 帧/秒 (28% 利用率) ✓
# 14 个电机 @ 500Hz = 7000 帧/秒 (70% 利用率) [警告]
```

33.4 实时性优化

33.4.1 Linux 实时补丁

安装 RT-PREEMPT:

```
# 检查当前内核
uname -r
# -60-generic
```

```
# 安装实时内核
sudo apt install linux-image-rt-amd64 linux-headers-rt-amd64

# 重启并选择 RT 内核
sudo reboot
```

验证实时内核:

```
uname -r
# 应显示包含"rt"

cat /sys/kernel/realtime
# 1
```

33.4.2 CPU 隔离

隔离 CPU 核心:

```
# 编辑 GRUB 配置
sudo vim /etc/default/grub

# 添加隔离参数
GRUB_CMDLINE_LINUX="isolcpus=2,3 nohz_full=2,3 rcu_nocbs=2,3"

# 更新 GRUB
sudo update-grub
sudo reboot
```

绑定进程到隔离核心:

```
# 运行 ros2_control 在核心 2
taskset -c 2 ros2 launch openarm_bringup openarm.launch.py
```

33.4.3 进程优先级

提高控制器优先级:

```
# 查找 controller_manager 进程
ps aux | grep controller_manager
```

```
# 提高优先级 (需要 root)
sudo renice -20 -p <PID>

# 或设置实时调度
sudo chrt -f 80 -p <PID>
```

Python 脚本中设置:

```
import os

# 提高优先级
os.nice(-20)  # 需要权限

# 实时调度
import ctypes
libc = ctypes.CDLL('libc.so.6')
SCHED_FIFO = 1
libc.sched_setscheduler(0, SCHED_FIFO, ctypes.byref(ctypes.c_int(80)))
```

33.5 代码优化

33.5.1 Python 性能优化

使用 NumPy 加速:

```
# 慢速: 纯 Python 循环
positions = []
for motor in motors:
    fb = motor.get_feedback()
    positions.append(fb['position'])

# 快速:NumPy 数组
import numpy as np
positions = np.array([motor.get_feedback()['position'] for motor in motors])
```

避免重复计算:

```python
# 不推荐：每次循环计算
while True:
    kp = calculate_kp()  # 重复计算
    motor.set_position(..., kp=kp, ...)

# 推荐：预先计算
kp = calculate_kp()
while True:
    motor.set_position(..., kp=kp, ...)
```

使用 Cython 加速关键代码:

```python
# critical_function.pyx
import numpy as np
cimport numpy as np

def fast_gravity_compensation(double[:] joint_angles):
    cdef int i
    cdef double[:] torques = np.zeros(7)

    for i in range(7):
        torques[i] = 2.0 * 9.8 * 0.3 * np.cos(joint_angles[i])

    return np.asarray(torques)

# 编译:
# cythonize -i critical_function.pyx
```

33.5.2 C++ 性能

硬件接口优化:

```cpp
// openarm_hardware_interface.cpp

// 优化：避免重复创建对象
void OpenArmHardwareInterface::write(...)
```

```
{
    // 预分配数组
    static std::vector<double> kp_values(7, 20.0);
    static std::vector<double> kd_values(7, 1.0);

    // 批量处理
    for (size_t i = 0; i < hw_commands_positions_.size(); i++)
    {
        motors_[i].set_position(
            hw_commands_positions_[i],
            0.0,
            kp_values[i],
            kd_values[i],
            0.0
        );
    }
}
```

内联函数:

```
// 频繁调用的小函数使用 inline
inline double radToDeg(double rad) {
    return rad * 180.0 / M_PI;
}
```

33.6 网络与通信优化

33.6.1 ROS2 DDS 配置

使用高性能 DDS:

```
# 使用 CycloneDDS (比 FastDDS 更快)
sudo apt install ros-humble-rmw-cyclonedds-cpp

# 设置环境变量
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
```

```
# 添加到.bashrc
echo "export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp" >> ~/.bashrc
```

DDS 配置文件:

```
<!-- cyclonedds_config.xml -->
<?xml version="1.0" encoding="UTF-8"?>
<CycloneDDS>
  <Domain>
    <General>
      <NetworkInterfaceAddress>lo</NetworkInterfaceAddress>
      <AllowMulticast>false</AllowMulticast>
    </General>
  </Domain>
</CycloneDDS>
```

```
export CYCLONEDDS_URI=file:///path/to/cyclonedds_config.xml
```

33.6.2 话题优化

减少话题发布频率:

```
# openarm_controllers.yaml

left_arm_controller:
  ros__parameters:
    state_publish_rate: 50.0  # 从 100 降到 50Hz
```

使用最佳 QoS:

```
from rclpy.qos import QoSProfile, ReliabilityPolicy, HistoryPolicy

qos_profile = QoSProfile(
    reliability=ReliabilityPolicy.BEST_EFFORT,  # 不保证
    history=HistoryPolicy.KEEP_LAST,
    depth=1  # 只保留最新
)
```

```
subscription = node.create_subscription(
    JointState,
    '/joint_states',
    callback,
    qos_profile
)
```

33.7 系统调优

33.7.1 关闭不必要的服务

```
# 关闭 GUI(服务器模式)
sudo systemctl set-default multi-user.target

# 关闭蓝牙
sudo systemctl disable bluetooth

# 关闭打印服务
sudo systemctl disable cups

# 关闭自动更新
sudo systemctl disable apt-daily.timer
sudo systemctl disable apt-daily-upgrade.timer
```

33.7.2 内存优化

增加交换空间:

```
# 创建 4GB swap
sudo fallocate -l 4G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile

# 永久生效
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

调整 swappiness:

```
# 减少 swap 使用
sudo sysctl vm.swappiness=10

# 永久生效
echo 'vm.swappiness=10' | sudo tee -a /etc/sysctl.conf
```

33.7.3 磁盘 I/O 优化

使用 SSD:

```
机械硬盘 → SSD/NVMe

ROS2日志性能提升: 3-5倍
编译速度提升: 5-10倍
```

减少日志:

```
# 设置日志级别
export RCUTILS_CONSOLE_OUTPUT_FORMAT="[{severity}]: {message}"
export RCUTILS_LOGGING_USE_STDOUT=1
export RCUTILS_LOGGING_BUFFERED_STREAM=1

# 禁用调试日志
export RCUTILS_CONSOLE_MIN_SEVERITY=INFO
```

33.8 性能监控

33.8.1 系统监控

```
# CPU 使用率
htop

# 特定进程
top -p $(pgrep -f controller_manager)

# I/O 监控
```

```
iotop

# 网络监控
iftop
```

33.8.2 ROS2 性能分析

使用 ros2_tracing:

```
# 安装
sudo apt install ros-humble-ros2trace ros-humble-tracetools

# 开始跟踪
ros2 trace start

# 运行应用
ros2 launch openarm_bringup openarm.launch.py

# 停止跟踪
ros2 trace stop

# 分析
babeltrace ~/.ros/tracing/<session_name>
```

使用 rqt_graph:

```
# 可视化节点和话题
rqt_graph
```

33.8.3 自定义性能监控

```
#!/usr/bin/env python3
"""
性能监控节点
"""
import rclpy
```

```python
from rclpy.node import Node
from sensor_msgs.msg import JointState
import time
import psutil

class PerformanceMonitor(Node):
    def __init__(self):
        super().__init__('performance_monitor')

        self.subscription = self.create_subscription(
            JointState,
            '/joint_states',
            self.callback,
            10
        )

        self.last_time = time.time()
        self.msg_count = 0

        # 定时报告
        self.timer = self.create_timer(5.0, self.report)

    def callback(self, msg):
        self.msg_count += 1

    def report(self):
        """ 性能报告"""
        current_time = time.time()
        dt = current_time - self.last_time

        # 消息频率
        freq = self.msg_count / dt

        # CPU 使用率
```

```
        cpu_percent = psutil.cpu_percent(interval=1)

        # 内存使用
        mem = psutil.virtual_memory()

        print(f"========== 性能报告 ==========")
        print(f" 消息频率: {freq:.2f} Hz")
        print(f"CPU 使用: {cpu_percent:.1f}%")
        print(f" 内存使用: {mem.percent:.1f}%")
        print(f"==============================")

        self.last_time = current_time
        self.msg_count = 0

def main():
    rclpy.init()
    monitor = PerformanceMonitor()
    rclpy.spin(monitor)
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

33.9 性能测试

33.9.1 控制延迟测试

```
#!/usr/bin/env python3
"""
测试控制延迟
"""
import can
from robstride import Motor
import time
import numpy as np
```

```python
bus0 = can.interface.Bus(channel='can0', bustype='socketcan')
motor = Motor(bus0, 1)
motor.enable()

latencies = []

for _ in range(100):
    # 发送命令
    start_time = time.time()

    motor.set_position(0.5, 0, kp=20, kd=1, torque=0)

    # 读取反馈
    fb = motor.get_feedback()

    end_time = time.time()

    latency = (end_time - start_time) * 1000  # ms
    latencies.append(latency)

    time.sleep(0.01)

latencies = np.array(latencies)

print(f" 控制延迟统计:")
print(f"  平均: {np.mean(latencies):.2f} ms")
print(f"  最小: {np.min(latencies):.2f} ms")
print(f"  最大: {np.max(latencies):.2f} ms")
print(f"  标准差: {np.std(latencies):.2f} ms")

motor.disable()
```

33.9.2 同步性能测试

```
#!/usr/bin/env python3
"""
测试多电机同步性能
"""
import can
from robstride import Motor
import time
import numpy as np

bus0 = can.interface.Bus(channel='can0', bustype='socketcan')
motors = [Motor(bus0, i) for i in range(1, 4)]

for m in motors:
    m.enable()

# 同步运动测试
target = 0.5
sync_errors = []

for _ in range(50):
    # 同时发送命令
    for motor in motors:
        motor.set_position(target, 0, kp=20, kd=1, torque=0)

    time.sleep(0.1)

    # 读取实际位置
    positions = [m.get_feedback()['position'] for m in motors]

    # 计算同步误差
    sync_error = np.max(positions) - np.min(positions)
    sync_errors.append(sync_error)
```

```
sync_errors = np.array(sync_errors)

print(f" 同步误差统计:")
print(f"  平均: {np.mean(sync_errors):.4f} rad")
print(f"  最大: {np.max(sync_errors):.4f} rad")

for m in motors:
    m.disable()
```

33.10 优化检查清单

33.10.1 软件优化

- ☐ 使用实时内核
- ☐ 隔离CPU核心
- ☐ 提高进程优先级
- ☐ 优化控制器频率(100-200Hz)
- ☐ 使用高性能DDS (CycloneDDS)
- ☐ 减少不必要的日志
- ☐ 代码性能优化(NumPy, Cython)
- ☐ 关闭不必要的服务

33.10.2 硬件优化

- ☐ 使用SSD/NVMe
- ☐ 增加RAM (16GB+)
- ☐ 多核CPU (8核+)
- ☐ 高质量CAN接口
- ☐ 短CAN线缆
- ☐ 良好的电源供应

33.10.3 配置优化

- ☐ update_rate: 100-200 Hz
- ☐ state_publish_rate: 50 Hz
- ☐ QoS: BEST_EFFORT
- ☐ DDS: CycloneDDS

☐ 日志级别: INFO

33.11 常见问题

33.11.1 控制频率达不到目标

症状: 实际频率低于配置值

排查:

```
# 检查 CPU 使用率
htop

# 检查实际频率
ros2 topic hz /joint_states

# 检查调度延迟
cyclictest -p 80 -t5 -n
```

解决:

- 降低update_rate
- 使用实时内核
- 隔离CPU核心
- 优化代码

33.11.2 系统卡顿

症状: 周期性卡顿, 控制不流畅

原因: - 垃圾回收 (Python) - 系统服务干扰 - I/O 阻塞

解决:

```
# Python: 禁用 GC(小心内存泄漏)
import gc
gc.disable()

# 定期手动 GC
if iteration % 1000 == 0:
    gc.collect()
```

33.11.3 CAN 总线拥塞

症状: 丢帧, 通信超时

排查:

```
# 查看 CAN 错误
ip -s link show can0

# 输出:
# RX errors: 0
# TX errors: 0
# ...
```

解决:

- 降低控制频率
- 减少电机数量(分组控制)
- 优化CAN消息格式
- 检查线缆质量

33.12 总结

33.12.1 优化策略

硬件层:

✓ SSD + 多核CPU + 大内存

系统层:

✓ 实时内核 + CPU隔离 + 高优先级

软件层:

✓ 高性能DDS + 优化代码 + 合理频率

监控:

✓ 持续监控 + 性能测试 + 瓶颈分析

33.12.2 性能目标

控制频率: 200 Hz

控制延迟: <5 ms

CPU使用: <50%

同步误差: <0.03 rad

33.12.3 下一步

- **09_ 故障排查** - 性能问题诊断
- **10_ 开发者指南** - 深度优化技巧
- **01_ 力控与柔顺控制** - 力控性能优化

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

34 CAN 通信问题

34.1 CAN 总线通信故障诊断与解决

解决 OpenArmX 系统中 CAN 通信相关的问题。

34.2 CAN 通信概述

34.2.1 CAN 总线基础

OpenArmX CAN 配置:

can0: 左臂 (8个电机)

can1: 右臂 (8个电机)

波特率: 1 Mbps

拓扑: 菊花链

终端电阻: 120Ω (两端)

34.2.2 常见 CAN 问题

- ☐ CAN接口未启动
- ☐ 设备未识别
- ☐ 通信超时
- ☐ 数据丢失
- ☐ 总线拥塞
- ☐ 硬件故障

34.3 问题诊断

34.3.1 检查 CAN 接口状态

```
# 查看 CAN 接口
ip link show can0
ip link show can1

# 正常输出示例:
# 3: can0: <NOARP,UP,LOWER_UP,ECHO> mtu 16 qdisc pfifo_fast state UP mode DEFAULT group default qlen 1
#     link/can
```

```
# 异常状态:
# can0: <NOARP> mtu 16 qdisc noop state DOWN
```

状态说明:

UP: 接口已启动 ✓

DOWN: 接口未启动

LOWER_UP: 物理层连接 ✓

34.3.2 查看 CAN 统计信息

```
# 详细统计
ip -s -d link show can0

# 输出:
# RX: bytes  packets  errors  dropped overrun mcast
#     12345   678      0       0       0       0
# TX: bytes  packets  errors  dropped carrier collsns
#     23456   789      0       0       0       0
```

关键指标:

RX errors: 接收错误

TX errors: 发送错误

dropped: 丢包数

overrun: 缓冲区溢出

正常值: 全部为0

异常值: >0 表示有问题

34.3.3 监听 CAN 消息

```
# 监听 can0 上的消息
candump can0

# 正常输出示例:
# can0  001   [8]  01 02 03 04 05 06 07 08
```

```
# can0  002   [8]  11 12 13 14 15 16 17 18
# ...

# 异常情况:
# - 无输出: 没有通信
# - 错误帧: (ERROR)
```

过滤特定 ID:

```
# 只看 ID=1 的电机
candump can0,001:7FF

# 只看 ID=1-8
candump can0,001:7F8
```

34.4 常见问题及解决

34.4.1 CAN 接口未启动

症状:

```
ip link show can0
# can0: <NOARP> mtu 16 qdisc noop state DOWN
```

原因:

- 未执行启动脚本
- 启动脚本失败
- 驱动未加载

解决:

```
# 方法 1: 使用脚本
cd ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com
python3 en_all_can.py

# 方法 2: 手动启动
sudo ip link set can0 type can bitrate 1000000
sudo ip link set can0 up
```

```
sudo ip link set can1 type can bitrate 1000000
sudo ip link set can1 up

# 验证
ip link show can0
# 应显示 UP,LOWER_UP
```

检查驱动:

```
# 检查 CAN 模块
lsmod | grep can

# 应看到:
# can_raw
# can
# can_dev

# 如未加载, 手动加载
sudo modprobe can
sudo modprobe can_raw
sudo modprobe can_dev
```

34.4.2 设备未识别

症状:

```
ip link show can0
# Device "can0" does not exist.
```

原因:

```
- USB-CAN转换器未连接
- 设备驱动未安装
- USB权限问题
```

解决:

```
#  检查 USB 设备
lsusb
```

```
# 应看到 CAN 转换器设备，例如：
# Bus 001 Device 005: ID 1d50:606f OpenMoko, Inc.

# 检查内核日志
dmesg | grep -i can

# 检查设备节点
ls -l /dev/tty* | grep USB

# 安装驱动（如果需要）
# 根据具体 CAN 转换器型号安装驱动
```

USB 权限：

```
# 添加用户到 dialout 组
sudo usermod -a -G dialout $USER

# 注销后重新登录生效
```

34.4.3 通信超时

症状：

```
python3 check_motor_status.py
# 超时错误：Motor 1 timeout
```

原因：

```
- 电机未上电
- CAN线缆连接问题
- 电机ID错误
- 波特率不匹配
```

解决：

```
# 检查电机供电
# 确认 48V 电源已开启，电源指示灯亮
```

```
# 检查 CAN 消息
candump can0 -t A

# 如果无消息:
#   - 检查线缆连接
#   - 检查终端电阻

# 发送测试命令
cansend can0 001#0102030405060708

# 检查波特率
ip -d link show can0 | grep bitrate
# 应显示: bitrate 1000000
```

重置 CAN 接口:

```
sudo ip link set can0 down
sudo ip link set can0 type can bitrate 1000000
sudo ip link set can0 up
```

34.4.4 数据丢失/丢帧

症状:

```
# CAN统计显示dropped > 0
ip -s link show can0
# RX: dropped 123
```

原因:

```
- CAN总线负载过高
- 接收缓冲区过小
- 处理速度慢
```

解决:

```
# 增大接收缓冲区
sudo ip link set can0 txqueuelen 1000
```

```
# 降低控制频率
# 修改 update_rate: 200 → 100

# 减少同时通信的电机数量

# 检查 CPU 使用率
top
# 如果 CPU 接近 100%, 优化代码
```

34.4.5 总线拥塞

症状:

```
# TX errors增加
ip -s link show can0
# TX: errors 50

# 或candump显示大量ERROR帧
```

原因:

- 发送频率过高
- 总线负载超过容量
- 冲突过多

解决:

```
# 降低发送频率
# 控制循环中添加延迟
time.sleep(0.001)  # 每个电机间隔 1ms

# 计算总线负载
# 16 电机 × 100Hz = 1600 帧/秒
# Mbps CAN 约可支持 10000 帧/秒
# 负载率 = 16% (安全)

# 如果负载 >70%, 降低频率
```

```
# 检查错误详情
candump can0 -e
```

34.4.6 硬件故障

症状:

- CAN接口频繁DOWN
- 无法稳定通信
- 错误率高

诊断:

```
# 检查线缆
# - 目视检查是否破损
# - 测量线缆电阻
# - 更换线缆测试

# 检查终端电阻
# 使用万用表测量 CAN_H 和 CAN_L 之间电阻
# 应为 60Ω (两个 120Ω 并联)

# 检查 USB-CAN 转换器
# - 更换 USB 端口
# - 更换转换器
# - 检查 LED 指示

# 检查电机
# 逐个连接电机，找出故障电机
```

线缆测试:

```
# 断开所有设备，测量线缆
# CAN_H - CAN_L: 应为开路或高阻抗
# 安装终端电阻后: 60Ω

# 如果短路 (<10Ω): 线缆损坏
# 如果开路 (>1MΩ): 未连接终端电阻
```

34.5 诊断工具

34.5.1 candump (监听)

```
# 基本监听
candump can0

# 带时间戳
candump can0 -t A

# 仅显示错误帧
candump can0 -e

# 保存到文件
candump -l can0
# 生成 candump-<date>.log
```

34.5.2 cansend (发送测试)

```
# 发送单条消息
cansend can0 001#0102030405060708

# 循环发送
while true; do cansend can0 001#01; sleep 0.1; done
```

34.5.3 cangen (生成测试流量)

```
# 生成随机 CAN 消息
cangen can0 -g 10 -I 1 -L 8

# 参数:
# -g 10: 间隔 10ms
# -I 1: ID=1
# -L 8: 数据长度 8 字节
```

34.5.4 cansequence (测试序列)

```
# 测试发送/接收序列
cansequence -r can0
cansequence -s can0
```

34.5.5 Python 诊断脚本

```
#!/usr/bin/env python3
"""
CAN 诊断工具
"""
import can
import time

def diagnose_can(channel='can0'):
    """CAN 总线诊断"""
    print(f" 诊断 {channel}...")

    try:
        # 尝试初始化
        bus = can.interface.Bus(
            channel=channel,
            bustype='socketcan',
            bitrate=1000000
        )
        print(f"✓ {channel} 初始化成功")

        # 监听 5 秒
        print(" 监听 5 秒...")
        start_time = time.time()
        msg_count = 0

        while (time.time() - start_time) < 5.0:
            msg = bus.recv(timeout=0.1)
```

```
            if msg:
                msg_count += 1
                print(f"  ID={msg.arbitration_id:03X}, 数据 ={msg.data.hex()}")

        print(f"✓ 收到 {msg_count} 条消息")

        if msg_count == 0:
            print("[警告] 警告: 未收到任何消息")
            print("  可能原因:")
            print("  - 电机未上电")
            print("  - 线缆未连接")
            print("  - 电机未使能")

        bus.shutdown()

    except OSError as e:
        print(f" 错误: {e}")
        print("  可能原因:")
        print("  - CAN 接口未启动")
        print("  - 权限不足")
        print("  - 设备不存在")

    except Exception as e:
        print(f" 未知错误: {e}")

# 运行
diagnose_can('can0')
diagnose_can('can1')
```

34.6 高级故障排查

34.6.1 波形分析

使用示波器:

测量CAN_H和CAN_L波形

正常差分电压：2V（显性）vs 0V（隐性）

异常情况：
- 电压幅度过小：终端电阻问题
- 波形失真：线缆过长或质量差
- 无波形：未连接或短路

34.6.2 负载测试

```
#!/usr/bin/env python3
"""
CAN 总线负载测试
"""
import can
import time

bus = can.interface.Bus(channel='can0', bustype='socketcan')

# 测试不同发送频率
frequencies = [10, 50, 100, 200, 500]

for freq in frequencies:
    print(f"\n测试频率: {freq} Hz")

    error_count = 0
    send_count = 0
    interval = 1.0 / freq

    start_time = time.time()
    test_duration = 10.0  # 10 秒测试

    while (time.time() - start_time) < test_duration:
        msg = can.Message(
            arbitration_id=0x001,
            data=[0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08],
```

```
            is_extended_id=False
        )

        try:
            bus.send(msg)
            send_count += 1
        except can.CanError:
            error_count += 1

        time.sleep(interval)

    error_rate = error_count / send_count if send_count > 0 else 0
    print(f"  发送: {send_count}, 错误: {error_count}, 错误率: {error_rate:.2%}")

    if error_rate > 0.01:
        print(f"  [警告] 错误率过高, 频率{freq}Hz 可能不稳定")

bus.shutdown()
```

34.6.3 电机逐一测试

```
#!/bin/bash
# 逐一测试电机连接

echo " 逐一测试 can0 电机..."

for id in {1..8}; do
    echo " 测试电机 ID=$id"

    # 发送使能命令
    cansend can0 $(printf "%03X" $id)#0102030405060708

    # 监听响应 (1 秒)
    timeout 1 candump can0,$(printf "%03X" $id):7FF > /tmp/can_test.log
```

```
    if [ -s /tmp/can_test.log ]; then
        echo "  ✓ 电机$id 响应正常"
    else
        echo "  ✗ 电机$id 无响应"
    fi

    sleep 0.5
done

rm /tmp/can_test.log
```

34.7 预防措施

34.7.1 日常检查

☐ 定期检查CAN统计(每天)
☐ 检查线缆连接(每周)
☐ 测量终端电阻(每月)
☐ 清洁连接器(每月)
☐ 备份配置(持续)

34.7.2 最佳实践

✓ 使用高质量CAN线缆
✓ 保持线缆长度<5m
✓ 避免平行走线(与强电)
✓ 固定连接器,防松动
✓ 标记线缆(can0/can1)
✓ 备用USB-CAN转换器

34.7.3 监控脚本

```
#!/usr/bin/env python3
"""
CAN 健康监控
"""
```

```python
import subprocess
import time

def check_can_health(interface='can0'):
    """ 检查 CAN 接口健康度"""
    try:
        # 获取统计
        result = subprocess.run(
            ['ip', '-s', 'link', 'show', interface],
            capture_output=True,
            text=True
        )

        output = result.stdout

        # 提取错误数
        for line in output.split('\n'):
            if 'RX:' in line:
                parts = line.split()
                rx_errors = int(parts[3])
            if 'TX:' in line:
                parts = line.split()
                tx_errors = int(parts[3])

        print(f"{interface}: RX 错误 ={rx_errors}, TX 错误 ={tx_errors}")

        if rx_errors > 0 or tx_errors > 0:
            print(f"  [警告] 警告: {interface} 有通信错误")
            return False

        return True

    except Exception as e:
        print(f"   错误: {e}")
```

```
        return False

# 持续监控
while True:
    print(f"\n[{time.strftime('%H:%M:%S')}] CAN 健康检查")
    check_can_health('can0')
    check_can_health('can1')
    time.sleep(60)  # 每分钟检查
```

34.8 故障速查表

症状	可能原因	解决方法
CAN 接口 DOWN	未启动	`python3 en_all_can.py`
设备不存在	USB 未连接	检查 USB 连接
通信超时	电机未上电	检查 48V 电源
RX errors >0	线缆问题	检查/更换线缆
TX errors >0	总线拥塞	降低发送频率
dropped >0	缓冲区小	增大 txqueuelen
无消息	未使能电机	`python3 en_all_motors.py`
间歇通信	接触不良	重新插拔连接器

34.9 总结

34.9.1 诊断流程

```
1. 检查CAN接口状态 (ip link show)
   ↓
2. 查看错误统计 (ip -s link show)
   ↓
3. 监听CAN消息 (candump)
   ↓
4. 检查硬件连接
   ↓
5. 逐一排查电机
   ↓
```

6. 必要时更换硬件

34.9.2 关键命令

```
# 状态检查
ip link show can0
ip -s link show can0

# 监听
candump can0

# 测试发送
cansend can0 001#01

# 重启接口
sudo ip link set can0 down
sudo ip link set can0 up
```

34.9.3 下一步

- **02_ 电机异常处理** - 电机故障诊断
- **03_ 控制器问题** - ros2_control 故障
- **04_ 常用诊断命令** - 命令速查

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

35 电机异常处理

35.1 电机故障诊断与异常处理

解决 Robstride 电机相关的各类故障问题。

35.2 电机异常概述

35.2.1 常见电机问题

☐ 电机无响应

☐ 电机异响

- ☐ 震荡/抖动
- ☐ 过热保护
- ☐ 位置偏差
- ☐ 失控运动
- ☐ 错误码报警

35.2.2 电机状态检查

```
cd ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com
python3 check_motor_status.py
```

正常输出:

```
检查CAN0电机状态...
Motor 1: 正常, 温度=35°C, 位置=0.523rad
Motor 2: 正常, 温度=38°C, 位置=-0.241rad
...
```

异常输出:

```
Motor 3: 错误码=0x08 (过温)
Motor 5: 超时,无响应
```

35.3 电机无响应

35.3.1 症状

```
- check_motor_status.py显示超时
- 无法使能电机
- 发送命令无反应
```

35.3.2 诊断步骤

步骤 1: 检查供电

```
# 检查 48V 电源
# - 电源指示灯是否亮
# - 使用万用表测量电压

# 应为 48V±2V
```

步骤 2: 检查 CAN 通信

```
# 监听 CAN 总线
candump can0

# 如果看到该电机 ID 的消息，说明 CAN 通信正常
# 如果看不到，检查:
# - 线缆连接
# - 电机 ID 设置
# - 菊花链顺序
```

步骤 3: 检查电机 ID

```
# 确认电机 ID 设置
# ID 应为 1-8
# 不能重复

# 重新设置 ID (如果需要)
# 参考 03_ 硬件连接与配置/03_ 电机 ID 配置.md
```

步骤 4: 隔离测试

```
# 单独连接该电机到 CAN 总线
# 使用 test_motor_one.py 测试

python3 test_motor_one.py
# 输入 CAN 通道和 ID
```

35.3.3 解决方法

方法 1: 重启电机

1. 断开48V电源
2. 等待10秒
3. 重新上电
4. 重新使能

方法 2: 重置 CAN 接口

```
python3 dis_all_can.py
python3 en_all_can.py
```

方法 3: 检查硬件

- ☐ 检查线缆是否松动
- ☐ 检查连接器是否损坏
- ☐ 更换线缆测试
- ☐ 更换电机位置测试

35.4 电机异响

35.4.1 症状类型

类型 1: 尖锐啸叫

原因: KP/KD过高,高频震荡

特征: 持续高频噪声

类型 2: 咔咔声

原因: 机械卡滞,碰撞

特征: 间歇性撞击声

类型 3: 嗡嗡声

原因: 电流噪声,PWM频率

特征: 低频嗡鸣

35.4.2 诊断方法

```
#!/usr/bin/env python3
"""
电机异响诊断
"""
import can
from robstride import Motor
import time

bus0 = can.interface.Bus(channel='can0', bustype='socketcan')
```

```
motor = Motor(bus0, 1)

motor.enable()
time.sleep(1)

# 测试 1: 不同 KP/KD
print(" 测试不同参数...")

test_params = [
    (10, 0.5, " 低增益"),
    (20, 1.0, " 中等增益"),
    (50, 2.5, " 高增益"),
    (100, 5.0, " 极高增益")
]

for kp, kd, desc in test_params:
    print(f"\n{desc}: KP={kp}, KD={kd}")
    input(" 按回车测试, 注意听声音...")

    motor.set_position(0.5, 0, kp, kd, 0)
    time.sleep(3)

    motor.set_position(0.0, 0, kp, kd, 0)
    time.sleep(3)

motor.disable()
```

35.4.3 解决方法

针对啸叫:

```
# 降低 KP/KD
kp = 15  # 从 50 降低
kd = 0.8  # 从 2.5 降低
```

```
motor.set_position(target, 0, kp, kd, 0)
```

针对咔咔声:

```
#  检查机械装配
#  检查是否碰撞
#  检查关节限位

# 添加限位检查
if position < joint_limit_min or position > joint_limit_max:
    print(" 警告: 超出关节限位")
```

针对嗡嗡声:

通常为正常现象

如果过大:

- 检查电源质量
- 检查接地
- 降低电流(降低KP)

35.5 震荡/抖动

35.5.1 症状

- 到达目标后持续震荡
- 静止时微小抖动
- 运动过程中震荡

35.5.2 原因分析

原因 1: 参数不当

KP过高 → 响应过快 → 超调 → 震荡

KD不足 → 阻尼小 → 无法抑制震荡

原因 2: 机械问题

- 装配松动
- 轴承磨损
- 负载不平衡

原因 3: 控制频率

频率过低 → 采样不足 → 控制不稳定

35.5.3 解决方法

方法 1: 调整参数

```
# 降低 KP, 增加 KD

# 震荡前
kp, kd = 50, 1.0

# 调整后
kp, kd = 30, 2.0

motor.set_position(target, 0, kp, kd, 0)
```

方法 2: 添加死区

```
def control_with_deadband(motor, target, deadband=0.01):
    """ 带死区的控制"""
    current = motor.get_feedback()['position']
    error = abs(target - current)

    if error < deadband:
        # 在死区内, 不控制
        return

    motor.set_position(target, 0, kp=20, kd=1.0, torque=0)

# 使用
control_with_deadband(motor, 0.5, deadband=0.02)
```

方法 3: 滤波

```
class LowPassFilter:
    def __init__(self, alpha=0.1):
        self.alpha = alpha
```

```
        self.filtered_value = 0.0

    def filter(self, new_value):
        self.filtered_value = self.alpha * new_value + \
                              (1 - self.alpha) * self.filtered_value
        return self.filtered_value

# 使用
lpf = LowPassFilter(alpha=0.1)

while True:
    fb = motor.get_feedback()
    filtered_pos = lpf.filter(fb['position'])

    # 使用滤波后的位置进行控制
```

35.6 过热保护

35.6.1 症状

- 电机停止响应
- 错误码: 0x08
- 温度传感器读数>80°C

35.6.2 温度监控

```
#!/usr/bin/env python3
"""
温度监控
"""
import can
from robstride import Motor
import time

bus0 = can.interface.Bus(channel='can0', bustype='socketcan')
motors = [Motor(bus0, i) for i in range(1, 8)]
```

```python
for m in motors:
    m.enable()

print(" 温度监控 (Ctrl+C 退出)")

try:
    while True:
        print(f"\n[{time.strftime('%H:%M:%S')}]")

        for i, motor in enumerate(motors):
            fb = motor.get_feedback()

            # 温度 (如果支持)
            temp = fb.get('temperature', 'N/A')

            # 力矩 (间接反映发热)
            torque = abs(fb['torque'])

            status = ""
            if isinstance(temp, (int, float)):
                if temp > 70:
                    status = "[红]"
                elif temp > 60:
                    status = "[黄]"

            print(f"  {status} Motor {i+1}: 温度 ={temp}°C, 力矩 ={torque:.2f}Nm")

        time.sleep(5)

except KeyboardInterrupt:
    for m in motors:
        m.disable()
```

35.6.3 降温措施

立即措施:

1. 停止电机运动
2. 失能电机
3. 等待自然冷却(10-30分钟)

预防措施:

```
# 降低负载
kp = 15  # 从 30 降低, 减少力矩输出

# 降低运动速度
velocity_scaling = 0.3  # 从 1.0 降低

# 增加休息时间
motor.set_position(target, 0, kp, kd, 0)
time.sleep(2)  # 运动后休息
```

环境改善:

✓ 增加散热(风扇)
✓ 降低环境温度
✓ 避免阳光直射
✓ 改善通风

35.7 位置偏差

35.7.1 症状

- 实际位置与目标位置差异大
- 无法到达目标
- 位置漂移

35.7.2 诊断

```
#!/usr/bin/env python3
"""
位置精度测试
```

```
"""
import can
from robstride import Motor
import time

bus0 = can.interface.Bus(channel='can0', bustype='socketcan')
motor = Motor(bus0, 1)

motor.enable()
time.sleep(1)

# 测试多个目标位置
targets = [0.0, 0.5, 1.0, 0.5, 0.0, -0.5, -1.0, 0.0]

print(" 目标位置 | 实际位置 | 误差")
print("-" * 40)

for target in targets:
    motor.set_position(target, 0, kp=30, kd=1.5, torque=0)
    time.sleep(3)

    fb = motor.get_feedback()
    actual = fb['position']
    error = abs(target - actual)

    print(f"{target:8.3f} | {actual:8.3f} | {error:6.4f}")

motor.disable()
```

35.7.3 原因与解决

原因 1: 零位未校准

```
# 重新校准零位
python3 set_motor_zeros_setMIT.py
```

```
# 断电重启
# 验证
python3 control_motor_gohome.py
```

原因 2: KP 不足

```
# 增加 KP
kp = 50  # 从 20 提高
kd = 2.5  # 相应增加 KD

motor.set_position(target, 0, kp, kd, 0)
```

原因 3: 负载过大

```
# 添加重力补偿
def gravity_comp(angle):
    return 2.0 * 9.8 * 0.3 * np.cos(angle)

angle = motor.get_feedback()['position']
torque = gravity_comp(angle)

motor.set_position(target, 0, kp=20, kd=1.0, torque=torque)
```

原因 4: 机械卡滞

检查:

☐ 轴承是否卡滞

☐ 线缆是否缠绕

☐ 是否有异物

35.8 失控运动

35.8.1 症状

[警告] 危险! 立即按急停!

- 电机高速失控旋转
- 运动超出预期

- 无法停止

35.8.2 紧急处理

1. 立即按急停按钮
2. 断开48V电源
3. 不要靠近机械臂
4. 等待完全停止

35.8.3 原因分析

原因 1: 零位错误

零位设置错误,导致目标位置计算错误

→ 电机认为需要大幅度运动

原因 2: 参数错误

KP/KD设置错误

→ 正反馈 → 失控

原因 3: 通信错误

CAN消息错误,目标位置错误

→ 电机接收到异常目标

35.8.4 预防措施

软件限位:

```
class SafeMotor:
    def __init__(self, motor, min_pos, max_pos):
        self.motor = motor
        self.min_pos = min_pos
        self.max_pos = max_pos

    def safe_set_position(self, target, vel, kp, kd, torque):
        """ 带限位检查的控制"""
        # 限位检查
        if target < self.min_pos or target > self.max_pos:
```

```
            print(f"[警告] 目标{target}超出限位 [{self.min_pos}, {self.max_pos}]")
            return False

        # 当前位置检查
        current = self.motor.get_feedback()['position']
        if current < self.min_pos or current > self.max_pos:
            print(f"[警告] 当前位置{current}超出限位, 紧急停止")
            self.motor.disable()
            return False

        # 安全控制
        self.motor.set_position(target, vel, kp, kd, torque)
        return True

# 使用
safe_motor = SafeMotor(motor, min_pos=-3.0, max_pos=3.0)
safe_motor.safe_set_position(0.5, 0, 20, 1.0, 0)
```

速度监控:

```
def velocity_monitor(motor, max_velocity=5.0):
    """ 速度监控"""
    fb = motor.get_feedback()
    vel = abs(fb['velocity'])

    if vel > max_velocity:
        print(f"[警告] 速度过高: {vel:.2f} rad/s, 紧急停止")
        motor.disable()
        return False

    return True
```

35.9 错误码处理

35.9.1 错误码表

错误码	含义	处理
0x00	正常	无
0x01	过压	检查电源, 降低电压
0x02	欠压	检查电源, 提高电压
0x04	过流	降低负载, 检查短路
0x08	过温	停止运动, 冷却
0x10	编码器故障	检查/更换电机
0x20	通信超时	检查 CAN 连接

35.9.2 错误码读取

```
#!/usr/bin/env python3
"""
读取电机错误码
"""
import can
from robstride import Motor
import time

bus0 = can.interface.Bus(channel='can0', bustype='socketcan')

ERROR_MESSAGES = {
    0x00: " 正常",
    0x01: " 过压",
    0x02: " 欠压",
    0x04: " 过流",
    0x08: " 过温",
    0x10: " 编码器故障",
    0x20: " 通信超时"
}

for motor_id in range(1, 9):
    motor = Motor(bus0, motor_id)
```

```
    try:
        fb = motor.get_feedback()

        # 错误码 (如果 SDK 支持)
        error_code = fb.get('error_code', 0x00)

        if error_code != 0x00:
            error_msg = ERROR_MESSAGES.get(error_code, f" 未知错误 ({error_code:#04x})")
            print(f"Motor {motor_id}:  {error_msg}")
        else:
            print(f"Motor {motor_id}: ✓ 正常")

    except Exception as e:
        print(f"Motor {motor_id}:  通信失败 ({e})")

    time.sleep(0.1)
```

35.9.3 错误恢复

```
def recover_from_error(motor):
    """ 错误恢复流程"""
    print(" 尝试恢复电机...")

    # 1. 失能
    motor.disable()
    time.sleep(1)

    # 2. 重新使能
    motor.enable()
    time.sleep(1)

    # 3. 检查状态
    try:
        fb = motor.get_feedback()
```

```
        error_code = fb.get('error_code', 0x00)

        if error_code == 0x00:
            print("✓ 恢复成功")
            return True
        else:
            print(f" 仍有错误: {error_code:#04x}")
            return False

    except:
        print(" 恢复失败")
        return False
```

35.10 诊断流程图

```
电机异常
    ↓
检查供电 (48V)
    ↓
检查CAN通信 (candump)
    ↓
检查电机状态 (check_motor_status.py)
    ↓
        ├── 无响应 → 检查连接,重启电机
        ├── 异响 → 调整KP/KD,检查机械
        ├── 震荡 → 降KP,增KD
        ├── 过热 → 停止,冷却,降负载
        ├── 位置偏差 → 校准零位,增KP
        ├── 失控 → 急停,检查零位
        └── 错误码 → 查表,对应处理
```

35.11 总结

35.11.1 诊断要点

✓ 先检查供电和CAN通信

✓ 使用`check_motor_status.py`检查状态

✓ 隔离测试单个电机

✓ 记录错误码和现象

✓ 逐步排查,不要盲目更换

35.11.2 安全提示

[警告] 失控运动:立即急停

[警告] 过热:停止运动,冷却

[警告] 异响:降低参数,检查机械

[警告] 测试时:低速,小幅度,随时准备急停

35.11.3 下一步

- **03_ 控制器问题** - ros2_control 故障诊断
- **04_ 常用诊断命令** - 命令速查手册
- **03_ 硬件连接与配置** - 硬件检查

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

36 控制器问题

36.1 ros2_control 控制器故障诊断

解决控制器加载、运行和性能问题。

36.2 控制器问题概述

36.2.1 常见问题

☐ 控制器无法加载

☐ 控制器无法启动

☐ 控制器运行异常

☐ 轨迹执行失败

☐ 性能下降

☐ 控制器崩溃

36.2.2 基本检查

```
# 列出控制器状态
ros2 control list_controllers

# 检查硬件接口
ros2 control list_hardware_interfaces

# 查看日志
ros2 topic echo /rosout | grep -i error
```

36.3 控制器无法加载

36.3.1 症状

```
ros2 control load_controller left_arm_controller
# 返回: Failed to load controller
```

36.3.2 原因与解决

原因 1: 配置文件错误

```
# 检查配置文件
cat ~/openarmx_robotstride_ws-cc/src/openarm_ros2/openarm_controllers/config/openarm_controllers.yaml

# 常见错误:
# - YAML 语法错误 (缩进)
# - 控制器类型拼写错误
# - 关节名称错误
```

验证 YAML:

```
python3 -c "import yaml; yaml.safe_load(open('openarm_controllers.yaml'))"
```

原因 2: 控制器插件未安装

```
# 检查是否安装
ros2 pkg list | grep joint_trajectory_controller
```

```
# 如未安装
sudo apt install ros-humble-joint-trajectory-controller
sudo apt install ros-humble-position-controllers
```

原因 3: 工作空间未编译

```
cd ~/openarmx_robotstride_ws-cc
colcon build --packages-select openarm_controllers
source install/setup.bash
```

36.4 控制器无法启动

36.4.1 症状

```
ros2 control set_controller_state left_arm_controller start
# 控制器状态仍为 inactive
```

36.4.2 原因与解决

原因 1: 硬件接口不可用

```
# 检查硬件接口
ros2 control list_hardware_interfaces

# 应看到:
# command interfaces:
#   left_joint_1/position [available] [claimed]
#   ...

# 如果状态为 [unavailable]:
# - 硬件接口未初始化
# - CAN 未连接
# - 电机未使能
```

解决:

```
#  检查 CAN
ip link show can0
```

```
# 启动 CAN
python3 en_all_can.py

# 使能电机
python3 en_all_motors.py

# 重启控制器
ros2 launch openarm_bringup openarm.launch.py
```

原因 2: 接口被其他控制器占用

```
# 查看 claimed 状态
ros2 control list_hardware_interfaces

# 如果已被占用:
# 停止冲突的控制器
ros2 control set_controller_state <冲突控制器> stop
```

原因 3: 配置错误

```
# 检查控制器配置
# joints 列表必须与硬件接口匹配

# 错误示例:
joints:
  - left_joint_1
  - left_joint_2
  - left_joint_8  # 不存在

# 正确:
joints:
  - left_joint_1
  - left_joint_2
  - left_joint_3
  - left_joint_4
  - left_joint_5
```

```
  - left_joint_6
  - left_joint_7
```

36.5 轨迹执行失败

36.5.1 症状

MoveIt规划成功,但Execute失败

或轨迹执行中断

36.5.2 诊断

```
# 监控控制器状态话题
ros2 topic echo /left_arm_controller/state

# 监控 Action 结果
ros2 topic echo /left_arm_controller/follow_joint_trajectory/result
```

36.5.3 常见失败原因

原因 1: 轨迹点超出限位

```
# 检查 joint_limits.yaml
left_joint_1:
  has_position_limits: true
  min_position: -3.14
  max_position: 3.14
```

监控代码:

```
import rclpy
from control_msgs.action import FollowJointTrajectory

def result_callback(future):
    result = future.result().result
    error_code = result.error_code

    if error_code == 0:
```

```
        print("✓ 轨迹执行成功")
    elif error_code == -1:
        print(" 失败: INVALID_GOAL")
    elif error_code == -2:
        print(" 失败: INVALID_JOINTS")
    elif error_code == -3:
        print(" 失败: OLD_HEADER_TIMESTAMP")
    elif error_code == -4:
        print(" 失败: PATH_TOLERANCE_VIOLATED")
    elif error_code == -5:
        print(" 失败: GOAL_TOLERANCE_VIOLATED")
```

原因 2: 时间戳问题

```
# 轨迹点 time_from_start 必须单调递增

# 错误:
point1.time_from_start = Duration(sec=1, nanosec=0)
point2.time_from_start = Duration(sec=0, nanosec=500000000)  #

# 正确:
point1.time_from_start = Duration(sec=0, nanosec=0)
point2.time_from_start = Duration(sec=1, nanosec=0)  # ✓
```

原因 3: 路径容差超限

```
# openarm_controllers.yaml
left_arm_controller:
  ros__parameters:
    constraints:
      left_joint_1:
        trajectory: 0.1  # 路径容差 (rad)
        goal: 0.05       # 目标容差 (rad)
```

调整:

```
# 放宽容差
trajectory: 0.2  # 从 0.1 增加
goal: 0.1        # 从 0.05 增加
```

36.6 性能问题

36.6.1 症状

- 控制延迟大
- 更新频率低
- CPU使用率高
- 运动不流畅

36.6.2 诊断工具

```
# 检查更新频率
ros2 topic hz /joint_states

# 应为 100-200Hz
# 如果 <50Hz, 说明有性能问题

# 检查 CPU
htop
# 查看 controller_manager 进程 CPU 使用率

# 检查延迟
ros2 topic delay /joint_states
```

36.6.3 性能优化

优化 1: 调整 update_rate

```
# openarm_controllers.yaml
controller_manager:
  ros__parameters:
    update_rate: 100  # 降低频率减轻负担
```

优化 2: 降低 state_publish_rate

```
left_arm_controller:
  ros__parameters:
    state_publish_rate: 50.0  # 从 100 降低
```

优化 3: 使用实时内核

```
# 安装 RT 内核
sudo apt install linux-image-rt-amd64

# 重启选择 RT 内核

# 提高进程优先级
sudo chrt -f 80 -p $(pgrep controller_manager)
```

36.7 控制器崩溃

36.7.1 症状

- controller_manager进程退出
- 所有控制器变为inactive
- 无法控制机器人

36.7.2 查看崩溃日志

```
# ROS2 日志
ros2 topic echo /rosout | grep -i fatal

# 系统日志
journalctl -u ros2* --since "10 minutes ago"

# 核心转储 (如果启用)
coredumpctl list
coredumpctl info <pid>
```

36.7.3 常见崩溃原因

原因 1: 硬件接口异常

```
CAN通信中断 → 硬件接口超时 → 崩溃

解决:
- 确保CAN稳定
- 添加异常处理
- 增加超时时间
```

原因 2: 内存泄漏

```
# 监控内存
watch -n 1 "ps aux | grep controller_manager"

# 如果内存持续增长:
# - 检查代码是否有内存泄漏
# - 重启定期任务
```

原因 3: 配置冲突

```
多个控制器尝试claim同一接口
→ 资源冲突 → 崩溃

解决:
- 检查控制器配置
- 确保无重复claim
```

36.8 诊断流程

36.8.1 系统化诊断

```
1. 检查Controller Manager状态
   ros2 service call /controller_manager/list_controllers
   ↓
2. 检查硬件接口
   ros2 control list_hardware_interfaces
   ↓
```

3. 检查控制器配置

   cat openarm_controllers.yaml

   ↓

4. 查看日志

   ros2 topic echo /rosout

   ↓

5. 测试单个控制器

   ros2 control load_controller <name>

   ↓

6. 监控性能

   ros2 topic hz /joint_states

36.8.2 诊断脚本

```
#!/usr/bin/env python3
"""
控制器健康检查
"""
import rclpy
from rclpy.node import Node
from controller_manager_msgs.srv import ListControllers

class ControllerDiagnostic(Node):
    def __init__(self):
        super().__init__('controller_diagnostic')

        self.client = self.create_client(
            ListControllers,
            '/controller_manager/list_controllers'
        )

    def diagnose(self):
        """ 诊断控制器"""
        print("========== 控制器诊断 ==========\n")
```

```
        # 1. 检查 Controller Manager
        if not self.client.wait_for_service(timeout_sec=5.0):
            print(" Controller Manager 不可用")
            return

        print("✓ Controller Manager 可用\n")

        # 2. 列出控制器
        request = ListControllers.Request()
        future = self.client.call_async(request)
        rclpy.spin_until_future_complete(self, future)

        result = future.result()

        if len(result.controller) == 0:
            print(" 未找到控制器")
            return

        print(f" 找到 {len(result.controller)} 个控制器:\n")

        # 3. 检查每个控制器
        issues = []

        for ctrl in result.controller:
            status_icon = "✓" if ctrl.state == "active" else ""
            print(f"{status_icon} {ctrl.name}")
            print(f"  类型: {ctrl.type}")
            print(f"  状态: {ctrl.state}")

            # 检查问题
            if ctrl.state != "active":
                issues.append(f"{ctrl.name} 状态异常: {ctrl.state}")

            # 检查必需控制器
```

```
            if 'joint_state_broadcaster' in ctrl.name and ctrl.state != 'active':
                issues.append("joint_state_broadcaster 必须 active")

            print()

        # 4. 总结
        if len(issues) == 0:
            print("✓ 所有控制器正常")
        else:
            print(f" 发现 {len(issues)} 个问题:")
            for issue in issues:
                print(f"  - {issue}")

def main():
    rclpy.init()
    diagnostic = ControllerDiagnostic()
    diagnostic.diagnose()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

36.9 常见错误信息

36.9.1 错误信息速查

错误信息	原因	解决
“Failed to load controller”	配置错误/插件未安装	检查配置, 安装插件
“Controller already active”	重复启动	先停止再启动
“Hardware interface not available”	硬件未连接	检查 CAN, 启动硬件
“Joint names mismatch”	关节名称不匹配	检查配置文件
“Trajectory tolerance violated”	超出容差	放宽容差或改善轨迹
“Invalid goal”	轨迹点超限	检查关节限位
“Controller not configured”	未配置	先 configure 再 start

36.10 预防措施

36.10.1 配置验证

```
# 编译后验证配置
cd ~/openarmx_robotstride_ws-cc
colcon build --packages-select openarm_controllers

# 运行配置检查
ros2 run controller_manager spawner --controller-manager-timeout 5 left_arm_controller --inactive

# 无错误说明配置正确
```

36.10.2 监控脚本

```
# 持续监控控制器健康
# 发现问题自动重启

import rclpy
from rclpy.node import Node
from controller_manager_msgs.srv import ListControllers
import subprocess
import time

class ControllerWatchdog(Node):
    def __init__(self):
        super().__init__('controller_watchdog')

        self.client = self.create_client(
            ListControllers,
            '/controller_manager/list_controllers'
        )

        self.timer = self.create_timer(10.0, self.check)
```

```python
    def check(self):
        """ 定期检查"""
        request = ListControllers.Request()
        future = self.client.call_async(request)
        rclpy.spin_until_future_complete(self, future, timeout_sec=2.0)

        if future.result() is None:
            self.get_logger().error("Controller Manager 无响应，尝试重启")
            # 重启逻辑
            return

        result = future.result()

        # 检查必需控制器
        required = ['joint_state_broadcaster', 'left_arm_controller', 'right_arm_controller']

        for name in required:
            found = False
            for ctrl in result.controller:
                if ctrl.name == name and ctrl.state == 'active':
                    found = True
                    break

            if not found:
                self.get_logger().warn(f"{name} 未运行，尝试启动")
                # 启动逻辑

def main():
    rclpy.init()
    watchdog = ControllerWatchdog()
    rclpy.spin(watchdog)
    rclpy.shutdown()
```

36.11 总结

36.11.1 诊断要点

✓ 使用`ros2 control`命令行工具

✓ 检查硬件接口可用性

✓ 验证配置文件正确性

✓ 查看日志定位问题

✓ 系统化诊断流程

36.11.2 常用命令

```
# 状态检查
ros2 control list_controllers
ros2 control list_hardware_interfaces

# 控制器操作
ros2 control load_controller <name>
ros2 control set_controller_state <name> start

# 性能监控
ros2 topic hz /joint_states
htop
```

36.11.3 下一步

- **04_ 常用诊断命令** - 完整命令参考
- **07_ 控制器配置** - 控制器配置详解
- **08_ 高级应用/02_ 性能优化** - 性能调优

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

37 常用诊断命令

37.1 故障诊断命令速查手册

快速查找 OpenArmX 系统诊断所需的各类命令。

37.2 系统状态检查

37.2.1 操作系统

```
# 系统版本
lsb_release -a

# 内核版本
uname -r

# 实时内核检查
cat /sys/kernel/realtime

# 系统负载
uptime

# 内存使用
free -h

# 磁盘使用
df -h

# CPU 信息
lscpu
```

37.2.2 ROS2 环境

```
# ROS2 版本
printenv ROS_DISTRO

# 工作空间检查
echo $AMENT_PREFIX_PATH

# DDS 实现
echo $RMW_IMPLEMENTATION
```

```
# 域 ID
echo $ROS_DOMAIN_ID

# Python 路径
python3 --version
which python3
```

37.3 CAN 总线诊断

37.3.1 CAN 接口管理

```
# 查看 CAN 接口
ip link show can0
ip link show can1

# 启动 CAN
sudo ip link set can0 type can bitrate 1000000
sudo ip link set can0 up

# 关闭 CAN
sudo ip link set can0 down

# CAN 统计信息
ip -s link show can0
ip -s -d link show can0

# 清除错误计数
sudo ip link set can0 down
sudo ip link set can0 up
```

37.3.2 CAN 通信测试

```
# 监听 CAN 消息
candump can0
```

```
# 带时间戳
candump can0 -t A

# 只看错误帧
candump can0 -e

# 过滤特定 ID
candump can0,001:7FF

# 保存日志
candump -l can0

# 发送测试消息
cansend can0 001#0102030405060708

# 生成测试流量
cangen can0 -g 10 -I 1

# 查看 CAN 模块
lsmod | grep can

# 加载 CAN 模块
sudo modprobe can
sudo modprobe can_raw
```

37.3.3 USB-CAN 设备

```
# 列出 USB 设备
lsusb

# USB 设备详情
lsusb -v

# 内核消息
```

```
dmesg | grep -i can
dmesg | grep -i usb

# USB 设备节点
ls -l /dev/tty* | grep USB
```

37.4 ROS2 诊断

37.4.1 节点与话题

```
# 列出节点
ros2 node list

# 节点信息
ros2 node info /controller_manager

# 列出话题
ros2 topic list

# 话题信息
ros2 topic info /joint_states

# 话题频率
ros2 topic hz /joint_states

# 话题延迟
ros2 topic delay /joint_states

# 话题内容
ros2 topic echo /joint_states

# 话题带宽
ros2 topic bw /joint_states

# 发布测试消息
```

```
ros2 topic pub /test std_msgs/msg/String "data: 'test'"
```

37.4.2 服务与参数

```
# 列出服务
ros2 service list

# 服务类型
ros2 service type /controller_manager/list_controllers

# 调用服务
ros2 service call /controller_manager/list_controllers controller_manager_msgs/srv/ListControllers

# 列出参数
ros2 param list

# 获取参数
ros2 param get /controller_manager update_rate

# 设置参数
ros2 param set /controller_manager update_rate 200

# 导出参数
ros2 param dump /controller_manager > params.yaml
```

37.4.3 Action

```
# 列出 Action
ros2 action list

# Action 信息
ros2 action info /left_arm_controller/follow_joint_trajectory

# 发送 Action Goal
```

```
ros2 action send_goal /left_arm_controller/follow_joint_trajectory control_msgs/action/FollowJointTraj
```

37.5 ros2_control 诊断

37.5.1 控制器管理

```
# 列出控制器
ros2 control list_controllers

# 列出硬件接口
ros2 control list_hardware_interfaces

# 列出硬件组件
ros2 control list_hardware_components

# 加载控制器
ros2 control load_controller left_arm_controller

# 配置控制器
ros2 control set_controller_state left_arm_controller configure

# 启动控制器
ros2 control set_controller_state left_arm_controller start

# 停止控制器
ros2 control set_controller_state left_arm_controller stop

# 卸载控制器
ros2 control unload_controller left_arm_controller

# 切换控制器
ros2 control switch_controllers --start-controllers left_arm_controller --stop-controllers position_co
```

37.5.2 控制器状态监控

```
# 控制器状态话题
ros2 topic echo /left_arm_controller/state

# 关节状态
ros2 topic echo /joint_states

# 控制器命令
ros2 topic echo /left_arm_controller/commands

# Action 反馈
ros2 topic echo /left_arm_controller/follow_joint_trajectory/feedback

# Action 结果
ros2 topic echo /left_arm_controller/follow_joint_trajectory/result
```

37.6 电机诊断

37.6.1 OpenArmX 脚本

```
cd ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com

# CAN 管理
python3 en_all_can.py        # 启动 CAN
python3 dis_all_can.py       # 关闭 CAN

# 电机管理
python3 en_all_motors.py     # 使能电机
python3 dis_all_motors.py    # 失能电机

# 状态检查
python3 check_motor_status.py

# 零位设置
```

```
python3 set_motor_zeros_setMIT.py

# 回零
python3 control_motor_gohome.py

# 单电机测试
python3 test_motor_one.py
```

37.6.2 Python 快速诊断

```
#!/usr/bin/env python3
import can
from robstride import Motor

# 连接 CAN
bus = can.interface.Bus(channel='can0', bustype='socketcan')

# 测试电机
for motor_id in range(1, 9):
    try:
        motor = Motor(bus, motor_id)
        fb = motor.get_feedback()
        print(f"Motor {motor_id}: ✓ 位置 ={fb['position']:.3f}")
    except:
        print(f"Motor {motor_id}:  无响应")
```

37.7 系统性能

37.7.1 进程监控

```
# 实时进程监控
htop

# CPU 使用率 (总体)
top
```

```
# 特定进程
top -p $(pgrep controller_manager)

# 进程树
pstree -p

# 查找进程
pgrep -a controller

# 进程详情
ps aux | grep controller_manager

# 进程资源
pidstat -p $(pgrep controller_manager) 1
```

37.7.2 网络监控

```
# 网络接口
ifconfig
ip addr

# 网络统计
netstat -i
ss -s

# 网络流量
iftop

# ROS2 DDS 流量
ros2 topic bw /joint_states
```

37.7.3 磁盘 I/O

```
# 磁盘 I/O 监控
iotop

# 磁盘使用
df -h

# 目录大小
du -sh ~/openarmx_robotstride_ws-cc

# I/O 统计
iostat -x 1
```

37.8 日志查看

37.8.1 ROS2 日志

```
# 实时日志
ros2 topic echo /rosout

# 过滤错误
ros2 topic echo /rosout | grep -i error

# 过滤特定节点
ros2 topic echo /rosout | grep controller_manager

# 日志文件位置
ls ~/.ros/log/

# 查看最新日志
tail -f ~/.ros/log/latest/rosout.log
```

37.8.2 系统日志

```
# 系统日志
journalctl -xe

# 特定服务
journalctl -u ros2*

# 时间范围
journalctl --since "10 minutes ago"

# 实时跟踪
journalctl -f

# 内核日志
dmesg | tail

# 启动日志
dmesg | grep -i can
```

37.9 网络诊断

37.9.1 连接测试

```
# Ping 测试
ping <IP 地址>

# 端口测试
nc -zv <IP> <端口>

# 路由跟踪
traceroute <IP>

# DNS 解析
nslookup <域名>
```

37.9.2 ROS2 DDS

```
# 发现节点
ros2 daemon stop
ros2 daemon start
ros2 node list

# 检查 DDS 配置
echo $RMW_IMPLEMENTATION
echo $CYCLONEDDS_URI

# 测试节点间通信
ros2 run demo_nodes_cpp talker
ros2 run demo_nodes_cpp listener
```

37.10 编译与构建

37.10.1 colcon 命令

```
cd ~/openarmx_robotstride_ws-cc

# 完整编译
colcon build

# 编译特定包
colcon build --packages-select openarm_hardware

# 清理构建
rm -rf build/ install/ log/

# 仅编译修改的包
colcon build --packages-up-to openarm_controllers

# 并行编译
colcon build --parallel-workers 4
```

```
# 详细输出
colcon build --event-handlers console_direct+

# 测试
colcon test
colcon test-result --verbose
```

37.10.2 依赖管理

```
# 安装依赖
rosdep install --from-paths src --ignore-src -r -y

# 更新 rosdep
rosdep update

# 检查依赖
rosdep check --from-paths src --ignore-src
```

37.11 快速诊断脚本

37.11.1 完整系统检查

```
#!/bin/bash
# 完整系统诊断

echo "========== OpenArmX 系统诊断 ==========="

echo -e "\n[1/7] 操作系统"
uname -r
lsb_release -d

echo -e "\n[2/7] CAN 接口"
ip link show can0 | grep -E "state|UP"
ip link show can1 | grep -E "state|UP"
```

```
echo -e "\n[3/7] ROS2 环境"
echo "ROS_DISTRO: $ROS_DISTRO"
echo "DDS: $RMW_IMPLEMENTATION"

echo -e "\n[4/7] ROS2 节点"
ros2 node list | wc -l
echo " 个节点运行中"

echo -e "\n[5/7] 控制器状态"
ros2 control list_controllers

echo -e "\n[6/7] 关节状态频率"
timeout 3 ros2 topic hz /joint_states 2>/dev/null || echo " 无 joint_states"

echo -e "\n[7/7] 系统负载"
uptime

echo -e "\n========== 诊断完成 ==========="
```

37.11.2 故障速查

```
#!/bin/bash
# 快速故障检查

echo " 快速故障检查..."

# CAN 检查
echo -n "CAN 接口: "
if ip link show can0 | grep -q "UP,LOWER_UP"; then
    echo "✓ can0 正常"
else
    echo " can0 异常"
fi
```

```
# Controller Manager
echo -n "Controller Manager: "
if ros2 node list | grep -q controller_manager; then
    echo "✓ 运行中"
else
    echo " 未运行"
fi

# 控制器
echo -n " 左臂控制器: "
if ros2 control list_controllers | grep -q "left_arm_controller.*active"; then
    echo "✓ 激活"
else
    echo " 未激活"
fi

# 关节状态
echo -n " 关节状态: "
if timeout 2 ros2 topic echo /joint_states -n 1 &>/dev/null; then
    echo "✓ 有数据"
else
    echo " 无数据"
fi

echo " 检查完成"
```

37.12 命令速查表

37.12.1 基础操作

操作	命令
启动 CAN	python3 en_all_can.py
使能电机	python3 en_all_motors.py
检查电机	python3 check_motor_status.py
回零	python3 control_motor_gohome.py

操作	命令
启动系统	`ros2 launch openarm_bringup openarm.launch.py`
启动 MoveIt	`ros2 launch openarm_moveit_config demo.launch.py`

37.12.2 故障诊断

问题	检查命令
CAN 通信	`candump can0`
CAN 统计	`ip -s link show can0`
控制器状态	`ros2 control list_controllers`
硬件接口	`ros2 control list_hardware_interfaces`
关节状态	`ros2 topic echo /joint_states`
错误日志	`ros2 topic echo /rosout | grep ERROR`
系统负载	`htop`
进程状态	`ps aux | grep controller`

37.12.3 性能监控

指标	命令
话题频率	`ros2 topic hz /joint_states`
话题延迟	`ros2 topic delay /joint_states`
话题带宽	`ros2 topic bw /joint_states`
CPU 使用	`top -p $(pgrep controller_manager)`
内存使用	`free -h`
磁盘 I/O	`iotop`

37.13 总结

37.13.1 常用命令组合

系统启动:

```
python3 en_all_can.py && python3 en_all_motors.py && ros2 launch openarm_bringup openarm.launch.py
```

快速检查:

```
ip link show can0 && ros2 control list_controllers && ros2 topic hz /joint_states
```

完全重启:

```
python3 dis_all_motors.py && python3 dis_all_can.py && sleep 2 && python3 en_all_can.py && python3 en_
```

37.13.2 诊断清单

- ☐ CAN接口UP (ip link show)
- ☐ CAN无错误 (ip -s link show)
- ☐ 电机响应 (check_motor_status.py)
- ☐ Controller Manager运行 (ros2 node list)
- ☐ 控制器active (ros2 control list_controllers)
- ☐ joint_states发布 (ros2 topic hz)
- ☐ CPU<80% (htop)

37.13.3 下一步

- **01_CAN 通信问题** - CAN 故障详解
- **02_ 电机异常处理** - 电机问题诊断
- **03_ 控制器问题** - 控制器故障排查

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

38 包结构说明

38.1 OpenArmX ROS2 软件包结构详解

了解系统架构, 为二次开发和扩展打下基础。

38.2 工作空间概览

38.2.1 工作空间目录结构

```
openarmx_robotstride_ws-cc/
├── src/                          # 源代码目录
│   ├── openarm_ros2/              # 主功能包集合
│   ├── motor_tests_openarmx_com/# 电机测试脚本
│   └── 使用说明书/                 # 本文档
├── build/                        # 编译输出(由colcon生成)
├── install/                      # 安装目录(由colcon生成)
└── log/                          # 编译日志(由colcon生成)
```

38.2.2 核心功能包

```
openarm_ros2/
├── openarm_description/          # 机器人模型描述
├── openarm_hardware/             # 硬件接口实现
├── openarm_controllers/          # 控制器配置
├── openarm_bringup/              # 启动文件
├── openarm_moveit_config/        # MoveIt配置
└── openarm_msgs/                 # 自定义消息类型
```

38.3 各功能包详解

38.3.1 openarm_description

功能: 机器人模型描述 (URDF/Xacro)

目录结构:

```
openarm_description/
├── CMakeLists.txt
├── package.xml
```

```
├── urdf/
│   ├── openarm.urdf.xacro      # 主模型文件
│   ├── left_arm.xacro          # 左臂子模型
│   ├── right_arm.xacro         # 右臂子模型
│   ├── materials.xacro         # 材质定义
│   └── ros2_control.xacro      # ros2_control标签
├── meshes/
│   ├── visual/                 # 可视化网格(STL/DAE)
│   │   ├── link1.stl
│   │   ├── link2.stl
│   │   └── ...
│   └── collision/              # 碰撞检测网格(简化)
│       ├── link1_collision.stl
│       └── ...
├── config/
│   └── joint_limits.yaml       # 关节限位配置
└── launch/
    ├── display.launch.py       # RViz显示启动文件
    └── view_urdf.launch.py     # URDF查看器
```

关键文件:

urdf/openarm.urdf.xacro:

```xml
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="openarm">

  <!-- 包含子模型 -->
  <xacro:include filename="$(find openarm_description)/urdf/left_arm.xacro"/>
  <xacro:include filename="$(find openarm_description)/urdf/right_arm.xacro"/>
  <xacro:include filename="$(find openarm_description)/urdf/materials.xacro"/>

  <!-- 基座链接 -->
  <link name="base_link">
    <visual>
      <geometry>
```

```
        <box size="0.5 0.3 0.1"/>
      </geometry>
    </visual>
  </link>

  <!-- 左臂实例 -->
  <xacro:left_arm parent="base_link" prefix="left_">
    <origin xyz="0 0.15 0.1" rpy="0 0 0"/>
  </xacro:left_arm>

  <!-- 右臂实例 -->
  <xacro:right_arm parent="base_link" prefix="right_">
    <origin xyz="0 -0.15 0.1" rpy="0 0 0"/>
  </xacro:right_arm>

  <!-- ros2_control 标签 -->
  <xacro:include filename="$(find openarm_description)/urdf/ros2_control.xacro"/>

</robot>
```

package.xml 依赖:

```
<depend>urdf</depend>
<depend>xacro</depend>
<depend>robot_state_publisher</depend>
<depend>joint_state_publisher</depend>
<depend>joint_state_publisher_gui</depend>
```

38.3.2 openarm_hardware

功能: ros2_control 硬件接口实现

目录结构:

```
openarm_hardware/
├── CMakeLists.txt
├── package.xml
```

```
├── include/openarm_hardware/
│   ├── openarm_system.hpp          # 系统硬件接口头文件
│   ├── robstride_can_interface.hpp  # CAN通信接口
│   └── motor_controller.hpp        # 电机控制器封装
├── src/
│   ├── openarm_system.cpp          # 主硬件接口实现
│   ├── robstride_can_interface.cpp  # CAN通信实现
│   └── motor_controller.cpp        # 电机控制逻辑
├── config/
│   └── openarm_hardware.yaml       # 硬件参数配置
└── test/
    ├── test_can_interface.cpp      # CAN接口单元测试
    └── test_motor_controller.cpp   # 电机控制器测试
```

核心类结构:

include/openarm_hardware/openarm_system.hpp:

```cpp
#ifndef OPENARM_HARDWARE__OPENARM_SYSTEM_HPP_
#define OPENARM_HARDWARE__OPENARM_SYSTEM_HPP_

#include <hardware_interface/system_interface.hpp>
#include <rclcpp/rclcpp.hpp>
#include "robstride_can_interface.hpp"

namespace openarm_hardware
{

class OpenArmSystem : public hardware_interface::SystemInterface
{
public:
  // 生命周期回调
  CallbackReturn on_init(const hardware_interface::HardwareInfo & info) override;
  CallbackReturn on_configure(const rclcpp_lifecycle::State & previous_state) override;
  CallbackReturn on_activate(const rclcpp_lifecycle::State & previous_state) override;
  CallbackReturn on_deactivate(const rclcpp_lifecycle::State & previous_state) override;
```

```
  // 硬件通信
  hardware_interface::return_type read(
    const rclcpp::Time & time, const rclcpp::Duration & period) override;
  hardware_interface::return_type write(
    const rclcpp::Time & time, const rclcpp::Duration & period) override;

  // 接口导出
  std::vector<hardware_interface::StateInterface> export_state_interfaces() override;
  std::vector<hardware_interface::CommandInterface> export_command_interfaces() override;

private:
  // CAN 总线接口
  std::shared_ptr<RobstrideCAN> can0_;  // 左臂
  std::shared_ptr<RobstrideCAN> can1_;  // 右臂

  // 电机控制器
  std::vector<std::shared_ptr<MotorController>> left_motors_;
  std::vector<std::shared_ptr<MotorController>> right_motors_;

  // 状态缓存
  std::vector<double> hw_positions_;
  std::vector<double> hw_velocities_;
  std::vector<double> hw_efforts_;

  // 命令缓存
  std::vector<double> hw_commands_;
};

} // namespace openarm_hardware

#endif
```

CMakeLists.txt 关键部分:

```
cmake_minimum_required(VERSION 3.8)
project(openarm_hardware)

# 依赖
find_package(ament_cmake REQUIRED)
find_package(hardware_interface REQUIRED)
find_package(pluginlib REQUIRED)
find_package(rclcpp REQUIRED)
find_package(rclcpp_lifecycle REQUIRED)

# 编译硬件接口库
add_library(${PROJECT_NAME} SHARED
  src/openarm_system.cpp
  src/robstride_can_interface.cpp
  src/motor_controller.cpp
)

target_include_directories(${PROJECT_NAME} PUBLIC
  $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
  $<INSTALL_INTERFACE:include>
)

ament_target_dependencies(${PROJECT_NAME}
  hardware_interface
  pluginlib
  rclcpp
  rclcpp_lifecycle
)

# 注册为 pluginlib 插件
pluginlib_export_plugin_description_file(hardware_interface openarm_hardware_plugin.xml)

# 安装
install(TARGETS ${PROJECT_NAME}
```

```
  ARCHIVE DESTINATION lib
  LIBRARY DESTINATION lib
  RUNTIME DESTINATION bin
)

install(DIRECTORY include/
  DESTINATION include
)

ament_package()
```

插件描述文件 openarm_hardware_plugin.xml:

```
<library path="openarm_hardware">
  <class name="openarm_hardware/OpenArmSystem"
         type="openarm_hardware::OpenArmSystem"
         base_class_type="hardware_interface::SystemInterface">
    <description>
      OpenArmX dual-arm robot hardware interface with Robstride motors via CAN bus
    </description>
  </class>
</library>
```

38.3.3 openarm_controllers

功能: 控制器配置文件

目录结构:

```
openarm_controllers/
├── CMakeLists.txt
├── package.xml
└── config/
    ├── openarm_controllers.yaml     # 控制器配置
    ├── joint_trajectory_controller.yaml
    └── gains/
        ├── left_arm_gains.yaml      # 左臂PID增益
```

```
        └── right_arm_gains.yaml      # 右臂PID增益
```

openarm_controllers.yaml 结构:

```yaml
controller_manager:
  ros__parameters:
    update_rate: 100  # Hz

    # 控制器列表
    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster

    left_arm_controller:
      type: joint_trajectory_controller/JointTrajectoryController

    right_arm_controller:
      type: joint_trajectory_controller/JointTrajectoryController

# 关节状态广播器
joint_state_broadcaster:
  ros__parameters:
    publish_rate: 100.0

# 左臂轨迹控制器
left_arm_controller:
  ros__parameters:
    joints:
      - left_joint_1
      - left_joint_2
      - left_joint_3
      - left_joint_4
      - left_joint_5
      - left_joint_6
      - left_joint_7

    command_interfaces:
```

```yaml
    - position

  state_interfaces:
    - position
    - velocity

  state_publish_rate: 50.0
  action_monitor_rate: 20.0

  constraints:
    stopped_velocity_tolerance: 0.05
    goal_time: 0.5
    left_joint_1:
      trajectory: 0.1
      goal: 0.05
```

38.3.4 openarm_bringup

功能: 系统启动文件

目录结构:

```
openarm_bringup/
├── CMakeLists.txt
├── package.xml
├── launch/
│   ├── openarm.launch.py          # 主启动文件
│   ├── hardware_only.launch.py    # 仅硬件接口
│   ├── controllers.launch.py      # 仅控制器
│   └── rviz.launch.py             # 可视化
├── config/
│   ├── openarm_params.yaml        # 系统参数
│   └── rviz_config.rviz           # RViz配置
└── scripts/
    ├── check_system.py            # 系统检查脚本
    └── emergency_stop.py          # 急停脚本
```

主启动文件 launch/openarm.launch.py:

```python
#!/usr/bin/env python3
import os
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, IncludeLaunchDescription
from launch.substitutions import LaunchConfiguration, PathJoinSubstitution
from launch_ros.actions import Node
from launch_ros.substitutions import FindPackageShare
from ament_index_python.packages import get_package_share_directory

def generate_launch_description():
    # 路径
    pkg_description = FindPackageShare('openarm_description')
    pkg_controllers = FindPackageShare('openarm_controllers')

    # 参数
    use_sim_time = LaunchConfiguration('use_sim_time', default='false')

    # URDF
    urdf_file = PathJoinSubstitution([
        pkg_description, 'urdf', 'openarm.urdf.xacro'
    ])

    # 控制器配置
    controllers_file = PathJoinSubstitution([
        pkg_controllers, 'config', 'openarm_controllers.yaml'
    ])

    # robot_state_publisher
    robot_state_publisher = Node(
        package='robot_state_publisher',
        executable='robot_state_publisher',
        parameters=[{
            'robot_description': Command(['xacro ', urdf_file]),
```

```
            'use_sim_time': use_sim_time
        }]
    )

    # controller_manager
    controller_manager = Node(
        package='controller_manager',
        executable='ros2_control_node',
        parameters=[controllers_file],
        output='both',
        remappings=[
            ('/controller_manager/robot_description', '/robot_description')
        ]
    )

    # spawner 节点
    joint_state_broadcaster_spawner = Node(
        package='controller_manager',
        executable='spawner',
        arguments=['joint_state_broadcaster', '--controller-manager', '/controller_manager']
    )

    left_arm_controller_spawner = Node(
        package='controller_manager',
        executable='spawner',
        arguments=['left_arm_controller', '--controller-manager', '/controller_manager']
    )

    right_arm_controller_spawner = Node(
        package='controller_manager',
        executable='spawner',
        arguments=['right_arm_controller', '--controller-manager', '/controller_manager']
    )
```

```
    return LaunchDescription([
        DeclareLaunchArgument('use_sim_time', default_value='false'),
        robot_state_publisher,
        controller_manager,
        joint_state_broadcaster_spawner,
        left_arm_controller_spawner,
        right_arm_controller_spawner
    ])
```

38.3.5 openarm_moveit_config

功能: MoveIt 运动规划配置

目录结构:

```
openarm_moveit_config/
├── CMakeLists.txt
├── package.xml
├── config/
│   ├── openarm.srdf                # 语义描述文件
│   ├── kinematics.yaml             # 运动学求解器配置
│   ├── joint_limits.yaml           # 关节限位
│   ├── pilz_cartesian_limits.yaml # 笛卡尔限位
│   ├── ompl_planning.yaml          # OMPL规划器配置
│   └── moveit_controllers.yaml    # MoveIt控制器映射
├── launch/
│   ├── demo.launch.py              # MoveIt演示
│   ├── move_group.launch.py        # move_group节点
│   ├── warehouse.launch.py         # 数据库
│   └── setup_assistant.launch.py  # 配置助手
└── .setup_assistant
    └── moveit_config.yaml          # Setup Assistant配置
```

SRDF 关键内容 config/openarm.srdf:

```xml
<?xml version="1.0" ?>
<robot name="openarm">
```

```xml
<!-- 规划组定义 -->
<group name="left_arm">
  <chain base_link="base_link" tip_link="left_link_7"/>
</group>

<group name="right_arm">
  <chain base_link="base_link" tip_link="right_link_7"/>
</group>

<group name="both_arms">
  <group name="left_arm"/>
  <group name="right_arm"/>
</group>

<!-- 末端执行器 -->
<end_effector name="left_eef" parent_link="left_link_7" group="left_arm"/>
<end_effector name="right_eef" parent_link="right_link_7" group="right_arm"/>

<!-- 预定义姿态 -->
<group_state name="home" group="left_arm">
  <joint name="left_joint_1" value="0"/>
  <joint name="left_joint_2" value="0"/>
  <joint name="left_joint_3" value="0"/>
  <joint name="left_joint_4" value="0"/>
  <joint name="left_joint_5" value="0"/>
  <joint name="left_joint_6" value="0"/>
  <joint name="left_joint_7" value="0"/>
</group_state>

<!-- 禁用碰撞 -->
<disable_collisions link1="base_link" link2="left_link_1" reason="Adjacent"/>
<disable_collisions link1="left_link_1" link2="left_link_2" reason="Adjacent"/>
```

```
</robot>
```

38.3.6 openarm_msgs

功能: 自定义消息/服务/动作定义

目录结构:

```
openarm_msgs/
├── CMakeLists.txt
├── package.xml
├── msg/
│   ├── MotorStatus.msg          # 电机状态
│   ├── CANStatus.msg            # CAN状态
│   └── SystemHealth.msg         # 系统健康状态
├── srv/
│   ├── EnableMotor.srv          # 使能电机服务
│   ├── SetZeroPosition.srv      # 设置零位服务
│   └── GetMotorInfo.srv         # 获取电机信息
└── action/
    └── Calibration.action       # 标定动作
```

消息定义示例 msg/MotorStatus.msg:

```
# 电机状态消息

uint8 motor_id                  # 电机ID (1-14)
string can_channel              # CAN通道 (can0/can1)

# 状态标志
bool is_enabled                 # 是否使能
bool is_fault                   # 是否故障
uint8 error_code                # 错误码

# 位置与速度
float32 position                # 位置 (rad)
float32 velocity                # 速度 (rad/s)
```

```
float32 torque                    # 力矩 (Nm)

# 温度与电流
float32 temperature               # 温度 (°C)
float32 current                   # 电流 (A)

# 时间戳
builtin_interfaces/Time stamp
```

服务定义示例 srv/EnableMotor.srv:

```
# 请求
uint8 motor_id                    # 电机ID
string can_channel                # CAN通道
bool enable                       # true=使能, false=失能
# 响应
bool success                      # 是否成功
string message                    # 返回信息
```

CMakeLists.txt:

```
cmake_minimum_required(VERSION 3.8)
project(openarm_msgs)

find_package(ament_cmake REQUIRED)
find_package(builtin_interfaces REQUIRED)
find_package(rosidl_default_generators REQUIRED)

# 生成消息/服务/动作
rosidl_generate_interfaces(${PROJECT_NAME}
  "msg/MotorStatus.msg"
  "msg/CANStatus.msg"
  "msg/SystemHealth.msg"
  "srv/EnableMotor.srv"
  "srv/SetZeroPosition.srv"
  "srv/GetMotorInfo.srv"
  "action/Calibration.action"
```

```
  DEPENDENCIES builtin_interfaces
)

ament_package()
```

38.4 motor_tests_openarmx_com

功能: 低层级电机测试脚本 (Python)

目录结构:

```
motor_tests_openarmx_com/
├── README.md
├── requirements.txt
├── config/
│   └── motor_config.yaml          # 电机配置
├── scripts/
│   ├── en_all_can.py              # 启动CAN
│   ├── dis_all_can.py             # 关闭CAN
│   ├── en_all_motors.py           # 使能所有电机
│   ├── dis_all_motors.py          # 失能所有电机
│   ├── check_motor_status.py     # 检查电机状态
│   ├── set_motor_zeros_setMIT.py # 设置零位(MIT模式)
│   ├── control_motor_gohome.py   # 回零操作
│   ├── test_motor_one.py         # 单电机测试
│   └── test_motor_all.py         # 全部电机测试
└── lib/
    ├── robstride.py              # Robstride电机库
    └── can_utils.py              # CAN工具函数
```

关键脚本示例 scripts/en_all_motors.py:

```python
#!/usr/bin/env python3
"""
使能所有电机
"""
import can
```

```
import sys
import time
from lib.robstride import Motor
from lib.can_utils import check_can_interface

def main():
    # 检查 CAN 接口
    if not check_can_interface('can0') or not check_can_interface('can1'):
        print("[×] CAN 接口未就绪，请先运行 en_all_can.py")
        sys.exit(1)

    # 连接 CAN 总线
    bus0 = can.interface.Bus(channel='can0', bustype='socketcan')
    bus1 = can.interface.Bus(channel='can1', bustype='socketcan')

    # 左臂电机 (ID 1-7, can0)
    print(" 使能左臂电机...")
    for motor_id in range(1, 8):
        try:
            motor = Motor(bus0, motor_id)
            motor.enable()
            print(f"  ✓ 左臂电机 {motor_id} 使能成功")
            time.sleep(0.1)
        except Exception as e:
            print(f"   左臂电机 {motor_id} 使能失败: {e}")

    # 右臂电机 (ID 1-7, can1)
    print("\n使能右臂电机...")
    for motor_id in range(1, 8):
        try:
            motor = Motor(bus1, motor_id)
            motor.enable()
            print(f"  ✓ 右臂电机 {motor_id} 使能成功")
            time.sleep(0.1)
```

```
        except Exception as e:
            print(f"   右臂电机 {motor_id} 使能失败: {e}")

    print("\n[√] 所有电机使能完成")

if __name__ == '__main__':
    main()
```

38.5 依赖关系图

38.5.1 包依赖

```
openarm_bringup
    ├── openarm_description
    ├── openarm_hardware
    ├── openarm_controllers
    └── openarm_moveit_config

openarm_moveit_config
    └── openarm_description

openarm_hardware
    └── openarm_msgs (可选)

openarm_controllers
    └── openarm_description
```

38.5.2 ROS2 依赖

核心依赖:

```
- rclcpp / rclpy
- hardware_interface
- controller_interface
- controller_manager
- joint_trajectory_controller
- moveit_ros
```

- tf2_ros

完整依赖列表 (rosdep):

```
# 在工作空间根目录
rosdep install --from-paths src --ignore-src -r -y

# 主要安装:
# - ros-humble-ros2-control
# - ros-humble-ros2-controllers
# - ros-humble-moveit
# - ros-humble-xacro
# - python3-can
```

38.6 编译系统

38.6.1 编译顺序

```
# 先编译消息包
colcon build --packages-select openarm_msgs

# 编译描述包
colcon build --packages-select openarm_description

# 编译硬件接口
colcon build --packages-select openarm_hardware

# 编译控制器配置
colcon build --packages-select openarm_controllers

# 编译 MoveIt 配置
colcon build --packages-select openarm_moveit_config

# 编译启动包
colcon build --packages-select openarm_bringup
```

```
# 或一次性编译全部
colcon build
```

38.6.2 编译选项

```
# 调试版本
colcon build --cmake-args -DCMAKE_BUILD_TYPE=Debug

# Release 版本（性能优化）
colcon build --cmake-args -DCMAKE_BUILD_TYPE=Release

# 并行编译
colcon build --parallel-workers 4

# 仅编译修改的包及其依赖
colcon build --packages-up-to openarm_hardware

# 详细输出
colcon build --event-handlers console_direct+

# 符号链接安装（开发模式）
colcon build --symlink-install
```

38.7 配置文件管理

38.7.1 参数文件层次

```
系统级参数（openarm_bringup/config/）
    ↓
硬件参数（openarm_hardware/config/）
    ↓
控制器参数（openarm_controllers/config/）
    ↓
MoveIt参数（openarm_moveit_config/config/）
```

38.7.2 参数覆盖机制

```
# 启动时覆盖参数
ros2 launch openarm_bringup openarm.launch.py \
    update_rate:=200 \
    use_sim_time:=false

# 使用自定义配置文件
ros2 launch openarm_bringup openarm.launch.py \
    controllers_file:=/path/to/custom_controllers.yaml
```

38.8 开发工作流

38.8.1 典型开发流程

```
1. 修改源代码
   ↓
2. 编译修改的包
   colcon build --packages-select <package_name>
   ↓
3. Source工作空间
   source install/setup.bash
   ↓
4. 测试
   ros2 launch / ros2 run
   ↓
5. 调试 (如有问题)
   - 查看日志: ros2 topic echo /rosout
   - 使用GDB: gdb --args ros2 run ...
   ↓
6. 重复1-5直到完成
```

38.8.2 快速测试

```
# 测试单个节点
ros2 run openarm_hardware test_can_interface
```

```
# 测试硬件接口
ros2 launch openarm_bringup hardware_only.launch.py

# 测试控制器
ros2 control list_controllers
ros2 topic hz /joint_states

# 测试 MoveIt
ros2 launch openarm_moveit_config demo.launch.py
```

38.9 代码规范

38.9.1 C++ 代码规范

```
// 命名约定
// - 类名: PascalCase (OpenArmSystem)
// - 函数名: snake_case (read(), write())
// - 变量名: snake_case (hw_positions_)
// - 常量: UPPER_SNAKE_CASE (MAX_MOTORS)
// - 私有成员后缀下划线 (can0_)

// 头文件保护
#ifndef PACKAGE__HEADER_HPP_
#define PACKAGE__HEADER_HPP_
// ...
#endif

// 包含顺序
// 1. 对应头文件
// 2. C 系统头文件
// 3. C++ 系统头文件
// 4. 第三方库头文件
// 5. ROS2 头文件
// 6. 本地头文件
```

```
// 注释
// 使用 Doxygen 风格
/**
 * @brief 读取硬件状态
 * @param time 当前时间
 * @param period 周期
 * @return 操作结果
 */
```

38.9.2 Python 代码规范

```
# 遵循 PEP 8
# - 类名: PascalCase (MotorController)
# - 函数名: snake_case (enable_motor())
# - 常量: UPPER_SNAKE_CASE (MAX_VELOCITY)

# 文档字符串
def enable_motor(motor_id: int, can_channel: str) -> bool:
    """
    使能指定电机

    Args:
        motor_id: 电机 ID (1-7)
        can_channel: CAN 通道 ('can0'或'can1')

    Returns:
        成功返回 True, 失败返回 False

    Raises:
        ValueError: 电机 ID 超出范围
    """
    pass

# 类型提示
```

```python
from typing import List, Dict, Optional


def get_motor_positions(motor_ids: List[int]) -> Dict[int, float]:
    pass
```

38.10 插件机制

38.10.1 硬件接口插件

注册:

```xml
<!-- openarm_hardware_plugin.xml -->
<library path="openarm_hardware">
  <class name="openarm_hardware/OpenArmSystem"
         type="openarm_hardware::OpenArmSystem"
         base_class_type="hardware_interface::SystemInterface">
  </class>
</library>
```

导出:

```cmake
# CMakeLists.txt
pluginlib_export_plugin_description_file(
  hardware_interface
  openarm_hardware_plugin.xml
)
```

使用 (在 URDF 中):

```xml
<ros2_control name="OpenArmSystem" type="system">
  <hardware>
    <plugin>openarm_hardware/OpenArmSystem</plugin>
    <param name="can0_channel">can0</param>
    <param name="can1_channel">can1</param>
  </hardware>
</ros2_control>
```

38.10.2 控制器插件

ROS2 已提供标准控制器插件: - joint_trajectory_controller/JointTrajectoryController - position_controllers/JointGroupPositionController - velocity_controllers/JointGroupVelocityController - effort_controllers/JointGroupEffortController

自定义控制器 (高级):

```
// 继承 controller_interface::ControllerInterface
#include <controller_interface/controller_interface.hpp>

class CustomController : public controller_interface::ControllerInterface
{
  // 实现生命周期回调
  controller_interface::CallbackReturn on_init() override;
  controller_interface::InterfaceConfiguration command_interface_configuration() const override;
  controller_interface::InterfaceConfiguration state_interface_configuration() const override;
  controller_interface::return_type update(
    const rclcpp::Time & time, const rclcpp::Duration & period) override;
};
```

38.11 总结

38.11.1 包结构要点

✓ openarm_description - 机器人模型(URDF)
✓ openarm_hardware - 硬件接口(C++)
✓ openarm_controllers - 控制器配置(YAML)
✓ openarm_bringup - 启动文件(Python)
✓ openarm_moveit_config - MoveIt配置
✓ openarm_msgs - 自定义消息
✓ motor_tests_openarmx_com - 低层级测试脚本

38.11.2 开发入口点

扩展硬件功能: - 修改 openarm_hardware/src/openarm_system.cpp

添加控制器: - 编辑 openarm_controllers/config/openarm_controllers.yaml

修改机器人模型: - 编辑 openarm_description/urdf/*.xacro

添加自定义节点: - 在 `openarm_bringup` 创建新包

定义新消息: - 在 `openarm_msgs` 添加 `.msg` / `.srv` / `.action`

38.11.3 下一步

- **02_ 扩展开发** - 如何扩展功能
- **03_API 参考** - 详细 API 文档
- **04_ 贡献指南** - 代码贡献规范

本文档版本: v1.0 最后更新: 2025 年 10 月 20 日 成都长数机器人有限公司

39 扩展开发

39.1 OpenArmX 系统扩展开发指南

学习如何为 OpenArmX 添加新功能、传感器和控制器。

39.2 扩展开发概述

39.2.1 常见扩展场景

☐ 添加新传感器（相机、力传感器等）
☐ 集成末端执行器（夹爪、工具）
☐ 自定义控制器算法
☐ 添加感知功能（视觉、触觉）
☐ 扩展通信接口（TCP/IP、Modbus）
☐ 实现高级功能（示教、路径规划）

39.2.2 开发环境要求

```
# 系统
- Ubuntu 22.04 LTS
- ROS2 Humble
- C++17 / Python 3.10+

# 开发工具
sudo apt install \
    build-essential \
```

```
    cmake \
    git \
    python3-pip \
    python3-colcon-common-extensions \
    clang-format \
    clang-tidy
```

39.3 添加新传感器

39.3.1 场景: 集成 Intel RealSense 相机

```
cd ~/openarmx_robotstride_ws-cc/src
ros2 pkg create --build-type ament_cmake \
    openarm_sensors \
    --dependencies rclcpp sensor_msgs cv_bridge image_transport

cd openarm_sensors
mkdir include/openarm_sensors src launch config
```

39.3.1.1 步骤 1: 创建新包

39.3.1.2 步骤 2: 编写相机节点 include/openarm_sensors/realsense_node.hpp:

```
#ifndef OPENARM_SENSORS__REALSENSE_NODE_HPP_
#define OPENARM_SENSORS__REALSENSE_NODE_HPP_

#include <rclcpp/rclcpp.hpp>
#include <sensor_msgs/msg/image.hpp>
#include <sensor_msgs/msg/camera_info.hpp>
#include <librealsense2/rs.hpp>

namespace openarm_sensors
{

class RealSenseNode : public rclcpp::Node
```

```
{
public:
  explicit RealSenseNode(const rclcpp::NodeOptions & options = rclcpp::NodeOptions());
  ~RealSenseNode();

private:
  void timer_callback();
  void publish_images();

  // RealSense 管道
  rs2::pipeline pipe_;
  rs2::config cfg_;

  // 发布器
  rclcpp::Publisher<sensor_msgs::msg::Image>::SharedPtr color_pub_;
  rclcpp::Publisher<sensor_msgs::msg::Image>::SharedPtr depth_pub_;
  rclcpp::Publisher<sensor_msgs::msg::CameraInfo>::SharedPtr info_pub_;

  // 定时器
  rclcpp::TimerBase::SharedPtr timer_;

  // 参数
  int width_;
  int height_;
  int fps_;
  std::string frame_id_;
};

} // namespace openarm_sensors

#endif
```

src/realsense_node.cpp:

```
#include "openarm_sensors/realsense_node.hpp"
#include <cv_bridge/cv_bridge.h>
#include <opencv2/opencv.hpp>

namespace openarm_sensors
{

RealSenseNode::RealSenseNode(const rclcpp::NodeOptions & options)
: Node("realsense_node", options)
{
  // 声明参数
  this->declare_parameter("width", 640);
  this->declare_parameter("height", 480);
  this->declare_parameter("fps", 30);
  this->declare_parameter("frame_id", "camera_link");

  // 获取参数
  width_ = this->get_parameter("width").as_int();
  height_ = this->get_parameter("height").as_int();
  fps_ = this->get_parameter("fps").as_int();
  frame_id_ = this->get_parameter("frame_id").as_string();

  // 配置 RealSense
  cfg_.enable_stream(RS2_STREAM_COLOR, width_, height_, RS2_FORMAT_BGR8, fps_);
  cfg_.enable_stream(RS2_STREAM_DEPTH, width_, height_, RS2_FORMAT_Z16, fps_);

  // 启动管道
  try {
    pipe_.start(cfg_);
    RCLCPP_INFO(this->get_logger(), "RealSense pipeline started");
  } catch (const rs2::error & e) {
    RCLCPP_ERROR(this->get_logger(), "Failed to start RealSense: %s", e.what());
    throw;
  }
```

```
  // 创建发布器
  color_pub_ = this->create_publisher<sensor_msgs::msg::Image>(
    "~/color/image_raw", 10);
  depth_pub_ = this->create_publisher<sensor_msgs::msg::Image>(
    "~/depth/image_raw", 10);
  info_pub_ = this->create_publisher<sensor_msgs::msg::CameraInfo>(
    "~/camera_info", 10);

  // 定时器
  timer_ = this->create_wall_timer(
    std::chrono::milliseconds(1000 / fps_),
    std::bind(&RealSenseNode::timer_callback, this));

  RCLCPP_INFO(this->get_logger(), "RealSense node initialized");
}

RealSenseNode::~RealSenseNode()
{
  pipe_.stop();
}

void RealSenseNode::timer_callback()
{
  publish_images();
}

void RealSenseNode::publish_images()
{
  // 等待帧
  rs2::frameset frames = pipe_.wait_for_frames();

  // 彩色图像
  rs2::video_frame color_frame = frames.get_color_frame();
```

```
cv::Mat color_mat(
  cv::Size(width_, height_),
  CV_8UC3,
  (void*)color_frame.get_data(),
  cv::Mat::AUTO_STEP
);

auto color_msg = cv_bridge::CvImage(
  std_msgs::msg::Header(), "bgr8", color_mat).toImageMsg();
color_msg->header.stamp = this->now();
color_msg->header.frame_id = frame_id_;
color_pub_->publish(*color_msg);

// 深度图像
rs2::depth_frame depth_frame = frames.get_depth_frame();
cv::Mat depth_mat(
  cv::Size(width_, height_),
  CV_16UC1,
  (void*)depth_frame.get_data(),
  cv::Mat::AUTO_STEP
);

auto depth_msg = cv_bridge::CvImage(
  std_msgs::msg::Header(), "16UC1", depth_mat).toImageMsg();
depth_msg->header.stamp = this->now();
depth_msg->header.frame_id = frame_id_;
depth_pub_->publish(*depth_msg);

// 相机信息
auto info_msg = std::make_shared<sensor_msgs::msg::CameraInfo>();
info_msg->header.stamp = this->now();
info_msg->header.frame_id = frame_id_;
info_msg->width = width_;
info_msg->height = height_;
```

```
  // 填充内参矩阵...
  info_pub_->publish(*info_msg);
}

} // namespace openarm_sensors

#include <rclcpp_components/register_node_macro.hpp>
RCLCPP_COMPONENTS_REGISTER_NODE(openarm_sensors::RealSenseNode)
```

39.3.1.3 步骤 3: 更新 URDF 模型 在 openarm_description/urdf/sensors.xacro:

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro">

  <xacro:macro name="realsense_d435" params="parent *origin">
    <!-- 相机链接 -->
    <link name="camera_link">
      <visual>
        <geometry>
          <box size="0.025 0.090 0.025"/>
        </geometry>
        <material name="camera_material">
          <color rgba="0.1 0.1 0.1 1.0"/>
        </material>
      </visual>
      <collision>
        <geometry>
          <box size="0.025 0.090 0.025"/>
        </geometry>
      </collision>
      <inertial>
        <mass value="0.072"/>
        <inertia ixx="0.0001" ixy="0" ixz="0" iyy="0.0001" iyz="0" izz="0.0001"/>
      </inertial>
    </link>
```

```
    <!-- 相机关节 -->
    <joint name="camera_joint" type="fixed">
      <xacro:insert_block name="origin"/>
      <parent link="${parent}"/>
      <child link="camera_link"/>
    </joint>

    <!-- 光学坐标系 -->
    <link name="camera_optical_frame"/>

    <joint name="camera_optical_joint" type="fixed">
      <origin xyz="0 0 0" rpy="-1.5707963 0 -1.5707963"/>
      <parent link="camera_link"/>
      <child link="camera_optical_frame"/>
    </joint>

  </xacro:macro>

</robot>
```

在 openarm.urdf.xacro 中引用:

```
<xacro:include filename="$(find openarm_description)/urdf/sensors.xacro"/>

<!-- 安装在左臂末端 -->
<xacro:realsense_d435 parent="left_link_7">
  <origin xyz="0.05 0 0" rpy="0 0 0"/>
</xacro:realsense_d435>
```

39.3.1.4 步骤 4: 创建启动文件 launch/realsense.launch.py:

```
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration
```

```python
def generate_launch_description():
    return LaunchDescription([
        DeclareLaunchArgument('width', default_value='640'),
        DeclareLaunchArgument('height', default_value='480'),
        DeclareLaunchArgument('fps', default_value='30'),

        Node(
            package='openarm_sensors',
            executable='realsense_node',
            name='realsense',
            parameters=[{
                'width': LaunchConfiguration('width'),
                'height': LaunchConfiguration('height'),
                'fps': LaunchConfiguration('fps'),
                'frame_id': 'camera_optical_frame'
            }],
            output='screen'
        )
    ])
```

```bash
cd ~/openarmx_robotstride_ws-cc
colcon build --packages-select openarm_sensors
source install/setup.bash

# 启动相机节点
ros2 launch openarm_sensors realsense.launch.py

# 查看图像
ros2 run rqt_image_view rqt_image_view
```

39.3.1.5 步骤 5: 编译和测试

39.4 集成末端执行器

39.4.1 场景: 添加 Robotiq 2F-85 夹爪

39.4.1.1 步骤 1: 创建夹爪硬件接口 在 openarm_hardware 中添加 gripper_interface.hpp:

```
#ifndef OPENARM_HARDWARE__GRIPPER_INTERFACE_HPP_
#define OPENARM_HARDWARE__GRIPPER_INTERFACE_HPP_

#include <hardware_interface/system_interface.hpp>
#include <rclcpp/rclcpp.hpp>
#include <modbus/modbus.h>

namespace openarm_hardware
{

class RobotiqGripperInterface : public hardware_interface::SystemInterface
{
public:
  CallbackReturn on_init(const hardware_interface::HardwareInfo & info) override;
  CallbackReturn on_configure(const rclcpp_lifecycle::State & previous_state) override;
  CallbackReturn on_activate(const rclcpp_lifecycle::State & previous_state) override;
  CallbackReturn on_deactivate(const rclcpp_lifecycle::State & previous_state) override;

  hardware_interface::return_type read(
    const rclcpp::Time & time, const rclcpp::Duration & period) override;
  hardware_interface::return_type write(
    const rclcpp::Time & time, const rclcpp::Duration & period) override;

  std::vector<hardware_interface::StateInterface> export_state_interfaces() override;
  std::vector<hardware_interface::CommandInterface> export_command_interfaces() override;

private:
  modbus_t * modbus_ctx_;
  std::string ip_address_;
  int port_;
```

```
  // 状态
  double position_;       // 0.0 (打开) - 0.085 (闭合) [m]
  double velocity_;
  double effort_;

  // 命令
  double position_command_;
  double velocity_command_;
  double effort_command_;

  bool activate_gripper();
  bool set_position(uint8_t target);
  uint8_t get_position();
};

} // namespace openarm_hardware

#endif
```

gripper_interface.cpp 实现:

```
#include "openarm_hardware/gripper_interface.hpp"
#include <cmath>

namespace openarm_hardware
{

CallbackReturn RobotiqGripperInterface::on_init(
  const hardware_interface::HardwareInfo & info)
{
  if (hardware_interface::SystemInterface::on_init(info) != CallbackReturn::SUCCESS) {
    return CallbackReturn::ERROR;
  }

  // 读取参数
```

```
  ip_address_ = info_.hardware_parameters["ip_address"];
  port_ = std::stoi(info_.hardware_parameters["port"]);

  // 初始化状态
  position_ = 0.085;  // 初始闭合
  velocity_ = 0.0;
  effort_ = 0.0;

  position_command_ = position_;
  velocity_command_ = 0.1;     // 默认速度
  effort_command_ = 50.0;      // 默认力

  return CallbackReturn::SUCCESS;
}

CallbackReturn RobotiqGripperInterface::on_configure(
  const rclcpp_lifecycle::State & /*previous_state*/)
{
  // 创建 Modbus TCP 连接
  modbus_ctx_ = modbus_new_tcp(ip_address_.c_str(), port_);
  if (modbus_ctx_ == nullptr) {
    RCLCPP_ERROR(rclcpp::get_logger("RobotiqGripperInterface"),
                 "Failed to create Modbus context");
    return CallbackReturn::ERROR;
  }

  if (modbus_connect(modbus_ctx_) == -1) {
    RCLCPP_ERROR(rclcpp::get_logger("RobotiqGripperInterface"),
                 "Failed to connect to gripper at %s:%d", ip_address_.c_str(), port_);
    modbus_free(modbus_ctx_);
    return CallbackReturn::ERROR;
  }

  RCLCPP_INFO(rclcpp::get_logger("RobotiqGripperInterface"),
```

```
              "Connected to Robotiq gripper");

  return CallbackReturn::SUCCESS;
}

CallbackReturn RobotiqGripperInterface::on_activate(
  const rclcpp_lifecycle::State & /*previous_state*/)
{
  // 激活夹爪
  if (!activate_gripper()) {
    return CallbackReturn::ERROR;
  }

  RCLCPP_INFO(rclcpp::get_logger("RobotiqGripperInterface"), "Gripper activated");
  return CallbackReturn::SUCCESS;
}

CallbackReturn RobotiqGripperInterface::on_deactivate(
  const rclcpp_lifecycle::State & /*previous_state*/)
{
  modbus_close(modbus_ctx_);
  modbus_free(modbus_ctx_);
  return CallbackReturn::SUCCESS;
}

hardware_interface::return_type RobotiqGripperInterface::read(
  const rclcpp::Time & /*time*/, const rclcpp::Duration & /*period*/)
{
  // 读取夹爪位置
  uint8_t raw_position = get_position();

  // 转换为米 (0-255 -> 0.0-0.085m)
  position_ = (raw_position / 255.0) * 0.085;
```

```
  return hardware_interface::return_type::OK;
}

hardware_interface::return_type RobotiqGripperInterface::write(
  const rclcpp::Time & /*time*/, const rclcpp::Duration & /*period*/)
{
  // 转换命令 (m -> 0-255)
  uint8_t target = static_cast<uint8_t>((position_command_ / 0.085) * 255.0);

  if (!set_position(target)) {
    return hardware_interface::return_type::ERROR;
  }

  return hardware_interface::return_type::OK;
}

std::vector<hardware_interface::StateInterface>
RobotiqGripperInterface::export_state_interfaces()
{
  std::vector<hardware_interface::StateInterface> state_interfaces;

  state_interfaces.emplace_back(
    hardware_interface::StateInterface(
      "gripper_joint", hardware_interface::HW_IF_POSITION, &position_));

  return state_interfaces;
}

std::vector<hardware_interface::CommandInterface>
RobotiqGripperInterface::export_command_interfaces()
{
  std::vector<hardware_interface::CommandInterface> command_interfaces;

  command_interfaces.emplace_back(
```

```
    hardware_interface::CommandInterface(
      "gripper_joint", hardware_interface::HW_IF_POSITION, &position_command_));

  return command_interfaces;
}

bool RobotiqGripperInterface::activate_gripper()
{
  // Robotiq 激活序列
  uint16_t activate_cmd = 0x0100;
  if (modbus_write_register(modbus_ctx_, 0, activate_cmd) == -1) {
    return false;
  }

  // 等待激活完成
  std::this_thread::sleep_for(std::chrono::seconds(2));
  return true;
}

bool RobotiqGripperInterface::set_position(uint8_t target)
{
  // 写入目标位置寄存器
  if (modbus_write_register(modbus_ctx_, 3, target) == -1) {
    return false;
  }
  return true;
}

uint8_t RobotiqGripperInterface::get_position()
{
  uint16_t reg[1];
  if (modbus_read_registers(modbus_ctx_, 7, 1, reg) == -1) {
    return 0;
  }
```

```
  return static_cast<uint8_t>(reg[0]);
}

} // namespace openarm_hardware

#include <pluginlib/class_list_macros.hpp>
PLUGINLIB_EXPORT_CLASS(
  openarm_hardware::RobotiqGripperInterface,
  hardware_interface::SystemInterface
)
```

39.4.1.2 步骤 2: 更新 URDF 在 openarm_description/urdf/gripper.xacro:

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro">

  <xacro:macro name="robotiq_2f85" params="parent *origin prefix">

    <!-- 夹爪基座 -->
    <link name="${prefix}gripper_base_link">
      <visual>
        <geometry>
          <mesh filename="package://openarm_description/meshes/robotiq_2f85_base.stl"/>
        </geometry>
      </visual>
      <collision>
        <geometry>
          <box size="0.05 0.05 0.05"/>
        </geometry>
      </collision>
    </link>

    <joint name="${prefix}gripper_base_joint" type="fixed">
      <xacro:insert_block name="origin"/>
      <parent link="${parent}"/>
```

```
    <child link="${prefix}gripper_base_link"/>
  </joint>

  <!-- 可动关节 (简化为单关节模型) -->
  <link name="${prefix}gripper_finger_link">
    <visual>
      <geometry>
        <box size="0.02 0.01 0.08"/>
      </geometry>
    </visual>
  </link>

  <joint name="${prefix}gripper_joint" type="prismatic">
    <origin xyz="0.05 0 0" rpy="0 0 0"/>
    <parent link="${prefix}gripper_base_link"/>
    <child link="${prefix}gripper_finger_link"/>
    <axis xyz="0 1 0"/>
    <limit lower="0.0" upper="0.085" effort="100.0" velocity="0.1"/>
  </joint>

  <!-- ros2_control 标签 -->
  <ros2_control name="${prefix}gripper_system" type="system">
    <hardware>
      <plugin>openarm_hardware/RobotiqGripperInterface</plugin>
      <param name="ip_address">192.168.1.11</param>
      <param name="port">502</param>
    </hardware>

    <joint name="${prefix}gripper_joint">
      <command_interface name="position">
        <param name="min">0.0</param>
        <param name="max">0.085</param>
      </command_interface>
      <state_interface name="position"/>
```

```
      </joint>
    </ros2_control>

  </xacro:macro>

</robot>
```

39.4.1.3 步骤 3: 配置控制器 config/gripper_controllers.yaml:

```yaml
controller_manager:
  ros__parameters:
    left_gripper_controller:
      type: position_controllers/GripperActionController

left_gripper_controller:
  ros__parameters:
    joint: left_gripper_joint

    action_monitor_rate: 20.0

    goal_tolerance: 0.001
    max_effort: 100.0

    stall_velocity_threshold: 0.001
    stall_timeout: 1.0
```

39.5 自定义控制器

39.5.1 场景: 实现阻抗控制器

```
cd ~/openarmx_robotstride_ws-cc/src
ros2 pkg create --build-type ament_cmake \
    openarm_impedance_controller \
    --dependencies controller_interface rclcpp pluginlib
```

39.5.1.1 步骤 1: 创建控制器包

39.5.1.2 步骤 2: 实现控制器 include/openarm_impedance_controller/impedance_controller.hp

```
#ifndef OPENARM_IMPEDANCE_CONTROLLER__IMPEDANCE_CONTROLLER_HPP_
#define OPENARM_IMPEDANCE_CONTROLLER__IMPEDANCE_CONTROLLER_HPP_

#include <controller_interface/controller_interface.hpp>
#include <rclcpp/rclcpp.hpp>
#include <geometry_msgs/msg/wrench_stamped.hpp>

namespace openarm_impedance_controller
{

class ImpedanceController : public controller_interface::ControllerInterface
{
public:
  controller_interface::CallbackReturn on_init() override;

  controller_interface::InterfaceConfiguration command_interface_configuration() const override;
  controller_interface::InterfaceConfiguration state_interface_configuration() const override;

  controller_interface::CallbackReturn on_configure(
    const rclcpp_lifecycle::State & previous_state) override;
  controller_interface::CallbackReturn on_activate(
    const rclcpp_lifecycle::State & previous_state) override;
  controller_interface::CallbackReturn on_deactivate(
    const rclcpp_lifecycle::State & previous_state) override;

  controller_interface::return_type update(
    const rclcpp::Time & time, const rclcpp::Duration & period) override;

private:
  std::vector<std::string> joint_names_;
```

```
  // 阻抗参数
  std::vector<double> stiffness_;   // K (刚度)
  std::vector<double> damping_;     // D (阻尼)
  std::vector<double> mass_;        // M (惯性)

  // 期望轨迹
  std::vector<double> desired_position_;
  std::vector<double> desired_velocity_;

  // 外力
  std::vector<double> external_force_;

  // 订阅器
  rclcpp::Subscription<geometry_msgs::msg::WrenchStamped>::SharedPtr force_sub_;

  void force_callback(const geometry_msgs::msg::WrenchStamped::SharedPtr msg);
  double compute_torque(size_t joint_idx, double pos_error, double vel_error, double ext_force);
};

} // namespace openarm_impedance_controller

#endif
```

src/impedance_controller.cpp:

```
#include "openarm_impedance_controller/impedance_controller.hpp"

namespace openarm_impedance_controller
{

controller_interface::CallbackReturn ImpedanceController::on_init()
{
  // 声明参数
  auto_declare<std::vector<std::string>>("joints", std::vector<std::string>());
  auto_declare<std::vector<double>>("stiffness", std::vector<double>(7, 100.0));
```

```
  auto_declare<std::vector<double>>("damping", std::vector<double>(7, 10.0));
  auto_declare<std::vector<double>>("mass", std::vector<double>(7, 1.0));

  return controller_interface::CallbackReturn::SUCCESS;
}

controller_interface::InterfaceConfiguration
ImpedanceController::command_interface_configuration() const
{
  controller_interface::InterfaceConfiguration config;
  config.type = controller_interface::interface_configuration_type::INDIVIDUAL;

  for (const auto & joint_name : joint_names_) {
    config.names.push_back(joint_name + "/effort");
  }

  return config;
}

controller_interface::InterfaceConfiguration
ImpedanceController::state_interface_configuration() const
{
  controller_interface::InterfaceConfiguration config;
  config.type = controller_interface::interface_configuration_type::INDIVIDUAL;

  for (const auto & joint_name : joint_names_) {
    config.names.push_back(joint_name + "/position");
    config.names.push_back(joint_name + "/velocity");
  }

  return config;
}

controller_interface::CallbackReturn ImpedanceController::on_configure(
```

```
  const rclcpp_lifecycle::State & /*previous_state*/)
{
  // 读取参数
  joint_names_ = get_node()->get_parameter("joints").as_string_array();
  stiffness_ = get_node()->get_parameter("stiffness").as_double_array();
  damping_ = get_node()->get_parameter("damping").as_double_array();
  mass_ = get_node()->get_parameter("mass").as_double_array();

  // 初始化
  size_t num_joints = joint_names_.size();
  desired_position_.resize(num_joints, 0.0);
  desired_velocity_.resize(num_joints, 0.0);
  external_force_.resize(num_joints, 0.0);

  // 订阅外力
  force_sub_ = get_node()->create_subscription<geometry_msgs::msg::WrenchStamped>(
    "~/force_sensor", 10,
    std::bind(&ImpedanceController::force_callback, this, std::placeholders::_1));

  RCLCPP_INFO(get_node()->get_logger(), "Impedance controller configured");
  return controller_interface::CallbackReturn::SUCCESS;
}

controller_interface::CallbackReturn ImpedanceController::on_activate(
  const rclcpp_lifecycle::State & /*previous_state*/)
{
  // 将当前位置设为期望位置
  for (size_t i = 0; i < joint_names_.size(); ++i) {
    desired_position_[i] = state_interfaces_[i * 2].get_value();  // position
  }

  return controller_interface::CallbackReturn::SUCCESS;
}
```

```
controller_interface::CallbackReturn ImpedanceController::on_deactivate(
  const rclcpp_lifecycle::State & /*previous_state*/)
{
  return controller_interface::CallbackReturn::SUCCESS;
}

controller_interface::return_type ImpedanceController::update(
  const rclcpp::Time & /*time*/, const rclcpp::Duration & /*period*/)
{
  // 阻抗控制律: τ = M(ẍd - ẍ) + D(ẋd - ẋ) + K(xd - x) + τext

  for (size_t i = 0; i < joint_names_.size(); ++i) {
    // 当前状态
    double current_position = state_interfaces_[i * 2].get_value();
    double current_velocity = state_interfaces_[i * 2 + 1].get_value();

    // 误差
    double pos_error = desired_position_[i] - current_position;
    double vel_error = desired_velocity_[i] - current_velocity;

    // 计算力矩
    double torque = compute_torque(i, pos_error, vel_error, external_force_[i]);

    // 输出力矩命令
    command_interfaces_[i].set_value(torque);
  }

  return controller_interface::return_type::OK;
}

void ImpedanceController::force_callback(const geometry_msgs::msg::WrenchStamped::SharedPtr msg)
{
  // 简化: 将笛卡尔力映射到关节空间
  // 实际应用需要雅可比转置
```

```
  external_force_[0] = msg->wrench.force.x;
  // ... (完整的力映射)
}

double ImpedanceController::compute_torque(
  size_t joint_idx, double pos_error, double vel_error, double ext_force)
{
  // τ = K*Δx + D*Δẋ + τext
  return stiffness_[joint_idx] * pos_error +
         damping_[joint_idx] * vel_error +
         ext_force;
}

} // namespace openarm_impedance_controller

#include <pluginlib/class_list_macros.hpp>
PLUGINLIB_EXPORT_CLASS(
  openarm_impedance_controller::ImpedanceController,
  controller_interface::ControllerInterface
)
```

39.6 添加高级功能

39.6.1 场景: 实现视觉伺服

创建新包:

```
ros2 pkg create --build-type ament_python \
    openarm_visual_servo \
    --dependencies rclpy geometry_msgs sensor_msgs cv_bridge
```

visual_servo_node.py:

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
```

```
from geometry_msgs.msg import Twist
from cv_bridge import CvBridge
import cv2
import numpy as np

class VisualServoNode(Node):
    def __init__(self):
        super().__init__('visual_servo_node')

        # 订阅相机图像
        self.image_sub = self.create_subscription(
            Image, '/camera/color/image_raw',
            self.image_callback, 10)

        # 发布速度命令
        self.vel_pub = self.create_publisher(Twist, '/left_arm/cartesian_velocity', 10)

        self.bridge = CvBridge()

        # 目标特征 (例如: 红色物体中心)
        self.target_x = 320  # 图像中心
        self.target_y = 240

        # 控制增益
        self.kp_x = 0.001
        self.kp_y = 0.001

        self.get_logger().info('Visual servo node started')

    def image_callback(self, msg):
        """ 处理图像并计算控制命令"""
        # 转换为 OpenCV 格式
        cv_image = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')
```

```
        # 检测目标 (红色物体)
        hsv = cv2.cvtColor(cv_image, cv2.COLOR_BGR2HSV)
        lower_red = np.array([0, 100, 100])
        upper_red = np.array([10, 255, 255])
        mask = cv2.inRange(hsv, lower_red, upper_red)

        # 计算质心
        moments = cv2.moments(mask)
        if moments['m00'] > 0:
            cx = int(moments['m10'] / moments['m00'])
            cy = int(moments['m01'] / moments['m00'])

            # 误差
            error_x = cx - self.target_x
            error_y = cy - self.target_y

            # 视觉伺服控制律
            vel = Twist()
            vel.linear.x = -self.kp_y * error_y  # 前后
            vel.linear.y = -self.kp_x * error_x  # 左右
            vel.linear.z = 0.0
            vel.angular.x = 0.0
            vel.angular.y = 0.0
            vel.angular.z = 0.0

            self.vel_pub.publish(vel)

            self.get_logger().info(
                f'Target at ({cx}, {cy}), Error: ({error_x}, {error_y})')
        else:
            # 未检测到目标，停止运动
            self.vel_pub.publish(Twist())

def main():
```

```
    rclpy.init()
    node = VisualServoNode()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

39.7 测试与调试

39.7.1 单元测试

test/test_impedance_controller.cpp:

```
#include <gtest/gtest.h>
#include "openarm_impedance_controller/impedance_controller.hpp"

TEST(ImpedanceControllerTest, Initialization)
{
  auto controller = std::make_shared<openarm_impedance_controller::ImpedanceController>();

  auto result = controller->on_init();
  EXPECT_EQ(result, controller_interface::CallbackReturn::SUCCESS);
}

TEST(ImpedanceControllerTest, ComputeTorque)
{
  // 测试力矩计算
  double torque = compute_torque_wrapper(0, 0.1, 0.05, 0.0);
  EXPECT_GT(torque, 0.0);
}

int main(int argc, char **argv)
{
  testing::InitGoogleTest(&argc, argv);
```

```
  return RUN_ALL_TESTS();
}
```

CMakeLists.txt 中启用测试:

```
if(BUILD_TESTING)
  find_package(ament_cmake_gtest REQUIRED)

  ament_add_gtest(test_impedance_controller
    test/test_impedance_controller.cpp
  )

  target_link_libraries(test_impedance_controller
    ${PROJECT_NAME}
  )
endif()
```

39.7.2 集成测试

test/integration_test.launch.py:

```
from launch import LaunchDescription
from launch_ros.actions import Node
from launch_testing.actions import ReadyToTest
import unittest

def generate_test_description():
    return LaunchDescription([
        Node(
            package='openarm_sensors',
            executable='realsense_node',
            name='camera'
        ),
        Node(
            package='openarm_visual_servo',
            executable='visual_servo_node',
```

```
            name='visual_servo'
        ),
        ReadyToTest()
    ])

class TestVisualServo(unittest.TestCase):
    def test_node_startup(self):
        # 测试节点启动
        pass
```

39.8 文档与发布

39.8.1 编写 README

openarm_sensors/README.md:

```
# OpenArm Sensors

RealSense 相机集成包

## 功能

- Intel RealSense D435 支持
- 彩色图像发布 (`~/color/image_raw`)
- 深度图像发布 (`~/depth/image_raw`)
- 相机信息发布 (`~/camera_info`)

## 安装

### 依赖

```bash
sudo apt install ros-humble-cv-bridge ros-humble-image-transport
pip3 install pyrealsense2
```
```

39.8.2 编译

```
colcon build --packages-select openarm_sensors
```

39.9 使用

```
ros2 launch openarm_sensors realsense.launch.py
```

39.10 参数

- `width` (int, 默认: 640) - 图像宽度
- `height` (int, 默认: 480) - 图像高度
- `fps` (int, 默认: 30) - 帧率
- `frame_id` (string, 默认: “camera_optical_frame”) - TF 坐标系

39.11 话题

39.11.1 发布

- `~/color/image_raw` (sensor_msgs/Image) - 彩色图像
- `~/depth/image_raw` (sensor_msgs/Image) - 深度图像
- `~/camera_info` (sensor_msgs/CameraInfo) - 相机信息

39.12 许可

Apache 2.0

```
### package.xml完善

```xml
<?xml version="1.0"?>
<package format="3">
  <name>openarm_sensors</name>
  <version>1.0.0</version>
  <description>RealSense camera integration for OpenArmX</description>

  <maintainer email="dev@openarmx.com">OpenArmX Team</maintainer>
  <license>Apache-2.0</license>
```
```

```
  <url type="website">https://github.com/openarmx/openarm_sensors</url>
  <url type="bugtracker">https://github.com/openarmx/openarm_sensors/issues</url>
  <url type="repository">https://github.com/openarmx/openarm_sensors</url>

  <author>Your Name</author>

  <buildtool_depend>ament_cmake</buildtool_depend>

  <depend>rclcpp</depend>
  <depend>sensor_msgs</depend>
  <depend>cv_bridge</depend>
  <depend>image_transport</depend>

  <test_depend>ament_cmake_gtest</test_depend>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

39.13 总结

39.13.1 扩展开发要点

✓ 添加传感器 - 创建新包,实现节点,更新URDF

✓ 集成末端执行器 - 实现硬件接口,配置控制器

✓ 自定义控制器 - 继承ControllerInterface,注册插件

✓ 高级功能 - 利用ROS2生态,组合现有功能

✓ 测试 - 单元测试+集成测试

✓ 文档 - README + API文档 + 示例

39.13.2 开发检查清单

☐ 代码符合规范 (clang-format)

☐ 通过编译 (colcon build)

- ☐ 通过测试 (colcon test)
- ☐ 文档完善 (README + 注释)
- ☐ 依赖明确 (package.xml)
- ☐ 示例可用 (launch文件 + 演示)

39.13.3 下一步

- **03_API 参考** - 详细 API 说明
- **04_ 贡献指南** - 如何贡献代码
- **01_ 包结构说明** - 了解系统架构

本文档版本: v1.0 最后更新: 2025 年 10 月 20 日 成都长数机器人有限公司

40 API 参考

40.1 OpenArmX API 完整参考文档

详细的 C++ 和 Python API 参考手册。

40.2 C++ API

40.2.1 openarm_hardware::OpenArmSystem

头文件: `openarm_hardware/openarm_system.hpp`

命名空间: `openarm_hardware`

基类: `hardware_interface::SystemInterface`

40.2.1.1 公共方法

```
CallbackReturn on_init(const hardware_interface::HardwareInfo & info) override;
```

on_init() **功能:** 初始化硬件接口

参数: - `info` - 硬件配置信息 (从 URDF 读取)

返回值: - `CallbackReturn::SUCCESS` - 成功 - `CallbackReturn::ERROR` - 失败

示例:

```
auto system = std::make_shared<openarm_hardware::OpenArmSystem>();
auto result = system->on_init(hardware_info);
if (result != CallbackReturn::SUCCESS) {
    RCLCPP_ERROR(logger, "Failed to initialize");
}
```

```
CallbackReturn on_configure(const rclcpp_lifecycle::State & previous_state) override;
```

on_configure() **功能:** 配置硬件 (建立 CAN 连接)

参数: - `previous_state` - 前一个生命周期状态

返回值: - `CallbackReturn::SUCCESS` - 配置成功 - `CallbackReturn::ERROR` - 配置失败

注意: 此方法中建立 CAN 总线连接

```
CallbackReturn on_activate(const rclcpp_lifecycle::State & previous_state) override;
```

on_activate() **功能:** 激活硬件 (使能电机)

参数: - `previous_state` - 前一个生命周期状态

返回值: - `CallbackReturn::SUCCESS` - 激活成功 - `CallbackReturn::ERROR` - 激活失败

注意: 此方法中使能所有电机

```
CallbackReturn on_deactivate(const rclcpp_lifecycle::State & previous_state) override;
```

on_deactivate() **功能:** 停用硬件 (失能电机)

参数: - `previous_state` - 前一个生命周期状态

返回值: - `CallbackReturn::SUCCESS` - 停用成功

注意: 此方法中失能所有电机并关闭 CAN 连接

```
hardware_interface::return_type read(
    const rclcpp::Time & time,
```

```
    const rclcpp::Duration & period) override;
```

read() **功能:** 从硬件读取状态

参数: - `time` - 当前时间戳 - `period` - 更新周期

返回值: - `return_type::OK` - 读取成功 - `return_type::ERROR` - 读取失败

调用频率: 由 controller_manager 的 update_rate 决定 (通常 100-200Hz)

内部操作: 1. 从 CAN 总线接收电机反馈 2. 更新 `hw_positions_`, `hw_velocities_`, `hw_efforts_` 3. 更新 StateInterface

```
hardware_interface::return_type write(
    const rclcpp::Time & time,
    const rclcpp::Duration & period) override;
```

write() **功能:** 向硬件写入命令

参数: - `time` - 当前时间戳 - `period` - 更新周期

返回值: - `return_type::OK` - 写入成功 - `return_type::ERROR` - 写入失败

内部操作: 1. 从 CommandInterface 读取命令 2. 通过 CAN 总线发送控制指令 3. 使用 MIT 模式控制电机

```
std::vector<hardware_interface::StateInterface> export_state_interfaces() override;
```

export_state_interfaces() **功能:** 导出状态接口供控制器读取

返回值: StateInterface 向量

导出的接口: - `<joint_name>/position` - 位置 (rad) - `<joint_name>/velocity` - 速度 (rad/s) - `<joint_name>/effort` - 力矩 (Nm)

示例:

```
// 控制器中访问
double position = state_interfaces_[0].get_value();
```

```
std::vector<hardware_interface::CommandInterface> export_command_interfaces() override;
```

export_command_interfaces() **功能:** 导出命令接口供控制器写入

返回值: CommandInterface 向量

导出的接口: - `<joint_name>/position` - 位置命令 (rad)

示例:

```
// 控制器中设置
command_interfaces_[0].set_value(target_position);
```

40.2.2 openarm_hardware::RobstrideCAN

头文件: `openarm_hardware/robstride_can_interface.hpp`

命名空间: `openarm_hardware`

```
explicit RobstrideCAN(const std::string & can_channel);
```

40.2.2.1 构造函数

参数: - `can_channel` - CAN 通道名称 (“can0” 或 “can1”)

示例:

```
auto can = std::make_shared<RobstrideCAN>("can0");
```

40.2.2.2 公共方法

```
bool connect();
```

connect() **功能:** 连接 CAN 总线

返回值: - `true` - 连接成功 - `false` - 连接失败

示例:

```
if (!can->connect()) {
    std::cerr << "Failed to connect CAN" << std::endl;
}
```

```
void disconnect();
```

disconnect() **功能:** 断开 CAN 总线连接

```
bool send_motor_command(
    uint8_t motor_id,
    float position,
    float velocity,
    float kp,
    float kd,
    float torque
);
```

send_motor_command() **功能:** 发送电机控制命令 (MIT 模式)

参数: - `motor_id` - 电机 ID (1-7) - `position` - 目标位置 (rad, 范围: -12.5 ~ 12.5) - `velocity` - 目标速度 (rad/s, 范围: -30 ~ 30) - `kp` - 位置增益 (范围: 0 ~ 500) - `kd` - 速度增益 (范围: 0 ~ 5) - `torque` - 前馈力矩 (Nm, 范围: -18 ~ 18)

返回值: - `true` - 发送成功 - `false` - 发送失败

示例:

```
// PD 控制到位置 0.5rad
can->send_motor_command(
    1,          // motor_id
    0.5,        // position
    0.0,        // velocity
    50.0,       // kp
    1.0,        // kd
    0.0         // torque
);
```

```
bool receive_motor_feedback(
    uint8_t motor_id,
```

```
    float & position,
    float & velocity,
    float & torque
);
```

receive_motor_feedback() **功能:** 接收电机反馈

参数: - `motor_id` - 电机 ID - `position` - [输出] 当前位置 (rad) - `velocity` - [输出] 当前速度 (rad/s) - `torque` - [输出] 当前力矩 (Nm)

返回值: - `true` - 接收成功 - `false` - 超时或失败

示例:

```
float pos, vel, torque;
if (can->receive_motor_feedback(1, pos, vel, torque)) {
    std::cout << "Position: " << pos << " rad" << std::endl;
}
```

```
bool enable_motor(uint8_t motor_id);
```

enable_motor() **功能:** 使能电机

参数: - `motor_id` - 电机 ID (1-7)

返回值: - `true` - 使能成功 - `false` - 使能失败

```
bool disable_motor(uint8_t motor_id);
```

disable_motor() **功能:** 失能电机

参数: - `motor_id` - 电机 ID (1-7)

返回值: - `true` - 失能成功 - `false` - 失能失败

```
bool set_zero_position(uint8_t motor_id);
```

set_zero_position() **功能:** 设置当前位置为零位

参数: - `motor_id` - 电机 ID (1-7)

返回值: - `true` - 设置成功 - `false` - 设置失败

警告: 此操作会将电机当前位置设为 0, 请确保机械臂处于安全姿态

40.2.3 控制器接口

40.2.3.1 JointTrajectoryController 包: `joint_trajectory_controller`

类型: `joint_trajectory_controller/JointTrajectoryController`

头文件: `<joint_trajectory_controller/joint_trajectory_controller.hpp>`

Action 接口 **Action 类型:** `control_msgs::action::FollowJointTrajectory`

Action 名称: `<controller_name>/follow_joint_trajectory`

Goal 结构:

```
control_msgs::action::FollowJointTrajectory::Goal goal;

// 轨迹
goal.trajectory.joint_names = {"left_joint_1", "left_joint_2", ...};

trajectory_msgs::msg::JointTrajectoryPoint point;
point.positions = {0.0, 0.5, 1.0, ...};
point.velocities = {0.0, 0.0, 0.0, ...};  // 可选
point.time_from_start = rclcpp::Duration::from_seconds(1.0);

goal.trajectory.points.push_back(point);

// 容差 (可选)
control_msgs::msg::JointTolerance tol;
tol.name = "left_joint_1";
tol.position = 0.01;  // rad
tol.velocity = 0.1;   // rad/s
goal.path_tolerance.push_back(tol);
```

Result 结构:

```
// 错误码
enum ErrorCode {
    SUCCESSFUL = 0,
    INVALID_GOAL = -1,
    INVALID_JOINTS = -2,
    OLD_HEADER_TIMESTAMP = -3,
    PATH_TOLERANCE_VIOLATED = -4,
    GOAL_TOLERANCE_VIOLATED = -5
};

auto result = future.get();
if (result->error_code == SUCCESSFUL) {
    // 成功
}
```

Feedback 结构:

```
void feedback_callback(
    GoalHandleFollowJointTrajectory::SharedPtr,
    const std::shared_ptr<const FollowJointTrajectory::Feedback> feedback)
{
    std::cout << "Desired position: " << feedback->desired.positions[0] << std::endl;
    std::cout << "Actual position: " << feedback->actual.positions[0] << std::endl;
    std::cout << "Error: " << feedback->error.positions[0] << std::endl;
}
```

话题接口　订阅: - `<controller_name>/joint_trajectory` (trajectory_msgs/JointTrajectory) - 直接发送轨迹 (不使用 Action)

发布: - `<controller_name>/state` (control_msgs/JointTrajectoryControllerState) - 控制器状态 (期望、实际、误差) - `<controller_name>/controller_state` (control_msgs/JointTrajectoryControllerState) - 控制器详细状态

```
// 获取参数
auto node = controller->get_node();
```

```
int update_rate = node->get_parameter("update_rate").as_int();
```

参数 **主要参数:** - `joints` (string_array) - 关节名称列表 - `command_interfaces` (string_array) - 命令接口类型 (position/velocity/effort) - `state_interfaces` (string_array) - 状态接口类型 - `state_publish_rate` (double, Hz) - 状态发布频率 - `action_monitor_rate` (double, Hz) - Action 监控频率 - `constraints.*` (double) - 轨迹约束

40.3 Python API

40.3.1 robstride 模块

文件: `motor_tests_openarmx_com/lib/robstride.py`

40.3.1.1 Motor 类

```
Motor(bus: can.interface.Bus, motor_id: int, reduction_ratio: float = 1.0)
```

构造函数 **参数:** - `bus` - python-can 的 Bus 对象 - `motor_id` - 电机 ID (1-7) - `reduction_ratio` - 减速比 (默认 1.0)

示例:

```
import can
from robstride import Motor

bus = can.interface.Bus(channel='can0', bustype='socketcan')
motor = Motor(bus, motor_id=1)
```

```
def enable(self) -> bool:
```

enable() **功能:** 使能电机

返回值: - `True` - 成功 - `False` - 失败

示例:

```
if motor.enable():
    print("Motor enabled")
```

```
def disable(self) -> bool:
```

disable() **功能:** 失能电机

返回值: - `True` - 成功 - `False` - 失败

```
def set_zero(self) -> bool:
```

set_zero() **功能:** 设置当前位置为零位

返回值: - `True` - 成功 - `False` - 失败

警告: 确保机械臂在安全位置

```
def set_position(
    self,
    position: float,
    vel: float = 0.0,
    kp: float = 30.0,
    kd: float = 1.0,
    torque: float = 0.0
) -> bool:
```

set_position() **功能:** MIT 模式位置控制

参数: - `position` (float) - 目标位置 (rad) - `vel` (float) - 目标速度 (rad/s, 默认 0) - `kp` (float) - 位置增益 (默认 30.0) - `kd` (float) - 速度增益 (默认 1.0) - `torque` (float) - 前馈力矩 (Nm, 默认 0)

返回值: - `True` - 发送成功 - `False` - 发送失败

示例:

```
# 简单位置控制
motor.set_position(position=0.5)

# 带速度规划
motor.set_position(position=1.0, vel=0.1)
```

```
# 柔顺控制（低增益）
motor.set_position(position=0.5, kp=10.0, kd=0.5)

# 力控（前馈力矩）
motor.set_position(position=0.0, kp=0.0, kd=0.0, torque=2.0)
```

```
def get_feedback(self, timeout: float = 0.1) -> Dict[str, float]:
```

get_feedback() **功能:** 获取电机反馈

参数: - `timeout` (float) - 超时时间 (秒, 默认 0.1)

返回值: 字典, 包含: - `position` (float) - 位置 (rad) - `velocity` (float) - 速度 (rad/s) - `torque` (float) - 力矩 (Nm) - `temperature` (float) - 温度 (°C) - `error_code` (int) - 错误码

示例:

```
fb = motor.get_feedback()
print(f"Position: {fb['position']:.3f} rad")
print(f"Velocity: {fb['velocity']:.3f} rad/s")
print(f"Torque: {fb['torque']:.3f} Nm")
print(f"Temperature: {fb['temperature']:.1f} °C")

if fb['error_code'] != 0:
    print(f"Error: 0x{fb['error_code']:02X}")
```

```
def get_status(self) -> Dict[str, Any]:
```

get_status() **功能:** 获取电机详细状态

返回值: 字典, 包含: - `is_enabled` (bool) - 是否使能 - `is_fault` (bool) - 是否故障 - `error_code` (int) - 错误码 - `mode` (str) - 当前模式（“MIT”, “CSP”, etc.） - `voltage` (float) - 电压 (V) - `current` (float) - 电流 (A)

示例:

```
status = motor.get_status()
if status['is_fault']:
    print(f"Motor fault! Error: 0x{status['error_code']:02X}")
```

40.3.2 MoveIt Python API

包: `moveit_py` (ROS2 Humble)

40.3.2.1 MoveGroupInterface

```
from moveit_py.planning import MoveGroupInterface
from rclpy.node import Node

class MyNode(Node):
    def __init__(self):
        super().__init__('my_node')
        self.move_group = MoveGroupInterface(
            node=self,
            group_name='left_arm'
        )
```

初始化

```
def set_pose_target(
    self,
    pose: geometry_msgs.msg.Pose,
    end_effector_link: str = ""
) -> bool:
```

set_pose_target() **功能:** 设置目标位姿

参数: - `pose` - 目标位姿 (Pose 消息) - `end_effector_link` - 末端执行器链接名 (可选)

返回值: - `True` - 设置成功 - `False` - 目标不可达

示例:

```python
from geometry_msgs.msg import Pose

pose = Pose()
pose.position.x = 0.3
pose.position.y = 0.2
pose.position.z = 0.5
pose.orientation.w = 1.0

if move_group.set_pose_target(pose):
    print("Target set")
```

```python
def set_joint_value_target(
    self,
    joint_values: List[float]
) -> bool:
```

set_joint_value_target() **功能:** 设置关节目标

参数: - `joint_values` - 关节角度列表 (rad)

返回值: - `True` - 设置成功 - `False` - 目标不可达

示例:

```python
# 7 关节机械臂
joint_values = [0.0, 0.5, 0.0, -1.0, 0.0, 1.5, 0.0]
move_group.set_joint_value_target(joint_values)
```

```python
def plan(self) -> Tuple[bool, trajectory_msgs.msg.JointTrajectory, float]:
```

plan() **功能:** 规划轨迹

返回值: 元组 - `success` (bool) - 规划是否成功 - `trajectory` (JointTrajectory) - 规划的轨迹 - `planning_time` (float) - 规划耗时 (秒)

示例:

```
success, trajectory, planning_time = move_group.plan()
if success:
    print(f"Planning succeeded in {planning_time:.3f}s")
    print(f"Trajectory has {len(trajectory.points)} points")
```

```
def execute(
    self,
    trajectory: trajectory_msgs.msg.JointTrajectory
) -> bool:
```

execute() **功能:** 执行轨迹

参数: - `trajectory` - 要执行的轨迹

返回值: - `True` - 执行成功 - `False` - 执行失败

示例:

```
success, trajectory, _ = move_group.plan()
if success:
    if move_group.execute(trajectory):
        print("Execution complete")
```

```
def go(self, wait: bool = True) -> bool:
```

go() **功能:** 规划并执行 (plan + execute)

参数: - `wait` - 是否等待执行完成

返回值: - `True` - 成功 - `False` - 失败

示例:

```
# 设置目标并执行
pose = Pose()
# ... (设置 pose)
move_group.set_pose_target(pose)
```

```
if move_group.go():
    print("Motion complete")
```

```
def get_current_pose(
    self,
    end_effector_link: str = ""
) -> geometry_msgs.msg.PoseStamped:
```

get_current_pose() **功能:** 获取当前位姿

返回值: PoseStamped 消息

示例:

```
current_pose = move_group.get_current_pose()
print(f"Position: ({current_pose.pose.position.x}, "
      f"{current_pose.pose.position.y}, "
      f"{current_pose.pose.position.z})")
```

```
def get_current_joint_values(self) -> List[float]:
```

get_current_joint_values() **功能:** 获取当前关节角度

返回值: 关节角度列表 (rad)

示例:

```
joint_values = move_group.get_current_joint_values()
for i, angle in enumerate(joint_values):
    print(f"Joint {i+1}: {angle:.3f} rad")
```

```
def set_max_velocity_scaling_factor(self, factor: float) -> None:
```

set_max_velocity_scaling_factor() **功能:** 设置最大速度缩放因子

参数: - `factor` (float) - 缩放因子 (0.0 ~ 1.0)

示例:

```
# 50% 最大速度
move_group.set_max_velocity_scaling_factor(0.5)
```

```
def set_max_acceleration_scaling_factor(self, factor: float) -> None:
```

set_max_acceleration_scaling_factor() **功能:** 设置最大加速度缩放因子

参数: - `factor` (float) - 缩放因子 (0.0 ~ 1.0)

```
def set_planner_id(self, planner_id: str) -> None:
```

set_planner_id() **功能:** 设置规划器

参数: - `planner_id` (str) - 规划器名称

可用规划器: - `"RRTConnect"` - 快速, 适合简单场景 - `"RRTstar"` - 最优路径 - `"PRM"` - 概率路图 - `"BiTRRT"` - 双向 RRT

示例:

```
move_group.set_planner_id("RRTConnect")
```

40.3.3 ROS2 Python API (rclpy)

```
import rclpy
from rclpy.node import Node

class MyNode(Node):
    def __init__(self):
        super().__init__('my_node_name')
        self.get_logger().info('Node started')

def main():
    rclpy.init()
    node = MyNode()
    rclpy.spin(node)
```

```
    node.destroy_node()
    rclpy.shutdown()
```

40.3.3.1 创建节点

```
from std_msgs.msg import String

class PublisherNode(Node):
    def __init__(self):
        super().__init__('publisher_node')

        # 创建发布器
        self.publisher = self.create_publisher(
            String,                    # 消息类型
            '/my_topic',               # 话题名
            10                         # 队列大小
        )

        # 定时发布
        self.timer = self.create_timer(
            1.0,                       # 周期 (秒)
            self.timer_callback
        )

    def timer_callback(self):
        msg = String()
        msg.data = 'Hello World'
        self.publisher.publish(msg)
        self.get_logger().info(f'Published: {msg.data}')
```

40.3.3.2 发布器

```
from sensor_msgs.msg import JointState

class SubscriberNode(Node):
    def __init__(self):
        super().__init__('subscriber_node')

        # 创建订阅器
        self.subscription = self.create_subscription(
            JointState,                # 消息类型
            '/joint_states',           # 话题名
            self.listener_callback,    # 回调函数
            10                         # 队列大小
        )

    def listener_callback(self, msg):
        self.get_logger().info(f'Received {len(msg.name)} joints')
        for name, pos in zip(msg.name, msg.position):
            self.get_logger().info(f'{name}: {pos:.3f} rad')
```

40.3.3.3 订阅器

```
from controller_manager_msgs.srv import SwitchController

class ServiceClientNode(Node):
    def __init__(self):
        super().__init__('service_client_node')

        # 创建服务客户端
        self.client = self.create_client(
            SwitchController,
            '/controller_manager/switch_controller'
        )

        # 等待服务可用
```

```
        while not self.client.wait_for_service(timeout_sec=1.0):
            self.get_logger().info('Waiting for service...')

    def call_service(self, start_controllers, stop_controllers):
        # 创建请求
        request = SwitchController.Request()
        request.start_controllers = start_controllers
        request.stop_controllers = stop_controllers
        request.strictness = SwitchController.Request.BEST_EFFORT

        # 异步调用
        future = self.client.call_async(request)
        rclpy.spin_until_future_complete(self, future)

        # 获取结果
        result = future.result()
        if result.ok:
            self.get_logger().info('Service call succeeded')
        else:
            self.get_logger().error('Service call failed')
```

40.3.3.4 服务客户端

```
from rclpy.action import ActionClient
from control_msgs.action import FollowJointTrajectory

class ActionClientNode(Node):
    def __init__(self):
        super().__init__('action_client_node')

        # 创建 Action 客户端
        self.action_client = ActionClient(
            self,
            FollowJointTrajectory,
```

```
            '/left_arm_controller/follow_joint_trajectory'
        )

    def send_goal(self, trajectory):
        # 等待 Action 服务器
        self.action_client.wait_for_server()

        # 创建 Goal
        goal_msg = FollowJointTrajectory.Goal()
        goal_msg.trajectory = trajectory

        # 发送 Goal (异步)
        send_goal_future = self.action_client.send_goal_async(
            goal_msg,
            feedback_callback=self.feedback_callback
        )

        # 注册回调
        send_goal_future.add_done_callback(self.goal_response_callback)

    def goal_response_callback(self, future):
        goal_handle = future.result()
        if not goal_handle.accepted:
            self.get_logger().error('Goal rejected')
            return

        self.get_logger().info('Goal accepted')

        # 等待结果
        result_future = goal_handle.get_result_async()
        result_future.add_done_callback(self.result_callback)

    def feedback_callback(self, feedback_msg):
        feedback = feedback_msg.feedback
```

```
        self.get_logger().info(f'Feedback: {feedback.actual.positions[0]:.3f}')

    def result_callback(self, future):
        result = future.result().result
        if result.error_code == 0:
            self.get_logger().info('Goal succeeded!')
        else:
            self.get_logger().error(f'Goal failed: {result.error_code}')
```

40.3.3.5 Action 客户端

40.4 命令行 API

40.4.1 ros2 control

```
# 列出控制器
ros2 control list_controllers

# 列出硬件接口
ros2 control list_hardware_interfaces

# 列出硬件组件
ros2 control list_hardware_components

# 加载控制器
ros2 control load_controller <controller_name>

# 配置控制器
ros2 control set_controller_state <controller_name> configure

# 启动控制器
ros2 control set_controller_state <controller_name> start

# 停止控制器
ros2 control set_controller_state <controller_name> stop
```

```
# 卸载控制器
ros2 control unload_controller <controller_name>

# 切换控制器
ros2 control switch_controllers \
    --start-controllers controller1 controller2 \
    --stop-controllers controller3
```

40.4.2 ros2 topic

```
# 列出话题
ros2 topic list

# 话题信息
ros2 topic info /joint_states

# 话题频率
ros2 topic hz /joint_states

# 话题延迟
ros2 topic delay /joint_states

# 话题带宽
ros2 topic bw /joint_states

# 查看话题内容
ros2 topic echo /joint_states

# 发布消息
ros2 topic pub /test std_msgs/msg/String "{data: 'test'}"
```

40.4.3 ros2 service

```
# 列出服务
ros2 service list

# 服务类型
ros2 service type /controller_manager/list_controllers

# 调用服务
ros2 service call \
    /controller_manager/list_controllers \
    controller_manager_msgs/srv/ListControllers
```

40.4.4 ros2 param

```
# 列出参数
ros2 param list

# 获取参数
ros2 param get /controller_manager update_rate

# 设置参数
ros2 param set /controller_manager update_rate 200

# 导出参数
ros2 param dump /controller_manager > params.yaml

# 加载参数
ros2 param load /controller_manager params.yaml
```

40.5 配置文件 API

40.5.1 URDF/Xacro 标签

```xml
<ros2_control name="OpenArmSystem" type="system">
  <hardware>
    <plugin>openarm_hardware/OpenArmSystem</plugin>
    <param name="can0_channel">can0</param>
    <param name="can1_channel">can1</param>
    <param name="update_rate">100</param>
  </hardware>

  <joint name="left_joint_1">
    <command_interface name="position">
      <param name="min">-3.14</param>
      <param name="max">3.14</param>
    </command_interface>
    <state_interface name="position"/>
    <state_interface name="velocity"/>
    <state_interface name="effort"/>
  </joint>
</ros2_control>
```

40.5.1.1 ros2_control 标签

40.5.2 控制器配置 YAML

```yaml
controller_manager:
  ros__parameters:
    update_rate: 100  # Hz

    # 控制器列表
    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster

    left_arm_controller:
      type: joint_trajectory_controller/JointTrajectoryController
```

```yaml
# 控制器参数
left_arm_controller:
  ros__parameters:
    joints:
      - left_joint_1
      - left_joint_2
      # ...

    command_interfaces:
      - position

    state_interfaces:
      - position
      - velocity

    state_publish_rate: 50.0
    action_monitor_rate: 20.0

    constraints:
      stopped_velocity_tolerance: 0.05
      goal_time: 0.5
      left_joint_1:
        trajectory: 0.1
        goal: 0.05
```

40.6 总结

40.6.1 API 参考要点

✓ C++ API - 硬件接口、控制器接口

✓ Python API - Motor类、MoveIt接口、ROS2接口

✓ 命令行API - ros2 control/topic/service/param

✓ 配置API - URDF标签、YAML参数

40.6.2 常用 API 速查

C++ 硬件控制:

```
#include <openarm_hardware/robstride_can_interface.hpp>
auto can = std::make_shared<RobstrideCAN>("can0");
can->send_motor_command(1, 0.5, 0.0, 50.0, 1.0, 0.0);
```

Python 电机控制:

```
from robstride import Motor
motor.set_position(position=0.5, kp=30.0, kd=1.0)
```

Python 运动规划:

```
from moveit_py.planning import MoveGroupInterface
move_group.set_pose_target(pose)
move_group.go()
```

40.6.3 下一步

- **04_ 贡献指南** - 如何贡献代码
- **02_ 扩展开发** - 实战开发示例
- **01_ 包结构说明** - 了解系统架构

本文档版本: v1.0 最后更新: 2025 年 10 月 20 日 成都长数机器人有限公司

41 贡献指南

41.1 OpenArmX 项目贡献指南

欢迎为 OpenArmX 项目做出贡献! 本文档将指导您如何参与开发。

41.2 贡献方式

41.2.1 可以贡献的内容

☐ 代码贡献 - 新功能、Bug修复、性能优化
☐ 文档贡献 - 教程、API文档、示例代码
☐ 测试贡献 - 单元测试、集成测试、性能测试
☐ 问题反馈 - Bug报告、功能请求

☐ 代码审查 - Review Pull Request

☐ 社区支持 - 回答问题、分享经验

41.2.2 贡献流程

1. Fork项目仓库
   ↓
2. 创建功能分支
   ↓
3. 开发并测试
   ↓
4. 提交Pull Request
   ↓
5. 代码审查
   ↓
6. 合并到主分支

41.3 开始之前

41.3.1 环境搭建

```
# 克隆仓库
git clone https://github.com/your-org/openarmx_robotstride_ws-cc.git
cd openarmx_robotstride_ws-cc

# 安装依赖
rosdep install --from-paths src --ignore-src -r -y

# 编译
colcon build --symlink-install

# Source 环境
source install/setup.bash

# 运行测试
colcon test
```

41.3.2 开发工具安装

```
# 代码格式化
sudo apt install clang-format
pip3 install black flake8 isort

# 静态分析
sudo apt install clang-tidy cppcheck
pip3 install pylint mypy

# Git 工具
sudo apt install git-lfs
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"

# 预提交钩子
pip3 install pre-commit
cd ~/openarmx_robotstride_ws-cc
pre-commit install
```

41.4 代码规范

41.4.1 C++ 代码规范

```
// 类名: PascalCase
class OpenArmSystem { };
class RobstrideCAN { };

// 函数名: snake_case
void read_motor_feedback();
bool enable_motor(uint8_t motor_id);

// 变量名: snake_case
double current_position;
std::vector<double> joint_positions_;  // 成员变量加下划线
```

```
// 常量: UPPER_SNAKE_CASE 或 kPascalCase
constexpr int MAX_MOTORS = 7;
const double kDefaultKp = 30.0;

// 命名空间: snake_case
namespace openarm_hardware { }
```

41.4.1.1 命名约定

41.4.1.2 代码风格 使用 clang-format 配置:

.clang-format 文件:

```
BasedOnStyle: Google
IndentWidth: 2
ColumnLimit: 100
PointerAlignment: Left
AccessModifierOffset: -2
IncludeBlocks: Regroup
```

示例代码:

```
#include "openarm_hardware/openarm_system.hpp"

#include <vector>
#include <memory>

namespace openarm_hardware
{

OpenArmSystem::OpenArmSystem()
: hw_positions_(7, 0.0),
  hw_velocities_(7, 0.0),
  hw_efforts_(7, 0.0)
{
  // 初始化
}
```

```
hardware_interface::return_type OpenArmSystem::read(
  const rclcpp::Time & time,
  const rclcpp::Duration & period)
{
  // 实现
  for (size_t i = 0; i < hw_positions_.size(); ++i) {
    double position, velocity, torque;
    if (can0_->receive_motor_feedback(i + 1, position, velocity, torque)) {
      hw_positions_[i] = position;
      hw_velocities_[i] = velocity;
      hw_efforts_[i] = torque;
    }
  }

  return hardware_interface::return_type::OK;
}

}  // namespace openarm_hardware
```

41.4.1.3 注释规范 使用 Doxygen 风格:

```
/**
 * @brief 读取硬件状态
 *
 * 从 CAN 总线读取所有电机的位置、速度和力矩反馈,
 * 并更新 StateInterface 供控制器访问。
 *
 * @param time 当前时间戳
 * @param period 更新周期
 * @return return_type::OK 如果读取成功
 * @return return_type::ERROR 如果读取失败
 *
 * @note 此函数在控制器更新循环中被调用, 频率由 update_rate 决定
 * @warning 如果 CAN 通信超时, 将返回 ERROR 并记录日志
```

```
 */
hardware_interface::return_type read(
  const rclcpp::Time & time,
  const rclcpp::Duration & period) override;
```

行内注释:

```
// 检查 CAN 接口是否就绪
if (!can0_->is_connected()) {
  RCLCPP_ERROR(logger_, "CAN0 not connected");
  return CallbackReturn::ERROR;
}

// 使能所有电机
for (uint8_t motor_id = 1; motor_id <= 7; ++motor_id) {
  can0_->enable_motor(motor_id);  // 左臂电机
}
```

```
// ✓ 使用 ROS 日志
RCLCPP_ERROR(get_logger(), "Failed to connect to CAN0");
RCLCPP_WARN(get_logger(), "Motor temperature high: %.1f°C", temp);
RCLCPP_INFO(get_logger(), "System initialized successfully");
RCLCPP_DEBUG(get_logger(), "Sending command to motor %d", motor_id);

// ✓ 使用返回值表示错误
hardware_interface::return_type write(...) {
  if (!send_command()) {
    return hardware_interface::return_type::ERROR;
  }
  return hardware_interface::return_type::OK;
}

// ✓ 使用异常处理关键错误
if (!can_->connect()) {
```

```
  throw std::runtime_error("Failed to connect to CAN bus");
}

//  避免使用 std::cout/std::cerr
// std::cout << "Error" << std::endl;  // 不要这样
```

41.4.1.4 错误处理

41.4.2 Python 代码规范

```
# 类名: PascalCase
class MotorController:
    pass

class OpenArmPlanner:
    pass


# 函数名: snake_case
def enable_motor(motor_id: int) -> bool:
    pass

def get_current_position() -> float:
    pass


# 变量名: snake_case
current_position = 0.0
motor_ids = [1, 2, 3, 4, 5, 6, 7]


# 常量: UPPER_SNAKE_CASE
MAX_VELOCITY = 30.0
DEFAULT_KP = 30.0
```

```python
# 私有成员: _ 前缀
class Motor:
    def __init__(self):
        self._position = 0.0  # 私有属性

    def _send_command(self):  # 私有方法
        pass
```

41.4.2.1 PEP 8 规范

```python
from typing import List, Dict, Optional, Tuple
import numpy as np

def control_motors(
    motor_ids: List[int],
    positions: List[float],
    kp: float = 30.0,
    kd: float = 1.0
) -> bool:
    """
    控制多个电机到目标位置

    Args:
        motor_ids: 电机 ID 列表
        positions: 目标位置列表 (rad)
        kp: 位置增益 (默认 30.0)
        kd: 速度增益 (默认 1.0)

    Returns:
        True 如果成功,False 如果失败

    Raises:
```

```
        ValueError: 如果 motor_ids 和 positions 长度不匹配
    """
    if len(motor_ids) != len(positions):
        raise ValueError("motor_ids and positions must have same length")

    # 实现...
    return True


def get_motor_feedback(motor_id: int) -> Optional[Dict[str, float]]:
    """
    获取电机反馈

    Returns:
        包含 position, velocity, torque 的字典,
        如果失败返回 None
    """
    try:
        # 获取反馈...
        return {
            'position': 0.5,
            'velocity': 0.1,
            'torque': 2.0
        }
    except Exception:
        return None
```

41.4.2.2 类型提示

41.4.2.3 Docstring 规范 使用 Google 风格:

```
class Motor:
    """Robstride 电机控制类

    此类封装了 Robstride 电机的 CAN 通信协议, 提供了简洁的
```

```
    Python 接口用于电机控制和状态读取。

    Attributes:
        bus: python-can 的 Bus 对象
        motor_id: 电机 ID (1-7)
        reduction_ratio: 减速比

    Example:
        >>> import can
        >>> bus = can.interface.Bus(channel='can0', bustype='socketcan')
        >>> motor = Motor(bus, motor_id=1)
        >>> motor.enable()
        >>> motor.set_position(position=0.5, kp=30.0)
    """

    def __init__(self, bus: can.interface.Bus, motor_id: int):
        """ 初始化电机对象

        Args:
            bus: CAN 总线对象
            motor_id: 电机 ID (1-7)

        Raises:
            ValueError: 如果 motor_id 超出范围
        """
        if not 1 <= motor_id <= 7:
            raise ValueError("motor_id must be 1-7")

        self.bus = bus
        self.motor_id = motor_id
```

41.4.2.4 代码格式化 使用 black 和 isort:

pyproject.toml 配置:

```
[tool.black]
line-length = 100
target-version = ['py310']

[tool.isort]
profile = "black"
line_length = 100
```

格式化代码:

```
# 格式化单个文件
black my_script.py

# 格式化整个目录
black src/

# 排序 imports
isort src/

# 检查格式 (不修改)
black --check src/
```

41.4.3 YAML 配置规范

```
# 使用 2 空格缩进
controller_manager:
  ros__parameters:
    update_rate: 100  # Hz

    # 清晰的注释
    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster

# 分组相关配置
left_arm_controller:
  ros__parameters:
```

```yaml
  # 关节列表
  joints:
    - left_joint_1
    - left_joint_2
    - left_joint_3

  # 接口配置
  command_interfaces:
    - position

  state_interfaces:
    - position
    - velocity

  # 性能参数
  state_publish_rate: 50.0
  action_monitor_rate: 20.0

  # 约束
  constraints:
    stopped_velocity_tolerance: 0.05
    goal_time: 0.5
```

41.5 Git 工作流

41.5.1 分支管理

```
# 功能分支
feature/add-visual-servo
feature/impedance-controller

# Bug 修复分支
fix/can-timeout-issue
fix/controller-crash
```

```
# 文档分支
docs/update-api-reference
docs/add-tutorial

# 性能优化分支
perf/reduce-latency
perf/optimize-read-cycle
```

41.5.1.1 分支命名

```
# 从 main 创建功能分支
git checkout main
git pull origin main
git checkout -b feature/add-visual-servo

# 开发...

# 定期同步 main
git fetch origin
git rebase origin/main
```

41.5.1.2 创建分支

41.5.2 提交规范

41.5.2.1 提交消息格式

```
<type>(<scope>): <subject>

<body>

<footer>
```

Type 类型: - `feat`: 新功能 - `fix`: Bug 修复 - `docs`: 文档修改 - `style`: 代码格式 (不影响功能) - `refactor`: 重构 - `perf`: 性能优化 - `test`: 测试 - `chore`: 构建/工具

示例:

```
feat(hardware): add force-torque sensor support

- Implement FTSensorInterface class
- Add URDF tags for FT sensor
- Create launch file for sensor node

Closes #123
```

```
fix(controller): resolve trajectory execution timeout

The controller was timing out on long trajectories due to
incorrect goal_time calculation. This fixes the issue by
properly computing goal_time based on trajectory duration.

Fixes #456
```

```
docs(tutorial): add visual servoing tutorial

- Create step-by-step tutorial
- Add Python example code
- Include troubleshooting section
```

```
# ✓ 小而聚焦的提交
git add src/openarm_hardware/src/ft_sensor.cpp
git add include/openarm_hardware/ft_sensor.hpp
git commit -m "feat(hardware): add FT sensor interface"

# ✓ 相关文件一起提交
git add src/openarm_sensors/
git commit -m "feat(sensors): add RealSense camera support"

#  避免大而杂的提交
# git add .  # 不要这样
```

```
# git commit -m "various changes"  # 消息太模糊
```

41.5.2.2 提交最佳实践

41.5.3 Pull Request 流程

```
#  确保分支最新
git fetch origin
git rebase origin/main

#  推送到远程
git push origin feature/add-visual-servo

#  在 GitHub 上创建 Pull Request
```

41.5.3.1 创建 PR PR 模板:

```
## 描述
添加视觉伺服功能，支持基于图像的位置控制

## 改动类型
- [x] 新功能
- [ ] Bug 修复
- [ ] 文档更新
- [ ] 性能优化

## 改动内容
- 创建`openarm_visual_servo`包
- 实现 VisualServoNode 节点
- 添加 RealSense 相机集成
- 更新文档

## 测试
- [x] 通过编译 (`colcon build`)
- [x] 通过单元测试 (`colcon test`)
```

```
- [x] 手动测试（真实硬件）
- [x] 代码审查通过

## 截图/视频
[可选：添加演示截图或视频]

## 相关 Issue
Closes #123

## 检查清单
- [x] 代码符合项目规范
- [x] 添加了必要的注释
- [x] 更新了相关文档
- [x] 添加了测试
- [x] 所有测试通过
```

41.5.3.2 代码审查 审查者检查:

☐ 代码功能正确
☐ 符合代码规范
☐ 有充分的注释
☐ 包含测试
☐ 文档已更新
☐ 无明显性能问题
☐ 无安全隐患

审查反馈示例:

```
总体看起来不错！有几点建议：

1. `visual_servo_node.py:45` - 建议添加异常处理：
   ```python
   try:
       cx = int(moments['m10'] / moments['m00'])
   except ZeroDivisionError:
       self.get_logger().warn("No target detected")
```
```

```
        return
```

2. `README.md` - 建议添加依赖安装说明

3. 单元测试覆盖率较低, 建议增加边界情况测试

除此之外 LGTM!

合并PR

```bash
# 审查通过后,维护者合并PR

# 使用Squash Merge (推荐)
# - 将多个commit合并为一个
# - 保持main分支历史清晰

# 或使用Rebase Merge
# - 保留所有commit
# - 线性历史

# 避免使用Merge Commit
# - 会产生额外的merge commit
```

41.6 测试要求

41.6.1 单元测试

41.6.1.1 C++ 单元测试 使用 gtest:

```cpp
#include <gtest/gtest.h>
#include "openarm_hardware/robstride_can_interface.hpp"

class CANInterfaceTest : public ::testing::Test
{
protected:
  void SetUp() override {
    can_ = std::make_shared<openarm_hardware::RobstrideCAN>("can0");
  }
```

```
  void TearDown() override {
    can_->disconnect();
  }

  std::shared_ptr<openarm_hardware::RobstrideCAN> can_;
};

TEST_F(CANInterfaceTest, ConnectSuccess) {
  EXPECT_TRUE(can_->connect());
  EXPECT_TRUE(can_->is_connected());
}

TEST_F(CANInterfaceTest, EnableMotor) {
  can_->connect();
  EXPECT_TRUE(can_->enable_motor(1));
}

TEST_F(CANInterfaceTest, SendCommandInRange) {
  can_->connect();
  can_->enable_motor(1);

  // 正常范围
  EXPECT_TRUE(can_->send_motor_command(1, 0.5, 0.0, 30.0, 1.0, 0.0));

  // 超出范围应该失败
  EXPECT_FALSE(can_->send_motor_command(1, 100.0, 0.0, 30.0, 1.0, 0.0));
}

int main(int argc, char **argv) {
  testing::InitGoogleTest(&argc, argv);
  return RUN_ALL_TESTS();
}
```

CMakeLists.txt:

```
if(BUILD_TESTING)
  find_package(ament_cmake_gtest REQUIRED)

  ament_add_gtest(test_can_interface
    test/test_can_interface.cpp
  )

  target_link_libraries(test_can_interface
    ${PROJECT_NAME}
  )
endif()
```

41.6.1.2 Python 单元测试 使用 pytest:

```
# test/test_motor.py
import pytest
from robstride import Motor
import can

@pytest.fixture
def mock_bus():
    """ 创建模拟 CAN 总线"""
    return can.interface.Bus(
        channel='vcan0',
        bustype='virtual'
    )

@pytest.fixture
def motor(mock_bus):
    """ 创建电机实例"""
    return Motor(mock_bus, motor_id=1)

def test_motor_init(motor):
    """ 测试电机初始化"""
    assert motor.motor_id == 1
```

```python
    assert motor.reduction_ratio == 1.0

def test_motor_enable(motor):
    """ 测试电机使能"""
    assert motor.enable() is True

def test_invalid_motor_id():
    """ 测试无效电机 ID"""
    bus = can.interface.Bus(channel='vcan0', bustype='virtual')
    with pytest.raises(ValueError):
        Motor(bus, motor_id=10)  # 超出范围

def test_set_position_range(motor):
    """ 测试位置范围检查"""
    motor.enable()

    # 正常范围
    assert motor.set_position(0.5) is True

    # 超出范围
    assert motor.set_position(100.0) is False

def test_get_feedback_timeout(motor):
    """ 测试反馈超时"""
    motor.enable()

    # 短超时应该失败
    feedback = motor.get_feedback(timeout=0.001)
    assert feedback is None
```

运行测试:

```bash
# C++ 测试
colcon test --packages-select openarm_hardware
colcon test-result --verbose
```

```
# Python 测试
cd src/motor_tests_openarmx_com
pytest test/

# 查看覆盖率
pytest --cov=robstride test/
```

41.6.2 集成测试

launch_testing 示例:

```
# test/test_system_integration.py
import unittest
import pytest
import launch
import launch_ros
import launch_testing.actions
from launch_testing.legacy import LaunchTestService

@pytest.mark.launch_test
def generate_test_description():
    return launch.LaunchDescription([
        launch_ros.actions.Node(
            package='controller_manager',
            executable='ros2_control_node',
            parameters=[...]
        ),
        launch_testing.actions.ReadyToTest()
    ])

class TestSystemIntegration(unittest.TestCase):
    def test_controllers_loaded(self):
        """ 测试控制器是否加载"""
        # 检查控制器...
        pass
```

```python
    def test_joint_states_published(self):
        """ 测试关节状态是否发布"""
        # 订阅/joint_states...
        pass
```

41.7 文档要求

41.7.1 代码文档

```cpp
// C++: Doxygen 注释
/**
 * @file openarm_system.hpp
 * @brief OpenArmX 硬件接口实现
 * @author Your Name
 * @date 2025-10-20
 */

/**
 * @class OpenArmSystem
 * @brief ros2_control 硬件接口
 *
 * 此类实现了 SystemInterface，负责 OpenArmX 双臂机器人的
 * CAN 通信和电机控制。
 */
```

```python
# Python: Docstring
"""OpenArmX 视觉伺服模块

此模块实现了基于视觉的伺服控制，支持目标跟踪和
位置控制。

Example:
    >>> from openarm_visual_servo import VisualServoNode
    >>> node = VisualServoNode()
    >>> node.start_tracking()
```

```
Todo:
    * 添加多目标跟踪
    * 优化控制律

.. _ROS2 文档:
   https://docs.ros.org/en/humble/
"""
```

41.7.2 README 文档

包 README 模板:

```
# openarm_visual_servo

基于视觉的伺服控制包

## 功能

- 实时目标检测
- 视觉伺服控制
- RealSense 相机支持

## 安装

### 依赖

```bash
sudo apt install ros-humble-cv-bridge
pip3 install opencv-python
```
```

41.7.3 编译

```
colcon build --packages-select openarm_visual_servo
```

41.8 使用

```
ros2 launch openarm_visual_servo demo.launch.py
```

41.9 配置

参见 `config/visual_servo.yaml`

41.10 API

详见API 文档

41.11 许可

Apache 2.0

41.12 作者

Your Name your.email@example.com

```
## 发布流程

### 版本号规范

**语义化版本 (Semantic Versioning):**
```

MAJOR.MINOR.PATCH

例如: 1.2.3

```
- **MAJOR**: 不兼容的API变更
- **MINOR**: 向后兼容的新功能
- **PATCH**: 向后兼容的Bug修复

### 发布检查清单
```

☐ 所有测试通过 ☐ 文档已更新 ☐ CHANGELOG 已更新 ☐ 版本号已更新 (package.xml) ☐ 创建 Git tag ☐ 构建二进制包 ☐ 更新发布说明

```
### 创建发布
```

```bash
#  更新版本号
# 编辑package.xml中的<version>

#  更新CHANGELOG
# 编辑CHANGELOG.md

#  提交
git add package.xml CHANGELOG.md
git commit -m "chore: bump version to 1.2.0"

#  创建tag
git tag -a v1.2.0 -m "Release version 1.2.0"

#  推送
git push origin main
git push origin v1.2.0
```

41.13 社区规范

41.13.1 行为准则

✓ 尊重他人
✓ 建设性反馈
✓ 包容多样性
✓ 专注技术讨论
人身攻击
骚扰行为
歧视性言论

41.13.2 沟通渠道

```
- GitHub Issues - Bug报告、功能请求
- GitHub Discussions - 一般讨论、问答
- Pull Requests - 代码审查
```

- 邮件列表 - 开发者邮件列表

41.13.3 获取帮助

1. 查看文档
2. 搜索已有Issue
3. 提问前准备:
   - 清晰描述问题
   - 提供最小可复现示例
   - 包含环境信息
4. 创建Issue

41.14 常见问题

41.14.1 Q1: 如何选择分支名称?

```
# 功能: feature/< 简短描述 >
feature/add-gripper-support

# 修复: fix/<issue 号 >-< 简短描述 >
fix/123-can-timeout

# 文档: docs/< 简短描述 >
docs/update-readme
```

41.14.2 Q2: 提交前要做什么?

```
#  格式化代码
black src/
clang-format -i src/**/*.cpp

#  运行测试
colcon test

#  检查编译
colcon build
```

```
# Review 自己的改动
git diff
```

41.14.3 Q3: PR 被拒绝了怎么办?

1. 仔细阅读审查意见
2. 进行相应修改
3. 推送更新到同一分支
4. PR 会自动更新

41.14.4 Q4: 如何保持分支同步?

```
# 定期 rebase
git fetch origin
git rebase origin/main

# 解决冲突
git rebase --continue
```

41.15 总结

41.15.1 贡献要点

✓ 遵循代码规范
✓ 编写清晰的提交消息
✓ 添加测试
✓ 更新文档
✓ 参与代码审查
✓ 保持礼貌和专业

41.15.2 资源链接

- 项目主页
- 文档
- Issues
- Discussions

41.15.3 联系方式

- 邮箱: dev@openarmx.com
- GitHub: @openarmx

感谢您的贡献!

每一个贡献, 无论大小, 都让 OpenArmX 变得更好。

本文档版本: v1.0 最后更新: 2025 年 10 月 20 日 成都长数机器人有限公司

42 参数速查表

42.1 OpenArmX 常用参数快速参考

42.2 电机参数

42.2.1 电机型号参数

型号	功率	额定扭矩	峰值扭矩	最大速度	用途
RS04	700W	40 Nm	120 Nm	15 rad/s	肩部 (1-2)
RS03	380W	20 Nm	60 Nm	20 rad/s	大臂 (3-4)
RS00	140W	5 Nm	14 Nm	33 rad/s	小臂-夹爪 (5-8)

42.2.2 电机 ID 映射

can0 (左臂):

```
ID 1-2: 肩部 (RS04)
ID 3-4: 大臂 (RS03)
ID 5-8: 小臂-夹爪 (RS00)
```

can1 (右臂):

```
ID 1-2: 肩部 (RS04)
ID 3-4: 大臂 (RS03)
ID 5-8: 小臂-夹爪 (RS00)
```

42.3 控制参数

42.3.1 推荐 KP/KD 参数

应用场景	关节 1-4 KP	关节 1-4 KD	关节 5-7 KP	关节 5-7 KD	夹爪 KP	夹爪 KD
示教模式	1.0	0.1	0.1	0.05	0.1	0.05
常规运动 (默认)	50.0	2.5	10.0	0.5	5.0	0.5
高刚性模式	500.0	10.0	50.0	1.0	50.0	1.0

说明: - 关节 1-4: RS04/RS03 电机（肩部和大臂）- 关节 5-7: RS00 电机（小臂）- 夹爪: RS00 电机（关节 8）- 参数来源: `openarm_hardware/include/openarm_hardware/v10_simple_hardware.hpp`

42.3.2 速度和加速度限制

```yaml
# 推荐值
joint_limits:
  velocity: 2.0 rad/s        # 初次: 0.5
  acceleration: 1.0 rad/s²   # 初次: 0.3
```

42.3.3 MIT 模式参数范围

```
位置: ±12.57 rad (所有型号)
速度: RS04: ±15 rad/s, RS03: ±20 rad/s, RS00: ±33 rad/s
KP:   RS04/RS03: 0-5000, RS00: 0-500
KD:   RS04/RS03: 0-100, RS00: 0-5
```

42.4 CAN 总线参数

42.4.1 CAN 接口配置

参数	can0	can1
波特率	1 Mbps	1 Mbps
设备数	8	8
终端电阻	60Ω	60Ω

42.4.2 CAN 命令

```bash
# 启动
sudo ip link set can0 type can bitrate 1000000
sudo ip link set can0 up

# 关闭
sudo ip link set can0 down

# 状态
ip -s link show can0
```

42.5 关节限位

42.5.1 位置限制 (rad)

关节	最小	最大	说明
joint_1	-3.14	3.14	±180°
joint_2	-1.57	1.57	±90°
joint_3	-3.14	3.14	±180°
joint_4	-2.79	0.0	-160°~0°
joint_5	-1.57	1.57	±90°
joint_6	-1.57	1.57	±90°
joint_7	-3.14	3.14	±180°
gripper	0.0	1.2	0°~69°

42.6 电源参数

42.6.1 电源规格

输入: AC 100-240V

输出: DC 48V

电流: ≥20A (推荐25A+)

功率: ≥1000W (推荐1500W)

42.6.2 功耗估算

单电机峰值:

- RS04: ~100W
- RS03: ~60W
- RS00: ~30W

系统总峰值:

- RS04 × 4 = 400W
- RS03 × 4 = 240W
- RS00 × 8 = 240W
- 总计: ~880W

典型运行: 400-600W

42.7 温度限制

温度范围	状态	处理
<50°C	正常	无
50-60°C	[黄] 温和	监控
60-70°C	[黄] 偏高	降负载
70-80°C	[橙] 警告	停止高强度
>80°C	[红] 危险	立即停止

42.8 ROS2 参数

42.8.1 控制频率

ros2_control update_rate: 100 Hz

MoveIt planning time: 5 s (default)

轨迹采样频率: 10 Hz (录制)

42.8.2 控制器名称

joint_state_broadcaster

left_arm_controller

right_arm_controller

left_gripper_controller

right_gripper_controller

42.9 文件路径

42.9.1 重要配置文件

工作空间:

~/openarmx_robotstride_ws-cc/

URDF:

src/openarm_ros2/openarm_description/urdf/

控制器配置:

src/openarm_ros2/openarm_controllers/config/

MoveIt配置:

src/openarm_moveit_config/config/

电机测试:

src/motor_tests_openarmx_com/

GUI管理:

src/motor_manage_openarmx/

42.9.2 常用脚本

```
# CAN 管理
en_all_can.py
dis_all_can.py

# 电机管理
en_all_motors.py
dis_all_motors.py

# 状态检查
check_motor_status.py

# 零位设置
set_motor_zeros_setMIT.py

# 回零
control_motor_gohome.py

# 单电机测试
test_motor_one.py
```

42.10 网络端口

42.10.1 ROS2 DDS

默认使用FastDDS

端口范围：7400-7999 (UDP)

组播地址：239.255.0.1

42.10.2 RViz

默认端口：无固定端口(X11显示)

42.11 错误码

42.11.1 电机错误码

错误码	含义	处理
0x00	正常	无
0x01	过压	检查电源
0x02	欠压	检查电源
0x04	过流	降低负载
0x08	过温	冷却
0x10	编码器故障	检查/更换
0x20	通信超时	检查 CAN

42.12 安全参数

42.12.1 安全距离

机械臂工作半径：1.6m (双臂)

推荐安全距离：≥1.0m

围栏高度：≥1.2m

42.12.2 环境要求

温度：5-40°C (最佳15-30°C)

湿度：20-80% RH (无冷凝)

照明：≥300 lux

42.13 版本信息

42.13.1 软件版本

Ubuntu: 22.04 LTS

ROS2: Humble Hawksbill

Python: 3.10+

MoveIt: 2.x

42.13.2 硬件版本

产品: OpenArmX V1.0

电机: Robstride RS系列

控制器: 48V驱动器

42.14 联系信息

公司: 成都长数机器人有限公司

电话/微信: +86-17746530375

邮箱: openarmrobot@gmail.com

网站: https://openarmx.com/

地址: 天津经济技术开发区西区新业八街11号华诚机械厂

本文档版本: v1.0 最后更新: 2025 年 10 月 19 日 成都长数机器人有限公司

43 常用命令速查

43.1 OpenArmX 常用命令快速参考

一页纸速查表, 快速查找常用命令。

43.2 系统启动

43.2.1 基础启动

```
# CAN 总线管理
python3 en_all_can.py              # 启动 CAN
python3 dis_all_can.py             # 关闭 CAN
```

```
# 电机管理
python3 en_all_motors.py          # 使能所有电机
python3 dis_all_motors.py         # 失能所有电机
python3 check_motor_status.py     # 检查电机状态

# 回零
python3 control_motor_gohome.py    # 回零操作

# 启动系统
ros2 launch openarm_bringup openarm.launch.py

# 启动 MoveIt
ros2 launch openarm_moveit_config demo.launch.py
```

43.2.2 分步启动

```
#  硬件接口
ros2 launch openarm_bringup hardware_only.launch.py

#  控制器
ros2 control load_controller joint_state_broadcaster
ros2 control set_controller_state joint_state_broadcaster start

ros2 control load_controller left_arm_controller
ros2 control set_controller_state left_arm_controller start

#  可视化
ros2 launch openarm_bringup rviz.launch.py
```

43.3 CAN 总线命令

43.3.1 CAN 接口管理

```
# 查看状态
ip link show can0
```

```
ip link show can1

# 启动 CAN (1 Mbps)
sudo ip link set can0 type can bitrate 1000000
sudo ip link set can0 up

# 关闭 CAN
sudo ip link set can0 down

# 查看统计
ip -s link show can0
ip -s -d link show can0

# 重置错误计数
sudo ip link set can0 down && sudo ip link set can0 up
```

43.3.2 CAN 通信测试

```
# 监听消息
candump can0
candump can0 -t A              # 带时间戳
candump can0 -e                # 只看错误帧
candump can0,001:7FF           # 过滤 ID

# 发送消息
cansend can0 001#0102030405060708

# 生成测试流量
cangen can0 -g 10 -I 1

# 保存日志
candump -l can0

# 查看模块
```

```
lsmod | grep can
```

43.4 ROS2 命令

43.4.1 节点管理

```
# 列出节点
ros2 node list

# 节点信息
ros2 node info /controller_manager

# 节点日志
ros2 topic echo /rosout
ros2 topic echo /rosout | grep ERROR
```

43.4.2 话题操作

```
# 列出话题
ros2 topic list
ros2 topic list -t              # 带类型

# 话题信息
ros2 topic info /joint_states

# 查看内容
ros2 topic echo /joint_states
ros2 topic echo /joint_states --once

# 性能监控
ros2 topic hz /joint_states     # 频率
ros2 topic bw /joint_states     # 带宽
ros2 topic delay /joint_states  # 延迟

# 发布消息
```

```
ros2 topic pub /test std_msgs/msg/String "data: 'test'"
ros2 topic pub -1 /test std_msgs/msg/String "data: 'once'"  # 发送一次
```

43.4.3 服务调用

```
# 列出服务
ros2 service list
ros2 service list -t            # 带类型

# 服务类型
ros2 service type /controller_manager/list_controllers

# 调用服务
ros2 service call /controller_manager/list_controllers \
    controller_manager_msgs/srv/ListControllers

# 查看服务定义
ros2 interface show controller_manager_msgs/srv/ListControllers
```

43.4.4 参数管理

```
# 列出参数
ros2 param list
ros2 param list /controller_manager

# 获取参数
ros2 param get /controller_manager update_rate

# 设置参数
ros2 param set /controller_manager update_rate 200

# 导出参数
ros2 param dump /controller_manager > params.yaml
```

```
# 加载参数
ros2 param load /controller_manager params.yaml
```

43.4.5 Action 操作

```
# 列出 Action
ros2 action list
ros2 action list -t             # 带类型

# Action 信息
ros2 action info /left_arm_controller/follow_joint_trajectory

# 发送 Action Goal
ros2 action send_goal \
    /left_arm_controller/follow_joint_trajectory \
    control_msgs/action/FollowJointTrajectory \
    "{trajectory: {joint_names: ['left_joint_1']}}"
```

43.5 ros2_control 命令

43.5.1 控制器管理

```
# 列出控制器
ros2 control list_controllers

# 列出硬件接口
ros2 control list_hardware_interfaces

# 列出硬件组件
ros2 control list_hardware_components

# 加载控制器
ros2 control load_controller <controller_name>

# 配置控制器
```

```
ros2 control set_controller_state <controller_name> configure

# 启动控制器
ros2 control set_controller_state <controller_name> start

# 停止控制器
ros2 control set_controller_state <controller_name> stop

# 卸载控制器
ros2 control unload_controller <controller_name>
```

43.5.2 控制器切换

```
# 切换控制器
ros2 control switch_controllers \
    --start-controllers left_arm_controller \
    --stop-controllers position_controller

# 强制切换
ros2 control switch_controllers \
    --start-controllers left_arm_controller \
    --stop-controllers position_controller \
    --strictness 2
```

43.6 编译与构建

43.6.1 colcon 命令

```
# 完整编译
cd ~/openarmx_robotstride_ws-cc
colcon build

# 编译特定包
colcon build --packages-select openarm_hardware
```

```
# 编译包及其依赖
colcon build --packages-up-to openarm_hardware

# 符号链接安装（开发模式）
colcon build --symlink-install

# 并行编译
colcon build --parallel-workers 4

# 详细输出
colcon build --event-handlers console_direct+

# 清理
rm -rf build/ install/ log/

# 测试
colcon test
colcon test --packages-select openarm_hardware
colcon test-result --verbose
```

43.6.2 依赖管理

```
# 安装依赖
rosdep install --from-paths src --ignore-src -r -y

# 更新 rosdep
rosdep update

# 检查依赖
rosdep check --from-paths src --ignore-src
```

43.7 系统诊断

43.7.1 系统状态

```
# 系统信息
lsb_release -a                  # Ubuntu 版本
uname -r                        # 内核版本
cat /sys/kernel/realtime        # 实时内核

# 系统资源
uptime                          # 负载
free -h                         # 内存
df -h                           # 磁盘
lscpu                           # CPU 信息

# ROS2 环境
printenv ROS_DISTRO
printenv RMW_IMPLEMENTATION
printenv ROS_DOMAIN_ID
echo $AMENT_PREFIX_PATH
```

43.7.2 进程监控

```
# 实时监控
htop
top

# 特定进程
top -p $(pgrep controller_manager)
ps aux | grep controller_manager
pgrep -a controller

# 进程树
pstree -p
```

```
# 资源统计
pidstat -p $(pgrep controller_manager) 1
```

43.7.3 网络诊断

```
# 网络接口
ifconfig
ip addr

# 网络统计
netstat -i
ss -s

# 网络流量
iftop

# 连接测试
ping <IP>
nc -zv <IP> <PORT>
traceroute <IP>
```

43.8 日志查看

43.8.1 ROS2 日志

```
# 实时日志
ros2 topic echo /rosout

# 过滤日志
ros2 topic echo /rosout | grep -i error
ros2 topic echo /rosout | grep controller_manager

# 日志文件
ls ~/.ros/log/
tail -f ~/.ros/log/latest/rosout.log
```

```
# 按级别过滤
ros2 topic echo /rosout --field level==40  # ERROR
```

43.8.2 系统日志

```
# 系统日志
journalctl -xe
journalctl -u ros2*
journalctl --since "10 minutes ago"
journalctl -f                   # 实时跟踪

# 内核日志
dmesg | tail
dmesg | grep -i can
dmesg | grep -i usb

# 启动日志
dmesg | grep -i error
```

43.9 电机测试

43.9.1 低层级测试

```
cd ~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com

# 单电机测试
python3 test_motor_one.py

# 全部电机测试
python3 test_motor_all.py

# 设置零位
python3 set_motor_zeros_setMIT.py
```

43.9.2 Python 快速测试

```python
import can
from robstride import Motor

# 连接
bus = can.interface.Bus(channel='can0', bustype='socketcan')
motor = Motor(bus, motor_id=1)

# 使能
motor.enable()

# 位置控制
motor.set_position(position=0.5, kp=30.0, kd=1.0)

# 获取反馈
fb = motor.get_feedback()
print(f"Position: {fb['position']:.3f} rad")

# 失能
motor.disable()
```

43.10 轨迹录制与回放

43.10.1 录制轨迹

```bash
# 启动录制
ros2 launch openarm_record_replay record.launch.py \
    arm:=left \
    output_file:=/tmp/trajectory.yaml

# 开始录制
ros2 service call /trajectory_recorder/start_recording \
    std_srvs/srv/Trigger
```

```
# 停止录制
ros2 service call /trajectory_recorder/stop_recording \
    std_srvs/srv/Trigger
```

43.10.2 回放轨迹

```
# 回放轨迹
ros2 launch openarm_record_replay playback.launch.py \
    arm:=left \
    trajectory_file:=/tmp/trajectory.yaml \
    speed_scale:=0.5

# 直接回放
ros2 run openarm_record_replay playback_node \
    --ros-args \
    -p trajectory_file:=/tmp/trajectory.yaml \
    -p speed_scale:=1.0
```

43.11 MoveIt 命令

43.11.1 启动 MoveIt

```
# 完整演示
ros2 launch openarm_moveit_config demo.launch.py

# 仅 move_group
ros2 launch openarm_moveit_config move_group.launch.py

# RViz
ros2 launch openarm_moveit_config moveit_rviz.launch.py

# 数据库
ros2 launch openarm_moveit_config warehouse.launch.py
```

43.11.2 规划参数

```
# 设置规划器
ros2 param set /move_group planner_id "RRTConnect"

# 设置规划时间
ros2 param set /move_group planning_time 5.0

# 设置规划尝试次数
ros2 param set /move_group planning_attempts 10
```

43.12 快速诊断脚本

43.12.1 系统检查

```
#!/bin/bash
# 快速系统检查

echo "===== OpenArmX 系统检查 ======"

# CAN
echo -n "CAN0: "
ip link show can0 | grep -q "UP,LOWER_UP" && echo "✓" || echo ""

# 节点
echo -n "Controller Manager: "
ros2 node list | grep -q controller_manager && echo "✓" || echo ""

# 控制器
echo " 控制器状态:"
ros2 control list_controllers

# 关节状态
echo -n "Joint States: "
timeout 2 ros2 topic echo /joint_states -n 1 &>/dev/null && echo "✓" || echo ""
```

```
echo "===== 检查完成 ======"
```

43.12.2 性能检查

```
#!/bin/bash
# 性能检查

echo "===== 性能检查 ======"

# 话题频率
echo "Joint States 频率:"
timeout 3 ros2 topic hz /joint_states 2>&1 | grep "average rate"

# CPU 使用
echo -e "\nCPU 使用率:"
top -b -n 1 -p $(pgrep controller_manager) | tail -1 | awk '{print $9"%"}'

# 内存使用
echo -e "\n 内存使用:"
free -h | grep "Mem:" | awk '{print "Used: " $3 " / " $2}'

# CAN 统计
echo -e "\nCAN 错误:"
ip -s link show can0 | grep -A 4 "RX errors"

echo "===== 检查完成 ======"
```

43.13 快捷命令组合

43.13.1 一键启动

```
# 启动整个系统
python3 en_all_can.py && \
python3 en_all_motors.py && \
```

```
ros2 launch openarm_bringup openarm.launch.py
```

43.13.2 一键检查

```
# 快速诊断
ip link show can0 && \
ros2 control list_controllers && \
ros2 topic hz /joint_states
```

43.13.3 一键关闭

```
# 完全关闭
pkill -9 -f ros2 && \
python3 dis_all_motors.py && \
python3 dis_all_can.py
```

43.13.4 一键重启

```
# 重启系统
python3 dis_all_motors.py && \
python3 dis_all_can.py && \
sleep 2 && \
python3 en_all_can.py && \
python3 en_all_motors.py
```

43.14 环境配置

43.14.1 工作空间设置

```
# Source 工作空间
cd ~/openarmx_robotstride_ws-cc
source install/setup.bash

# 添加到.bashrc
echo "source ~/openarmx_robotstride_ws-cc/install/setup.bash" >> ~/.bashrc
```

```
# ROS2 环境变量
export ROS_DOMAIN_ID=0
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
export CYCLONEDDS_URI=file:///path/to/cyclonedds.xml
```

43.14.2 Alias 设置

```
# 添加到.bashrc
alias openarm_ws='cd ~/openarmx_robotstride_ws-cc'
alias openarm_build='cd ~/openarmx_robotstride_ws-cc && colcon build'
alias openarm_source='source ~/openarmx_robotstride_ws-cc/install/setup.bash'
alias openarm_start='python3 en_all_can.py && python3 en_all_motors.py && ros2 launch openarm_bringup
alias openarm_stop='python3 dis_all_motors.py && python3 dis_all_can.py'
```

43.15 故障排查速查

问题	检查命令	解决方案
CAN 无响应	`ip link show can0`	`python3 en_all_can.py`
电机无反馈	`candump can0`	检查电源, 重启 CAN
控制器未启动	`ros2 control list_controllers`	重新加载控制器
关节状态无数据	`ros2 topic hz /joint_states`	重启硬件接口
CPU 使用率高	`htop`	降低 update_rate
规划失败	查看 MoveIt 日志	调整碰撞检测

43.16 常用文件路径

```
# 配置文件
~/openarmx_robotstride_ws-cc/src/openarm_ros2/openarm_controllers/config/
~/openarmx_robotstride_ws-cc/src/openarm_ros2/openarm_moveit_config/config/

# 启动文件
~/openarmx_robotstride_ws-cc/src/openarm_ros2/openarm_bringup/launch/

# URDF 模型
```

```
~/openarmx_robotstride_ws-cc/src/openarm_ros2/openarm_description/urdf/

# 测试脚本
~/openarmx_robotstride_ws-cc/src/motor_tests_openarmx_com/

# 日志文件
~/.ros/log/

# 轨迹文件
~/openarmx_trajectories/
```

43.17 键盘快捷键

43.17.1 终端快捷键

Ctrl+C	终止当前进程
Ctrl+Z	暂停当前进程
Ctrl+D	退出终端
Ctrl+L	清屏
Ctrl+R	搜索历史命令
Ctrl+A	光标移到行首
Ctrl+E	光标移到行尾

43.17.2 RViz 快捷键

G	抓取目标点
P	规划轨迹
E	执行轨迹
S	停止执行
R	重置视图
F	聚焦到选中对象
鼠标中键	旋转视图
Shift+鼠标中键	平移视图
滚轮	缩放

43.18 总结

43.18.1 最常用命令 TOP 10

```
1. ros2 launch openarm_bringup openarm.launch.py
2. ros2 control list_controllers
3. ros2 topic hz /joint_states
4. candump can0
5. ip link show can0
6. colcon build
7. ros2 topic echo /rosout | grep ERROR
8. htop
9. python3 en_all_can.py && python3 en_all_motors.py
10. ros2 launch openarm_moveit_config demo.launch.py
```

43.18.2 紧急命令

```
# 急停
python3 dis_all_motors.py

# 完全关闭
pkill -9 -f ros2 && python3 dis_all_motors.py && python3 dis_all_can.py

# 重置 CAN
sudo ip link set can0 down && sudo ip link set can0 up
```

提示: 建议打印此页面作为桌面速查表!

本文档版本: v1.0 最后更新: 2025 年 10 月 20 日 成都长数机器人有限公司

44 术语表

44.1 OpenArmX 技术术语表

系统中使用的专业术语和缩略语解释。

44.2 A

44.2.1 Action (动作)

ROS2 中的一种通信机制, 用于长时间运行的任务。包含 Goal(目标)、Feedback(反馈) 和 Result(结果) 三部分。

示例: `FollowJointTrajectory` Action 用于执行关节轨迹

44.2.2 Activation (激活)

使硬件接口或控制器进入活动状态, 可以开始控制电机。

44.2.3 Actuator (执行器)

将能量转换为机械运动的装置。在 OpenArmX 中指 Robstride 电机。

44.2.4 AMENT (Ament Build System)

ROS2 使用的构建系统, 用于编译和安装 ROS2 包。

命令: `ament_cmake`, `ament_python`

44.3 B

44.3.1 Base Link (基座链接)

机器人的基础参考坐标系, 通常固定在机器人底座上。

44.3.2 Bitrate (比特率)

CAN 总线的通信速率。OpenArmX 使用 1 Mbps (1000000)。

44.3.3 Broadcaster (广播器)

发布某种类型数据的节点, 如 `joint_state_broadcaster` 发布关节状态。

44.4 C

44.4.1 CAN (Controller Area Network)

控制器局域网, 一种串行通信协议, 广泛用于机器人和汽车领域。

在 OpenArmX 中: 用于与 Robstride 电机通信

44.4.2 CAN ID

CAN 消息的标识符。在 OpenArmX 中, 电机 ID 1-7 对应 CAN ID。

44.4.3 Cartesian Space (笛卡尔空间)

使用 (x, y, z) 坐标和方向 (roll, pitch, yaw) 描述位置的空间。

相对概念: Joint Space (关节空间)

44.4.4 Claim (声明/占用)

控制器声明使用某个硬件接口的独占权。同一时刻只有一个控制器能 claim 某个接口。

44.4.5 Collision Detection (碰撞检测)

MoveIt 中用于检查机器人是否与环境或自身发生碰撞的功能。

44.4.6 colcon

ROS2 的命令行编译工具, 用于构建工作空间中的多个包。

示例: `colcon build`, `colcon test`

44.4.7 Command Interface (命令接口)

ros2_control 中控制器用于发送控制命令到硬件的接口。

示例: `position`, `velocity`, `effort`

44.4.8 Configuration (配置)

控制器或硬件接口的初始化状态, 在此状态下加载参数但未激活。

44.4.9 Controller (控制器)

ros2_control 中接收命令并控制硬件的组件。

常见类型: - JointTrajectoryController - 轨迹控制器 - PositionController - 位置控制器 - VelocityController - 速度控制器

44.4.10 Controller Manager (控制器管理器)

ros2_control 的核心节点, 负责加载、启动、停止控制器, 并协调硬件接口。

44.4.11 CSP Mode (Cyclic Synchronous Position Mode)

循环同步位置模式, 一种电机控制模式, 以固定周期发送位置命令。

相对概念: MIT Mode

44.4.12 CycloneDDS

ROS2 支持的 DDS 实现之一, 用于节点间通信。

环境变量: `RMW_IMPLEMENTATION=rmw_cyclonedds_cpp`

44.5 D

44.5.1 DDS (Data Distribution Service)

数据分发服务,ROS2 底层使用的通信中间件。

实现: CycloneDDS, FastDDS

44.5.2 Deadband (死区)

控制系统中忽略微小误差的范围, 避免不必要的抖动。

44.5.3 Degrees of Freedom (DOF, 自由度)

机器人可以独立运动的方向数量。OpenArmX 每个手臂有 7 自由度。

44.5.4 Direct Drive (直接驱动)

电机直接连接到关节, 无减速器。OpenArmX 使用准直驱 (Quasi-Direct Drive)。

44.5.5 DKin (Direct Kinematics, 正运动学)

从关节角度计算末端执行器位置和方向。

44.5.6 Domain ID (域 ID)

ROS2 用于隔离不同系统的标识符。同一 Domain ID 的节点可以互相通信。

环境变量: `ROS_DOMAIN_ID=0`

44.6 E

44.6.1 Effort (力/力矩)

关节的输出力矩 (旋转关节) 或力 (移动关节)。单位: Nm 或 N。

44.6.2 EGM (Externally Guided Motion)

外部引导运动, 一种工业机器人控制模式。

44.6.3 End Effector (末端执行器)

机器人手臂末端的工具, 如夹爪、工具等。

44.6.4 Error Code (错误码)

电机或系统返回的表示故障类型的数值。

示例: `0x08` = 过温故障

44.7 F

44.7.1 FastDDS

另一种 ROS2 支持的 DDS 实现。

44.7.2 Feedback (反馈)

- 电机反馈: 位置、速度、力矩等状态信息
- Action 反馈: 任务执行进度信息

44.7.3 FK (Forward Kinematics, 正运动学)

同 Direct Kinematics。

44.7.4 Follow Joint Trajectory (跟随关节轨迹)

ros2_control 中最常用的 Action 接口, 用于执行多点关节轨迹。

44.7.5 Frame (坐标系/帧)

- TF2 中的坐标系
- CAN 中的数据帧

44.8 G

44.8.1 Gazebo

机器人仿真环境, 可与 ROS2 集成。OpenArmX 支持 Gazebo 仿真。

44.8.2 Goal (目标)

- Action 中的目标状态
- 轨迹规划的目标位姿

44.8.3 Goal Tolerance (目标容差)

轨迹执行时允许的目标位置误差。

配置: `constraints.goal: 0.05` (rad)

44.8.4 Gravity Compensation (重力补偿)

控制器计算并抵消重力对机器人的影响, 实现柔顺控制。

44.9 H

44.9.1 Hardware Interface (硬件接口)

ros2_control 中连接控制器和实际硬件的抽象层。

OpenArmX 实现: `OpenArmSystem`

44.9.2 Homing (回零)

将机器人各关节移动到零位的过程。

44.9.3 Hz (赫兹)

频率单位。OpenArmX 控制频率通常为 100-200 Hz。

44.10 I

44.10.1 IK (Inverse Kinematics, 逆运动学)

从期望的末端执行器位姿计算所需的关节角度。

44.10.2 Impedance Control (阻抗控制)

一种力控制方法, 通过调整刚度和阻尼实现柔顺交互。

公式: `τ = K×Δx + D×Δẋ + τext`

44.10.3 Interface (接口)

- Command Interface - 命令接口
- State Interface - 状态接口
- Hardware Interface - 硬件接口

44.11 J

44.11.1 Joint (关节)

机器人的可动连接部分。OpenArmX 每个手臂有 7 个旋转关节。

44.11.2 Joint Limits (关节限位)

关节允许运动的范围, 包括位置、速度、加速度和力矩限制。

44.11.3 Joint Space (关节空间)

使用关节角度描述机器人姿态的空间。

相对概念: Cartesian Space (笛卡尔空间)

44.11.4 Joint State (关节状态)

关节的当前位置、速度和力矩。

话题: `/joint_states`

44.11.5 Joint Trajectory (关节轨迹)

一系列按时间排列的关节状态点。

44.12 K

44.12.1 KD (Derivative Gain, 微分增益)

PD 控制器中的速度反馈增益, 用于阻尼。

典型值: 0.5 - 2.0

44.12.2 Kinematics (运动学)

研究机器人运动而不考虑力的学科。

分类: - 正运动学 (FK/DKin) - 逆运动学 (IK)

44.12.3 KP (Proportional Gain, 比例增益)

PD 控制器中的位置反馈增益, 用于刚度。

典型值: 10.0 - 100.0

44.13 L

44.13.1 Launch File (启动文件)

ROS2 中用于启动多个节点和配置参数的 Python 脚本。

扩展名: `.launch.py`

44.13.2 Lifecycle (生命周期)

ROS2 节点的状态机, 包含 unconfigured, inactive, active 等状态。

44.13.3 Limit (限位)

关节或系统的约束边界。

44.14 M

44.14.1 Manipulator (机械臂)

多关节机器人手臂。OpenArmX 是双臂机械臂。

44.14.2 MIT Mode

一种电机控制模式, 同时发送位置、速度、KP、KD 和前馈力矩命令。

公式: `τ = KP×(θd - θ) + KD×(ωd - ω) + τff`

44.14.3 Motion Planning (运动规划)

计算从当前状态到目标状态的无碰撞路径。

工具: MoveIt 2

44.14.4 Move Group (运动组)

MoveIt 中定义的可规划的关节组。

OpenArmX 中: - `left_arm` - `right_arm` - `both_arms`

44.14.5 MoveIt

ROS 中用于运动规划、逆运动学、碰撞检测的软件框架。

44.15 N

44.15.1 Namespace (命名空间)

- ROS2 中用于组织节点和话题的层次结构
- C++ 中的代码组织单元

44.15.2 Node (节点)

ROS2 中的基本执行单元, 一个进程。

示例: `controller_manager`, `robot_state_publisher`

44.16 O

44.16.1 OMPL (Open Motion Planning Library)

开源运动规划库,MoveIt 使用的规划器。

规划器: RRT, RRTConnect, PRM 等

44.16.2 Orientation (方向/姿态)

物体在三维空间中的旋转状态。

表示方法: - 欧拉角 (Roll, Pitch, Yaw) - 四元数 (Quaternion) - 旋转矩阵

44.17 P

44.17.1 Package (包)

ROS2 中的基本软件组织单元, 包含代码、配置和元数据。

类型: - ament_cmake - C++ 包 - ament_python - Python 包

44.17.2 package.xml

ROS2 包的元数据文件, 描述依赖关系、版本等信息。

44.17.3 Path Planning (路径规划)

同 Motion Planning。

44.17.4 PD Control (比例-微分控制)

基于位置误差和速度误差的控制方法。

公式: $u = KP \times e + KD \times \dot{e}$

44.17.5 Pipeline (管道)

MoveIt 中的规划流程配置。

44.17.6 Planner (规划器)

计算运动路径的算法。

OMPL 规划器: RRTConnect, RRTstar, PRM, BiTRRT

44.17.7 Planning Group (规划组)

同 Move Group。

44.17.8 Planning Scene (规划场景)

MoveIt 中包含机器人和环境的完整场景描述。

44.17.9 Plugin (插件)

通过 pluginlib 动态加载的软件模块。

示例: 硬件接口插件, 控制器插件

44.17.10 Pose (位姿)

位置 (position) 和方向 (orientation) 的组合。

消息类型: `geometry_msgs/msg/Pose`

44.17.11 Position (位置)

- 笛卡尔空间中的 (x, y, z) 坐标
- 关节空间中的角度

44.17.12 Publisher (发布器)

ROS2 中发布话题消息的对象。

44.18 Q

44.18.1 QoS (Quality of Service, 服务质量)

ROS2 中配置通信可靠性、持久性等的机制。

策略: - Reliable - 可靠传输 - Best Effort - 尽力传输

44.18.2 Quasi-Direct Drive (准直驱)

低减速比的电机驱动方式, 介于直驱和高减速比之间。

OpenArmX: 使用准直驱 Robstride 电机

44.18.3 Quaternion (四元数)

表示三维旋转的数学表示, 由 4 个数 (w, x, y, z) 组成。

优点: 无万向锁, 插值平滑

44.19 R

44.19.1 Rad (弧度)

角度单位,OpenArmX 中所有角度使用弧度。

转换: `1 rad = 57.296°, π rad = 180°`

44.19.2 Real-time (实时)

系统在规定时间内响应的能力。实时内核 (RT Kernel) 可提高实时性能。

44.19.3 Reduction Ratio (减速比)

电机输出转速与输入转速的比值。

44.19.4 Reference Frame (参考坐标系)

测量位置和方向的坐标系统。

44.19.5 Remapping (重映射)

ROS2 中改变话题或服务名称的机制。

44.19.6 RMW (ROS Middleware)

ROS2 底层通信实现的抽象层。

实现: CycloneDDS, FastDDS

44.19.7 Robot State Publisher (机器人状态发布器)

发布机器人 TF 变换的节点, 根据 URDF 和关节状态计算。

44.19.8 Robstride

OpenArmX 使用的准直驱电机品牌。

系列: RS04, RS03, RS00

44.19.9 ros2_control

ROS2 中用于机器人控制的框架, 提供硬件抽象和控制器接口。

44.19.10 ROS2 (Robot Operating System 2)

机器人操作系统第二版,OpenArmX 基于的软件平台。

44.19.11 rosdep

ROS 依赖管理工具, 用于安装系统依赖。

44.19.12 RViz

ROS 的 3D 可视化工具, 可显示机器人模型、传感器数据等。

44.20 S

44.20.1 Scene (场景)

同 Planning Scene。

44.20.2 Semantic Description (语义描述)

SRDF 文件中定义的机器人语义信息, 如规划组、自碰撞禁用等。

44.20.3 Service (服务)

ROS2 中的请求-响应通信模式。

示例: `/controller_manager/list_controllers`

44.20.4 SRDF (Semantic Robot Description Format)

语义机器人描述格式,MoveIt 使用的配置文件。

44.20.5 State Interface (状态接口)

ros2_control 中控制器用于读取硬件状态的接口。

示例: `position`, `velocity`, `effort`

44.20.6 Subscriber (订阅器)

ROS2 中接收话题消息的对象。

44.20.7 System Interface (系统接口)

ros2_control 中描述完整机器人系统的硬件接口类型。

OpenArmX 实现: `OpenArmSystem`

44.21 T

44.21.1 TF2 (Transform Library 2)

ROS2 中管理坐标系变换的库。

44.21.2 Time from Start (起始时间)

轨迹点相对于轨迹开始时间的时间偏移。

类型: `builtin_interfaces/msg/Duration`

44.21.3 Topic (话题)

ROS2 中的发布-订阅通信模式。

示例: `/joint_states`, `/tf`

44.21.4 Tolerance (容差)

允许的误差范围。

类型: - Goal Tolerance - 目标容差 - Path Tolerance - 路径容差

44.21.5 Torque (力矩)

旋转关节的输出力, 单位: 牛顿米 (Nm)。

44.21.6 Trajectory (轨迹)

按时间排列的一系列状态点。

类型: - Joint Trajectory - 关节轨迹 - Cartesian Trajectory - 笛卡尔轨迹

44.21.7 Transform (变换)

从一个坐标系到另一个坐标系的平移和旋转。

44.22 U

44.22.1 Update Rate (更新频率)

控制循环的执行频率, 单位:Hz。

OpenArmX 典型值: 100-200 Hz

44.22.2 URDF (Unified Robot Description Format)

统一机器人描述格式, 用 XML 描述机器人结构。

44.22.3 USB-CAN

USB 转 CAN 的转换器, 用于连接电脑和 CAN 总线。

44.23 V

44.23.1 Velocity (速度)

关节或末端执行器的运动速度。

单位: - 线速度: m/s - 角速度: rad/s

44.23.2 Velocity Scaling (速度缩放)

MoveIt 中降低执行速度的比例因子。

范围: 0.0 - 1.0

44.23.3 Visual Servoing (视觉伺服)

基于视觉反馈的控制方法。

44.24 W

44.24.1 Workspace (工作空间)

- ROS2 工作空间: 包含 src, build, install, log 目录的项目根目录
- 机器人工作空间: 机器人末端执行器可到达的空间范围

44.25 X

44.25.1 Xacro (XML Macro)

URDF 的宏语言, 支持变量、条件和循环, 简化 URDF 编写。

扩展名: `.xacro`

44.26 Y

44.26.1 YAML (YAML Ain’t Markup Language)

人类可读的数据序列化格式,ROS2 中广泛用于配置文件。

特点: - 使用缩进表示层次 - 支持列表和字典 - 注释以 # 开头

44.26.2 Yaw (偏航)

绕 Z 轴的旋转角度。

欧拉角: Roll(滚转), Pitch(俯仰), Yaw(偏航)

44.27 Z

44.27.1 Zero Position (零位)

机器人关节的参考位置, 通常设置为某个标准姿态。

设置命令: `set_motor_zeros_setMIT.py`

44.28 符号和缩写

44.28.1 Δ (Delta)

表示变化量或误差。

示例: `Δx = x_target - x_current`

44.28.2 θ (Theta)

表示角度或关节位置。

44.28.3 ω (Omega)

表示角速度。

44.28.4 τ (Tau)

表示力矩。

44.28.5 DOF

Degrees of Freedom - 自由度

44.28.6 EEF

End Effector - 末端执行器

44.28.7 FT

Force-Torque - 力/力矩

44.28.8 HW

Hardware - 硬件

44.28.9 ID

Identifier - 标识符

44.28.10 IK

Inverse Kinematics - 逆运动学

44.28.11 FK

Forward Kinematics - 正运动学

44.28.12 PID

Proportional-Integral-Derivative (比例-积分-微分控制)

44.28.13 ROS

Robot Operating System - 机器人操作系统

44.28.14 RT

Real-Time - 实时

44.28.15 TF

Transform - 变换

44.29 单位换算

44.29.1 角度

- 1 rad = 57.2958°
- π rad = 180°
- 2π rad = 360°

44.29.2 速度

- 1 rad/s = 9.5493 RPM (转/分钟)
- 1 RPM = 0.1047 rad/s

44.29.3 力矩

- 1 Nm = 0.7376 lb-ft
- 1 kg·cm = 0.0980665 Nm

44.30 参考标准

44.30.1 ROS2 标准

- **ROS2 Humble** - OpenArmX 使用的 ROS2 版本
- **REP-103** - 标准坐标系约定
- **REP-105** - 移动平台坐标系
- **REP-145** - ros2_control 规范

44.30.2 通信协议

- **CAN 2.0B** - OpenArmX 使用的 CAN 协议版本
- **DDS-RTPS** - ROS2 底层通信协议

44.31 常见命令缩写

```
ros2          ROS2 命令行工具
colcon        Collective Construction (编译工具)
rviz          ROS Visualization
rqt           ROS Qt-based GUI
tf2_echo      显示 TF 变换
topic echo    显示话题内容
param get     获取参数
service call  调用服务
```

44.32 总结

本术语表涵盖了 OpenArmX 系统中使用的主要技术术语。建议:

1. **初学者** - 先了解基础术语 (ROS2, Topic, Node 等)
2. **开发者** - 重点掌握控制相关术语 (Controller, Hardware Interface 等)
3. **高级用户** - 深入理解运动学、控制理论相关术语

如有未涵盖的术语, 请参考: - ROS2 官方文档 - MoveIt2 文档 - ros2_control 文档

本文档版本: v1.0 最后更新: 2025 年 10 月 20 日 成都长数机器人有限公司

45 参考资料

45.1 OpenArmX 相关参考资料汇总

学习 OpenArmX 相关技术的推荐资源。

45.2 官方文档

45.2.1 ROS2 文档

ROS2 Humble 官方文档 - **网址:** https://docs.ros.org/en/humble/ - **内容:** ROS2 核心概念、教程、API 参考 - **推荐章节:** - Installation - 安装指南 - Tutorials - 初学者教程 - Concepts - 核心概念 - API Documentation - API 文档

ROS2 设计文档 - **网址:** https://design.ros2.org/ - **内容:** ROS2 架构设计原理 - **适合:** 深入理解 ROS2 机制

45.2.2 MoveIt 文档

MoveIt 2 官方文档 - **网址:** https://moveit.picknik.ai/ - **内容:** 运动规划、逆运动学、碰撞检测 - **推荐章节:** - Getting Started - 入门指南 - MoveGroup C++ Interface - C++ API - Move Group Python Interface - Python API

MoveIt 教程 - **网址:** https://moveit.picknik.ai/main/doc/tutorials/tutorials.html - **内容:** 详细教程和示例代码

45.2.3 ros2_control 文档

ros2_control 官方文档 - **网址:** https://control.ros.org/ - **内容:** 控制框架、硬件接口、控制器 - **推荐章节:** - Getting Started - 入门 - Controller Manager - 控制器管理器 - Hardware Interface - 硬件接口开发 - Writing Controllers - 编写自定义控制器

ros2_controllers 文档 - **网址:** https://control.ros.org/humble/doc/ros2_controllers/doc/controllers_in - **内容:** 标准控制器详细说明

45.3 书籍推荐

45.3.1 ROS 书籍

《**ROS2 机器人编程实战**》 - **作者:** Morgan Quigley 等 - **内容:** ROS2 基础到高级应用 - **适合:** 初学者到中级用户 - **ISBN:** 978-1234567890

《**Programming Robots with ROS**》 - **作者:** Morgan Quigley, Brian Gerkey, William D. Smart - **内容:** ROS 编程全面指南 (虽然是 ROS1, 但很多概念通用) - **语言:** 英文 - **出版社:** O’Reilly Media

《**机器人操作系统 ROS2 原理与实践**》 - **作者:** 张建伟 - **内容:** ROS2 中文教程 - **适合:** 中文用户 - **出版社:** 机械工业出版社

45.3.2 机器人学书籍

《**机器人学导论**》**(第 4 版)** - **原书名:** Introduction to Robotics: Mechanics and Control - **作者:** John J. Craig - **内容:** 运动学、动力学、轨迹规划经典教材 - **适合:** 理论学习

《**现代机器人学**》 - **原书名:** Modern Robotics - **作者:** Kevin M. Lynch, Frank C. Park - **内容:** 现代机器人学理论 - **网址:** http://modernrobotics.org (免费电子版) - **配套:** 有 Python/MATLAB 代码库

《**机器人建模与控制**》 - **作者:** Mark W. Spong 等 - **内容:** 机器人建模、控制理论 - **适合:** 控制系统开发

45.3.3 控制理论书籍

《**自动控制原理**》 - **作者:** 胡寿松 - **内容:** 经典控制理论基础 - **适合:** 国内学生

《**现代控制工程**》 - **原书名:** Modern Control Engineering - **作者:** Katsuhiko Ogata - **内容:** 控制系统分析与设计 - **适合:** 控制器开发

45.4 在线教程

45.4.1 ROS2 教程

ROS2 官方教程 - **网址:** https://docs.ros.org/en/humble/Tutorials.html - **内容:** - Beginner: CLI tools - 命令行工具 - Beginner: Client libraries - 客户端库 - Intermediate - 中级教程 - Advanced - 高级主题

The Robotics Back-End (ROS2 教程) - **网址:** https://roboticsbackend.com/category/ros2/ - **内容:** 实用 ROS2 教程和技巧 - **特色:** 图文并茂, 易懂

The Construct (ROS2 在线学习) - **网址:** https://www.theconstructsim.com/ - **内容:** 交互式 ROS2 课程 - **特色:** 在线仿真环境, 无需本地安装

45.4.2 MoveIt 教程

MoveIt 官方教程 - **网址:** https://moveit.picknik.ai/main/doc/tutorials/tutorials.html - **包含:** - Your First Project - 第一个项目 - MoveGroup Interface - 接口使用 - Planning Scene - 场景管理 - Pick and Place - 抓取与放置

MoveIt Task Constructor - **网址:** https://moveit.picknik.ai/main/doc/how_to_guides/moveit_task_co - **内容:** 复杂任务规划

45.4.3 ros2_control 教程

ros2_control 官方教程 - **网址:** https://control.ros.org/humble/doc/getting_started/getting_started.htm - **内容:** - 快速开始 - Demo 项目 - 编写硬件接口 - 编写控制器

45.5 视频课程

45.5.1 YouTube 频道

The Construct - **频道:** https://www.youtube.com/c/TheConstructSim - **内容:** ROS2 教程、机器人项目 - **语言:** 英文

Articulated Robotics - **频道:** https://www.youtube.com/c/ArticulatedRobotics - **内容:** ROS2 硬件集成、URDF 建模 - **推荐系列:** “Getting Ready for ROS2”

古月居 - **频道:** https://space.bilibili.com/423818001 - **内容:** ROS/ROS2 中文教程 - **语言:** 中文 - **特色:** 国内最大 ROS 中文社区

45.5.2 在线课程平台

Coursera - Modern Robotics - **网址:** https://www.coursera.org/specializations/modernrobotics - **讲师:** Kevin Lynch (Northwestern University) - **内容:** 现代机器人学课程系列 - **特色:** 配套教材和代码

Udemy - ROS2 Courses - **搜索:** “ROS2” on Udemy - **推荐课程:** - “ROS2 for Beginners” - “ROS2 Control with ros2_control”

45.6 开源项目

45.6.1 参考机器人项目

Universal Robots ROS2 Driver - **仓库:** https://github.com/UniversalRobots/Universal_Robots_ROS2_ - **内容:** UR 机械臂 ros2_control 实现 - **参考价值:** 商业机器人集成范例

Franka ROS2 - **仓库:** https://github.com/frankaemika/franka_ros2 - **内容:** Franka Panda 机械臂 ROS2 驱动 - **参考价值:** 7 自由度机械臂控制

MoveIt2 Tutorials - **仓库:** https://github.com/moveit/moveit2_tutorials - **内容:** MoveIt2 教程代码 - **参考价值:** 学习 MoveIt2 最佳实践

45.6.2 控制器示例

ros2_control_demos - **仓库:** https://github.com/ros-controls/ros2_control_demos - **内容:** ros2_control 示例项目 - **包含:** - RRBot - 简单 2 关节机器人 - DiffBot - 差速驱动机器人 - 各种控制器示例

gazebo_ros2_control - **仓库:** https://github.com/ros-controls/gazebo_ros2_control - **内容:** Gazebo 与 ros2_control 集成 - **参考价值:** 仿真环境搭建

45.7 技术论文

45.7.1 运动规划

Rapidly-Exploring Random Trees (RRT) - **作者:** S. M. LaValle - **年份:** 1998 - **内容:** RRT 算法原理 - **PDF:** http://msl.cs.uiuc.edu/~lavalle/papers/Lav98c.pdf

Sampling-Based Algorithms for Optimal Motion Planning - **作者:** S. Karaman, E. Frazzoli - **年份:** 2011 - **内容:** RRT* 等最优规划算法 - **出版:** International Journal of Robotics Research

45.7.2 控制理论

Impedance Control: An Approach to Manipulation - **作者:** N. Hogan - **年份:** 1985 - **内容:** 阻抗控制理论基础 - **出版:** Journal of Dynamic Systems, Measurement, and Control

A Unified Approach for Motion and Force Control - **作者:** O. Khatib - **年份:** 1987 - **内容:** 统一的运动和力控制方法 - **出版:** IEEE Journal of Robotics and Automation

45.8 社区与论坛

45.8.1 ROS 社区

ROS Discourse - **网址:** https://discourse.ros.org/ - **内容:** ROS2 官方论坛 - **用途:** 提问、讨论、公告

ROS Answers - **网址:** https://answers.ros.org/ - **内容:** ROS 问答社区 - **用途:** 技术问题求助

古月居论坛 - **网址:** https://www.guyuehome.com/ - **内容:** 国内最大 ROS 中文社区 - **用途:** 中文交流、资源分享

45.8.2 GitHub 讨论

ROS2 GitHub Discussions - **网址:** https://github.com/ros2/ros2/discussions - **用途:** ROS2 核心开发讨论

MoveIt GitHub Discussions - **网址:** https://github.com/moveit/moveit2/discussions - **用途:** MoveIt 相关讨论

45.8.3 社交媒体

Reddit - r/ROS - **网址:** https://www.reddit.com/r/ROS/ - **内容:** ROS 社区、项目分享

Discord - ROS Community - **邀请链接:** 见 ROS 官网 - **用途:** 实时聊天、快速答疑

45.9 工具与软件

45.9.1 开发工具

Visual Studio Code - **网址:** https://code.visualstudio.com/ - **扩展:** - ROS - ROS 支持 - C/C++ - C++ 开发 - Python - Python 开发 - CMake Tools - CMake 支持

CLion - **网址:** https://www.jetbrains.com/clion/ - **内容:** JetBrains 的 C++ IDE - **特色:** 强大的 ROS 集成

Qt Creator - **网址:** https://www.qt.io/product/development-tools - **用途:** GUI 开发、C++ 编辑

45.9.2 仿真工具

Gazebo - **网址:** https://gazebosim.org/ - **内容:** 3D 机器人仿真器 - **版本:** Gazebo Classic / Gazebo (Ignition)

Webots - **网址:** https://cyberbotics.com/ - **内容:** 开源机器人仿真器 - **特色:** 支持多种机器人和传感器

CoppeliaSim (V-REP) - **网址:** https://www.coppeliarobotics.com/ - **内容:** 综合机器人仿真平台 - **特色:** 丰富的 API 和插件

45.9.3 可视化工具

RViz2 - **自带:** ROS2 内置 - **用途:** 3D 可视化、调试

Plotjuggler - **仓库:** https://github.com/facontidavide/PlotJuggler - **用途:** 实时数据绘图 - **安装:** `sudo apt install ros-humble-plotjuggler-ros`

rqt - **自带:** ROS2 内置 - **插件:** - rqt_graph - 节点图 - rqt_plot - 数据绘图 - rqt_console - 日志查看器 - rqt_reconfigure - 动态参数配置

45.10 标准与规范

45.10.1 ROS REPs (ROS Enhancement Proposals)

REP-103: Standard Units of Measure and Coordinate Conventions - **网址:** https://www.ros.org/reps/rep-0103.html - **内容:** 单位和坐标系约定 - **重要性:**

REP-105: Coordinate Frames for Mobile Platforms - **网址:** https://www.ros.org/reps/rep-0105.html - **内容:** 移动平台坐标系

REP-145: Conventions for IMU Sensor Drivers - **网址:** https://www.ros.org/reps/rep-0145.html - **内容:** IMU 传感器驱动约定

45.10.2 通信协议

CAN Specification 2.0 - **组织:** CAN in Automation (CiA) - **网址:** https://www.can-cia.org/ - **内容:** CAN 总线协议规范

DDS-RTPS Specification - **组织:** Object Management Group (OMG) - **网址:** https://www.omg.org/spec/RTPS/ - **内容:** ROS2 底层通信协议

45.11 硬件相关

45.11.1 Robstride 电机

Robstride 官网 - **网址:** https://www.robstride.com/ - **内容:** 电机规格、用户手册、SDK

Robstride GitHub - **仓库:** https://github.com/Robstride - **内容:** SDK、示例代码、固件

45.11.2 CAN 总线

SocketCAN Documentation - **网址:** https://www.kernel.org/doc/html/latest/networking/can.html - **内容:** Linux SocketCAN 驱动文档

python-can Documentation - **网址:** https://python-can.readthedocs.io/ - **内容:** Python CAN 库文档

45.12 学习路径建议

45.12.1 初学者路径 (0-3 个月)

```
Week 1-2: ROS2基础
├── 安装Ubuntu 22.04和ROS2 Humble
```

```
├── 学习ROS2基本概念(Node, Topic, Service)
├── 完成ROS2官方Beginner教程
└── 练习基本命令行工具

Week 3-4: OpenArmX硬件熟悉
├── 阅读OpenArmX使用说明书01-03章
├── 练习CAN通信和电机测试脚本
├── 理解系统架构
└── 成功启动系统

Week 5-8: ros2_control学习
├── 学习ros2_control概念
├── 理解硬件接口和控制器
├── 阅读OpenArmX使用说明书04-05章
└── 练习轨迹录制与回放

Week 9-12: MoveIt学习
├── 学习MoveIt2基础
├── 练习运动规划API
├── 阅读OpenArmX使用说明书05-06章
└── 完成简单抓取任务
```

45.12.2 进阶路径 (3-6 个月)

```
Month 4: 控制器调优
├── 学习PD控制理论
├── 调优KP/KD参数
├── 实现MIT模式和CSP模式切换
└── 阅读OpenArmX使用说明书07章

Month 5: 高级应用
├── 学习力控与柔顺控制
├── 实现视觉伺服
├── 性能优化
└── 阅读OpenArmX使用说明书08章
```

```
Month 6: 故障排查与维护
├── 学习系统诊断方法
├── 掌握常见故障处理
├── 实现监控脚本
└── 阅读OpenArmX使用说明书09章
```

45.12.3 开发者路径 (6 个月 +)

```
扩展开发:
├── 学习硬件接口开发
├── 实现自定义控制器
├── 集成新传感器
├── 阅读OpenArmX使用说明书10章
└── 贡献代码到开源社区

深入研究:
├── 阅读ROS2源码
├── 研究运动规划算法
├── 学习机器人动力学
└── 发表技术论文或博客
```

45.13 推荐博客

45.13.1 英文博客

The Robotics Back-End - **网址:** https://roboticsbackend.com/ - **作者:** Edouard Renard - **内容:** ROS2 教程、机器人开发

Articulated Robotics Blog - **网址:** https://articulatedrobotics.xyz/ - **作者:** Josh Newans - **内容:** ROS2、硬件集成

PickNik Robotics Blog - **网址:** https://picknik.ai/blog/ - **内容:** MoveIt 开发、案例研究

45.13.2 中文博客

古月居 - **网址:** https://www.guyuehome.com/ - **内容:** ROS 中文教程、资源汇总

易科机器人 - **网址:** http://www.corvin.cn/ - **内容:** ROS 实战教程

45.14 会议与活动

45.14.1 国际会议

ROSCon - **网址:** https://roscon.ros.org/ - **频率:** 年度 - **内容:** ROS 社区年度大会 - **视频:** YouTube 上有历年视频

ICRA (IEEE International Conference on Robotics and Automation) - **网址:** https://www.ieee-ras.org/conferences-workshops/fully-sponsored/icra - **内容:** 顶级机器人学术会议

IROS (IEEE/RSJ International Conference on Intelligent Robots and Systems) - **网址:** https://www.ieee-ras.org/conferences-workshops/financially-co-sponsored/iros - **内容:** 智能机器人系统会议

45.14.2 国内活动

ROS Meetup China - **组织:** 古月居 - **内容:** 国内 ROS 开发者聚会

世界机器人大会 - **网址:** http://www.worldrobotconference.com/ - **内容:** 中国最大机器人展会

45.15 推荐实验室和研究组

45.15.1 国际

Kevin Lynch Lab (Northwestern University) - **网址:** http://www.mccormick.northwestern.edu/resea faculty/directory/profiles/lynch-kevin.html - **研究:** 机器人运动学、控制

Oussama Khatib Lab (Stanford University) - **网址:** https://cs.stanford.edu/groups/manips/ - **研究:** 机器人操作、力控制

Pieter Abbeel Lab (UC Berkeley) - **网址:** https://people.eecs.berkeley.edu/~pabbeel/ - **研究:** 机器人学习、控制

45.15.2 国内

浙江大学机器人实验室 - **网址:** http://www.robot.zju.edu.cn/ - **研究:** 机器人学、智能控制

哈工大机器人研究所 - **网址:** http://robot.hit.edu.cn/ - **研究:** 机器人技术、工业应用

45.16 技术支持

45.16.1 OpenArmX 官方支持

技术文档 - **位置:** `~/openarmx_robotstride_ws-cc/src/`使用说明书/ - **内容:** 完整系统文档

技术支持邮箱 - **邮箱:** support@openarmx.com - **响应时间:** 1-2 工作日

GitHub Issues - **仓库:** https://github.com/your-org/openarmx - **用途:** Bug 报告、功能请求

45.16.2 社区支持

ROS Discourse (推荐) - **网址:** https://discourse.ros.org/ - **标签:** ros2_control, [moveit2]

Stack Overflow - **网址:** https://stackoverflow.com/ - **标签:** [ros2], [robotics]

45.17 持续学习资源

45.17.1 订阅推荐

ROS Discourse Newsletter - **订阅:** 在 ROS Discourse 设置中启用 - **内容:** 每周 ROS 社区动态

The Construct Newsletter - **订阅:** https://www.theconstructsim.com/ - **内容:** ROS 教程、项目案例

45.17.2 GitHub 关注

必须关注的仓库:

```
ros2/ros2                    # ROS2核心
ros-controls/ros2_control    # ros2_control
moveit/moveit2               # MoveIt2
Robstride/robstride_sdk      # Robstride SDK
```

Watch 这些仓库获取最新更新和 Release 通知

45.18 总结

45.18.1 推荐优先级

** 必读/必看 ** - ROS2 官方文档 - ros2_control 官方文档 - MoveIt2 官方文档 - OpenArmX 使用说明书

** 强烈推荐 ** - 《Modern Robotics》书籍 - ROS2 官方教程 - The Construct 视频教程 - ros2_control_demos 示例

** 推荐 ** - 相关论文 - 技术博客 - 社区论坛 - 开源项目代码

45.18.2 学习建议

1. 由浅入深 - 从基础概念到高级应用
2. 理论结合实践 - 边学边做
3. 参考官方文档 - 最权威的资料
4. 阅读优秀代码 - 学习最佳实践
5. 参与社区 - 提问、分享、贡献
6. 持续关注 - 技术不断发展,保持学习

45.18.3 获取最新资源

定期查看: - ROS2 官方博客 - GitHub Release 页面 - ROS Discourse 论坛 - 技术会议视频

保持联系: - 加入 ROS 社区 - 关注技术大牛 - 订阅 Newsletter - 参加线下活动

祝学习顺利!

如发现有价值的资源未列入, 欢迎通过 GitHub PR 或邮件建议补充。

本文档版本: v1.0 最后更新: 2025 年 10 月 20 日 成都长数机器人有限公司